



C-style C++ Summary 2024

Grimy86

Author Note

This summary is entirely possible thanks to the authors of the Learncpp website (**Alex, Nascardriver** and **James C.**) who made their knowledge available for public use. The summary itself has some more information in addition to the information from Learncpp. I will also be splitting this summary up to cover chapters that are related to C and keep the modern C++ style chapters to a minimum.

1	KEYWORDS	4
2	INTRODUCTION / GETTING STARTED	14
3	BASICS	31
4	FUNCTIONS AND FILES	65
5	DEBUGGING	109
6	FUNDAMENTAL DATA TYPES	110
7	CONSTANTS AND STRINGS	145
8	OPERATORS	216
9	OPTIONAL CHAPTER: BIT MANIPULATION	237
10	SCOPE, DURATION AND LINKAGE	250
11	CONTROL FLOW	302
12	ERROR DETECTION AND HANDLING	348
13	TYPE CONVERSION, TYPE ALIASES, AND TYPE DEDUCTION	382
14	FUNCTION OVERLOADING AND FUNCTION TEMPLATES	418
15	COMPOUND TYPES: REFERENCES AND POINTERS	462
16	COMPOUND TYPES: ENUMS AND STRUCTS	536
17	SUMMARY INTERMEZZO	585
18	C-STYLE: ARRAYS AND STRINGS	586
19	DYNAMIC ALLOCATION	603
20	FUNCTIONS	617

1 Keywords

Computer program, application

Hardware, physical computer parts

Bit, binary digit

Assembler, a program that translates assembly language into machine language

Compiler, a program (or collection of programs) that reads source code (typically written in a high-level language) and translates it into some other language (typically a low-level language, such as assembly or machine language, etc.).

Interpreter, a program that directly executes the instructions in the source code without requiring them to be compiled into an executable first.

ANSI, the American National Standards Institute

ISO, the International Organization for Standardization

Language standard or **language specification**, each new formal release of the language.

Linker, the linker's job is to combine all of the object files and produce the desired output file (typically an executable file). This process is called **linking**.

Building, the full process of converting source code files into an output file.

Build configurations or **build target**, a collection of project settings that determines how your IDE will build your project.

Compiler extensions, many compilers implement their own changes to the language, often to enhance compatibility with other versions of the language, or for historical reasons.

Diagnostics, if you have done something that definitively violates the rules of the language, the compiler is required to emit a diagnostic message.

Statement, a type of instruction that causes the program to perform some action.

Identifier, the name of a function (or object, type, template, etc.).

Random Access Memory (RAM), the main memory in a computer

Object, a region of storage (usually memory) that can store a value, and has other associated properties. Typically refers to an unnamed object in memory, a variable, or a function.

Variable, an object that has been named (**identifier**).

Definition, a special kind of declaration statement for creating variables.

Instantiation, the object gets created and assigned a memory address.

Instance, instantiated object

Data type (Type), determines what kind of value (e.g. a number, a letter, text, etc.) the object will store.

Assignment, the process of giving a value to a variable **after** it has been defined.

Initialization, giving an initial value to an object at the point of definition.

Uniform or brace initialization, list initialization

Zero initialization, default initialization

Buffer, a region of memory set aside to store a collection of requested data.

Flushing, transferring collected data from the buffer to its destination.

Uninitialized, an object that hasn't been given a known value.

Undefined behavior (UB), the result of executing code whose behavior is not well-defined by the C++ language.

Implementation-defined behavior, the result of executing code whose behavior is defined by the compiler.

Unspecified behavior, this is almost identical to implementation-defined behavior. The only difference here is that the implementation (**compiler**) doesn't have to document why it behaves a certain way.

Reserved words, words reserved to the C++ language such as keywords.

Operator Arity, the number of operands that an operator takes as input.

PEMDAS, Parenthesis → Exponents → Multiplication & Division → Addition & Subtraction.

Side effects, an operator (or function) that has some observable effect beyond producing a return value.

Metasyntactic variable or **placeholder names**, a placeholder name for a function or variable when the name is unimportant to the demonstration of some concept.

Status codes or **exit codes**, the return value of a function is passed back to the caller of the function, the status code is passed back to the caller. This return value has a defined meaning (e.g. `EXIT_SUCCESS`).

DRY, don't repeat yourself

WET, write everything twice

Modular programming, the ability to write a function, test it, ensure that it works, and then know that we can reuse it as many times as we want and it will continue to work (so long as we don't modify the function -- at which point we'll have to retest it).

Unreferenced parameters, functions that have parameters that are not used in the body of the function.

Lifetime, an object's lifetime is defined to be the time between its creation and destruction. (**runtime** property)

Scope, an identifier's scope determines where the identifier can be seen and used within the source code. (**compile-time** property)

Out of scope, an identifier is out of scope anywhere it cannot be accessed within the code.

Going out of scope, we say an object goes out of scope at the end of the scope (the end curly brace) in which the object was instantiated.

Temporary object or **anonymous object**, an unnamed object that is created by the compiler to store a value temporarily.

One task rule or **separation of concerns (SoC)**, a design principle to separate an application into units, with minimal overlapping between the functions of the individual units.

Function declaration statement or **function prototype**, the function declaration consists of the function's return type, name, and parameter types, terminated with a semicolon. The names of the parameters can be optionally included. The function body is not included in the declaration.

Pure declaration, In C++, all definitions are declarations. Conversely, not all declarations are definitions. Declarations that aren't definitions are called pure declarations.

One definition rule (ODR), Only one definition of any variable, function, class type, enumeration type, concept(since C++20) or template is allowed in any one translation unit (some of these may have multiple declarations, but only one definition is allowed).

Qualified name, an identifier that includes a namespace prefix.

Using-directive statement, allows us to access the names in a namespace without using a namespace prefix.

Preprocessor directive or directives, instructions that start with a # symbol and end with a newline (NOT a semicolon).

Translation unit, when the preprocessor has finished processing a code file, the result is called a translation unit.

Translation, the entire process of **preprocessing**, **compiling**, and **linking**.

Transitive includes, when your code file #includes the first header file, you'll also get any other header files that the first header file includes (and any header files those include, and so on). These additional header files are sometimes called transitive includes, as they're included implicitly rather than explicitly.

Header guard or include guard, conditional compilation directives that ensure proper transitive includes, avoiding **ODR** violations.

Call stack, a list of all the active functions that have been called to get to the current point of execution. The call stack includes an entry for each function called, as well as which line of code will be returned to when the function returns. Whenever a new function is called, that function is added to the top of the call stack. When the current function returns to the caller, it is removed from the top of the call stack, and control returns to the function just below it.

Fundamental data types or primitive types, data types that are built into and supported by C++ (void, std::nullptr_t, integral types, floating-point types).

IEEE, Institute of Electrical and Electronics Engineers

Sign, the attribute of being positive, negative, or zero is called the number's sign.

Sign bit, a single bit used to store the sign of the number.

Magnitude bits, the non-sign bits that determine the magnitude of the number.

Shorthand types, types that do not use a (int) suffix or signed prefix.

Fixed-width integers, int types that have a fixed-width across CPU architectures. C99 defined them in the **stdint.h**. Unfortunately, they may not be defined on all architectures and may be slower in terms of processing speed.

Fast integers, fast types defined by C++ to address the issue of processing speed with fixed-width integers defined in C99. These can be found in the **cstdint** header along with the **stdint.h** types.

Least integers, least types defined by C++ to address the issue of architecture support with fixed-width integers defined in C99. They give you the **smallest** (least) integer type.

std::size_t, an alias for an implementation-defined unsigned integral type.

Significant digits or **significant figures**, refers to scientific notation of the digits in the significand (the part before the 'e'). The more significant digits, the more precise a number is.

Condition or **conditional expression**, an expression that evaluates to a Boolean value.

Early return, a return statement that is not the last statement in a function.

Integral type, any type where the underlying value is stored as an integer. Obviously types like int, short, etc. but less obvious types like bool (0 or 1), chars (ASCII values).

ASCII, American Standard Code for Information Interchange

Type qualifier or **qualifier**, a keyword that is applied to a type that modifies how that type behaves. As of C++23, C++ only has two type qualifiers: **const** and **volatile**.

Cv-qualifier, in technical documentation, the const and volatile qualifiers are often referred to as cv-qualifiers.

As-if rule, it says that the compiler can modify a program however it likes in order to produce more optimized code, so long as those modifications do not affect a program's "**observable behavior**".

Constant variable or **named constants**, variables with fixed values that cannot be changed.

Constant expression, expression that contains only compile-time constants and / or operators / functions that support **compile-time evaluation**.

Runtime expression, expression that is not a constant expression.

Compile-time constant, a constant whose value must be known at **compile time**.

Runtime constant, Const variables that are not compile-time constants. Runtime constants **cannot be used in a constant expression**.

Immediate functions, a function that must evaluate at **compile-time**, otherwise a compile error will result.

Inline expansion, a process where a function call is replaced by the code from the called function's definition.

Read-only, we can access and use the value being viewed, but we cannot modify it.

Dangling view, a `std::string_view` object that is viewing a string that has been destroyed. This can result in unexpected and undefined behavior.

Substring, a sequence of characters that are part of a string and appear contiguously.

Value computation, the execution of operators in an expression to produce a value.

Storage duration or duration, a variable's duration determines what rules govern when and how a variable will be created (instantiated) and destroyed. In most cases, a variable's storage duration directly determines its **lifetime**.

Linkage, an identifier property that determines whether a declaration of that same identifier in a different scope refers to the same object (or function).

Internal variables, global variables with internal linkage.

Execution path or **path**, The specific sequence of statements that the CPU executes.

Spaghetti code, code that has a path of execution that resembles a bowl of spaghetti (all tangled and twisted), making it extremely difficult to follow the logic of such code.

Iteration, each time a loop executes.

Early return, A return statement that is not the last statement in a function.

Halt, control flow statement that terminates the program.

Normal termination, means the program has exited in an expected way. Note that the term normal termination **does not imply anything about whether the program was successful**.

Software testing or **software validation**, the process of determining whether or not the software actually works as expected.

Unit testing, testing a small part of your code in isolation to ensure that “unit” of code is correct. Each unit test is designed to ensure that a particular behavior of the unit is correct.

Integration testing, once each of your units has been tested in isolation, they can be integrated into your program and retested to make sure they were integrated properly.

Sentinel value, a value that has some special meaning in the context of a function or algorithm.

Conversion, the process of producing a new value of some type from a value of a different type.

Standard conversions, Rules defined by the C++ language standard about how different fundamental types (and in some cases, compound types) can be converted to other types.

Value-preserving conversion or **safe conversion**, conversion where every possible source value can be converted into an equal value of the destination type.

Type casting operators or **casts**, operators that can be used by the programmer to request that the compiler perform a type conversion. Because casts are explicit requests by the programmer, this form of type conversion is often called an **explicit type conversion**.

Overloaded function or **overload**, each function sharing a name (in the same scope).

Overload resolution, the process of matching function calls to a specific overloaded function.

Placeholder type, represents some type that is not known at the time the template is written, but that will be provided later.

Function template instantiation, the process of creating functions (with specific types) from function templates (with template types).

Implicit instantiation, a function instantiated due to a function call.

Specialization or **Function instance**, A function that is instantiated from a template. Function instances are normal functions in all regards.

Primary template, the template from which a specialization is produced.

Value category, the value category of an expression (or subexpression) is a property of expressions that indicates whether an expression resolves to a value, a function, or an object of some kind.

L-value, an lvalue in C++ is something you can identify and refer to, like a variable or a function. It usually has a specific location in memory, so you can get its address and use it in multiple places in your code. Essentially, an lvalue is something that persists beyond a single expression and can be reused.

R-value, an rvalue in C++ is a temporary value that doesn't have a specific memory address you can refer to. It's typically the result of an expression, like 5 or $a + b$, and only exists for a brief moment, usually within a single line of code. Unlike lvalues, rvalues can't be assigned to because they don't have a lasting location in memory.

Reference, alias for an existing object. Once a reference has been defined, any operation on the reference is applied to the object being referenced.

Reference binding, when a reference is initialized with an object (or function), we say it is bound to that object (or function). The process by which such a reference is bound is called reference binding.

Referent, the object (or function) being referenced.

Pointer, an object that holds a memory address (typically of another variable) as its value. This allows us to store the address of some other object to use later.

Raw pointer or **Smart pointer**, These are the pointers we are talking about in modern C++.

Wild pointer, a pointer that has not been initialized. Wild pointers contain a garbage address, and dereferencing a wild pointer will result in undefined behavior. Because of this, you should always initialize your pointers to a known value.

Null pointer, a pointer that holds no value and therefore points to nothing.

Reference Lifetime Extension, a feature in C++ that allows temporary objects (rvalues) to have their lifetimes extended when they are bound to a const reference.

type definition, the definition for a program-defined type.

Underlying type or **base**, The specific integral type used to represent the value of enumerators.

Aggregate data type or **aggregate**, Any type that can contain multiple data members. Some types of aggregates allow members to have different types (e.g. structs), while others require that all members must be of a single type (e.g. arrays).

Padding, For performance reasons, the compiler will sometimes add gaps into structures.

RAII, Resource Acquisition Is Initialization

Void pointer or **generic pointer**, special type of pointer that can be pointed at objects of any data type.

callback functions, functions used as arguments to another function.

C++

2 Introduction / Getting started

2.1 Introduction to programming languages

A **computer program** (also commonly called an **application**) is a **set of instructions** that the computer can perform in order to perform some task(s). The process of creating a program is called programming. Programmers typically create programs by producing source code (commonly shortened to code), which is a list of commands typed into one or more text files.

The collection of **physical** computer **parts** that make up a computer and execute programs is called the **hardware**. When a computer program is loaded into memory and the hardware sequentially executes each instruction, this is called **running** or **executing** the program.

2.1.1 Machine language

The collection of physical computer parts that make up a computer and execute programs is called the **hardware**. When a computer program is loaded into memory and the hardware sequentially executes each instruction, this is called **running** or **executing** the program.

Example: **1011000 0110001**

Each instruction is composed of a sequence of 1s and 0s. Each individual 0 or 1 is called a **binary digit** or **bit** for short. The number of bits that make up a single command varies.

Example:

- Some CPUs process instructions are always 32 bits long
- Other CPUs from the x86/x64 family have instructions that can be variable in length

Each set of binary digits is interpreted by the CPU into a command to do a very specific job, such as **compare these two numbers**, or **put this number in that memory location**.

However, because different **CPUs have different instruction sets**, instructions that were written for one CPU type could not be used on a CPU that didn't share the same instruction set. Back in the day this meant that programs generally weren't **portable**.

2.1.2 Assembly language

Because machine language is so hard for humans to read and understand, assembly language was invented. In an assembly language, each instruction is identified by a short abbreviation (**rather than a set of bits**), and names and other numbers can be used.

Here is the same instruction as above in assembly language: **mov al, 061h**

This makes assembly much easier to read and write than machine language. However, the **CPU cannot understand assembly language directly**. Instead, the assembly **program must be translated** into machine language before it can be executed by the computer. This is done by using a program called an **assembler**.

Assembly languages still require a lot of instructions to do even simple tasks. While the individual instructions themselves are somewhat human readable, understanding what an entire program is doing can be challenging. Assembly isn't portable, a program written in assembly for one CPU will likely not work on hardware that uses a different instruction set, and would have to be rewritten or extensively modified.

2.1.3 High-level languages

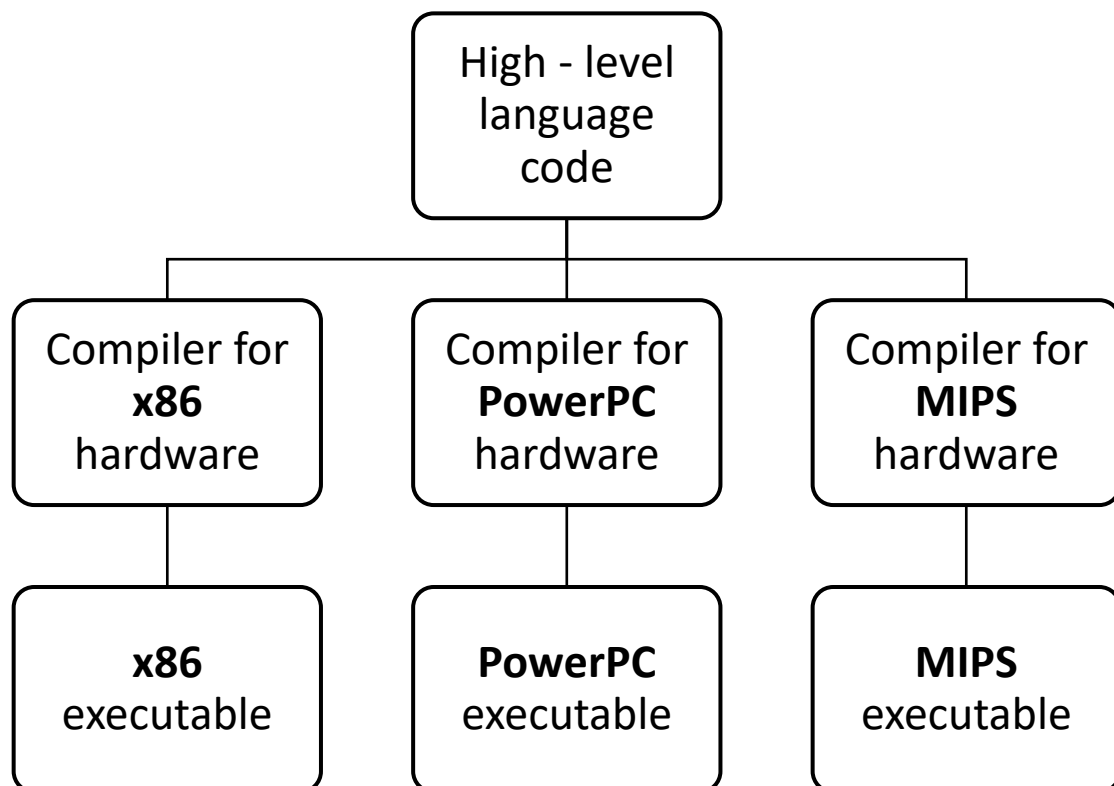
Now that we know some of the downsides:

- Hard to read
- Not portable
- A lot of work for simple instructions

This is where higher-level languages come into play. To address the readability and portability concerns, new programming languages such as C, C++, Pascal (and later, languages such as Java, Javascript, and Perl) were developed. These languages are called **high level languages**, as they are designed to allow the programmer to write programs without having to be as concerned about what kind of computer the program will be run on.

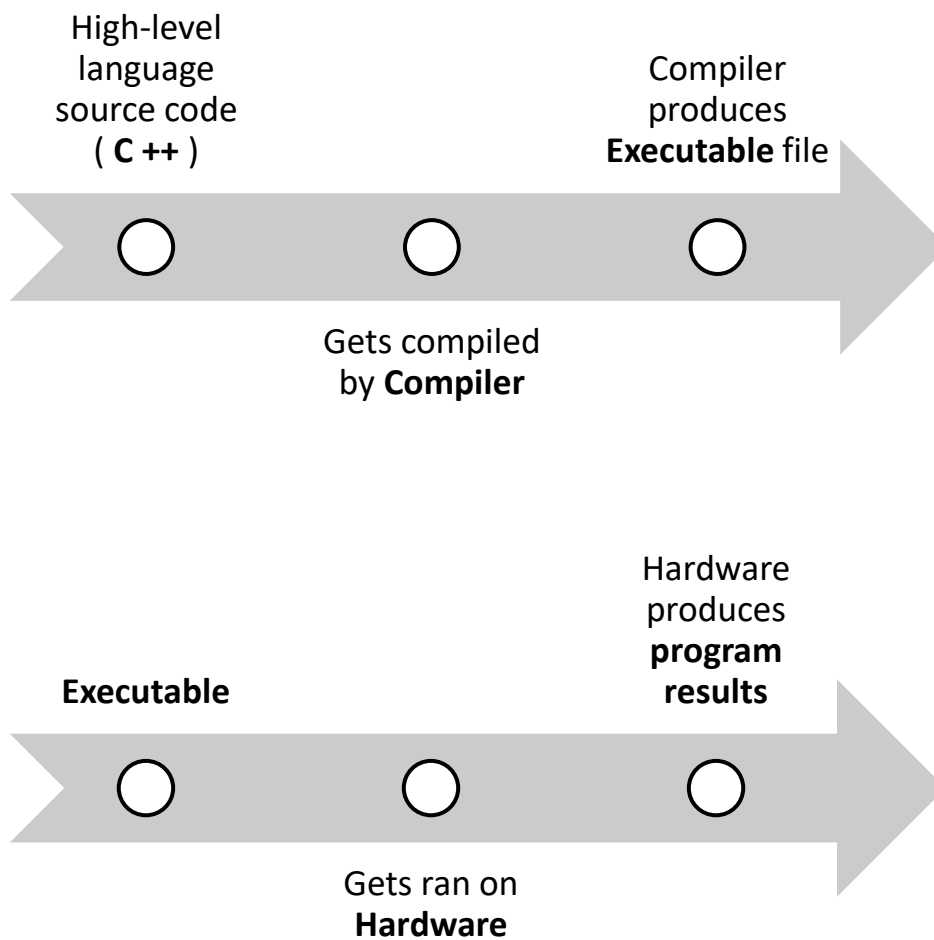
Here is the same instruction as above in C/C++: **a = 97;**

Much like assembly programs, programs written in high level languages must be translated into a format the computer can understand before they can be run. There are two primary ways this is done: compiling and interpreting. Most languages can be compiled or interpreted. Traditionally languages like **C, C++, and Pascal** are compiled, whereas "**scripting**" languages like **Perl** and **Javascript** tend to be interpreted. **Some languages, like Java, use a mix of the two**. Programs can be compiled (or interpreted) for many different systems, **and you don't have to change the program to run on different CPUs** (you just recompile for that CPU).



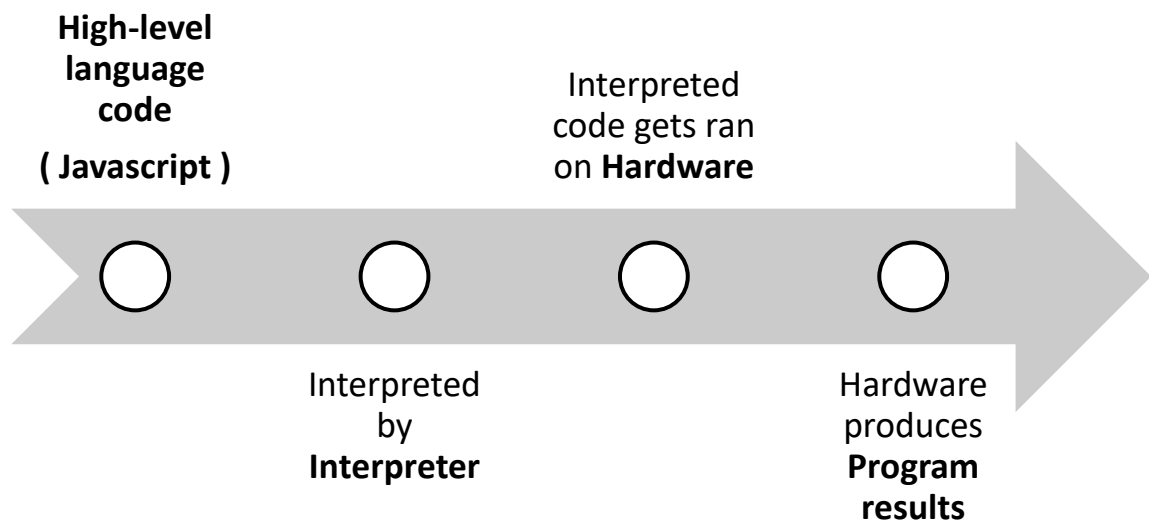
2.1.3.1 Compiling

A **compiler** is a program (or collection of programs) that reads source code (typically written in a high-level language) and **translates it into** some other language (typically a **low-level language**, such as assembly or machine language, etc.). Most often, these low-level language files are then combined into an executable file (containing machine language instructions) that can be run or distributed to others. Notably, **running** the executable file **does not require** the **compiler** to be installed (because the **code has already been compiled and put into the executable file**).



2.1.3.2 Interpreter

An **interpreter** is a program that **directly executes** the **instructions** in the source code without requiring them to be compiled into an executable first. Interpreters tend to be **more flexible than compilers**, but are **less efficient** when running programs because the interpreting process needs to be done every time the program is run. **This also means the interpreter must be installed on every machine where an interpreted program will be run.**



2.2 Introduction to C / C++

2.2.1 *Before C++, there was C*

C was created by **Dennis Ritchie** in 1972, primarily to **serve as a systems programming language** (To write Operating Systems with).

Goals for C:

- Minimalistic
- Easy to compile
- Efficient access to memory
- Efficient code
- Self-contained (Not reliant on other languages)

2.2.1.1 Brief history: C

Essentially, it was designed to give the programmer a lot of control, while still encouraging hardware and operating systems independence.

2.2.1.1.1 1973: *Unix*

It was so flexible that in 1973 the **Unix** operating system was written in C and Assembly.

2.2.1.1.2 1978: *“The C Programming Language”*

In 1978, Brian **Kernighan** and Dennis **Ritchie** published a book called **“The C Programming Language”**. This book, which was commonly known as **K&R**, provided an informal specification for the language and **became a de facto standard**.

2.2.1.1.3 1983: Forming of ANSI committee

In 1983, the American National Standards Institute (ANSI) formed a committee to establish a formal standard for C.

2.2.1.1.4 1989: Release: C89 standard / ANSI C

In 1989 (committees take forever to do anything), they finished, and released the C89 standard, more commonly known as ANSI C.

2.2.1.1.5 1990: Release: C90 standard: An adoption of ANSI C by the ISO

In 1990 the International Organization for Standardization (ISO) adopted ANSI C (with a few minor modifications). This version of C became known as C90. Compilers eventually became ANSI C/C90 compliant, and programs desiring maximum portability were coded to this standard.

2.2.1.1.6 1999: Release: C99 standard: An extended version of C90

In 1999, the ISO committee released a new version of C called C99. C99 adopted many features which had already made their way into compilers as extensions, or had been implemented in C++.

2.2.1.1.7 Later releases

- 2011: **C11**
- 2017: **C17**
- 2023: **C23**

2.2.1.2 Brief history: C++

C++ was created by **Bjarne Stroustrup** in 1979, primarily to **serve as an extension to C**. It is best thought of as a “superset” of C. Though new releases of C like the **C99 standard** introduced new features that weren’t in C++.

Goals for C:

- More features
- Object-oriented

2.2.1.2.1 1998: C++ standardized by the ISO Committee

C++ was standardized in 1998 by the ISO committee (this means the ISO standards committee approved a document describing the C++ language, to help ensure all compilers adhere to the same set of standards)

2.2.1.2.2 2003: Release: C++03 standard

A minor update to the language was released in 2003 (called C++03).

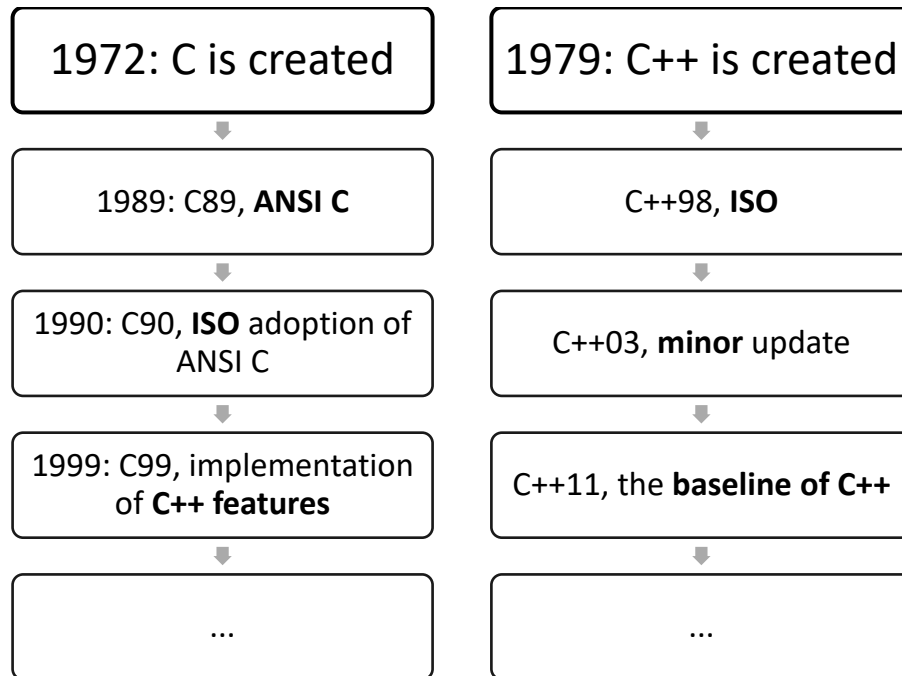
2.2.1.2.3 Later releases

Five major updates to the C++ language (**C++11, C++14, C++17, C++20, and C++23**) have been made since then, each adding additional functionality. **C++11 in particular added a huge number of new capabilities, and is widely considered to be the new baseline version of the language.** Future upgrades to the language are expected every three or so years.

Each new formal release of the language is called a **language standard** (or **language specification**).

Standards are named after the year they are released in. For example, there is no C++15, because there was no new standard in 2015.

2.2.1.3 Summary of language standards

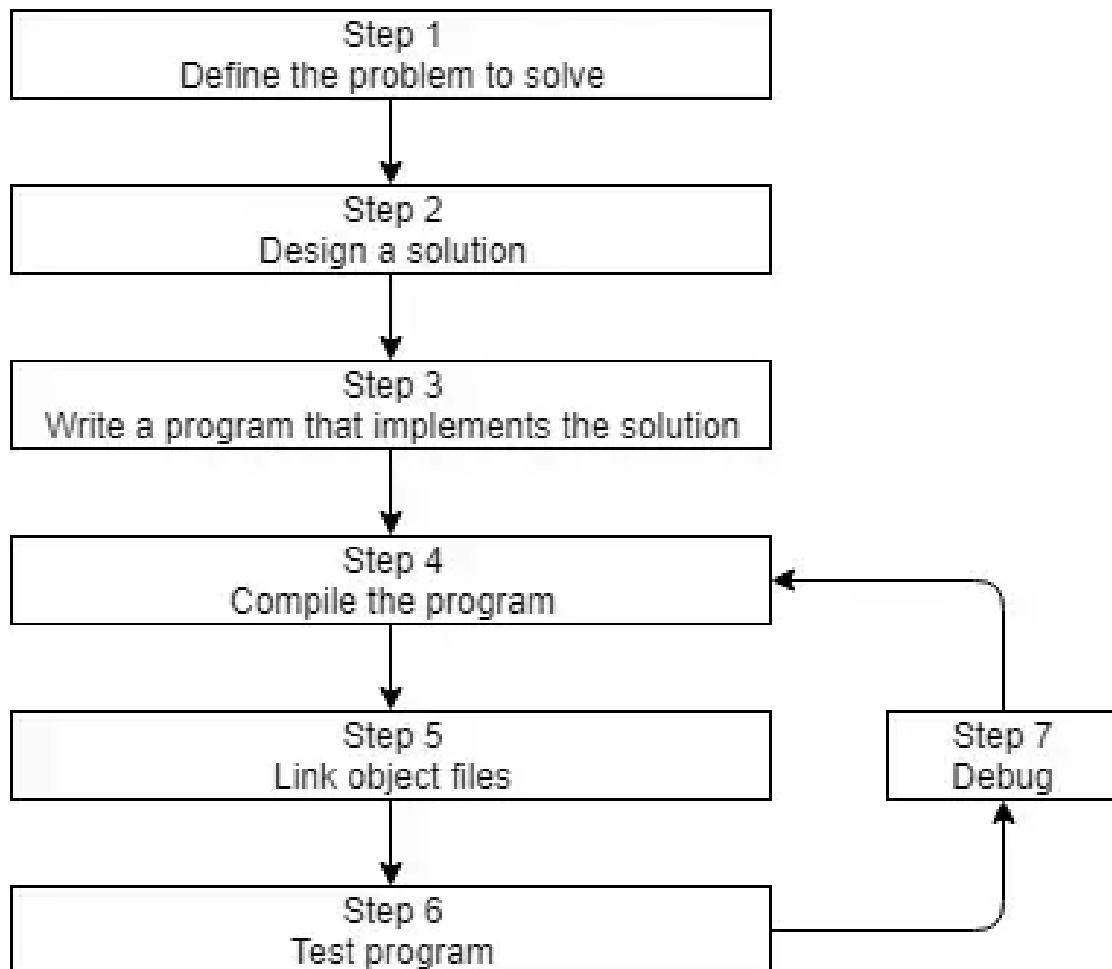


2.2.2 C / C++ philosophy

The underlying design philosophy of C and C++ can be summed up as “**trust the programmer**” -- which is **both wonderful and dangerous**. C++ is designed to allow the programmer a **high degree of freedom** to do what they want. However, **this also means the language often won’t stop you from doing things that don’t make sense**, because it will assume you’re doing so for some reason it doesn’t understand. There are quite a few pitfalls that new programmers are likely to fall into if caught unaware. **This is one of the primary reasons why knowing what you shouldn’t do in C/C++ is almost as important as knowing what you should do.**

2.3 Introduction to C++ development

2.3.1 C++ development flow



2.4 Introduction to the compiler, linker and libraries

2.4.1 Compiling your source code

In order to compile C++ source code files, we **use a C++ compiler**. The C++ compiler **sequentially goes through each** source code (.cpp) **file** in your program. The compiler does two important tasks:

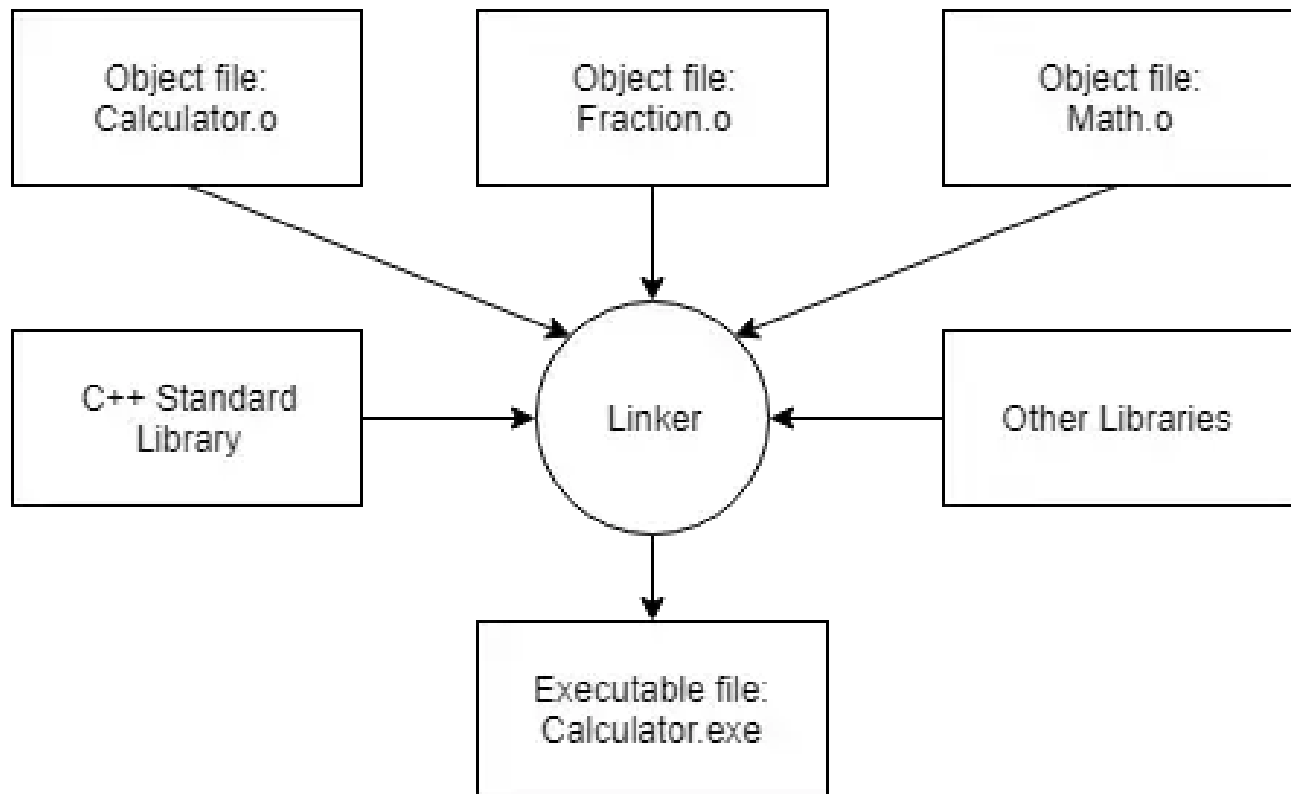
- **Checks** your C++ **code** to make sure it follows the rules of the C++ language.
 - **If it does not, the compiler will give you an error** (and the corresponding line number) to help pinpoint what needs fixing. **The compilation process will also be aborted until the error is fixed.**
- **Translates** your C++ **code** into machine language instructions.
 - These instructions are **stored in an intermediate** file called an **object file**. The **object file also contains metadata that is required or useful in subsequent steps**. These files carry the **.o / .obj extension** and will have the **same name as your .cpp Files**.

2.4.2 Linking object files and libraries

After the compiler has successfully finished, another program called the **linker** kicks in. The **linker's job is to combine all of the object files and produce the desired output file** (typically an executable file). This process is called **linking**.

The linker's flow:

- Read each object file to make sure they are valid
- Resolve cross-file dependencies
 - These are things you use something from one file to the other
 - **If the linker is unable to connect a reference** to something with its definition, you'll get a **linker error**, and **the linking process will abort**.
- Linking library files
 - A **library file** is a collection of **precompiled code** that has been "packaged up" for reuse in other programs.
 - C++ comes with an extensive library called the **C++ Standard Library** (usually shortened to *standard library*) that provides a set of useful capabilities for use in your programs.
 - You can also optionally **link other libraries**.
 - Example: C++ doesn't let you play sounds by standard. And figuring out how to code sound is a lot of work. Instead, you'd probably download a library that already knew how to do those things, and use that.



2.4.3 Building

Because there are multiple steps involved, the term **building** is often **used to refer to the full process** of converting source code files into an executable that can be run. A specific executable produced as the result of building is sometimes called a **build**.

2.4.3.1 Build types

- **Build**
 - **compiles** all *modified* code files in the project or workspace/solution, and **then links the object files into an executable**.
 - If no code files have been modified since the last build, this option does nothing.
- **Clean**
 - **removes all cached objects and executables** so the next time the project is built, all files will be recompiled and a new executable produced.
- **Rebuild**
 - does a “clean”, followed by a “build”.
- **Compile**
 - recompiles a **single code file** (regardless of whether it has been cached previously).
 - This option **does not invoke the linker** or produce an executable.
- **Run/start**
 - **executes the executable from a prior build**.
 - Some IDEs (e.g. Visual Studio) will **invoke a “build” before doing a “run”** to ensure you are running the latest version of your code.

For this summary I’m not going to go into IDE’s to use or how to build your first program or what IDE’s look like.

2.5 Configuring your compiler

2.5.1 *Build configurations*

A **build configuration** (also called a **build target**) is a collection of project settings that determines how your IDE will build your project. The build configuration typically includes things like what the executable will be named, what directories the IDE will look in for other code and library files, whether to keep or strip out debugging information, how much to have the compiler optimize your program, etc. Generally, you will want to leave these settings at their default values unless you have a specific reason to change something.

When you create a new project in your IDE, most IDEs will set up two different build configurations for you: a release configuration, and a debug configuration.

2.5.1.1 Debug configuration

The **debug configuration** is designed to help you debug your program, and is generally the one you will use when writing your programs. This configuration turns off all optimizations, and includes debugging information, which makes your programs larger and slower, but much easier to debug. The debug configuration is usually selected as the active configuration by default. We'll talk more about debugging techniques in a later lesson.

2.5.1.2 Release configuration

The **release configuration** is designed to be used when releasing your program to the public. This version is typically optimized for size and performance, and doesn't contain the extra debugging information. Because the release configuration includes all optimizations, this mode is also useful for testing the performance of your code (which we'll show you how to do later in the tutorial series).

2.5.2 *Compiler extensions*

The C++ standard defines rules about how programs should behave in specific circumstances. And in most cases, compilers will follow these rules. However, many compilers implement their own changes to the language, often to enhance compatibility with other versions of the language (e.g. C99), or for historical reasons. These compiler-specific behaviors are called **compiler extensions**.

Writing a program that makes use of a compiler extension **allows you to write programs that are incompatible with the C++ standard**. Programs using non-standard extensions **generally will not compile on other compilers** (that don't support those same extensions), or if they do, they may not run correctly.

Frustratingly, compiler extensions are **often enabled by default**. This is particularly damaging for new learners, who may think some behavior that works is part of official C++ standard, when in fact **their compiler is simply over-permissive**.

Because compiler extensions are never necessary, and cause your programs to be non-compliant with C++ standards, **we recommend turning compiler extensions off**.

In visual studio this setting is called the **Conformance mode** and **belongs to the C/C++ language settings**. It should be set to Yes or Permissive- (non-permissive).

2.5.3 *Warnings and error levels*

When you write your programs, the compiler will check to ensure you've followed the rules of the C++ language (assuming you've turned off compiler extensions).

2.5.3.1 **Diagnostics**

If you have done something that definitively violates the rules of the language, the compiler is required to emit a **diagnostic message** (often called a **diagnostic** for short). The C++ standard does not define how diagnostic messages should be categorized or worded. However, there are some common conventions that compilers have adopted.

2.5.3.2 Errors

If compilation cannot continue due to the violation, then the compiler will emit an **error**, providing both line number containing the error, and some text about what was expected vs what was found. Errors stop the compilation from proceeding. **The actual error may be on that line, or on a preceding line.** Once you've identified and fixed the erroneous line(s) of code, you can try compiling again.

2.5.3.3 Warnings

If compilation can continue despite the violation, the compiler may decide to emit either an error or a **warning**. Warnings are similar to errors, but they do not halt compilation.

In some cases, the compiler may identify code that does not violate the rules of the language, but that it believes could be incorrect. In such cases, the compiler may decide to emit a warning as a notice to the programmer that something seems amiss.

Best practice: Don't let warnings pile up, resolve them immediately (as if they were errors).

By default, most compilers will only generate warnings about the most obvious issues. However, **you can request your compiler be more assertive** about providing warnings for things it finds strange.

Best practice: Turn your warning levels up to the maximum, especially when learning.

2.5.3.4 Treating warnings as errors

It is also possible to tell your compiler to treat all warnings **as if they were errors** (in which case, the compiler will halt compilation if it finds any warnings). **This is a good way to enforce the recommendation that you should fix all warnings (if you lack self-discipline, which most of us do).**

Best practice: Enable "Treat warnings as errors". This will force you to resolve all issues causing warnings.

2.5.4 *Choosing a language standard*

With many different versions of C++ available (C++98, C++03, C++11, C++14, C++17, C++20, C++23, etc.) how does your compiler know which one to use? **Generally, a compiler will pick a standard to default to. Typically,** the default is ***not* the most recent** language standard -- many default to C++14, which is missing many of the latest and greatest features.

2.5.4.1 Code names for in-progress language standards

Finalized language standards are named after the years in which they are finalized (e.g. C++17 was finalized in 2017).

However, **when a new language standard is being worked on**, it's not clear in what year the finalization will take place. **Consequently, upcoming language standards are given code names**, which are then replaced by the actual names upon finalization of the standard. You may still see the code names used in places (particularly for the latest in-progress language standard that doesn't have a finalized name yet).

2.5.4.2 Which language standard should you use?

In professional environments, it's common to choose a language standard that is one or two versions back from the latest standard (e.g. if C++20 were the latest version, that means C++14 or C++17). This is typically done to ensure the compiler makers have had a chance to resolve defects, and so that best practices for new features are well understood. Where relevant, this also helps ensure better cross-platform compatibility, as compilers on some platforms may not provide full support for newer language standards immediately.

For personal projects and while learning, we recommend choosing the latest finalized standard, as there is little downside to doing so.

All the information regarding the C++ standard can be found on: <https://isocpp.org/>.

Even after a language standard is finalized, compilers supporting that language standard often still have missing, partial, or buggy support for certain features.

3 Basics

3.1 Statements and the structure of a program

3.1.1 Statements

A computer program is a **sequence** of instructions that tell the computer what to do.

A **statement** is a type of instruction that causes the program to *perform some action*.

Statements are by far the most common type of instruction in a C++ program. This is because they are the **smallest independent unit of computation** in the C++ language. In that regard, they act much like sentences do in natural language. When we want to convey an idea to another person, we typically write or speak in sentences (not in random words or syllables). In C++, when we want to have our program do something, we typically write statements.

Most (but not all) statements in C++ end in a semicolon. If you see a line that ends in a semicolon, it's probably a statement.

In a high-level language such as C++, a single statement may compile into many machine language instructions.

3.1.1.1 Different kinds of statements

- Declaration
- Jump
- Expression
- Compound
- Selection (Conditionals)
- Iteration (Loops)
- Try blocks

3.1.2 *Functions and the main function*

In C++, **statements are typically grouped into units called functions**. A function is a collection of statements that get executed sequentially (in order, from top to bottom). As you learn to write your own programs, you'll be able to create your own functions and mix and match statements in any way you please (we'll show how in a future lesson).

Important rule:

Every C++ program must have a **special function named main (all lower-case letters)**. When the program is run, the statements inside of main are **executed in sequential order**. Programs typically terminate (finish running) after the last statement inside function main has been executed (though programs may abort early in some circumstances, or do some cleanup afterwards).

3.1.3 *Identifiers*

In programming, the name of a **function (or object, type, template, etc.)** is called its **identifier**.

3.1.4 *Syntax and syntax errors*

In English, sentences are constructed according to specific grammatical rules that you probably learned in English class in school. For example, normal sentences end in a period. **The rules that govern how sentences are constructed in a language is called syntax**. If you forget the period and run two sentences together, this is a violation of the English language syntax.

C++ has a syntax too: rules about how your programs must be constructed in order to be considered valid. When you compile your program, the compiler is responsible for making sure your program follows the basic syntax of the C++ language. If you violate a rule, the compiler will complain when you try to compile your program, and issue you a syntax error.

3.2 **Comments**

A comment is like leaving a note in your source code. It is ignored by the compiler and is mostly to help programmers document the code in some way.

3.2.1 *Single-line comments*

We start by writing the `//` symbol. Everything before that gets compiled but everything after the symbol up until the new line is a comment.

3.2.2 *Multi-line comments*

We start by writing the `/*` symbol and end with the `*/` symbol. Everything in between the first and the last symbol gets ignored by the compiler. The use here is to have a “section” where you can write comments.

3.2.3 *Best practice*

Comment your code liberally, and write your comments as if speaking to someone who has no idea what the code does. Don't assume you'll remember why you made specific choices.

3.3 Introduction to objects and variables

You learned that the **majority of instructions in a program are statements**, and that **functions are groups of statements** that execute sequentially.

3.3.1 Key Insight

Programs are collections of instructions that manipulate data to produce a desired result.

3.3.2 Random Access Memory (RAM)

The main memory in a computer is called **Random Access Memory** (often called **RAM** for short). When we run a program, the operating system loads the program into RAM. Any **data that is hardcoded** into the program itself (e.g. text such as “Hello, world!”) is **loaded at this point**.

The operating system **also reserves some additional RAM for the program to use** while it is running. Common uses for this memory are to store values entered by the user, to store data read in from a file or network, or to store values calculated while the program is running (e.g. the sum of two values) so they can be used again later.

You can think of RAM as a series of numbered boxes that can be used to store data while the program is running.

In some older programming languages (like Applesoft BASIC), you could directly access these boxes (e.g. you could write a statement to “go get the value stored in mailbox number 7532”).

3.3.2.1 Quick summary

- Executable get ran
 - OS directly loads hardcoded values into memory
 - OS reserves additional space for data to be manipulated while the program is running

3.3.3 Objects & Variables

In C++, **direct memory access is discouraged**. Instead, we **access memory indirectly through an object**. An object is a **region of storage** (usually memory) that can store a value, and has other associated properties (that we'll cover in future lessons). How the compiler and operating system work to assign memory to objects is beyond the scope of this lesson. But the key point here is that rather than say "go get the value stored in mailbox number 7532", we can say, "go get the value stored by this object". **This means we can focus on using objects to store and retrieve values, and not have to worry about where in memory those objects are actually being placed.**

3.3.3.1 Key insight

An **object** is used **to store a value in memory**. A **variable is an object** that has been named (identifier). Naming our objects let us refer to them again later in the program.

3.3.3.2 Nomenclature

In general programming, the term *object* typically refers to an unnamed object in memory, a variable, or a function. **In C++, the term *object* has a narrower definition that excludes functions.**

3.3.3.3 Instantiation & Instances, a runtime occurrence

In order to create a variable, we use a special kind of declaration statement called a **definition**.

At compile time, when the compiler sees this statement, **it makes a note to itself** that we are defining a variable, giving it the name *x*, and that it is of type *int* (more on types in a moment). **From that point forward** (with some limitations that we'll talk about in a future lesson), **whenever the compiler sees the identifier *x*, it will know that we're referencing this variable.**

When the program is run (called **runtime**), the variable will be instantiated. **Instantiation** is a fancy word that **means the object will be created and assigned a memory address**. Variables must be instantiated before they can be used to store values. For the sake of example, **let's say that variable *x* is instantiated at memory location 140. Whenever the program uses variable *x*, it will access the value in memory location 140.** An instantiated object is sometimes called an **instance**.

3.3.4 Data types

A **data type** (more commonly just called a **type**) determines what kind of value (e.g. a number, a letter, text, etc.) the object will store. In C++, the type of a variable must be known at **compile-time** (when the program is compiled), and that type cannot be changed without recompiling the program. This means an integer variable can only hold integer values. If you want to store some other kind of value, you'll need to use a different type. Or do type conversion, which is something we'll get into more depth in, later on.

3.3.4.1 Defining a variable of a certain type

Int a;	← This is a definition
Int a, b;	← This defines two variables
Int a, int b;	← Wrong, compiler error
Int a, double b;	← Wrong, compiler error

In both cases these variables are of the integer type which allows them to store a numeric value into memory (not floating-point numeric values). There is **no need to specify the type twice** when defining variables, since it's in the same statement the compiler will know what type, it is.

Best practice: Although the language allows you to do so, **avoid defining multiple variables of the same type in a single statement**. Instead, define each variable in a separate statement on its own line (and then **use a single-line comment to document what it is used for**).

3.4 Variable assignment

3.4.1 Quick recap

In previous lessons we covered:

- **Instantiation** (the process of an object getting created and assigned to its memory address by the runtime system).
- **Definition** (Defining a variable by assigning it a type and identifier).

3.4.2 Variable assignment

After a variable has been defined, you can give it a value (in a separate statement) using the = *operator*. This process is called **assignment**, and the = *operator* is called the **assignment operator**.

Float pi;	← Definition
pi = 3.14f;	← Assignment

By default, assignment copies the value on the right-hand side of the = *operator* to the variable on the left-hand side of the operator. This is called **copy assignment**. This can be used to manipulate our variable.

Example: Since the type is already defined and the compiler knows about it we can access the variable by its **identifier (name)** and copy a different value into it.

pi = 3.141592f;	← Not definition but assigned a different value
-----------------	---

3.5 Variable initialization

One downside of **assignment** is that it **requires at least two statements**: one to define the variable, and another to assign the value. **These two steps can be combined**. When an object is defined, you can optionally give it an initial value. **The process of specifying an initial value for an object is called initialization**, and the syntax used to initialize an object is called an **initializer**.

Float pi;	← Definition
pi = 3.14f;	← Assignment
Float pi = 3.14f;	← Defined & Assigned → Initialized

3.5.1 Forms of initialization

int a; // no initializer, just a definition (default initialization by compiler)
int b = 5; // initial value after equals sign (copy initialization)
int c (6); // initial value in parenthesis (direct initialization)
 // List initialization methods (C++11) (preferred)
int d { 7 }; // initial value in braces (direct list initialization)
int e = { 8 }; // initial value in braces after equals sign (copy list initialization)
int f {}; // initializer is empty braces (value initialization)

3.5.1.1 Default initialization

When no initializer is provided (such as for variable `a` above), this is called **default initialization**. In most cases, default initialization performs no initialization, and **leaves a variable with an indeterminate value**.

3.5.1.2 Copy initialization

When an initial value is provided after an equals sign, this is called **copy initialization**. This form of initialization was inherited from C. Much like copy assignment, this copies the value on the right-hand side of the equals into the variable being created on the left-hand side.

Copy initialization had fallen out of favor in modern C++ due to being less efficient than other forms of initialization for some complex types. However, C++17 remedied the bulk of these issues, and copy initialization is now finding new advocates. **You will also find it used in older code (especially code ported from C), or by developers who simply think it looks more natural and is easier to read.**

3.5.1.3 Direct initialization

When an initial value is provided inside parenthesis, this is called **direct initialization**.

Direct initialization was initially introduced to allow for more efficient initialization of complex objects (those with class types, which we'll cover in a future chapter). **Just like copy initialization, direct initialization had fallen out of favor in modern C++,** largely due to being superseded by list initialization. However, we now know that list initialization has a few quirks of its own, and so direct initialization is once again finding use in certain cases.

3.5.1.4 List initialization

The modern way to initialize objects in C++ is to use a form of initialization that makes use of curly braces. This is called **list initialization** (or **uniform initialization** or **brace initialization**). List initialization has an added benefit: “narrowing conversions” in list initialization are ill-formed. **This means that if you try to brace initialize a variable using a value that the variable cannot safely hold, the compiler is required to produce a diagnostic (usually an error).** When a variable is initialized using empty braces, value initialization takes place. **In most cases, value initialization will initialize the variable to zero (or empty, if that’s more appropriate for a given type).** In such cases where zeroing occurs, this is called **zero initialization**.

Essentially writing:

<code>int a {};</code> is the same as writing <code>int a = 0;</code>

3.5.1.5 Best practice

Prefer direct list initialization (or value initialization) for initializing your variables.

- **Compiler errors** (a good way to identify if the type is compatible with the value you are trying to assign to it)
- **Zero initialization > Default initialization**

Prepend the `[[maybe_unused]]` attribute if you initialize a variable but **don’t want the compiler warning you that it’s not being used.**

3.6 Introduction to iostream

3.6.1 *The input/output library*

The input/output library (io library) is **part of the C++ standard library** that deals with basic input and output. We'll use the functionality in this library to get input from the keyboard and output data to the console. **The io part of iostream stands for input/output.**

To use the functionality defined within the iostream library, we need to include the iostream header at the top of any code file that uses the content defined in iostream.

```
#include <iostream>
```

```
// rest of code that uses iostream functionality here
```

3.6.2 *Cout: Character output*

The *iostream* library contains a few predefined variables for us to use. One of the most useful is **std::cout**, which allows us to send data to the console to be printed as text. *cout* stands for “character output”.

```
#include <iostream> // for std::cout

int main()

{

    std::cout << "Hello world!"; // print Hello world! to console

    return 0;

}
```

In this program, we have included *iostream* so that we have access to *std::cout*. Inside our *main* function, we use *std::cout*, along with the **insertion operator (<<)**, to send the text *Hello world!* to the console to be printed. To print more than one thing on the same line, the insertion operator (<<) can be used multiple times in a single statement to concatenate (link together) multiple pieces of output.

```
int X {};
```



```
std::cout << "The variable named X is equal to: " << X;
```

3.6.2.1 Cout Buffer

Consider a rollercoaster ride at your favorite amusement park. Passengers show up (at some variable rate) and get in line. Periodically, a train arrives and boards passengers (up to the maximum capacity of the train). When the train is full, or when enough time has passed, the train departs with a batch of passengers, and the ride commences. Any passengers unable to board the current train wait for the next one.

This analogy is similar to how output sent to `std::cout` is typically processed in C++. Statements in our program request that output be sent to the console. However, that output is typically not sent to the console immediately. Instead, the requested output “gets in line”, and is stored in a region of memory set aside to collect such requests (called a **buffer**). Periodically, the buffer is **flushed**, meaning all of the data collected in the buffer is transferred to its destination (in this case, the console).

3.6.3 Endl: End line

So, what happens when we write the following?

```
std::cout << "Hi!";  
  
std::cout << "My name is Alex.";
```

Result: Hi!My name is Alex. ← Notice there is **no space between sentence 1 and 2**, there is **also no newline**. It all gets written on the same line.

If we want to print separate lines of output to the console, we need to tell the console to move the cursor to the next line. We can do that by outputting a newline. A **newline** is an **OS-specific** character or sequence of characters that moves the cursor to the start of the next line.

```
std::cout << "Hi!" << std::endl; // std::endl will cause the cursor to move to the next line  
  
std::cout << "My name is Alex." << std::endl;  
  
Hi!  
  
My name is Alex.
```

3.6.3.1 Endl vs. \n

Using `std::endl` is often inefficient, as it actually does **two jobs**: it outputs a **newline** (moving the cursor to the next line of the console), **and** it **flushes the buffer** (which is slow). If we output multiple lines of text ending with `std::endl`, we will get multiple flushes, which is slow and probably unnecessary.

When outputting text to the console, we typically don't need to explicitly flush the buffer ourselves. **C++'s output system is designed to self-flush periodically**, and it's both simpler and more efficient to let it flush itself.

To output a newline without flushing the output buffer, we use `\n` (inside either single or double quotes), which is a special symbol that the compiler interprets as a newline character. `\n` moves the cursor to the next line of the console without causing a flush, so it will typically perform better. `\n` is also more concise to type and can be embedded into existing double-quoted text.

3.6.3.2 Best practice

- Output a newline **whenever a line of output is complete**.
- Prefer `\n` over `std::endl`

3.6.4 Cin: Character input

`std::cin` is another predefined variable that is defined in the `iostream` library.

Whereas `std::cout` prints data to the console using the insertion operator (`<<`), `std::cin` (which stands for "character input") reads input from keyboard using the **extraction operator** (`>>`). The input must be stored in a variable to be used.

```
int x{};           // define variable x to hold user input (and value-initialize it)

std::cin >> x;     // get number from keyboard and store it in variable x

std::cout << "You entered " << x << '\n';
```


3.6.4.1 Best practice

There's some debate over whether it's necessary to initialize a variable immediately before you give it a user provided value via another source (e.g. `std::cin`), since the user-provided value will just overwrite the initialization value. In line with our previous recommendation that variables should always be initialized, **best practice is to initialize the variable first.**

3.7 Uninitialized variables and undefined behavior

Unlike some programming languages, C/C++ does not automatically initialize most variables to a given value (such as zero). When a variable that is not initialized is given a memory address to use to store data, **the default value of that variable is whatever (garbage) value happens to already be in that memory address!** A variable that has not been given a known value (through initialization or assignment) is called an **uninitialized variable**.

3.7.1 Nomenclature

Many readers expect the terms “initialized” and “uninitialized” to be strict opposites, but they really aren’t! In common language, “initialized” means the object was provided with an initial value at the point of definition. “Uninitialized” means the object has not been given a known value yet (through any means, including assignment). Therefore, an object that is not initialized but is then assigned a value is no longer uninitialized (because it has been given a known value).

To recap:

- **Initialized** = The object is given a known value at the point of definition.
- **Assignment** = The object is given a known value beyond the point of definition.
- **Uninitialized** = The object has not been given a known value yet.

3.7.2 As an aside

This lack of initialization is a performance optimization inherited from C, back when computers were slow. Imagine a case where you were going to read in 100,000 values from a file. **In such case, you might create 100,000 variables, then fill them with data from the file.**

If C++ initialized all of those variables with default values upon creation, this would result in 100,000 initializations (which would be slow), and for little benefit (since you’re overwriting those values anyway). For now, **you should always initialize your variables because the cost of doing so is minuscule compared to the benefit.** Once you are more comfortable with the language, there may be certain cases where you omit the initialization for optimization purposes. But this should always be done selectively and intentionally.

3.7.3 *WARNING: Debug mode initializes with preset values*

Some compilers, such as Visual Studio, *will* initialize the contents of memory to some preset value when you're using a debug build configuration. This will not happen when using a release build configuration.

3.7.4 *Undefined behavior*

Using the value from an uninitialized variable is our first example of undefined behavior. **Undefined behavior** (often abbreviated **UB**) is the result of executing code whose behavior is not well-defined by the C++ language. In this case, the C++ language doesn't have any rules determining what happens if you use the value of a variable that has not been given a known value. Consequently, if you actually do this, undefined behavior will result.

3.7.5 *Implementation-defined behavior*

Implementation-defined behavior means the behavior of some syntax is **left up to the implementation (the compiler) to define**. Such behaviors must be consistent and documented, but different compilers may produce different results.

Example:

<code>sizeof(int)</code> // the number of bytes assigned to an integer type depends on the compiler

3.7.6 *Unspecified behavior*

This is almost identical to implementation-defined behavior. The only difference here is that the **implementation (compiler) doesn't have to document why it behaves a certain way**.

3.7.7 *Best practice*

Avoid implementation-defined and unspecified behavior whenever possible, as they may cause your program to malfunction on other implementations.

3.8 Keywords and naming identifiers

3.8.1 Keywords

A - C	D - P	R - Z
alignas (C++11)	decltype (C++11)	constexpr (reflection TS)
alignof (C++11)	default (1)	register (2)
and	delete (1)	reinterpret_cast
and_eq	do	requires (C++20)
asm	double	return
atomic_cancel (TMTS)	dynamic_cast	short
atomic_commit (TMTS)	else	signed
atomic_noexcept (TMTS)	enum (1)	sizeof (1)
auto (1) (2) (3) (4)	explicit	static
bitand	export (1) (3)	static_assert (C++11)
bitor	extern (1)	static_cast
bool	false	struct (1)
break	float	switch
case	for (1)	synchronized (TMTS)
catch	friend	template
char	goto	this (4)
char8_t (C++20)	if (2) (4)	thread_local (C++11)
char16_t (C++11)	inline (1)	throw
char32_t (C++11)	int	true
class (1)	long	try
compl	mutable (1)	typedef
concept (C++20)	namespace	typeid
const	new	typename
constexpr (C++11)	noexcept (C++11)	union
constinit (C++20)	not	unsigned
const_cast	not_eq	using (1)
continue	nullptr (C++11)	virtual
co_await (C++20)	operator (4)	void
co_return (C++20)	or	volatile
co_yield (C++20)	or_eq	wchar_t
	private (3)	while
	protected	xor
	public	xor_eq

- (1) — meaning changed or new meaning added in C++11.
- (2) — meaning changed or new meaning added in C++17.
- (3) — meaning changed or new meaning added in C++20.
- (4) — new meaning added in C++23.

3.8.2 Identifiers

3.8.2.1 Naming rules

Now that you know how you can name a variable, let's talk about how you should name a variable (or function). First, it is a convention in C++ that **variable names should begin with a lowercase letter**. If the variable name is a **single word or acronym, the whole thing should be written in lowercase letters**.

- Names **must begin with a letter or an underscore**
- Names **cannot contain whitespaces or special characters** like !, #, %, etc.
- **Reserved words** (like C++ keywords, such as int) **cannot be used** as names
- Note: names are case-sensitive

3.8.2.2 Beste practice:

When working in an existing program, use the conventions of that program (even if they don't conform to modern best practices). Use modern best practices when you're writing new programs. In any case, **avoid abbreviations (unless they are common/unambiguous)**. Although they reduce the time you need to write your code, they make your code harder to read.

3.9 Whitespace and basic formatting

Whitespace is a term that refers to characters that are used for formatting purposes. In C++, this refers primarily to **spaces, tabs, and newlines**.

Whitespace in C++ is generally **used for 3 things**:

- Separating certain language elements
- Inside text
- Formatting code

3.9.1.1 Whitespace rules

- **Separate language elements** such as keywords and identifiers
- **Comments** on a newline have to be re-assigned to be comments again: `//`
- **Preprocessor** using directives must be **on separate lines**
- Quoted text
 - **Spaces** in between tags **are taken literally** by the compiler
 - **Newlines** within text is only allowed in a certain format, just pressing enter in text such as a string is **not allowed**.

```
int x; // int and x must be whitespace separated
```

```
int
```

```
x; // this is also valid
```

The whitespace between the **keyword** `int` and the **identifier** `x` makes it so that the compiler doesn't see `intx`, it would see `intx` as an identifier. It doesn't matter how much whitespace there is between them as the compiler will strip this out

```
int main()
{
    std::cout << "Hello
    World!\n";

    return 0;
}
```

← Incorrect newline implementation

Error List		
Entire Solution		
13 Errors 0 Warnings 0		
	Code	Description
✖	C2065	'n': undeclared identifier
✖	C2065	'World': undeclared identifier
abc	E0065	expected a ';'
✖	C2017	illegal escape sequence
abc	E0008	missing closing quote
abc	E0008	missing closing quote
✖	C2001	newline in constant
✖	C2001	newline in constant
✖	C2143	syntax error: missing ';' before '!'
✖	C2143	syntax error: missing ';' before 'return'
✖	C2143	syntax error: missing ';' before 'string'
✖	C2146	syntax error: missing ';' before identifier 'World'
abc	E0007	unrecognized token

← Error output

```
int main()
{
    std::cout << "Hello"
               << "World!\n";

    return 0;
}
```

← Correct newline implementation

3.9.1.2 Using whitespace to format code

Whitespace is otherwise generally ignored. This means we can use whitespace wherever we like to format our code in order to make it easier to read.

Examples: less readable → more readable

```
1 | #include <iostream>
2 | int main(){std::cout<<"Hello world";return 0;}
```

```
1 | #include <iostream>
2 | int main() {
3 |     std::cout << "Hello world";
4 |     return 0;
5 | }
```

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     std::cout << "Hello world";
6 |
7 |     return 0;
8 | }
```


3.9.1.3 Basic formatting

Unlike some other languages, **C++ does not enforce any kind of formatting restrictions** on the programmer. For this reason, we say that **C++ is a whitespace-independent language**.

This is a mixed blessing. On one hand, it's nice to have the freedom to do whatever you like. On the other hand, many different methods of formatting C++ programs have been developed throughout the years, and you will find (sometimes significant and distracting) disagreement on which ones are best. **Our basic rule of thumb is that the best styles are the ones that produce the most readable code, and provide the most consistency.**

3.9.1.4 Recommendations

1. It's fine to use **either tabs or spaces** for indentation (most IDEs have a setting where you can convert a tab press into the appropriate number of spaces).

Developers who prefer spaces tend to do so because it ensures that code is precisely aligned as intended regardless of which editor or settings are used.

Proponents of using tabs wonder why you wouldn't use the character designed to do indentation for indentation, especially as you can set the width to whatever your personal preference is.

2. There are **two conventional styles** for function braces each with their own variations

K & R or **C-style**, puts the opening bracket on the same line as the statement

Reduces the amount of vertical whitespace (as you aren't devoting an entire line to an opening curly brace), so you can fit more code on a screen. This enhances code comprehension, as you don't need to scroll as much to understand what the code is doing.

```
int main(){
    std::cout << "Hello World!\n";

    return 0;
}
```

Allman, put the opening bracket on a separate line right under the statement

This **enhances readability**, and is **less error prone** since your brace pairs should always be indented at the same level. If you get a compiler error due to a brace mismatch, it's very easy to see where the issue came from.

```
int main()
{
    std::cout << "Hello World!\n";

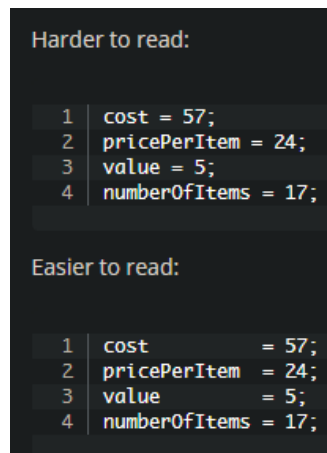
    return 0;
}
```

- Each statement within curly braces should start one tab in from the opening brace of the function it belongs to
- Lines should not be too long. Typically, 80 characters** has been the de facto standard for the maximum length a line should be. If a line is going to be longer, it should be split (**at a reasonable spot**) into multiple lines.
- If a long line is **split** with an operator (e.g. << or +), **the operator should be placed at the beginning of the next line**, not the end of the current line.

```
int main()
{
    std::cout << "Hello"
               << "World!\n";

    return 0;
}
```

6. Use whitespace to make your code easier to read by aligning values or comments or adding spacing between blocks of code.



```
Harder to read:
1 | cost = 57;
2 | pricePerItem = 24;
3 | value = 5;
4 | numberOfItems = 17;

Easier to read:
1 | cost      = 57;
2 | pricePerItem = 24;
3 | value      = 5;
4 | numberOfItems = 17;
```

7. Use the built-in automatic formatting feature for consistency, if available.

3.9.1.5 Style guides

A **style guide** is a concise, opinionated document containing (sometimes arbitrary) programming conventions, formatting guidelines, and best practices. The goal of a style guide is to ensure that all developers on a project are programming in a consistent manner.

- C++ Core Guidelines by Bjarne Stroustrup and Herb Sutter
- Google
- LLVM
- GCC / GNU

We generally **favor the C++ Core Guidelines**, as they are up to date and widely applicable.

3.10 Introduction to literals

Consider the following two statements:

```
int main()
{
    std::cout << "Hello world!";
    int x{ 5 };

    return 0;
}
```

“Hello world!” and ‘5’ are **literal constants**. They are fixed values inserted directly into the source code. Literals and variables both have a value (and a type). Unlike a variable (whose value can be set and changed through initialization and assignment respectively), **the value of a literal is fixed (5 is always 5)**. This is why literals are called constants.

3.10.1 Compiler-related

```
3  int main()
4  {
5      std::cout << 5 << '\n'; // print the value of a literal
6
7      int x{ 5 };
8      std::cout << x << '\n'; // print the value of a variable
9      return 0;
10 }
```

On line 7, we’re creating a variable named `x`, and initializing it with value 5. **The compiler will generate code that copies the literal value 5 into whatever memory location is given to `x`.**

On line 8, **when we print `x`, the compiler will generate code that causes `std::cout` to print the value at the memory location of `x` (which has value 5).**

Thus, both output statements do the same thing (print the value 5). But **in the case of the literal, the value 5 can be printed directly**. In the case of the variable, the value 5 must be fetched from the memory the variable represents. This also explains why a literal is constant while a variable can be changed. A literal’s value is placed directly in the executable, and the executable itself can’t be changed after it is created. A variable’s value is placed in memory, and the value of memory can be changed while the executable is running.

3.11 Introduction to Operators

In mathematics, an **operation** is a process involving zero or more input values (called **operands**) that produces a new value (called an *output value*). The specific operation to be performed is denoted by a symbol called an **operator**.

You are likely already quite familiar with standard arithmetic operators from common usage in mathematics, including addition (+), subtraction (-), multiplication (*), and division (/). In C++, assignment (=) is an operator as well, as are insertion (<<), extraction (>>), and equality (==). While most operators have symbols for names (e.g. +, or ==), there are **also a number of operators that are keywords** (e.g. **new**, **delete**, and **throw**).

The number of operands that an operator takes as input is called the operator's **arity**. Few people know what this word means, so don't drop it in a conversation and expect anybody to have any idea what you're talking about. **Operators in C++ come in four different arities:**

- **Unary**, acts on **one** operand
An example of a unary operator is the - operator. For example, given -5, operator- takes literal operand 5 and flips its sign to produce new output value -5.
- **Binary**, acts on **two** operands (often named the left and right operand)
An example of a binary operator is the + operator. For example, given 3 + 4, operator+ takes the left operand 3 and the right operand 4 and applies mathematical addition to produce new output value 7. The insertion (<<) and extraction (>>) operators are binary operators, taking std::cout or std::cin on the left side, and the value to output or variable to input to on the right side.
- **Ternary**, acts on **three** operands
There is only one of these in C++ (the conditional operator), which we'll cover later.
- **Nullary**, acts on **zero** operands
There is also only one of these in C++ (the throw operator), which we'll also cover later.

Note that some operators have more than one meaning depending on how they are used. For example, operator- has two contexts. It can be used in unary form to invert a number's sign (e.g. to convert 5 to -5, or vice versa), or it can be used in binary form to do subtraction (e.g. 4 - 3).

3.11.1 Chaining operators

Operators can be **chained together such that the output of one operator can be used as the input for another operator**. For example, given the following: `2 * 3 + 4`, the multiplication operator goes first, and converts left operand 2 and right operand 3 into return value 6 (which becomes the left operand for the plus operator). Next, the plus operator executes, and converts left operand 6 and right operand 4 into new value 10.

We'll talk more about the order in which operators execute when we do a deep dive into the topic of operators. For now, it's enough to know that the arithmetic operators execute in the same order as they do in standard mathematics: **Parenthesis first, then Exponents, then Multiplication & Division, then Addition & Subtraction**. This ordering is sometimes abbreviated ***PEMDAS***, or expanded to the mnemonic "Please Excuse My Dear Aunt Sally".

3.11.2 Return values and side effects

Most operators in C++ just use their operands to calculate a return value. There are a few operators that do not produce return values (such as `delete` and `throw`). We'll cover what these do later. Some operators have additional behaviors. An operator (or function) that has some observable effect beyond producing a return value is said to have a **side effect**.

For example, when `x = 5` is evaluated, **the assignment operator has the side effect of assigning the value 5 to variable x**. The changed value of `x` is observable (e.g. by printing the value of `x`) even after the operator has finished executing. `std::cout << 5` **has the side effect of printing 5 to the console**. We can observe the fact that 5 has been printed to the console even after `std::cout << 5` has finished executing.

Operators with side effects are usually called for the behavior of the side effect rather than for the return value (if any) those operators produce.

3.11.3 Nomenclature

In common language, the term “side effect” is typically used to mean a secondary (often negative or unexpected) result of some other thing happening (such as taking medicine). For example, a common side effect of taking oral antibiotics is diarrhea. As such, we often think of side effects as things we want to avoid, or things that are incidental to the primary goal.

In C++, **the term “side effect” has a different meaning: it is an observable effect of an operator or function beyond producing a return value.**

Since assignment has the observable effect of changing the value of an object, this is considered a side effect. We use certain operators (e.g. the assignment operator) primarily for their side effects. In such cases, the side effect is both beneficial and predictable (and it is the return value that is often incidental).

3.12 Introduction to expressions

An **expression** is a non-empty **sequence of literals, variables, operators, and function calls** that calculates a single value. The process of executing an expression is called **evaluation**, and the single value produced is called the **result** of the expression. When an expression is evaluated, **each of the terms inside the expression are evaluated**, until a single value remains.

While most expressions are used to calculate a value, **expressions can also identify an object** (which can be evaluated to get the value held by the object) or a function.

Example:

```
2           // 2 is a literal that evaluates to value 2
"Hello world!" // "Hello world!" is a literal that evaluates to text "Hello world!"
x           // x is a variable that evaluates to the value of x
2 + 3       // operator+ uses operands 2 and 3 to evaluate to value 5
five()      // evaluates to the return value of function five()
```

As you can see, literals evaluate to their own values. Variables evaluate to the value of the variable. Operators (such as **operator+**) use their operands to evaluate to some other value. We haven't covered function calls yet, but in the context of an expression, function calls evaluate to whatever value the function returns. **Expressions involving operators with side effects are a little trickier:**

```
x = 5       // x = 5 has side effect of assigning 5 to x, evaluates to x
x = 2 + 3    // has side effect of assigning 5 to x, evaluates to x
std::cout << x // has side effect of printing value of x to console, evaluates to std::cout
```

Note that **expressions do not end in a semicolon**, and **cannot be compiled by themselves**. For example, if you were to try compiling the expression `x = 5`, your compiler would complain (probably about a missing semicolon). Rather, expressions are always evaluated as part of statements.

If you were to break `int x {2+3};` down into syntax, it would look like this:

```
type identifier {expression};
```

Type could be any valid type (we chose `int`). **identifier** could be any valid name (we chose `x`). And **expression** could be any valid expression (we chose `2 + 3`, which uses two literals and an operator).

Key insight: Wherever you can use a single value in C++, you can use a value-producing expression instead, and the expression will be evaluated to produce a single value.

3.12.1 Useless expression statements

We can also make expression statements that compile but have no effect. For example, the expression statement (**`2 * 3;`**) is an expression statement whose **expression evaluates to the result value of 6, which is then discarded**. While **syntactically valid**, such expression statements are useless. **Some compilers (such as gcc and Clang) will produce warnings** if they can detect that an expression statement is useless.

3.12.2 Subexpressions, full expressions and compound expressions

Simplifying a bit, a **subexpression** is an expression used as an operand. For example, the **subexpressions of `x = 4 + 5`** are **`x`** and **`4 + 5`**. The **subexpressions of `4 + 5`** are **`4`** and **`5`**.

A **full expression** is an expression that is not a subexpression. **`x = 4 + 5`** is the full expression.

In casual language, a **compound expression** is an expression that contains two or more uses of operators. **`x = 4 + 5`** is a **compound expression because it contains two uses of operators (`operator=` and `operator+`)**

3.12.3 Recap

What is the difference between a statement and an expression?

Statements are used when we want the program to **perform an action**.

Expressions are used when we want the program to **calculate a value**.

3.13 Chapter summary

A **statement** is a type of instruction that causes the program to perform some action. Statements are often terminated by a semicolon.

A **function** is a collection of statements that execute sequentially. Every C++ program must include a special function named `main`. When you run your program, execution starts at the top of the `main` function.

In programming, the name of a function (or object, type, template, etc.) is called its **identifier**.

An **object** is a **region of storage** (usually memory) that can store a value, and has other associated properties. In C++, **direct memory access is discouraged**. Instead, **we access memory indirectly through an object**.

A **variable is an object** that has been named (identifier). Naming our objects let us refer to them again later in the program.

To create a variable, we use a **definition statement**. Later we can **assign** it a value. When the program is run, each defined variable is **instantiated**, which means it is assigned a memory address. The process of **specifying an initial value** for an object is called **initialization**, and the syntax used to initialize an object is called an **initializer**.

The rules that govern how elements of the C++ language are constructed is called **syntax**. A **syntax error** occurs when you violate the grammatical rules of the language.

Every C++ program must have a special function named `main` (all lower - case letters) which is of the `int` type. (because the number being returned is the exit status code of `main`)

```
int main()
{
    return 0;
}
```

Comments allow the programmer to leave notes in the code. C++ supports two types of comments. Line comments start with a `//` and run to the end of the line. Block comments start with a `/*` and go to the paired `*/` symbol. Don't nest block comments.

Data is any information that can be moved, processed, or stored by a computer. A single piece of data is called a **value**.

A **data type** tells the compiler how to interpret a piece of data into a meaningful value. An **integer** is a number that can be written without a fractional component.

Initialization Type	Example	Note
Default initialization	<code>int x;</code>	In most cases, leaves variable with indeterminate value
Copy initialization	<code>int x = 5;</code>	
Direct initialization	<code>int x (5);</code>	
Direct list initialization	<code>int x { 5 };</code>	Narrowing conversions disallowed
Copy list initialization	<code>int x = { 5 };</code>	Narrowing conversions disallowed
Value initialization	<code>int x {};</code>	Usually performs zero initialization

Direct initialization is sometimes called **parenthesis initialization**.

List initialization (including value initialization) is sometimes called **uniform initialization** or **brace initialization**. You should prefer brace initialization over the other initialization forms, and prefer initialization over assignment.

A **literal constant** is a fixed value inserted directly into the source code. Examples are 5 and “Hello world!”.

An **operation** is a process involving zero or more input values, called **operands**. The specific operation to be performed is denoted by the provided **operator**. The result of an operation produces an output value.

Unary operators take one operand. **Binary** operators take two operands, often called left and right. **Ternary** operators take three operands. **Nullary** operators take zero operands.

An **expression** is a sequence of literals, variables, operators, and function calls that are evaluated to produce a single output value. The calculation of this output value is called **evaluation**. The value produced is the **result** of the expression.

An **expression statement** is an expression that has been turned into a statement by placing a semicolon at the end of the expression.

When writing programs, add a few lines or a function, compile, resolve any errors, and make sure it works. Don’t wait until you’ve written an entire program before compiling it for the first time!

First-draft programs are often messy and imperfect. Most code requires cleanup and refinement to get great!

4 Functions and files

4.1 Introduction to functions

In the last chapter, we defined a function as a **collection of statements** that execute sequentially. While that is certainly true, that definition doesn't provide much insight into why functions are useful. Let's update our definition: A function is a **reusable sequence of statements** designed to do a particular job.

You already know that every executable program must have a function named `main` (which is where the program starts execution when it is run). However, **as programs start to get longer and longer, putting all the code inside the main function becomes increasingly hard to manage.** Functions provide a way for us to **split our programs into small, modular chunks that are easier to organize, test, and use.** Most programs use many functions. The C++ standard library comes with plenty of already-written functions for you to use -- however, it's just as common to write your own. Functions that you write yourself are called **user-defined functions**.

A program will be executing statements sequentially inside one function when it encounters a **function call**. A function call is an expression that tells the CPU to **interrupt the current function and execute the called function**. The CPU **"puts a bookmark"** at the current point of execution, and then **calls** (executes) the function named in the function call. When the called function ends, **the CPU returns back to the point it bookmarked**, and resumes execution.

The function initiating the function call is the **caller**, and the function being called is the **callee** or **called** function.

4.1.1 User-defined functions

Functions that you write yourself are called **user-defined functions**.

The start of a user-defined function is informally called the **function header**, and it tells the compiler about the existence of a function, the function's name (**identifier**), and some other information that we'll cover in future lessons (like the return type and parameter types).

The **parentheses** after the identifier tell the compiler that we're **defining a function**.

The curly braces and statements in-between are called the **function body**. This is where the statements that determine what your function does will go.

```
4 void PrintTheSumOf(int a, int b)
5 {
6     // Great, we just landed here from main()
7     // Now let's execute this sequentially
8
9     std::cout << "Inside of PrintTheSumOf()" << "\n";
10    // Ok here we print some text to show we are inside of this function
11
12
13    int sum{ a + b };
14    // Here we initialize a variable (named object) with the integer type
15    // It's a good thing that we assign it a value as to avoid undefined behaviour
16    // In this case the name is sum and it holds the expression a operator+ b (our initializer)
17    // Now or compiler will instantiate it!
18
19    std::cout << sum << "\n";
20    // This statement is little more complicated
21    // We use cout and operator<< to display the sum and then add a new line
22
23    std::cout << "Ending of PrintTheSumOf()" << std::endl;
24 }
25
26
27 int main()
28 {
29     // Because this is main, this is the actual start of the program
30
31     PrintTheSumOf(3 ,7);
32     // We encounter a fuction, we know this because of the parenthesis
33     // It seems to be doing something with two literal numbers
34     // Good thing it has a name (identifier), let's jump there
35     // But first we take note of this location so it's easy to jump back when we're done
36     // READ PrintTheSumOf()
37
38     std::cout << "back inside of main()" << std::endl;
39
40     return 0;
41     // This is the absolute end of our program
42     // Because main is a special function there is a lot going on
43     // The literal 0 here mean the program can exit with succes
44     // The literal 1 would mean the program exits with failure
45 }
46
```

4.1.1.1 Functions can call functions that call other functions

You've already seen that function **main** can call another function. Any function can call any other function. In the following program, function **main** calls function **doA**, which calls function **doB**:

```
1  #include <iostream> // for std::cout
2
3  void doB()
4  {
5      std::cout << "In doB()\n";
6  }
7
8
9  void doA()
10 {
11     std::cout << "Starting doA()\n";
12
13     doB();
14
15     std::cout << "Ending doA()\n";
16 }
17
18 // Definition of function main()
19 int main()
20 {
21     std::cout << "Starting main()\n";
22
23     doA();
24
25     std::cout << "Ending main()\n";
26
27     return 0;
28 }
```

4.1.1.2 Nested functions are not supported

Unlike some other programming languages, in C++, functions cannot be defined inside other functions. The following program is not legal:

```
1  #include <iostream>
2
3  int main()
4  {
5      void foo() // Illegal: this function is nested inside function main()
6      {
7          std::cout << "foo!\n";
8      }
9
10     foo(); // function call to foo()
11     return 0;
12 }
```

The proper way to write the above program is:

```
1  #include <iostream>
2
3  void foo() // no longer inside of main()
4  {
5      std::cout << "foo!\n";
6  }
7
8  int main()
9  {
10     foo();
11     return 0;
12 }
```

4.1.1.3 As an aside...

“foo” is a meaningless word that is often used as a **placeholder name for a function or variable** when the name is unimportant to the **demonstration of some concept**. Such words are called **metasyntactic variables** (though in common language they’re often called “**placeholder names**” since nobody can remember the term “metasyntactic variable”). Others include: bar, baz and 3-letter words that end in “oo”, such as goo, moo and boo.

For those interested in etymology (how words evolve), RFC 3092 is an interesting read.

4.2 Function return values (value-returning functions)

When you write a user-defined function, you get to determine whether your function will return a value back to the caller or not. To return a value back to the caller, two things are needed.

First, your function has to indicate **what type of value will be returned**. This is done by **setting the function's return type**, which is the type that is defined before the function's name. Note that this doesn't determine what specific value is returned -- it only determines what type of value will be returned. Void is a special case that doesn't have to return anything. We explore functions that return void further in the next lesson.

Second, inside the function that will return a value, we use a **return statement** to indicate the specific value being returned to the caller. The specific value returned from a function is called the **return value**. When the return statement is executed, the function exits immediately, and the return value is **copied** from the function back to the caller. **This process is called return by value.**

When a called function returns a value, **the caller may decide to use that value in an expression or statement** (e.g. by using it to initialize a variable, or sending it to `std::cout`) or ignore it (by doing nothing else). If the caller ignores the return value, it is discarded (nothing is done with it).

```
1 | #include <iostream>
2 |
3 | int getValueFromUser() // this function now returns an integer value
4 | {
5 |     std::cout << "Enter an integer: ";
6 |     int input{};
7 |     std::cin >> input;
8 |
9 |     return input; // return the value the user entered back to the caller
10 | }
11 |
12 | int main()
13 | {
14 |     int num { getValueFromUser() }; // initialize num with the return value of getValueFromUser()
15 |
16 |     std::cout << num << " doubled is: " << num * 2 << '\n';
17 |
18 |     return 0;
19 | }
```

When this program executes, the first statement in `main` will create an `int` variable named `num`. When the program goes to initialize `num`, it will see that there is a function call to `getValueFromUser()`, so it will go execute that function. Function `getValueFromUser`, asks the user to enter a value, and then it returns that value back to the caller (`main`). This return value is used as the initialization value for variable `num`.

4.2.1 Revisiting main()

You now have the conceptual tools to understand how the `main()` function actually works. When the program is executed, the operating system makes a function call to `main()`. Execution then jumps to the top of `main()`. The statements in `main()` are executed sequentially. **Finally, `main()` returns an integer value (usually 0), and your program terminates.**

C++ disallows calling the `main()` function explicitly. **C** does allow `main()` to be called explicitly, so some C++ compilers will allow this for compatibility reasons. For now, you should also **define your `main()` function at the bottom of your code file**, below other functions, and avoid calling it explicitly.

4.2.1.1 Status codes

The **return value** from `main()` is sometimes called a **status code** (also sometimes called an **exit code**, or rarely a return code). Just like the return value of a function is passed back to the caller of the function, the status code is passed back to the caller of the program. The caller of the program can then use this status code to determine whether your program ran successfully or not.

0: Indicates that the program ran successfully without any errors. This is the standard value for a successful program termination. Alternatively, we can write **`EXIT_SUCCESS`** (defined in the standard library).

1: Indicate that the program encountered some form of error or abnormal termination. Alternatively, we can write **`EXIT_FAILURE`** (defined in the standard library).

```
#include <stdlib.h>

int main()
{
    return EXIT_FAILURE;
}
```

4.2.2 *A value-returning function that does not return a value will produce undefined behavior*

A function that returns a value is called a **value-returning function**. A function is value-returning if the return type is anything other than void. **A value-returning function must return a value of that type (using a return statement)**, otherwise undefined behavior will result.

In most cases, compilers will detect if you've forgotten to return a value. However, in some complicated cases, the compiler may not be able to properly determine whether your function returns a value or not in all cases, so you should not rely on this.

The only exception to the rule that a value-returning function must return a value via a return statement is for function **main()**. The function **main()** will **implicitly return the value 0 if no return statement is provided**. That said, it is best practice to explicitly return a value from **main**, both to show your intent, and for consistency with other functions (which will exhibit undefined behavior if a return value is not specified).

4.2.3 *Functions can only return a single value*

A value-returning function can only return a single value back to the caller each time it is called. Note that the value provided in a return statement doesn't need to be literal -- it can be the result of any valid expression, including a variable or even a call to another function that returns a value. There are various ways to work around the limitation of functions only being able to return a single value, which we'll cover in future lessons.

4.2.4 *The function author can decide what the return value means*

The meaning of the value returned by a function is determined by the function's author. Some functions use return values as status codes, to indicate whether they succeeded or failed. Other functions return a calculated or selected value. Other functions return nothing.

Because of the wide variety of possibilities here, it's a good idea to document your function with a comment indicating what the return values mean.

4.2.5 Reusing functions

While this program works, it's a little redundant. In fact, this program violates one of the central tenets of good programming: **Don't Repeat Yourself** (often abbreviated **DRY**).

Why is repeated code bad? If we wanted to change the text "Enter an integer:" to something else, we'd have to update it in two locations. And what if we wanted to initialize 10 variables instead of 2? That would be a lot of **redundant code** (making our programs longer and harder to understand), and a lot of **room for typos to creep in**.

```
1  #include <iostream>
2
3  int getValueFromUser()
4  {
5      std::cout << "Enter an integer: ";
6      int input{};
7      std::cin >> input;
8
9      return input;
10 }
11
12 int main()
13 {
14     int x{ getValueFromUser() }; // first call to getValueFromUser
15     int y{ getValueFromUser() }; // second call to getValueFromUser
16
17     std::cout << x << " + " << y << " = " << x + y << '\n';
18
19     return 0;
20 }
```

In this program, we call **getValueFromUser** twice, once to initialize variable **x**, and once to initialize variable **y**. That saves us from duplicating the code to get user input, and reduces the odds of making a mistake. Once we know **getValueFromUser** works, we can call it as many times as we desire.

This is the **essence of modular programming**: the ability to write a function, test it, ensure that it works, and then know that we can reuse it as many times as we want and it will continue to work (so long as we don't modify the function -- at which point we'll have to retest it).

4.2.6 *Best practice*

Follow **DRY**: “**don’t repeat yourself**”. If you need to do something more than once, consider how to modify your code to **remove as much redundancy as possible**. Variables can be used to store the results of calculations that need to be used more than once (so we don’t have to repeat the calculation). Functions can be used to define a sequence of statements we want to execute more than once. And loops (which we’ll cover in a later chapter) can be used to execute a statement more than once.

Like all best practices, **DRY is meant to be a guideline, not an absolute**. Reader Yariv has noted that DRY can harm overall comprehension when code is broken into pieces that are too small.

The opposite of DRY is WET (“Write everything twice”).

4.3 Void functions (non-value returning functions)

Functions are **not required to return a value** back to the caller. To tell the compiler that a function does not return a value, a return type of void is used. A function that does not return a value is called a **non-value returning function** (or a **void function**). Trying to return a value from a non-value returning function will result in a **compilation error**.

```
void printHi()
{
    std::cout << "Hi" << '\n';

    // This function does not return a value so no return statement is needed
}

int main()
{
    printHi(); // okay: function printHi() is called, no value is returned

    return 0;
}
```

4.3.1 Void return statement

A void function will automatically return to the caller at the end of the function. No return statement is required.

A return statement (with no return value) **can be used** in a void function -- such a statement will cause the function to return to the caller at the point where the return statement is executed. This is the same thing that happens at the end of the function anyway. Consequently, putting an empty return statement at the end of a void function is **redundant**.

Best practice:

Do **not** put a return statement at the end of a non-value returning function.

4.4 Introduction to function parameters and arguments

In the previous lesson, we learned that we could have a function return a value back to the function's caller. Now we need a way to **pass values of objects in order to manipulate them in our functions**, this is where parameters and arguments come in.

In many cases, it is useful to be able to pass information to a function being called, so that the function has data to work with. For example, if we wanted to write a function to add two numbers, we need some way to tell the function which two numbers to add when we call it. Otherwise, how would the function know what to add? We do that via function **parameters and arguments**.

A function **parameter** is a variable **used in the header** of a function. Function parameters work almost identically to variables defined inside the function, but with **one difference: they are initialized with a value provided by the caller of the function**.

Function parameters are defined in the function header by placing them in between the parenthesis after the function name, with multiple parameters being separated by commas.

An **argument** is a value that is passed from the caller to the function when a function call is made.

```
✓ #include <iostream>
  [ #include <string>

  ✓ void foo(std::string sentence)
    {
      std::cout << sentence << '\n';
    }

  ✓ int main()
    {
      foo("Hello World!");

      return 0;
    }
  [ ]
```

In the example above we ask for a sentence to be passed of the type `std::string`, the variable `sentence` is the **parameter**. The literal `"Hello World!"` is the **argument** we pass.

4.4.1 How parameters and arguments work together

When a function is called, all of the **parameters of the function are created as variables**, and the **value of each of the arguments is copied into the matching parameter** (using **copy initialization**).

This process is called **pass by value**. Function parameters that utilize pass by value are called **value parameters**.

Note that the **number of arguments must generally match the number of function parameters**, or the compiler will throw an error. The argument passed to a function can be any valid expression (as the argument is essentially just an initializer for the parameter, and initializers can be any valid expression).

4.4.2 Using return values as arguments

Now we step it up a bit, in previous lesson we discussed return values. We could also use these by passing them to another function.

```
1  #include <iostream>
2
3  int getValueFromUser()
4  {
5      std::cout << "Enter an integer: ";
6      int input{};
7      std::cin >> input;
8
9      return input;
10 }
11
12 void printDouble(int value)
13 {
14     std::cout << value << " doubled is: " << value * 2 << '\n';
15 }
16
17 int main()
18 {
19     printDouble(getValueFromUser());
20
21     return 0;
22 }
```


4.4.3 How parameters and return values work together

By using both parameters and a return value, we can create functions that take data as input, do some calculation with it, and return the value to the caller.

```
1  #include <iostream>
2
3  int add(int x, int y)
4  {
5      return x + y;
6  }
7
8  int main()
9  {
10     std::cout << add(4, 5) << std::endl;
11     return 0;
12 }
```

The diagram illustrates the execution of the `add` function within the `main` function. Annotations show the arguments `x = 4` and `y = 5` being passed to the `add` function. The return value of the function is `9`, which is then printed by the `main` function.

4.4.4 Unreferenced parameters

In certain cases, you will encounter functions that have parameters that are not used in the body of the function. These are called unreferenced parameters.

This can happen when a function parameter was once used, but is not used any longer. Just like with unused local variables, your compiler will probably warn you.

If the unused function parameter were simply removed, then any existing call to the function would break (because the function call would be supplying more arguments than the function could accept).

```
1 void doSomething(int count) // warning: unreferenced parameter count
2 {
3     // This function used to do something with count but it is not used any longer
4 }
5
6 int main()
7 {
8     doSomething(4);
9
10    return 0;
11 }
```

In a function definition, the **name of a function parameter is optional**. Therefore, in cases where a function parameter needs to exist but is not used in the body of the function, you can simply omit the name. A parameter without a name is called an **unnamed parameter**.

```
1 void doSomething(int) // ok: unnamed parameter will not generate warning
2 {
3 }
```

Best practice:

When a function parameter **exists but is not used in the body of the function, do not give it a name**. You can optionally put a name inside a comment.

4.5 Introduction to local scope

Variables defined inside the body of a function are called **local variables** (as opposed to global variables, which we'll discuss in the future).

4.5.1 Lifetime

In Introduction to objects and variables, we discussed how a variable definition such as `int x;` causes the variable to be instantiated (created) when this statement is executed. **Function parameters are created and initialized when the function is entered, and variables within the function body are created and initialized at the point of definition.**

```
int add(int x, int y) // x and y created and initialized here
{
    int z{ x + y };    // z created and initialized here

    return z;
} // z, y, and x destroyed here
```

The natural follow-up question is, “**so when is an instantiated variable destroyed?**”. Local variables are destroyed in the **opposite order of creation at the end of the set of curly braces** in which it is defined (or for a function parameter, at the end of the function).

Much like a person's lifetime is defined to be the time between their birth and death, an object's **lifetime** is defined to be the time between its creation and destruction. Note that **variable creation and destruction happen when the program is running** (called **runtime**), not at compile time. Therefore, **lifetime is a runtime property**.

4.5.1.1 What happens when an object is destroyed?

In most cases, **nothing**. The **destroyed object becomes invalid**, and any **further use of the object will result in undefined behavior**. At some point after destruction, the memory used by the object will be freed up for reuse.

Advanced:

If the object is a **class type object**, prior to destruction, a special function called a **destructor** is **invoked**. In many cases, the destructor does nothing, in which case no cost is incurred.

4.5.2 Local scope

An identifier's **scope** determines where the identifier can be seen and used within the source code. When an identifier **can be seen and used**, we say **it is in scope**. When an identifier **cannot be seen**, we cannot use it, and we say **it is out of scope**.

Scope is a compile-time property, and trying to use an identifier when it is not in scope will result in a compile error.

A local variable's scope begins at the point of variable definition, and stops at the end of the set of curly braces in which it is defined (or for function parameters, at the end of the function). **This ensures variables cannot be used before the point of definition** (even if the compiler opts to create them before then). Local variables defined in one function are also not in scope in other functions that are called.

4.5.2.1 "Out of scope" vs "going out of scope"

The terms "out of scope" and "going out of scope" can be confusing to new programmers.

An identifier is **out of scope anywhere it cannot be accessed** within the code.

The term "going out of scope" is **typically applied to objects** rather than identifiers. We say an object **goes out of scope at the end of the scope** (the end curly brace) in which the object was instantiated. A local variable's **lifetime ends at the point where it goes out of scope**, so local variables are destroyed at this point. Note that **not all types of variables are destroyed** when they go out of scope.

```
1 | #include <iostream>
2 |
3 | int add(int x, int y) // x and y are created and enter scope here
4 | {
5 |     // x and y are usable only within add()
6 |     return x + y;
7 | } // y and x go out of scope and are destroyed here
8 |
9 | int main()
10 | {
11 |     int a{ 5 }; // a is created, initialized, and enters scope here
12 |     int b{ 6 }; // b is created, initialized, and enters scope here
13 |
14 |     // a and b are usable only within main()
15 |
16 |     std::cout << add(a, b) << '\n'; // calls add(5, 6), where x=5 and y=6
17 |
18 |     return 0;
19 | } // b and a go out of scope and are destroyed here
```

4.5.2.2 Functional separation

Names used for function parameters or variables declared in a function body are only visible within the function that declares them. This means **local variables within a function can be named without regard for the names of variables in other functions**. This helps keep functions independent.

We could have variable `x` in function `add()` and variable `x` in `main()` and they wouldn't interfere because of functional separation.

4.5.2.3 Where to define local variables

In modern C++, the best practice is that local variables inside the function body should be defined as close to their first use as reasonable.

4.5.2.4 As an aside

Due to the limitations of older, more primitive compilers, the C language used to require all local variables be defined at the top of the function. The equivalent C++ program using that style would look like this:

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     int x{}, y{}, sum{}; // how are these used?
6 |
7 |     std::cout << "Enter an integer: ";
8 |     std::cin >> x;
9 |
10 |    std::cout << "Enter another integer: ";
11 |    std::cin >> y;
12 |
13 |    sum = x + y;
14 |    std::cout << "The sum is: " << sum << '\n';
15 |
16 |    return 0;
17 | }
```

This style is suboptimal for several reasons:

- The intended use of these variables isn't apparent at the point of definition. You have to scan through the entire function to determine where and how each variable is used.
- The intended initialization value may not be available at the top of the function (e.g. we can't initialize sum to its intended value because we don't know the value of x and y yet).
- There may be many lines between a variable's initializer and its first use. If we don't remember what value it was initialized with, we will have to scroll back to the top of the function, which is distracting.

This restriction was lifted in the C99 language standard.

4.5.3 Introduction to temporary objects

A **temporary object** (also sometimes called an **anonymous object**) is an unnamed object that is created by the compiler to store a value temporarily.

Temporary objects have no scope at all (this makes sense, since scope is a property of an identifier, and temporary objects have no identifier).

Temporary objects are **destroyed at the end of the full expression** in which they are created. In the case where a temporary object is used to initialize a variable, the **initialization happens before the destruction of the temporary**.

Example:

The **caller** receives a **copy of the value** so that it has a value it can use even after input is destroyed.

But where is the value that is copied back to the caller stored? We haven't defined any variables in `main()`. The answer is that the **return value is stored in a temporary object**. This temporary object is then passed to the next function that needs it.

4.6 Why functions are useful, and how to use them effectively

This chapter I'm not going into detail as it's been hinted at before why functions are useful. However here is a quick summary:

- Organization
- Reusability
- Testing
- Extensibility
- Abstraction

New programmers often combine calculating a value and printing the calculated value into a single function. However, **this violates the "one task" rule of thumb for functions**

A function that calculates a value should return the value to the caller and let the caller decide what to do with the calculated value (such as call another function to print the value).

Some others might name this as **separation of concerns**.

4.7 Forward declarations and definitions

Take a look at this seemingly innocent sample program:

```
1  #include <iostream>
2
3  int main()
4  {
5      std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
6      return 0;
7  }
8
9  int add(int x, int y)
10 {
11     return x + y;
12 }
```

Output: add.cpp(5) : error C3861: 'add': identifier not found

The reason this program doesn't compile is because the **compiler compiles** the contents of code files **sequentially**. This means **top to bottom**.

When the compiler reaches the function call to `add` on line 5 of `main`, it doesn't know what `add` is, because we haven't defined `add` until line 9! That produces the error, identifier not found.

This is somewhat misleading, given that `add` wasn't ever defined in the first place. Despite this, it's useful to generally note that it is fairly common for a single error to produce many redundant or related errors or warnings. It can sometimes be hard to tell whether any error or warning beyond the first is a consequence of the first issue, or whether it is an independent issue that needs to be resolved separately.

Best practice: When addressing compilation errors or warnings in your programs, **resolve the first issue** listed and then compile again.

To fix this problem, we need to address the fact that the compiler doesn't know what `add` is.

There are two common ways to address the issue.

- Reorder the function definitions
- **Use a forward declaration**

We might not always have the option available to reorder in a sequential manner.

4.7.1 Forward declaration

A **forward declaration** allows us to tell the compiler about the existence of an identifier before actually defining the identifier.

In the case of functions, this allows us to tell the compiler about the existence of a function before we define the function's body. This way, when the compiler encounters a call to the function, it'll understand that we're making a function call, and can check to ensure we're calling the function correctly, even if it doesn't yet know how or where the function is defined.

To write a forward declaration for a function, we use a **function declaration statement** (also called a **function prototype**). The function declaration consists of the **function's return type, name, and parameter types, terminated with a semicolon**. The names of the parameters can be optionally included. The function body is not included in the declaration.

```
1  #include <iostream>
2
3  int add(int x, int y); // forward declaration of add() (using a function declaration)
4
5  int main()
6  {
7      std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n'; // this works because we forward declared add() above
8      return 0;
9  }
10
11 int add(int x, int y) // even though the body of add() isn't defined until here
12 {
13     return x + y;
14 }
```

It is worth noting that function declarations **do not need to specify the names of the parameters** (as they are not considered to be part of the function declaration).

Beste practice: Keep the parameter names in your function declarations.

4.7.2 *Why forward declarations?*

You may be wondering why we would use a forward declaration if we could just reorder the functions to make our programs work.

Most often, forward declarations are used to tell the compiler about the existence of some **function** that has been **defined in a different code file**. Reordering isn't possible in this scenario because the caller and the callee are in completely different files!

Forward declarations can also be used to define our functions in an **order-agnostic manner**. This allows us to define functions in whatever order **maximizes organization** (e.g. by clustering related functions together) or reader understanding.

Less often, there are times when we have **two functions that call each other**. Reordering isn't possible in this case either, as there is no way to reorder the functions such that each is before the other. Forward declarations give us a way to resolve such circular dependencies.

4.7.3 *Forgetting the function body*

New programmers often wonder **what happens if they forward declare a function but do not define it**.

The answer is: it **depends**. If a forward declaration is made, but the **function is never called, the program will compile and run fine**. However, if a forward declaration is made and **the function is called**, but the program never defines the function, the **program will compile okay**, but the **linker will complain that it can't resolve the function call**.

4.7.4 Identifier declaration

Forward declarations are most often used with functions. However, **forward declarations can also be used with other identifiers** in C++, **such as variables and types**. Variables and types have a different syntax for forward declaration, so we'll cover these in future lessons.

4.7.5 Declarations vs. definitions

In C++, you'll frequently hear the words "**declaration**" and "**definition**" used, and often interchangeably. What do they mean? You now have enough fundamental knowledge to understand the difference between the two.

A **declaration** tells the compiler about the existence of an identifier and its associated type information.

A **definition** is a declaration that actually implements (for functions and types) or instantiates (for variables) the identifier.

In C++, **all definitions are declarations**. Conversely, **not all declarations are definitions**. Declarations that aren't definitions are called **pure declarations**. Types of pure declarations include forward declarations for function, variables, and types.

Term	Technical Meaning	Examples
Declaration	Tells compiler about an identifier and its associated type information.	<code>void foo();</code> // function forward declaration (no body) <code>void goo() {};</code> // function definition (has body) <code>int x;</code> // variable definition
Definition	Implements a function or instantiates a variable. Definitions are also declarations.	<code>void foo() { };</code> // function definition (has body) <code>int x;</code> // variable definition
Pure declaration	A declaration that isn't a definition.	<code>void foo();</code> // function forward declaration (no body)
Initialization	Provides an initial value for a defined object.	<code>int x { 2 };</code> // 2 is the initializer

4.7.6 The one definition rule (ODR)

The one definition rule (or **ODR** for short) is a well-known rule in C++. The ODR has three parts:

- Within a **file**, each function, variable, type, or template **in a given scope** can only have **one definition**. Definitions occurring in different scopes (e.g. local variables defined inside different functions, or functions defined inside different namespaces) do not violate this rule.
- Within a **program**, each function or variable in a given scope can only have one definition. This rule exists because programs can have more than one file (we'll cover this in the next lesson). **Functions and variables not visible to the linker are excluded from this rule** (discussed further in lesson: Internal linkage).
- **Types, templates, inline functions, and inline variables are allowed to have duplicate definitions** in different files, so long as each definition is identical. We haven't covered what most of these things are yet, so don't worry about this for now -- we'll bring it back up when it's relevant.

```
1 | int add(int x, int y)
2 | {
3 |     return x + y;
4 | }
5 |
6 | int add(int x, int y) // violation of ODR, we've already defined function add(int, int)
7 | {
8 |     return x + y;
9 | }
10 |
11 | int main()
12 | {
13 |     int x{};
14 |     int x{ 5 }; // violation of ODR, we've already defined x
15 | }
```

4.8 Programs with multiple code files

As programs get larger, it is common to split them into multiple files for organizational or reusability purposes. One advantage of working with an IDE is that they make working with multiple files much easier. You already know how to create and compile single-file projects. Adding new files to existing projects is very easy.

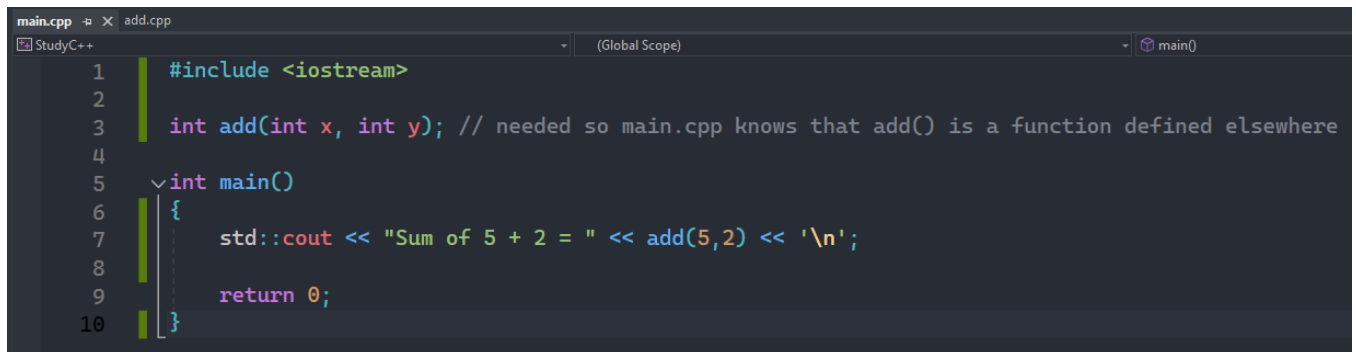
Let's say we **define** the function `add(int x, int y)` in a **separate file** named `add.cpp`. For context we'll use this in the **main function inside of our main.cpp file**.

Your compiler **may compile either** `add.cpp` or `main.cpp` **first**. Either way, `main.cpp` will **fail to compile**. The reason is **it doesn't know what identifier add is**.

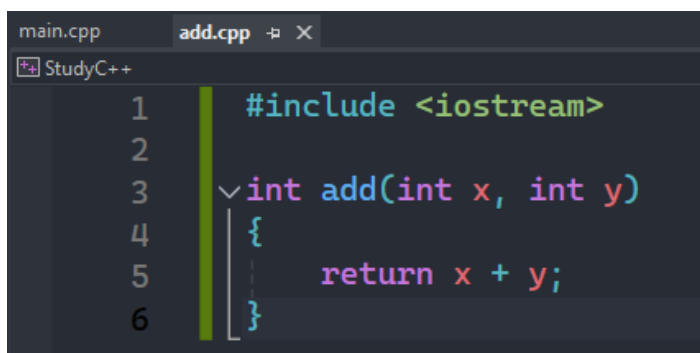
Remember, the compiler **compiles each file individually**. It does not know about the contents of other code files, or remember anything it has seen from previously compiled code files. So even though the compiler may have seen the definition of function `add` previously (if it compiled `add.cpp` first), **it doesn't remember**.

This **limited visibility and short memory is intentional**, for a few reasons:

- It allows the source files of a project to be **compiled in any order**.
- When we change a source file, **only that source file needs to be recompiled**.
- It **reduces the possibility of naming conflicts** between identifiers in different files.



```
main.cpp x add.cpp
StudyC++ (Global Scope) main()
1 #include <iostream>
2
3 int add(int x, int y); // needed so main.cpp knows that add() is a function defined elsewhere
4
5 int main()
6 {
7     std::cout << "Sum of 5 + 2 = " << add(5,2) << '\n';
8
9     return 0;
10 }
```



```
main.cpp add.cpp x
StudyC++
1 #include <iostream>
2
3 int add(int x, int y)
4 {
5     return x + y;
6 }
```

Now, when the compiler is compiling main.cpp, **it will know what identifier add is** and be satisfied. **The linker will connect the function call to add in main.cpp to the definition of function add in add.cpp.**

Using this method, we can give files access to functions that live in another file.

4.9 Naming collisions and an introduction to namespaces

Let's say you are driving to a friend's house for the first time, and the address given to you is 245 Front Street in Mill City. Upon reaching Mill City, you take out your map, only to discover that **Mill City actually has two different Front Streets across town from each other!** Which one would you go to? Unless there were some additional clues to help you decide (e.g. you remember your friend's house is near the river) you'd have to call your friend and ask for more information. Because this would be **confusing and inefficient** (particularly for your mail carrier), in most countries, all street names and house addresses within a city are **required to be unique**.

Similarly, **C++ requires that all identifiers be non-ambiguous**. If two identical identifiers are introduced into the same program in a way that the compiler or linker can't tell them apart, the compiler or linker will produce an error. This error is generally referred to as a **naming collision** (or **naming conflict**).

If the colliding identifiers are introduced into the **same file**, the **result will be a compiler error**. If the colliding identifiers are introduced into **separate files** belonging to the same program, the **result will be a linker error**.

Most naming collisions occur in two cases:

- Two (or more) identically named functions (or global variables) are introduced into **separate files** belonging to the same program. This will result in a **linker error**, as shown above.
- Two (or more) identically named functions (or global variables) are introduced into the **same file**. This will result in a **compiler error**.

As programs get larger and use more identifiers, the odds of a naming collision being introduced increases significantly. The good news is that **C++ provides plenty of mechanisms for avoiding naming collisions**. **Local scope**, which keeps local variables defined inside functions from conflicting with each other, is one such mechanism. **But local scope doesn't work for function names. So how do we keep function names from conflicting with each other?**

4.9.1 *Scope regions*

Back to our address analogy for a moment, having two Front Streets was only problematic because those streets existed within the same city. On the other hand, if you had to deliver mail to two addresses, one at 245 Front Street in Mill City, and another address at 245 Front Street in Jonesville, there would be no confusion about where to go. Put another way, cities **provide groupings that allow us to disambiguate** addresses that might otherwise conflict with each other.

A **scope region** is an **area of source code** where **all declared identifiers are considered distinct from names declared in other scopes** (much like the cities in our analogy). Two identifiers with the same name can be declared in separate scope regions without causing a naming conflict. However, within a given scope region, all identifiers must be unique, otherwise a naming collision will result.

4.9.1.1 **Function body**

The **body of a function is one example of a scope region**. Two identically-named identifiers can be defined in separate functions without issue -- because each function provides a separate scope region, there is no collision. However, if you try to define two identically-named identifiers within the same function, a naming collision will result, and the compiler will complain.

4.9.1.2 **Namespaces**

A **namespace** provides **another type of scope region** (called **namespace scope**) that allows you to declare names inside of it for the purpose of disambiguation. Any names declared inside the namespace won't be mistaken for identical names in other scopes.

A name declared in a scope region (such as a namespace) won't be mistaken for an identical name declared in another scope.

Unlike functions (which are designed to contain **executable statements**), **only declarations and definitions** can **appear in** the scope of a **namespace**. For example, two identically named functions can be defined inside separate namespaces, and no naming collision will occur.

Key insight: Only declarations and definitions can appear in the scope of a namespace (not executable statements). However, a function can be defined inside a namespace, and that function can contain executable statements.

4.9.1.3 The global namespace

In C++, **any name** that is **not defined inside a class, function, or a namespace** is considered to be part of an implicitly-defined namespace called the **global namespace** (sometimes also called the **global scope**).

We discuss the global namespace in more detail later on.

For now, there are **two things** you should know:

- Identifiers declared inside the global scope are **in scope from the point of declaration to the end of the file**.
- Although variables **can be defined in the global namespace, this should generally be avoided**.

4.9.1.4 The std namespace

When C++ was originally designed, all of the identifiers in the C++ standard library (including `std::cin` and `std::cout`) were available to be used without the `std::` prefix (they were part of the global namespace).

However, this meant that any identifier in the standard library could potentially conflict with any name you picked for your own identifiers (also defined in the global namespace). Code that was working might suddenly have a naming conflict when you `#included` a new file from the standard library. Or worse, programs that would compile under one version of C++ might not compile under a future version of C++, as new identifiers introduced into the standard library could have a naming conflict with already written code.

So, **C++ moved all of the functionality in the standard library** into a namespace **named std** (short for “**standard**”).

It turns out that `std::cout`’s name isn’t really `std::cout`. It’s actually just `cout`, and `std` is the **name of the namespace that identifier cout is part of**. Because `cout` is defined in the `std` namespace, the name `cout` won’t conflict with any objects or functions named `cout` that we create outside of the `std` namespace (such as in the global namespace).

When accessing an identifier that is defined in a namespace (e.g. `std::cout`), you need to tell the compiler that we’re looking for an identifier defined inside the namespace (`std`).

4.9.1.4.1 *Explicit namespace qualifier std::*

The most straightforward way to tell the compiler that we want to use `cout` from the `std` namespace is by explicitly using the **`std::` prefix**.

The `::` symbol is an operator called the **scope resolution operator**. The identifier to the **left** of the `::` symbol **identifies the namespace** that the name to the right of the `::` symbol is contained within. If no identifier to the left of the `::` symbol is provided, the global namespace is assumed.

When an identifier includes a namespace prefix, the identifier is called a **qualified name**.

Best practice: Do this. Explicitly use the scope resolution operator to tell the compiler where the identifier is and avoid naming collisions.

4.9.1.4.2 *Using namespace std*

Another way to access identifiers inside a namespace is to use a **using-directive statement**.

A **using directive** allows us to **access the names in a namespace without using a namespace prefix**.

Many texts, tutorials, and even some IDEs recommend or use a using-directive at the top of the program. However, used in this way, **this is a bad practice, and highly discouraged**.

When using a using-directive in this manner, any identifier we define may conflict with any identically named identifier in the `std` namespace. Even worse, while an identifier name may not conflict today, it may conflict with new identifiers added to the `std` namespace in future language revisions. **This was the whole point of moving all of the identifiers in the standard library into the `std` namespace in the first place!**

4.9.2 *Curly braces and indented code*

In C++, curly braces are **often used to delineate a scope region** that is nested within another scope region (braces are also used for some non-scope-related purposes, such as list initialization).

For example, a function defined inside the global scope region uses curly braces to separate the scope region of the function from the global scope.

In certain cases, identifiers defined outside the curly braces may still be part of the scope defined by the curly braces rather than the surrounding scope -- **function parameters** are a good example of this.

4.10 Introduction to the preprocessor

When you compile your project, you might expect that the compiler compiles each code file exactly as you've written it. **This actually isn't the case.**

Instead, prior to compilation, each code (.cpp) file goes through a **preprocessing** phase. In this phase, a program called the **preprocessor** makes various changes to the text of the code file. The preprocessor does not actually modify the original code files in any way -- rather, all changes made by the preprocessor happen either temporarily in-memory or using temporary files.

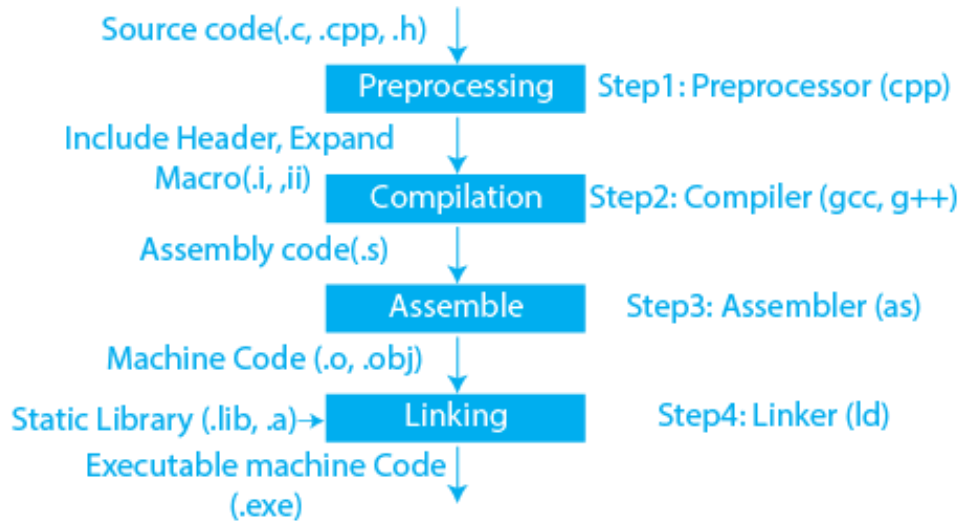
Historically, the preprocessor was a separate program from the compiler, **but in modern compilers, the preprocessor may be built right into the compiler itself.**

Most of what the preprocessor does is fairly uninteresting. For example, it strips out comments, and ensures each code file ends in a newline. However, the preprocessor does have one very important role: it is what processes **#include** directives.

When the preprocessor has finished processing a code file, the result is called a translation unit. This **translation unit** is what is then compiled by the compiler.

The entire process of preprocessing, compiling, and linking is called translation.

4.10.1 Translation process



4.10.2 Preprocessor directives

When the preprocessor runs, it scans through the code file (from top to bottom), looking for preprocessor directives. **Preprocessor directives** (often just called **directives**) are **instructions that start with a # symbol and end with a newline (NOT a semicolon)**. These directives tell the preprocessor to perform certain text manipulation tasks. Note that the preprocessor does not understand C++ syntax -- instead, the directives have their own syntax (which in some cases resembles C++ syntax, and in other cases, not so much).

4.10.2.1 Include

You've already seen the **#include** directive in action (generally to `#include <iostream>`). When you **#include a file, the preprocessor replaces the #include directive with the contents of the included file**. The included contents are then preprocessed (**which may result in additional #includes being preprocessed recursively**), then the rest of the file is preprocessed.

Once the preprocessor has finished processing the code file plus all of the #included content, the result is called a **translation unit**. The translation unit is what is sent to the compiler to be compiled.

4.10.2.2 Macro defines

The **#define directive** can be used to create a **macro**. In C++, **a macro is a rule that defines how input text is converted into replacement output text.**

There are **two basic types** of macros: **object-like macros, and function-like macros.**

Function-like macros act like functions, and serve a similar purpose. Their use is generally **considered unsafe**, and almost anything they can do can be done by a normal function.

Object-like macros can be defined in one of two ways:

- **#define IDENTIFIER**
 - Used for **conditional compilation**
 - **#if**
 - **#ifdef**
 - **#ifndef**
 - **#endif**
- **#define IDENTIFIER substitution_text**

The top definition has no substitution text, whereas the bottom one does. Because **these are** preprocessor **directives (not statements)**, note that **neither form ends with a semicolon.**

The identifier for a macro uses the same naming rules as normal identifiers: they can use letters, numbers, and underscores, cannot start with a number, and should not start with an underscore. By convention, macro names are typically all upper-case, separated by underscores.

4.11 Header files

As programs grow larger (and make use of more files), it becomes increasingly tedious to have to forward declare every function you want to use that is defined in a different file. Wouldn't it be nice if you could **put all your forward declarations in one place and then import them when you need them?**

C++ code files (with a .cpp extension) are not the only files commonly seen in C++ programs. The other type of file is called a **header file**. Header files **usually have a .h extension**, but you will **occasionally see them with a .hpp extension** or no extension at all. The primary purpose of a header file is to propagate declarations to code (.cpp) files.

When you **#include** a file, the **content** of the included file is **inserted at the point of inclusion**. This provides a useful way to pull in declarations from another file.

Note: including definitions in a header file results in a violation of the one-definition rule.

Best practice:

- Always include header guards (we'll cover these next lesson).
- Do not define variables and functions in header files (for now).
- Give a header file the same name as the source file it's associated with (e.g. grades.h is paired with grades.cpp).
- Each header file should have a specific job
- Be mindful of which headers you need to explicitly include for the functionality that you are using in your code files, to avoid inadvertent transitive includes.
- A header file should #include any other headers containing functionality it needs. Such a header should compile successfully when #included into a .cpp file by itself.
- Only #include what you need (don't include everything just because you can).
- Do not #include .cpp files.
- Prefer putting documentation on what something does or how to use it in the header. It's more likely to be seen there. Documentation describing how something works should remain in the source files.

4.11.1 Standard library header files

Consider the following program:

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     std::cout << "Hello, world!";
6 |     return 0;
7 | }
```

This program prints “Hello, world!” to the console using `std::cout`. However, **this program never provided a definition or declaration for `std::cout`**, so how does the compiler know what `std::cout` is?

The answer is that **`std::cout` has been forward declared in the “`iostream`” header file**. When we `#include <iostream>`, **we’re requesting that the preprocessor copy all of the content (including forward declarations for `std::cout`) from the file named “`iostream`” into the file doing the `#include`**.

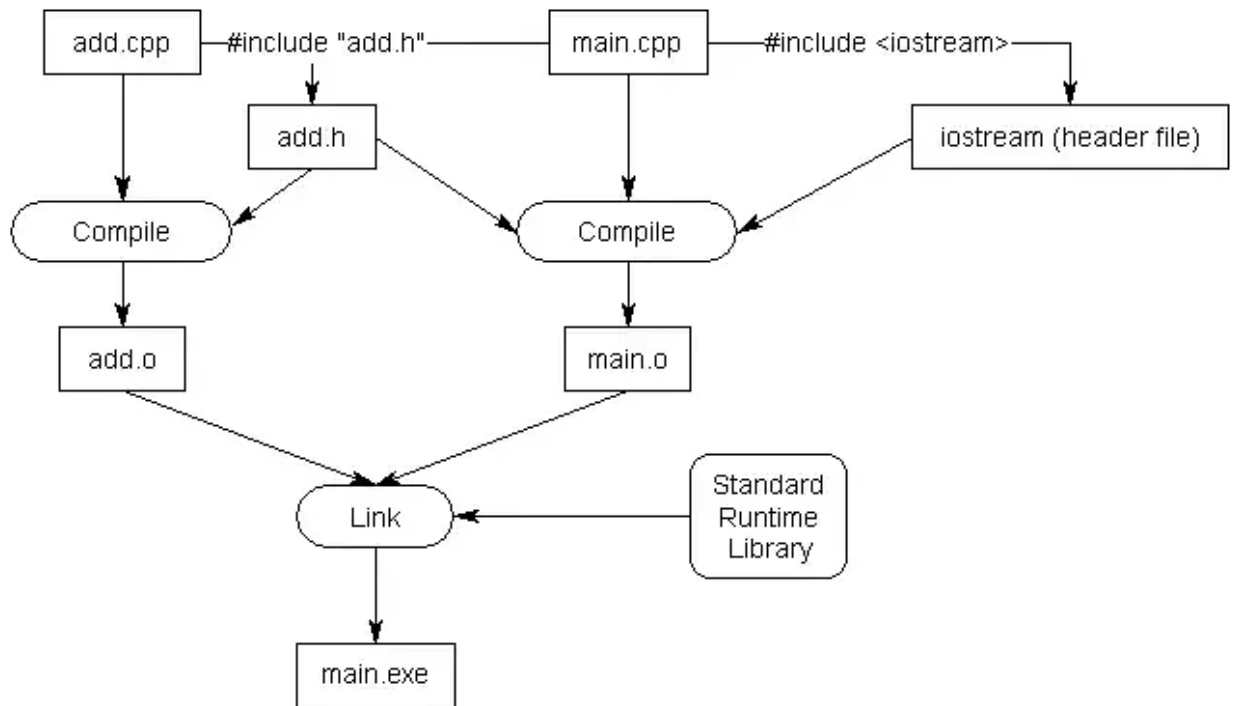
We use angled brackets here to help the preprocessor. This way we tell it that we didn’t write it ourselves and it will start looking in the **include directories**. These are configured according to the **IDE**, they can also be modified / configured according to the needs of a project.

4.11.2 Why doesn’t `iostream` have a `.h` extension?

`iostream.h` is a different header file than `iostream`! **When C++ was first created, all of the headers in the standard library ended in a `.h` suffix**.

When the language was standardized by the ANSI committee, they decided to move all of the names used in the standard library into the **`std` namespace to help avoid naming conflicts** with user-declared identifiers. However, this presented a problem: if they moved all the names into the `std` namespace, none of the old programs (that included `iostream.h`) would work anymore!

To work around this issue, C++ introduced new header files that lack the `.h` extension. These new header files declare all names inside the `std` namespace. This way, older programs that include `#include <iostream.h>` do not need to be rewritten, and newer programs can `#include <iostream>`.



4.11.3 *#include order*

To maximize the chance that missing includes will be flagged by compiler, order your `#includes` as follows:

1. The paired header file
2. Other headers from your project
3. 3rd party library headers
4. Standard library headers

The headers for each grouping should be sorted alphabetically (unless the documentation for a 3rd party library instructs you to do otherwise).

4.11.4 Including header files from other directories

Another common question involves how to include header files from other directories.

One (**bad**) way to do this is to include a **relative path** to the header file you want to include as part of the `#include` line.

For example:

- `#include "headers/myHeader.h"`
- `#include "../moreHeaders/myOtherHeader.h"`

While this will compile (assuming the files exist in those relative directories), the downside of this approach is that **it requires you to reflect your directory structure in your code**. If you ever update your directory structure, your code won't work anymore.

A better method is to tell your compiler or IDE that you have a bunch of header files in some other location, so that it will look there when it can't find them in the current directory. This can generally be done by setting an include path or search directory in your IDE project settings.

The nice thing about this approach is that if you ever change your directory structure, you only have to change a single compiler or IDE setting instead of every code file.

4.11.5 Headers may include other headers

It's common that a header file will need a declaration or definition that lives in a different header file. Because of this, **header files will often #include other header files**.

When your code file #includes the first header file, **you'll also get any other header files that the first header file includes** (and any header files those include, and so on). These additional header files are sometimes **called transitive includes**, as they're **included implicitly** rather than explicitly.

The content of these transitive includes are available for use in your code file. However, **you generally should not rely on the content of headers that are included transitively** (unless reference documentation indicates that those **transitive includes** are required). The implementation of header files may change over time, or be different across different systems. If that happens, your code may only compile on certain systems, or may compile now but not in the future. This is easily avoided by explicitly including all of the header files the content of your code file requires.

4.12 Header guards

4.12.1 *The duplicate definition problem*

In Forward declarations and definitions, we noted that a variable or function identifier can only have one definition (the one definition rule).

Thus, a program that defines a variable identifier more than once will cause a compile error. Similarly, programs that define a function more than once will also cause a compile error.

While these programs are easy to fix (remove the duplicate definition), with header files, it's quite easy to end up in a situation where a definition in a header file gets included more than once. This can happen when a header file `#includes` another header file (which is common).

square.h:

```
1 | int getSquareSides()
2 | {
3 |     return 4;
4 | }
```

wave.h:

```
1 | #include "square.h"
```

main.cpp:

```
1 | #include "square.h"
2 | #include "wave.h"
3 |
4 | int main()
5 | {
6 |     return 0;
7 | }
```

This seemingly innocent looking program won't compile! Here's what's happening. First, main.cpp #includes square.h, which copies the definition for function getSquareSides into main.cpp. Then main.cpp #includes wave.h, which #includes square.h itself. This copies contents of square.h (including the definition for function getSquareSides) into wave.h, which then gets copied into main.cpp.

The good news is that we can avoid the above problem via a mechanism called a **header guard** (also called an **include guard**). Header guards are conditional compilation directives that take the following form:

```
1 | #ifndef SOME_UNIQUE_NAME_HERE
2 | #define SOME_UNIQUE_NAME_HERE
3 |
4 | // your declarations (and certain types of definitions) here
5 |
6 | #endif
```

When this header is #included, the preprocessor will check whether SOME_UNIQUE_NAME_HERE has been previously defined in this translation unit. **If this is the first time we're including the header, SOME_UNIQUE_NAME_HERE will not have been defined. Consequently, it #defines SOME_UNIQUE_NAME_HERE** and includes the contents of the file. If the header is included again into the same file, SOME_UNIQUE_NAME_HERE will already have been defined from the first time the contents of the header were included, and the contents of the header will be ignored (thanks to the #ifndef).

All of your header files should have header guards on them. SOME_UNIQUE_NAME_HERE can be any name you want, but **by convention is set to the full filename** of the header file, typed in all caps, using underscores for spaces or punctuation. For example, square.h would have the header guard:

```
1 | #ifndef SQUARE_H
2 | #define SQUARE_H
3 |
4 | int getSquareSides()
5 | {
6 |     return 4;
7 | }
8 |
9 | #endif
```

Even the standard library headers use header guards.

SUMMARY:

- Header guards **prevent a header file from being included multiple times within the same translation unit** (a single .cpp file and all the headers it includes).
- They **do not prevent the same header file from being included once in multiple translation units** (different .cpp files).
- **Duplicate declarations** are fine -- but even if your header file is composed of all declarations (no definitions) it's still a best practice to include header guards.

BEST PRACTICE:

This is the reason why we **shouldn't put functions in a header file**.

4.12.2 *Can't we just avoid definitions in header files?*

We've generally told you not to include function definitions in your headers. So, you may be wondering why you should include header guards if they protect you from something you shouldn't do.

There are quite a few cases we'll show you in the future where it's necessary to put non-function definitions in a header file. For example, C++ will let you create your own types. These **custom types are typically defined in header files**, so the type definitions can be propagated out to the code files that need to use them. Without a header guard, a code file could end up with multiple (identical) copies of a given type definition, which the compiler will flag as an error.

So even though it's not strictly necessary to have header guards at this point in the tutorial series, **we're establishing good habits now, so you don't have to unlearn bad habits** later.

4.12.3 *#pragma once*

Modern compilers support a simpler, alternate form of header guards using the `#pragma` preprocessor directive.

#pragma once serves the same purpose as header guards: to avoid a header file from being included multiple times. With traditional header guards, the developer is responsible for guarding the header (by using preprocessor directives `#ifndef`, `#define`, and `#endif`). **With #pragma once, we're requesting that the compiler guard the header.** How exactly it does this is an implementation-specific detail.

For most projects, #pragma once works fine, and many developers now prefer it because it is easier and less error-prone. Many IDEs will also auto-include `#pragma once` at the top of a new header file generated through the IDE.

Because #pragma once is not defined by the C++ standard, it is possible that some compilers may not implement it. For this reason, **some development houses (such as Google) recommend using traditional header guards.** In this tutorial series, **we will favor header guards,** as **they are the most conventional way to guard headers.** However, support for `#pragma once` is fairly ubiquitous at this point, and **if you wish to use #pragma once instead, that is generally accepted in modern C++.**

4.13 How to design your first programs

4.13.1 Steps

1. Define your goal
 - In order to write a successful program, you first need to define what your goal is. Ideally, you should be able to state this in a sentence or two. It is often useful to express this as a user-facing outcome.
2. Define the requirements
 - While defining your problem helps you determine what outcome you want, it's still vague. The next step is to think about requirements.
 - Requirements is a fancy word for both the constraints that your solution needs to abide by (e.g. budget, timeline, space, memory, etc....), as well as the capabilities that the program must exhibit in order to meet the users' needs. Note that your requirements should similarly be focused on the "what", not the "how".
3. Define your tools, targets and backup plan
 - System architecture
 - Backup plans for your project
 - What parts of the standard library
 - Solo or co-op coding?
 - Define a testing / feedback / release strategy
4. Break hard problems down into easy problems
5. Figure out the sequence of events

4.13.2 Advice

- Keep your programs **simple to start**.
- **Add** features **over time**.
- **Focus on one area** at a time.
- **Test each piece of code** as you go.
- **Don't** invest in **perfecting early code**.
- **Optimize for maintainability**, not performance.

4.14 Chapter 2 summary

A **function** is a reusable sequence of statements designed to do a particular job. Functions you write yourself are called **user-defined** functions.

A **function call** is an expression that tells the CPU to execute a function. The function initiating the function call is the **caller**, and the function being called is the **callee** or **called** function. Do not forget to include parenthesis when making a function call.

The curly braces and statements in a function definition are called the **function body**.

A function that returns a value is called a **value-returning function**. The **return type** of a function indicates the type of value that the function will return. The **return statement** determines the specific **return value** that is returned to the caller. A return value is copied from the function back to the caller -- this process is called **return by value**. Failure to return a value from a non-void function will result in undefined behavior.

The return value from function main is called a **status code**, and it tells the operating system (and any other programs that called yours) whether your program executed successfully or not. By consensus a return value of 0 means success, and a non-zero return value means failure.

Practice **DRY** programming -- “don’t repeat yourself”. Make use of variables and functions to remove redundant code.

Functions with a return type of **void** do not return a value to the caller. A function that does not return a value is called a void function or non-value returning function. Void functions can’t be called where a value is required.

A return statement that is not the last statement in a function is called an **early return**. Such a statement causes the function to return to the caller immediately.

A **function parameter** is a variable used in a function where the value is provided by the caller of the function. An **argument** is the specific value passed from the caller to the function. When an argument is copied into the parameter, this is called **pass by value**.

Function parameters and variables defined inside the function body are called **local variables**. The time in which a variable exists is called its **lifetime**. Variables are created and destroyed at **runtime**, which is when the program is running. A variable’s **scope** determines where it can be seen and used. When a variable can be seen and used, we say it is **in scope**. When it cannot be seen, it cannot be used, and we say it is **out of scope**. **Scope is a compile-time property**, meaning it is enforced at compile time.

Whitespace refers to characters used for formatting purposes. In C++, this includes spaces, tabs, and newlines.

A **forward declaration** allows us to tell the compiler about the existence of an identifier before actually defining the identifier. To write a forward declaration for a function, we use a **function prototype**, which includes the function's return type, name, and parameters, but no function body, followed by a semicolon.

A **definition** actually implements (for functions and types) or instantiates (for variables) an identifier. A **declaration** is a statement that tells the compiler about the existence of the identifier. In C++, all definitions serve as declarations. **Pure declarations** are declarations that are not also definitions (such as function prototypes).

Most non-trivial programs contain multiple files.

When two identifiers are introduced into the same program in a way that the compiler or linker can't tell them apart, the compiler or linker will error due to a **naming collision**. A **namespace** guarantees that all identifiers within the namespace are unique. The std namespace is one such namespace.

The **preprocessor** is a process that runs on the code before it is compiled. **Directives** are special instructions to the preprocessor. Directives start with a # symbol and end with a newline. A **macro** is a rule that defines how input text is converted to a replacement output text.

Header files are files designed to propagate declarations to code files. When using the #include directive, the #include directive is replaced by the contents of the included file. When including headers, use angled brackets when including system headers (e.g. those in the C++ standard library), and use double quotes when including user-defined headers (the ones you write). When including system headers, include the versions with no .h extension if they exist.

Header guards prevent the contents of a header from being included more than once into a given code file. They do not prevent the contents of a header from being included into multiple different code files.

5 Debugging

5.1 Syntax and semantic errors

A **syntax error** occurs when you write a statement that is **not valid according to the grammar** of the C++ language.

A **semantic error** occurs when a statement is **syntactically valid**, but **does not do what the programmer intended**.

5.2 Process

- Find the root cause of the problem
 - Find the line of code that's not working correctly
- Understand the problem
 - Why does it occur?
 - Should it even be fixed or is the behavior correct?
- Determine a fix
- Write the fix.
- Retest to ensure the problem has been fixed.
- Retest to ensure no new problems have emerged.

5.3 Strategy

1. Figure out how to reproduce the problem
2. Run the program and gather information to narrow down where the problem is
3. Repeat the prior step until you find the problem

5.3.1 Tactics

1. Commenting out the code
2. Validate the code flow
3. Print out values
4. Use integrated debug mode
 - a. Breakpoints
 - b. Stepping
 - c. Watching variables
 - d. Watch the call stack

6 Fundamental data types

6.1 Introduction to fundamental data types

6.1.1 *Bits, bytes, and memory addressing*

Computers have random access memory (RAM) that is available for programs to use. When a variable is defined, **a piece of that memory is set aside** for that variable.

The smallest unit of memory is a **binary digit** (also called a **bit**), which can hold a value of 0 or 1. You can think of a bit as being like a traditional light switch -- either the light is off (0), or it is on (1). There is no in-between. If you were to look at a random segment of memory, all you would see is ...011010100101010... or some combination thereof.

Memory is organized into sequential units called **memory addresses** (or **addresses** for short). Similar to how a street address can be used to find a given house on a street, the memory address allows us to find and access the contents of memory at a particular location.

Perhaps surprisingly, in modern computer architectures, each bit does not get its own unique memory address. This is because the number of memory addresses is limited, and the need to access data bit-by-bit is rare. Instead, each memory address holds 1 byte of data. A byte is a group of bits that are operated on as a unit. The modern standard is that a **byte** is comprised of **8 sequential bits**.

6.1.2 Data types

Because all data on a computer is just a sequence of bits, we use a **data type** (often called a “**type**” for short) to tell the compiler how to interpret the contents of memory in some meaningful way. You have already seen one example of a data type: the integer. When we declare a variable as an integer, we are telling the compiler “The piece of memory that this variable uses is going to be interpreted as an integer value”.

When you give an object a value, **the compiler and CPU** take care of **encoding** your value **into** the appropriate **sequence of bits** for that data type, **which are then stored in memory** (remember: memory can only store bits). For example, if you assign an integer object the value 65, that value is converted to the sequence of bits 0100 0001 and stored in the memory assigned to the object.

Conversely, when the object is evaluated to produce a value, that sequence of bits is reconstituted back into the original value. Meaning that 0100 0001 is converted back into the value 65.

Fortunately, the compiler and CPU do all the hard work here, so **you generally don’t need to worry about how values get converted into bit sequences and back.**

6.1.3 Fundamental data types

C++ comes with built-in support for many different data types. These are called **fundamental data types**, but are often informally called **basic types**, **primitive types**, or **built-in types**.

Types	Category	Meaning	Example
float double long double	Floating Point	a number with a fractional part	3.14159
bool	Integral (Boolean)	true or false	true
char wchar_t char8_t (C++20) char16_t (C++11) char32_t (C++11)	Integral (Character)	a single character of text	'c'
short int int long int long long int (C++11)	Integral (Integer)	positive and negative whole numbers, including 0	64
std::nullptr_t (C++11)	Null Pointer	a null pointer	nullptr
void	Void	no type	n/a

6.1.4 The _t suffix

Many of the types defined in newer versions of C++ (e.g. `std::nullptr_t`) use a `_t` suffix. **This suffix means “type”, and it’s a common nomenclature applied to modern types.**

If you see something with a `_t` suffix, it’s probably a type. But many types don’t have a `_t` suffix, so this isn’t consistently applied.

6.2 Void

Void is the easiest of the data types to explain. Basically, void means “**no type**”!

Void is our first example of an **incomplete type**.

An incomplete type is a type that has been declared but not yet defined. The compiler knows about the existence of such types, but does not have enough information to determine how much memory to allocate for objects of that type. void is intentionally incomplete since it represents the lack of a type, and thus cannot be defined.

Void is a bit of a special type and it's only used in certain contexts.

For example: a void type variable doesn't work. This is because it is an incomplete type and they **cannot be instantiated**.

6.2.1 Void contexts

- Most commonly, void is used to indicate that a **function does not return a value**.
 - Note: a **value-returning return statement will cause a compiler error**.
- Deprecated functions that don't take parameters (**C-specific**)
 - We put **void as a parameter** without an identifier.
 - Note: Although this will compile in C++ (for backwards compatibility reasons), **this use of keyword void is considered deprecated in C++**.
 - **Best practice:** Empty parameter list instead of void. (**implies an implicit void**)
- Void pointers
 - Will be discussed later on since it's an advanced use of the keyword.

6.3 Object sizes and the sizeof operator

6.3.1 Object sizes

To recap, memory on modern machines is typically organized into **byte sized units**, with **each byte** of memory having a **unique address**.

Up to this point, it has been useful to think of memory as a bunch of cubbyholes or mailboxes where we can put and retrieve information, and variables as names for accessing those cubbyholes or mailboxes. However, **this analogy is not quite correct in one regard**.

Most objects actually take up more than 1 byte of memory. A single object may use 1, 2, 4, 8, or even more consecutive memory addresses. The amount of memory that an object uses is based on its data type.

Because we typically access memory through variable names (and not directly via memory addresses), the compiler is able to hide the details of how many bytes a given object uses from us. When we access some variable *x*, the compiler knows how many bytes of data to retrieve (based on the type of variable *x*), and can handle that task for us. **Even so, there are several reasons it is useful to know how much memory an object uses.**

First, **the more memory an object uses, the more information it can hold.**

A single bit can hold **2 possible values**, a 0, or a 1

2 bits can hold **4 possible values**

3 bits can hold **8 possible values**

To generalize, an object with *n* bits (where ***n* is an amount**) can hold 2^n (2 to the power of *n*, also commonly written **2^n**) unique values.

Example 1:

2-byte object (2 bytes = 8 bits)

Formula: $2^n = 2^8 = 256$ possible values, range 0 - 255

Example 2:

4-byte integer (4 bytes = 32 bits)

Formula: $2^n = 2^{32} = 4294967296$ possible values, range 0 - 4294967295

Thus, the size of the object puts a limit on the amount of unique values it can store.

Second, **computers have a finite amount of free memory**. Every time we define an object, a small portion of that free memory is used for as long as the object is in existence. Because modern computers have a lot of memory, this impact is usually negligible. **However, for programs that need a large amount of objects or data (e.g. a game that is rendering millions of polygons), the difference between using 1-byte and 8-byte objects can be significant.**

6.3.2 Fundamental data type sizes

The obvious next question is “how much memory do variables of different data types take?”.

Surprisingly the **C++ standard does not define the exact size in bits**. However, it **does define a minimum size** (in bits).

In this tutorial series, we will take a simplified view, by making some reasonable assumptions that are generally true for modern architectures:

- **A byte is 8 bits.**
- Memory is byte addressable, so the **smallest object is 1 byte**
- Floating point support is IEEE-754 compliant.
- We are on a **32-bit or 64-bit architecture**.

Category	Type	Minimum Size	Typical Size	Note
Boolean	bool	1 byte	1 byte	
character	char	1 byte	1 byte	always exactly 1 byte
	wchar_t	1 byte	2 or 4 bytes	
	char8_t	1 byte	1 byte	
	char16_t	2 bytes	2 bytes	
	char32_t	4 bytes	4 bytes	
integer	short	2 bytes	2 bytes	
	int	2 bytes	4 bytes	
	long	4 bytes	4 or 8 bytes	
	long long	8 bytes	8 bytes	
floating point	float	4 bytes	4 bytes	
	double	8 bytes	8 bytes	
	long double	8 bytes	8, 12, or 16 bytes	
pointer	std::nullptr_t	4 bytes	4 or 8 bytes	

Tip:

For maximum portability, you shouldn't assume that variables are larger than the specified minimum size.

Alternatively, if you want to assume that a type has a certain size (e.g. that an int is at least 4 bytes), you can use `static_assert` to have the compiler fail a build if it is compiled on an architecture where this assumption is not true. We cover how to do this in the `Assert` and `static_assert` lesson.

6.3.3 *The sizeof operator*

In order to determine the size of data types on a particular machine, C++ provides an operator named `sizeof`. The **`sizeof`** operator is a unary operator that takes either a type or a variable, and returns its size in bytes.

You can also use the `sizeof` operator on a **variable name**.

`sizeof` **does not include dynamically allocated memory** used by an object. We discuss dynamic memory allocation in a future lesson.

6.4 Signed integers

An integer is an integral type that can represent positive and negative whole numbers, including 0. C++ has **4 primary fundamental integer types** available for use:

- **Short int** **16 bits**
- **Int** **16 – 32 bits** (Typically, 32 bits on modern architectures)
- **Long int** **32 bits**
- **Long long int** **64 bits**

The key difference between the various integer types is that **they have varying sizes**.

Technically, the **bool** and **char** types **are considered to be integral types** (because **these types store their values as integer values**). For the purpose of the next few lessons, we'll exclude these types from our discussion.

When writing negative numbers in everyday life, we use a negative sign. For example, -3 means "negative 3". We'd also typically recognize +3 as "positive 3" (though common convention dictates that we typically omit plus prefixes).

This attribute of being positive, negative, or zero is called the number's sign.

By default, integers in C++ are **signed**, which means the number's **sign is stored as part of the number**. Therefore, a signed integer can hold both positive and negative numbers (and 0).

In binary representation, a single bit (called the **sign bit**) is used to store the sign of the number. The non-sign bits (called the **magnitude bits**) determine the magnitude of the number.

Best practice for defining:

Prefer the **shorthand types** that do not use the int suffix or signed prefix.

6.4.1 Signed integer ranges

As you learned in the last section, a variable with n bits can hold 2^n possible values. But which specific values? We call the set of specific values that a data type can hold its **range**. The range of an integer variable is determined by **two factors**: its size (in bits), and whether it is signed or not.

By definition, an 8-bit signed integer has a range of -128 to 127. This means a signed integer can store any integer value between -128 and 127 **(inclusive) safely**.

Here's a table containing the range of signed integers of different sizes:

Size/Type	Range
8 bit signed	-128 to 127
16 bit signed	-32,768 to 32,767
32 bit signed	-2,147,483,648 to 2,147,483,647
64 bit signed	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

For the math inclined, an n-bit signed variable has a range of $-(2^{n-1})$ to 2^{n-1} .

6.4.2 *Overflow*

What happens if we try to assign a value that is higher than the range of the fundamental data type?

The C++20 standard makes this blanket statement: “If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the **behavior is undefined**”. Colloquially, this is called **overflow**.

If an arithmetic operation (such as addition or multiplication) attempts to create a value outside the range that can be represented, this is called **integer overflow** (or **arithmetic overflow**). For signed integers, integer overflow will result in undefined behavior.

In general, **overflow results in information being lost**, which is almost never desirable. If there is any suspicion that an object might need to store a value that falls outside its range, use a type with a bigger range!

6.4.3 *Integer division*

When doing division with two integers (called **integer division**), C++ always produces an integer result. Since integers can't hold fractional values, any fractional portion is simply dropped (not rounded!).

Warning: Be careful when using integer division, as **you will lose any fractional parts of the quotient**. However, if it's what you want, integer division is safe to use, as the results are predictable.

6.4.4 Two's compliment

Two's compliment representation is a way of representing positive and negative numbers in binary.

Let's take 001000 for example: first we calculate the indexes separately because they make part of the whole number. In binary if any number at a certain position is set to 1 then we add them to know what the whole byte evaluates to.

6.4.4.1 Positive two's compliment

$$\begin{array}{ccccccc} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow \\ -2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 & 2^0 \end{array}$$

In this case 2^3 is 8. Also note that the 0 with the most value is signed (negative)

6.4.5 Negative two's compliment

The same rules as before apply except this time the signed bit is set to 1 and for this value it counts -2^5 (-32). This is important because when we add all of the positive magnitude bits we then also subtract the signed bit.

$$\begin{array}{l} 101100 \\ = -2^5 + 2^3 + 2^2 \\ = -32 + 8 + 4 \\ = -20 \end{array}$$

6.5 Unsigned integers, and why to avoid them

We covered signed integers, which are a set of types that can hold positive and negative whole numbers, including 0.

C++ also supports unsigned integers. Unsigned integers are integers that can only hold non-negative whole numbers.

To define an unsigned integer, we **use the unsigned keyword**. By convention, this is placed before the type.

A 1-byte unsigned integer has a range of 0 to 255. Compare this to the 1-byte signed integer range of -128 to 127. **Both can store 256 different values, but signed integers use half of their range for negative numbers, whereas unsigned integers can store positive numbers that are twice as large.**

When no negative numbers are required, unsigned integers are well-suited for networking and systems with little memory, because unsigned integers can store more positive numbers without taking up extra memory.

6.5.1 Overflow

Important note: Oddly, the **C++ standard explicitly says “a computation involving unsigned operands can never overflow”**. This is contrary to general programming consensus that integer overflow encompasses both signed and unsigned use cases. **Given that most programmers would consider this overflow, we’ll call this overflow despite the C++ standard’s statements to the contrary.**

If an unsigned value is out of range, it is divided by one greater than the largest number of the type, and only the **remainder kept**.

6.5.2 Controversy over unsigned numbers

Many developers (and some large development houses, such as Google) believe that developers should generally avoid unsigned integers. This is largely **because of two behaviors** that can cause problems.

First, with signed values, it takes a little work to accidentally overflow the top or bottom of the range because those values are far from 0. **With unsigned numbers, it is much easier to overflow the bottom of the range, because the bottom of the range is 0, which is close to where the majority of our values are.** Another common **unwanted wrap-around happens when an unsigned integer is repeatedly decremented by 1**, until it tries to decrement to a negative number. You'll see an example of this when loops are introduced.

Second, and more insidiously, **unexpected behavior can result when you mix signed and unsigned integers.** In C++, if a mathematical operation (e.g. arithmetic or comparison) has one signed integer and one unsigned integer, the **signed integer will usually be converted to an unsigned integer. And the result will thus be unsigned.**

Best practice: Favor signed numbers over unsigned numbers for holding quantities (even quantities that should be non-negative) and mathematical operations. Avoid mixing signed and unsigned numbers.

6.5.3 So, when should you use unsigned numbers?

There are still a few cases in C++ where it's okay / necessary to use unsigned numbers.

First, unsigned numbers are preferred when dealing with **bit manipulation** (covered in chapter 0 -- that's a capital 'o', not a '0'). They are also useful when well-defined wrap-around behavior is required (**useful in some algorithms like encryption and random number generation**).

Second, use of unsigned numbers is still unavoidable in some cases, mainly those having to do with **array indexing**. We'll talk more about this in the lessons on arrays and array indexing.

Also note that if you're developing for an embedded system (e.g. an Arduino) or some other **processor/memory limited context**, use of unsigned numbers is more common and accepted (and in some cases, unavoidable) for performance reasons.

6.6 Fixed-width integers and `size_t`

we covered that C++ only guarantees that integer variables will have a minimum size -- but they could be larger, depending on the target system.

6.6.1 *Why isn't the size of the integer variables fixed?*

The short answer is that this goes back to the early days of C, when computers were slow and performance was of the utmost concern. C opted to intentionally leave the size of an integer open so that the compiler implementers could pick a size for `int` that performs best on the target computer architecture.

6.6.2 *Fixed-width integers*

To address the above issues, **C99 defined a set of fixed-width integers** (in the **`stdint.h`** header) that are guaranteed to be the same size on any architecture.

<code>std::int8_t</code>	1 byte signed	-128 to 127	Treated like a signed char on many systems.
<code>std::uint8_t</code>	1 byte unsigned	0 to 255	Treated like an unsigned char on many systems.
<code>std::int16_t</code>	2 byte signed	-32,768 to 32,767	
<code>std::uint16_t</code>	2 byte unsigned	0 to 65,535	
<code>std::int32_t</code>	4 byte signed	-2,147,483,648 to 2,147,483,647	
<code>std::uint32_t</code>	4 byte unsigned	0 to 4,294,967,295	
<code>std::int64_t</code>	8 byte signed	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	
<code>std::uint64_t</code>	8 byte unsigned	0 to 18,446,744,073,709,551,615	

6.6.3 *Fixed-width integers downsides*

The fixed-width integers have **two downsides** that are typically raised.

First, the fixed-width integers are **not guaranteed to be defined on all architectures**. They **only exist on systems where there are fundamental types matching their widths** and following a certain binary representation. Your program will fail to compile on any such architecture that does not support a fixed-width integer that your program is using. However, given that most modern architectures have standardized around 8/16/32/64-bit variables, this is unlikely to be a problem unless your program needs to be portable to some exotic mainframe or embedded architectures.

Second, if you use a fixed-width integer, **it may be slower than a wider type on some architectures**. For example, if you need an integer that is guaranteed to be 32-bits, you might decide to use `std::int32_t`, but your CPU might actually be faster at processing 64-bit integers. However, just because your CPU can process a given type faster doesn't mean your program will be faster overall -- **modern programs are often constrained by memory usage rather than CPU, and the larger memory footprint may slow your program more than the faster CPU processing accelerates it. It's hard to know without actually measuring.**

6.6.4 *std::int8_t and std::uint8_t likely behave like chars instead of integers*

Due to an oversight in the C++ specification, most compilers define and treat `std::int8_t` and `std::uint8_t` (and the corresponding fast and least fixed-width types) identically to types signed char and unsigned char respectively. This means these 8-bit types may (or may not) behave differently than the rest of the fixed-width types, which can lead to errors. **This behavior is system-dependent, so a program that behaves correctly on one architecture may not compile or behave correctly on another architecture.**

6.6.5 Fast and least integers

To help address the above downsides, C++ also defines two alternative sets of integers that are **guaranteed to be defined**.

The **fast types** (`std::int_fast#_t` and `std::uint_fast#_t`) provide the fastest signed/unsigned integer type with a width of at least # bits (**where # = 8, 16, 32, or 64**). **For example**, `std::int_fast32_t` will give you the fastest signed integer type that's at least 32 bits. **By fastest, we mean the integral type that can be processed most quickly by the CPU.**

The **least types** (`std::int_least#_t` and `std::uint_least#_t`) provide the smallest signed/unsigned integer type with a width of at least # bits (**where # = 8, 16, 32, or 64**). **For example**, `std::uint_least32_t` will give you the **smallest** unsigned integer type that's at least 32 bits.

However, these **fast and least integers have their own downsides**: First, not many programmers actually use them, and a **lack of familiarity can lead to errors**. Second, the **fast types can lead to memory wastage, as their actual size may be larger than indicated by their name**. More seriously, because the **size of the fast/least integers can vary**, it's possible that your program may exhibit different behaviors on architectures where they resolve to different sizes.

Summary:

- **Fixed-width integers** aimed to standardize integer sizes across platforms but **aren't universally supported** and can be **suboptimal in performance on some architectures**.
- **Fast and least integers** were introduced to provide more flexible options that adapt to the target architecture's strengths but come with their own trade-offs, like potential **memory inefficiency (fast) and varying sizes (least)**.
- The **8-bit fixed-width, fast and least integer types** are **often treated like chars instead of integer values** (and this may vary per system). Prefer the 16-bit fixed integral types for most cases.

6.6.6 Best practices

- Prefer **int** when the size of the integer doesn't matter (e.g. the number will always fit within the range of a 2-byte signed integer) and the variable is short-lived (e.g. destroyed at the end of the function). For example, if you're asking the user to enter their age, or counting from 1 to 10, it doesn't matter whether int is 16 or 32 bits (the numbers will fit either way). **This will cover the vast majority of the cases you're likely to run across.**
- Prefer **std::int#_t** when storing a quantity that needs a guaranteed range.
- Prefer **std::uint#_t** when doing bit manipulation or where well-defined wrap-around behavior is required.

Avoid the following when possible:

- short and long integers, **use a fixed-width type** instead.
- **Unsigned types for holding quantities.**
- The **8-bit fixed-width integer types.**
- The fast and least fixed-width types.
- Any compiler-specific fixed-width integers
 - For example, Visual Studio defines `__int8`, `__int16`, etc...

6.6.7 Std::size_t

The **sizeof** operator **returns** a value of type **std::size_t**. `std::size_t` is an alias for an **implementation-defined unsigned integral type**.

In other words, **the compiler decides** if `std::size_t` is an unsigned int, an unsigned long, an unsigned long long, etc...

For advanced readers: `std::size_t` is actually a typedef.

Best practice: If you use `std::size_t` explicitly in your code, **#include** one of the **headers that defines `std::size_t`** (we recommend `<cstdlib>`).

Using sizeof does not require a header (even though it return a value whose type is `std::size_t`).

6.7 Introduction to scientific notation

Scientific notation is a useful shorthand for writing lengthy numbers in a concise manner. And although scientific notation may seem foreign at first, understanding scientific notation will help you understand how floating-point numbers work, and more importantly, what their limitations are.

Numbers in scientific notation take the following form: ***significand* $\times 10^{\text{exponent}}$**

By convention, numbers in scientific notation are written with one digit before the decimal point, and the rest of the digits afterward.

Consider the mass of the Earth. In decimal notation, we'd write this as **59722000000000000000000 kg**. That's a really large number (too big to fit even in an 8-byte integer). It's also hard to read (is that 19 or 20 zeros?). Even with separators (5,972,200,000,000,000,000,000) the number is still hard to read.

In scientific notation, this would be written as **5.9722×10^{24} kg**, which is **much easier to read**. Scientific notation has the added benefit of making it easier to compare the magnitude of two extremely large or small numbers simply by comparing the exponent.

Because it can be hard to type or display exponents in C++, we use the letter 'e' (or sometimes 'E') to represent the "**times 10 to the power of**" part of the equation. For example, **1.2×10^4 would be written as 1.2e4**, and 5.9722×10^{24} would be written as 5.9722e24.

6.7.1 Significant digits

Let's say you need to know the value of the mathematical constant **pi** for some equation, but you've forgotten. You ask two people. One tells you the value of pi is **3.14**. The other tells you the value of pi is **3.14159**. Both of these values are "correct", but **the latter is far more precise**.

Here's the most important thing to understand about scientific notation: The digits in the significand (the part before the 'e') are called the **significant digits** (or **significant figures**). The more significant digits, the more precise a number is.

In scientific notation, we'd write 3.14 as 3.14e0. Since there are **3 numbers** in the significand, this number **has 3 significant digits**. 3.14159 would be written as 3.14159e0. Since there are 6 numbers in the significand, this number has 6 significant digits.

6.7.2 How to convert decimal numbers to scientific notation

Use the following procedure:

- Your **exponent starts at zero**.
- **Slide** the decimal **left** or right **so there is only one non-zero digit** to the left of the decimal.
 - Each place you **slide** the decimal to the **left increases the exponent by 1**.
 - Each place you **slide** the decimal to the **right decreases the exponent by 1**.
- **Trim** off any leading **zeros (on the left end of the significand)**
- **Trim** off any trailing **zeros (on the right end of the significand)** only **if the original number had no decimal point**. We're assuming they're not significant. If you have additional information to suggest they are significant, you can keep them.

6.8 Floating point numbers

Integers are great for counting whole numbers, but sometimes we need to store very large (positive or negative) numbers, or **numbers with a fractional component**. A **floating-point type** variable is a variable that can hold a number with a fractional component, such as 4320.0, -3.33, or 0.01226. The floating part of the name floating point refers to the fact that the decimal point can **"float"** -- that is, it can support a variable number of digits before and after the decimal point.

When writing floating point numbers in your code, the decimal separator must be a decimal point. If you're from a country that uses a decimal comma, you'll need to get used to using a decimal point instead.

6.8.1 Floating point data types

There are **three standard floating point data types**: a **single-precision float**, a **double-precision double**, and an **extended-precision long double**. As with integers, C++ does not define the actual size of these types.

Floating point data types are **always signed** (can hold positive and negative values).

Float	4 bytes
double	8 bytes
long double	8, 12, or 16 bytes

On modern architectures, floating point representation for **float and double almost always follows IEEE 754 binary format** (created by William Kahan). In this format, **a float is 4 bytes and a double is 8 bytes**.

Long double is a strange one. Depending on the implementation, it may be equivalent to an IEEE 754 double (8 bytes), **it may be a 16-byte IEEE 754 binary format value, or it may be an 80-bit (typically padded to 12 bytes) non-IEEE 754 binary format value.** **We recommend avoiding long double.**

6.8.2 Best practice

Always make sure the type of your literals match the type of the variables they're being assigned to or used to initialize. Otherwise, an unnecessary conversion will result, possibly with a loss of precision.

e.g.:

double pi { 3.14 }; no suffix means double type by default

float pi { 3.14f };

6.8.3 Floating point range

Assuming IEEE 754 representation for 4-, 8-, and 16-byte representations:

Size	Range	Precision
4 bytes	$\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ and 0.0	6-9 significant digits, typically 7
8 bytes	$\pm 2.23 \times 10^{-308}$ to $\pm 1.80 \times 10^{308}$ and 0.0	15-18 significant digits, typically 16
80-bits (typically uses 12 or 16 bytes)	$\pm 3.36 \times 10^{-4932}$ to $\pm 1.18 \times 10^{4932}$ and 0.0	18-21 significant digits
16 bytes	$\pm 3.36 \times 10^{-4932}$ to $\pm 1.18 \times 10^{4932}$ and 0.0	33-36 significant digits

6.8.4 *Floating point precision*

Consider the fraction $1/3$. The decimal representation of this number is 0.333333333333... with **3's going out to infinity**. If you were writing this number on a piece of paper, your arm would get tired at some point, and you'd eventually stop writing. And the number you were left with would be close to 0.3333333333.... (with 3's going out to infinity) but not exactly.

On a computer, an **infinite precision number would require infinite memory to store**, and **we typically only have 4 or 8 bytes per value**. This limited memory means floating point numbers can only store a certain number of significant digits -- any additional significant digits are either lost or represented imprecisely. The number that is actually stored may be close to the desired number, but not exact.

The **precision** of a floating-point type **defines how many significant digits** it can represent **without information loss**.

The number of digits of precision a floating-point type has depends on both the size (**floats have less precision than doubles**) and the particular value being stored (some values can be represented more precisely than others).

For example, a float has 6 to 9 digits of precision. This means that **a float can exactly represent any number with up to 6 significant digits**. A number with **7 to 9** significant digits **may or may not be represented exactly** depending on the specific value. And a number with more than 9 digits of precision will definitely not be represented exactly.

Double values have between **15 and 18 digits of precision**, with most double values having at least 16 significant digits. **Long double** has a minimum precision of **15, 18, or 33 significant digits** depending on how many bytes it occupies.

6.8.5 *Outputting floating point values*

When outputting floating point numbers, **std::cout** has a **default precision of 6**.

that is, it **assumes** all **floating-point variables are only significant to 6 digits** (the minimum precision of a float), and hence **it will truncate anything after that**.

Also **note that std::cout will switch to outputting** numbers in **scientific notation in some cases**. Depending on the compiler, the exponent will typically be padded to a minimum number of digits.

We can **override** the **default precision that std::cout shows** by using an output manipulator function named **std::setprecision()**. **Output manipulators** alter how data is output, and are defined in the **iomanip** header.

Output manipulators (and input manipulators) are **sticky** -- meaning if you set them, they will remain set. The **one exception is std::setw**. Some IO operations reset **std::setw**, so **std::setw** should be used every time it is needed.

Best practice:

Favor double over float **unless space is at a premium**, as the lack of precision in a float will often lead to inaccuracies.

Also remember that **rounding errors make** floating point **comparisons tricky**. E.g.: Comparing 0.1 to 0.10000001 will be tricky and if you want this to evaluate to true you'd need to implement your own comparison logic or reduce precision on the comparison.

6.8.6 *NaN and Inf*

There are **two special categories** of floating-point numbers. These special categories are tied to **zero division**. The first is **Inf**, which represents **infinity**. **Inf can be positive or negative**.

The second is **NaN**, which stands for "**Not a Number**". There are several different kinds of NaN (which we won't discuss here). NaN and Inf are only available if the compiler uses a specific format (IEEE 754) for floating point numbers.

Inf stands for infinity, and **Ind** stands for **indeterminate**. Note that the **results of printing Inf and NaN are platform specific**, so your results may vary.

Best practice: **Avoid division by 0.0 altogether**, even if your compiler supports it.

6.9 Boolean values

In real-life, it's common to ask or be asked questions that can be answered with "yes" or "no". "Is an apple a fruit?" Yes. "Do you like asparagus?" No.

Now consider a similar statement that can be answered with a **"true" or "false"**: "Apples are a fruit". It's clearly true. Or how about, "I like asparagus". Absolutely false (yuck!).

These kinds of sentences that have only two possible outcomes: yes/true, or no/false are so common, that many programming languages include a **special type for dealing with them**. That type is called a **Boolean type** (note: Boolean is properly capitalized in the English language because it's named after its inventor, George Boole).

6.10 Boolean variables

Boolean variables are variables that can have only two possible values: **true**, and **false**.

To declare a Boolean variable, we use the keyword **bool**. E.g.: **bool b1 { true };**

When we print Boolean values, `std::cout` prints **0 for false**, and **1 for true**.

If you want `std::cout` to print true or false instead of 0 or 1, you can use **`std::boolalpha`**.

You can use **`std::noboolalpha`** to turn it back off.

6.11 Integer to Boolean conversion

When using uniform initialization, you can initialize a variable using **integer literals 0 (for false) and 1 (for true)** (but **you really should be using false and true** instead). **Other integer literals cause compilation errors.**

6.11.1 *Inputting Boolean values*

Inputting Boolean values using `std::cin` sometimes trips new programmers up.

It turns out that **`std::cin` only accepts two inputs for Boolean variables: 0 and 1** (not true or false). Any other inputs will cause `std::cin` to silently fail.

However, when **`std::boolalpha`** is enabled, **0 and 1 will no longer be interpreted as Booleans inputs (they both resolve to false as does any non-true input).**

Warning: Enabling `std::boolalpha` will **only allow lower-cased false or true to be accepted**. Variations with capital letters will not be accepted.

6.11.2 *Boolean return values*

Boolean values are often used as the return values for functions **that check whether something is true or not** (often called validation). Such functions are typically named starting with the word **is** (e.g. `isEqual`) or **has** (e.g. `hasCommonDivisor`).

6.12 Introduction to if statements

Consider a case where you want to do something only **IF** a certain condition is true.

Such conditions are common in programming, as they allow us to implement conditional behavior into our programs. The simplest kind of conditional statement in C++ is called an **if statement**. An if statement allows us to execute one (or more) lines of code only if some condition is true.

A **condition** (also called a **conditional expression**) is an expression that evaluates to a Boolean value.

Syntax:

```
if (health == 0)
{
    Respawn();
}
```

If the condition of an if statement evaluates to Boolean value true, then true_statement is executed. If the condition instead evaluates to Boolean value **false**, then **true_statement** is **skipped**.

6.12.1 If else example

```
int main()
{
    int x{ 6 };

    if (x < 5)
    {
        std::cout << x << " = Below five \n";
    }
    else
    {
        std::cout << x << " = Above five \n";
    }

    return 0;
}
```

6.12.2 Chaining if statements

Sometimes we want to check if several things are true or false in sequence. We can do so by chaining an if-statement (or if-else) to a prior if-else, like so:

```
int main()
{
    int x{ 5 };

    if (x == 5)
    {
        std::cout << x << " = Equal to five \n";
    }
    else if (x < 5)
    {
        std::cout << x << " = Below five \n";
    }
    else
    {
        std::cout << x << " = Above five \n";
    }

    return 0;
}
```

6.12.3 Boolean return values and if statements

Like we said in the last chapter on boolean values and **validation through control-flow statements** like if statements go hand in hand.

```
bool IsEqual(int x)
{
    return x == 5;
}

bool IsBelow(int x)
{
    return x < 5;
}

int main()
{
    int x{ 6 };

    if (IsEqual(x))
    {
        std::cout << x << " = Equal to five \n";
    }
    else if (IsBelow(x))
    {
        std::cout << x << " = Below five \n";
    }
    else
    {
        std::cout << x << " = Above five \n";
    }

    return 0;
}
```

6.12.4 Non-Boolean conditionals

In all of the examples above, our conditionals have been either Boolean values (true or false), Boolean variables, or functions that return a Boolean value. What happens if your conditional is an expression that does not evaluate to a Boolean value?

In such a case, **the conditional expression is converted to a Boolean value:**

- **non-zero** values get **converted to Boolean true (also negative values!)**
- **zero-values** get **converted to Boolean false.**

6.12.5 If-statements and early returns

A return statement that is not the last statement in a function is called an **early return**. Such a **statement will cause the function to return to the caller** when the return statement is executed (**before the function would otherwise return to the caller**, hence, “early”). Make sure early return statements have a condition attached to them. **Unconditional early returns aren’t useful.**

```
bool IsBelowFive(int x)
{
    if (x < 5)
    {
        return true;
    }

    return false;
}
```

6.13 Chars

To this point, the fundamental data types we've looked at have been used to hold numbers (integers and floating points) or true/false values (Booleans). But what if we want to store letters or punctuation?

The **char** data type was designed to hold **a single character**. A character can be a single letter, **number**, **symbol**, or **whitespace**.

The char data type is an **integral type (which is still a fundamental type)**, meaning **the underlying value is stored as an integer**. Similar to how a Boolean value **0** is interpreted as **false** and non-zero is interpreted as **true**, **the integer stored by a char variable are interpreted as an ASCII character**.

ASCII stands for American Standard Code for Information Interchange, and it defines a particular way to represent English characters (plus a few other symbols) as **numbers between 0 and 127** (called an ASCII code or code point). For example, ASCII code 97 is interpreted as the character 'a'.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

6.13.1 Initializing chars

You can initialize char variables using character literals or integer values:

```
char ch2{ 'a' };
```

```
char ch1{ 97 };
```

Warning: **Be careful not to mix up character numbers** with integer numbers. The following two initializations are not the same:

```
1 char ch{5}; // initialize with integer 5 (stored as integer 5)
2 char ch{'5'}; // initialize with code point for '5' (stored as integer 53)
```

6.13.2 Char size, range and default sign

Char is defined by C++ to always be **1 byte in size**. By default, **a char may be signed or unsigned** (though it's **usually signed**). **If you're using chars to hold ASCII characters, you don't need to specify a sign** (since both signed and unsigned chars can hold values between **0 and 127**).

If you're using a char to hold small integers (something you should not do unless you're explicitly optimizing for space), you should always specify whether it is signed or unsigned. A **signed char can hold a number between -128 and 127**. An unsigned char can hold a number between **0 and 255**.

6.13.3 Escape sequences

There are some sequences of characters in C++ that have special meaning. These characters are called **escape sequences**. An escape sequence **starts with a '\ (backslash)** character, and then a following letter or number.

Warning: Escape sequences start with a backslash (\), not a forward slash (/). If you use a forward slash by accident, it may still compile, but will not yield the desired result.

Name	Symbol	Meaning
Alert	\a	Makes an alert, such as a beep
Backspace	\b	Moves the cursor back one space
Formfeed	\f	Moves the cursor to next logical page
Newline	\n	Moves cursor to next line
Carriage return	\r	Moves cursor to beginning of line
Horizontal tab	\t	Prints a horizontal tab
Vertical tab	\v	Prints a vertical tab
Single quote	\'	Prints a single quote
Double quote	\"	Prints a double quote
Backslash	\\	Prints a backslash.
Question mark	\?	Prints a question mark. No longer relevant. You can use question marks unescaped.
Octal number	\(number)	Translates into char represented by octal
Hex number	\x(number)	Translates into char represented by hex number

6.13.4 *What's the difference between putting symbols in single and double quotes?*

Single **chars** are always put in **single quotes** (e.g. 'a', '+', '5'). A char can only represent one symbol (e.g. the letter a, the plus symbol, the number 5). **Text between double quotes** (e.g. "Hello, world!") is treated as a **string** of multiple characters.

Best practice: **Put stand-alone chars in single quotes** (e.g. 't' or '\n', **not "t" or "\n"**). This **helps the compiler optimize** more effectively.

6.13.5 *Avoid multicharacter literals*

For backwards compatibility reasons, many **C++ compilers support multicharacter literals**, which are **char literals that contain multiple characters** (e.g. '56'). If supported, these have an **implementation-defined** value (meaning it varies depending on the compiler). Because they are not part of the C++ standard, and their value is not strictly defined, **multicharacter literals should be avoided**.

6.13.6 *What about the other char types, wchar_t, char8_t, char16_t, and char32_t?*

wchar_t should be avoided in almost all cases (except when interfacing with the Windows API). Its size is implementation defined, and is not reliable. It has largely been **deprecated**.

Much like ASCII maps the integers 0-127 to American English characters, other character encoding standards exist to map integers (of varying sizes) to characters in other languages. The most well-known mapping outside of ASCII is the **Unicode standard**, which maps over 144,000 integers to characters in many different languages. Because Unicode contains so many code points, a single Unicode code point needs 32-bits to represent a character (called UTF-32). However, Unicode characters can also be encoded using multiple 16-bit or 8-bit characters (**called UTF-16 and UTF-8 respectively**).

char16_t and char32_t were added to C++11 to provide explicit support for 16-bit and 32-bit Unicode characters. These char types have the same size as std::uint_least16_t and std::uint_least32_t respectively (but are distinct types). **char8_t was added in C++20 to provide support for 8-bit Unicode (UTF-8)**. It is a distinct type that uses the same representation as unsigned char.

You won't need to use char8_t, char16_t, or char32_t unless you're planning on making your program Unicode compatible. Unicode and localization are generally outside the scope of these tutorials, so we won't cover it further.

6.14 Introduction to type conversion and static_cast

Consider the following program:

```
1  #include <iostream>
2
3  void print(double x) // print takes a double parameter
4  {
5      std::cout << x << '\n';
6  }
7
8  int main()
9  {
10     print(5); // what happens when we pass an int value?
11
12     return 0;
13 }
```

In the above example, the `print()` function has a **parameter of type double** but the caller is passing in the value 5 which is of type `int`. What happens in this case?

In most cases, C++ will allow us to convert values of one fundamental type to another fundamental type. The process of converting a value from one type to another type is called **type conversion**. Thus, the `int` argument 5 will be converted to double value 5.0 and then copied into parameter `x`. The `print()` function will print this value, resulting in the following output: 5.

6.14.1 Type conversion produces a new value

Even though it is called a conversion, a **type conversion does not actually change the value or type** of the value being converted. Instead, the value to be converted is used as input, and the conversion results in a new value of the target type (via direct initialization).

6.14.2 *Implicit type conversion warnings*

Although **implicit type conversion** is sufficient for most cases where type conversion is needed, there are a **few cases where it is not**.

Assume we're passing a **floating-point type to an integer type**. Unlike the initial example, when this program is compiled, your compiler will generate some kind of a warning about a possible loss of data.

And because you have **"treat warnings as errors"** turned on (you do, right?), your **compiler will abort the compilation** process.

Because converting a floating-point value to an integral value results in any fractional component being dropped, the compiler will warn us when it does an implicit type conversion from a floating point to an integral value. This happens even if we were to pass in a floating-point value with no fractional component, like 5.0 -- no actual loss of value occurs during the conversion to integral value 5 in this specific case, but the compiler may still warn us that the conversion is unsafe.

6.14.3 *An introduction to explicit type conversion via the static_cast operator*

What if we intentionally wanted to pass a double value to a function taking an integer (knowing that the converted value would drop any fractional component?) **Turning off "treat warnings as errors" just to make our program compile is a bad idea**, because then we'll have warnings every time we compile (which we will quickly learn to ignore), and we **risk overlooking warnings about more serious issues**.

C++ supports a second method of type conversion, called **explicit type conversion**. Explicit type conversion **allow us (the programmer) to explicitly tell the compiler to convert a value from one type to another type**, and that we take full responsibility for the result of that conversion. If such a conversion results in the loss of value, the compiler will not warn us.

To perform an explicit type conversion, in most cases we'll use the **static_cast operator**. The syntax for the static cast looks a little funny:

```
static_cast<new_type>(expression)
```

`static_cast` takes the value from an expression as input, and returns that value converted into the type specified by `new_type` (e.g. `int`, `bool`, `char`, `double`).

Key insight: Whenever you see C++ syntax (**excluding** the **preprocessor**) that makes use of angled brackets (<>), the thing **between the angled brackets will most likely be a type**. This is typically how C++ deals with code that need a parameterized type.

```
4  void print(int x)
5  {
6      std::cout << x << '\n';
7  }
8
9  int main()
10 {
11     print(static_cast<int>(5.5)); // explicitly convert double value 5.5 to an int
12
13     return 0;
14 }
```

Because we're now explicitly requesting that double value 5.5 be converted to an int value, the compiler will not generate a warning about a possible loss of data upon compilation (meaning we can leave "treat warnings as errors" enabled).

C++ supports other types of casts. We talk more about the different types of casts in the future.

There are some other special reasons to use the static_cast operator:

- Converting **char** to **int**
- Converting **unsigned** numbers to **signed** numbers
- std::int8_t and std::uint8_t likely **behave like chars** instead of integers

6.15 Fundamental data types summary

The smallest unit of memory is a **binary digit**, also called a **bit**. The smallest unit amount of memory that can be addressed (accessed) directly is a byte. The **modern standard** is that a **byte equals 8 bits**.

A **data type tells the compiler how to interpret the contents of memory** in some meaningful way.

C++ comes with support for many fundamental data types, including **floating point numbers, integers, boolean, chars, null pointers, and void**.

Void is used to indicate **no type**. It is **primarily used to indicate that a function does not return a value**.

Different types take different amounts of memory, and the amount of memory used may vary by machine.

The **sizeof** operator can be **used to return the size of a type in bytes**.

Signed integers are used for holding positive and negative whole numbers, including 0. The set of values that a specific data type can hold is called its range. When using integers, keep an eye out for overflow and integer division problems.

Unsigned integers only hold positive numbers (and 0), and should generally be avoided unless you're doing bit-level manipulation.

Fixed-width integers are integers with guaranteed sizes, but they may not exist on all architectures. The `std::int8_t` and `std::uint8_t` should generally be avoided, as they tend to behave like chars instead of integers.

size_t is an unsigned integral type that is used to **represent the size or length of objects**.

Scientific notation is a shorthand way of writing lengthy numbers. C++ supports scientific notation in conjunction with floating point numbers. The digits in the significand (the part before the e) are called the significant digits.

Floating point is a set of types designed to hold real numbers (including those with a fractional component). The precision of a number defines how many significant digits it can represent without information loss. A **rounding error** can occur when too many significant digits are stored in a floating-point number that can't hold that much precision.

Rounding errors happen all the time, even with simple numbers such as 0.1. Because of this, you shouldn't compare floating point numbers directly.

The **Boolean** type is used to store a true or false value.

If statements allow us to execute one or more lines of code if some condition is true. The **conditional expression** of an if-statement is interpreted as a boolean value. An **else statement** can be used to execute a statement when a prior if-statement condition evaluates to false.

Char is used to store values that are interpreted as an ASCII character. When using chars, be careful not to mix up ASCII code values and numbers. Printing a char as an integer value requires use of `static_cast`.

Angled brackets are typically used in C++ to represent something that needs a **parameterizable type**. This is used with `static_cast` to determine what data type the argument should be converted to (e.g. `static_cast<int>(x)` will return the value of x as an int).

7 Constants and Strings

7.1 Constant variables (named constants)

In programming, a **constant** is a **value that may not be changed** during the program's execution.

C++ supports two different kinds of constants:

- **Named constants** are constant values that **are associated with an identifier**. These are also sometimes called symbolic constants, or occasionally just constants.
- **Literal constants** are constant values that **are not associated with an identifier**.

Types of named constants:

- Constant **variables**.
- **Object-like macros with substitution text** (introduced in: Introduction to the preprocessor, with additional coverage in this lesson).
- **Enumerated** constants (covered in lesson 13.2 -- Unscoped enumerations).

So far, all of the variables we've seen have been non-constant -- **that is, their values can be changed at any time** (typically **by assigning a new value**).

However, there are many cases where it is useful to define variables with values that cannot be changed. For example, consider the gravity of Earth (near the surface): 9.8 meters/second². This isn't likely to change any time. **Defining this value as a constant helps ensure that this value isn't accidentally changed**. Constants also have other benefits that we'll explore in subsequent lessons.

To declare a constant variable, we place the **const keyword** (called a "**const qualifier**") **adjacent to the object's type**:

```
const double gravity {9.8};
```

7.1.1 *Const placing*

Although C++ will accept the const qualifier **either before or after the type**, it's much **more common to use const before the type** because it better follows standard English language convention where modifiers come before the object being modified (e.g. a "a green ball", not a "a ball green").

Due to the way that the compiler parses more complex declarations, some developers prefer placing the const after the type (because it is slightly more consistent). **This style is called "east const"**. While this style has some advocates (and some reasonable points), it **has not caught on significantly**.

Best practice: Place const before the type (because it is more conventional to do so).

7.1.2 *Const variables must be initialized*

Const variables **must be initialized** when you define them, and then that value cannot be changed via assignment.

Note that const variables **can be initialized from other variables** (including non-const ones).

7.1.3 *Naming conventions*

There are a number of different naming conventions that are used for const variables.

- C-style:
 - Programmers who have transitioned from C often prefer **underscored, upper-case names for const variables** (e.g. EARTH_GRAVITY).
- C++-style:
 - More common in C++ is to use **intercapped names with a 'k' prefix** (e.g. kEarthGravity).

However, because const variables act like normal variables (except they cannot be assigned to), there is no reason that they need a special naming convention. For this reason, we prefer using the same naming convention that we use for non-const variables (e.g. earthGravity).

7.1.4 *Const function parameters*

Function parameters can be made constants via the `const` keyword.

Note: We do not need to provide an explicit initializer for our `const` parameter. The value of the argument in the function call will be used as the initializer for `const` parameters.

Making a function parameter constant **enlists the compiler's help to ensure that the parameter's value is not changed inside the function**. However, in modern C++ we don't make value parameters `const` because we generally don't care if the function changes the value of the parameter (since **it's just a copy that will be destroyed at the end of the function** anyway). The `const` keyword also adds a small amount of unnecessary clutter to the function prototype.

Best practice: **Don't** use `const` when **passing by value**. Later we'll talk about two other ways to pass arguments to functions: **pass by reference**, and **pass by address**. When using either of these methods, **proper use of `const` is important**.

7.1.5 *Const return values*

A function's **return value** may also be made `const`.

For fundamental types, the `const` qualifier on a return type is simply ignored (your compiler may generate a warning).

For other types (which we'll cover later), there is typically little point in returning `const` objects by value, because they are temporary copies that will be destroyed anyway. Returning a `const` value **can also impede certain kinds of compiler optimizations** (involving move semantics), which **can result in lower performance**.

Best practice: **Don't** use `const` when **returning by value**.

7.1.6 Object-like macros with substitution text

In: Introduction to the preprocessor, we discussed object-like macros with substitution text.

```
1 #include <iostream>
2
3 #define MY_NAME "Alex"
4
5 int main()
6 {
7     std::cout << "My name is: " << MY_NAME << '\n';
8
9     return 0;
10 }
```

When the preprocessor processes the file containing this code, it will replace `MY_NAME` (on line 7) with `"Alex"`. Note that `MY_NAME` is a name, and the substitution text is a constant value, so **object-like macros with substitution text are also named constants**.

7.1.7 Prefer constant variables to preprocessor macros

So why not use preprocessor macros for named constants? **There are (at least) three major problems.**

The biggest issue is that **macros don't follow normal C++ scoping rules**. Once a macro is `#defined`, all subsequent occurrences of the macro's name in the current file will be replaced. If that name is used elsewhere, you'll get macro substitution where you didn't want it. This will most likely lead to strange compilation errors.

Second, it is often **harder to debug code using macros**. Although your source code will have the macro's name, the compiler and debugger never see the macro because it has already been replaced before they run. **Many debuggers are unable to inspect a macro's value**, and often have limited capabilities when working with macros.

Third, **macro substitution behaves differently than everything else in C++**. Inadvertent mistakes can be easily made as a result.

Best practice: Prefer **constant variables over object-like macros with substitution text**.

7.1.8 *Using constant variables throughout a multi-file program*

In many applications, a given named constant needs to be used throughout your code (not just in one file). These can include physics or mathematical constants that don't change (e.g. pi or Avogadro's number), or application-specific "tuning" values (e.g. friction or gravity coefficients). **Instead of redefining them every time** they are needed, it's better to **declare them once in a central location** and use them wherever needed. That way, if you ever need to change them, you only need to change them in one place.

There are **multiple ways to facilitate this** within C++ -- we cover this topic in full detail in lesson: Sharing global constants across multiple files (using inline variables).

7.1.9 *Type qualifiers*

A **type qualifier** (sometimes called a **qualifier** for short) is a keyword that is applied to a type that **modifies how that type behaves**. The `const` used to declare a constant variable is called a **const type qualifier** (or **const qualifier** for short).

As of C++23, C++ only has **two type qualifiers**: **const** and **volatile**.

The **volatile** qualifier is used to tell the compiler that an **object may have its value changed at any time**. This rarely-used qualifier disables certain types of optimizations.

In technical documentation, the `const` and `volatile` qualifiers are often referred to as `cv-qualifiers`. The following terms are also used in the C++ standard:

- A **cv-unqualified** type is a type with no type qualifiers (e.g. `int`).
- A **cv-qualified** type is a type with one or more type qualifiers applied (e.g. `const int`).
- A **possibly cv-qualified** type is a type that may be `cv-unqualified` or `cv-qualified`.

These terms are not used much outside of technical documentation, so they are listed here for reference, not as something you need to remember.

7.2 Literals

Literals are **values** that are **inserted directly into the code**. Literals are sometimes called **literal constants** because their meaning cannot be redefined (5 always means the integral value 5).

Just like objects have a type, **all literals have a type**. The type of a literal is **deduced from the literal's value**. For example, a literal that is a whole number (e.g. 5) is deduced to be of type `int`.

```
1 return 5; // 5 is an integer literal
2 bool myNameIsAlex { true }; // true is a boolean literal
3 double d { 3.4 }; // 3.4 is a double literal
4 std::cout << "Hello, world!"; // "Hello, world!" is a C-style string literal
```

7.2.1 Literal suffixes

If the default type of a literal is not as desired, **you can change the type of a literal by adding a suffix**. In most cases, suffixes aren't needed (**except for f**). It's important to **know what** the default types (floating-point types are **double** and integral types are **int**) are and what **literal types you are trying to use**. Mistakenly **using the wrong type requires the compiler to do conversion**.

For example: `long c { 7 }; // ok: compiler will convert int value 7 to long value 7`

Here are some of the more common suffixes:

Data type	Suffix	Meaning
integral	u or U	unsigned int
integral	l or L	long
integral	ul, uL, Ul, UL, lu, lU, Lu, LU	unsigned long
integral	ll or LL	long long
integral	ull, uLL, Ull, ULL, llu, llU, LLu, LLU	unsigned long long
integral	z or Z	The signed version of <code>std::size_t</code> (C++23)
integral	uz, uZ, Uz, UZ, zu, zU, Zu, ZU	<code>std::size_t</code> (C++23)
floating point	f or F	float
floating point	l or L	long double
string	s	<code>std::string</code>
string	sv	<code>std::string_view</code>

The **s** and **sv** suffixes **require an additional line of code to use**. We cover these in further lessons.

Additional (rarely used) **suffixes exist** for complex numbers and chrono (time) literals.

Excepting the **f** suffix, **suffixes are most often used in cases where type deduction is involved**.

7.2.2 *Suffix casting*

Most of the suffixes are not case sensitive.

The exceptions are:

- **s** and **sv** must be lower case.
- **Two consecutive l or L** characters must have the same casing.

Because lower-case L can look like numeric 1 in some fonts, some developers prefer to use upper-case literals. Others use lower case suffixes except for L. **Best practice** here is to **prefer literal suffix L (upper case) over l (lower case)**.

7.2.3 *String literals*

In programming, a string is a **collection of sequential characters** used to represent text (such as names, words, and sentences). **The very first C++ program most people write probably printed hello world without using any variables.**

"Hello, world!" is a **string literal**. String literals are placed between double quotes to identify them as strings (as opposed to **char literals**, which are placed **between single quotes**).

Because strings are commonly used in programs, most modern programming languages include a fundamental string data type. **For historical reasons, strings are not a fundamental type in C++**. Rather, they have a strange, complicated type that is hard to work with (we'll cover how/why in a future lesson, once we've covered more fundamentals required to explain how they work). **Such strings are often called C strings or C-style strings, as they are inherited from the C-language.**

There are two non-obvious things worth knowing about C-style string literals:

All **C-style string literals have an implicit null terminator**. Consider a string such as "hello". While this C-style string appears to only have five characters, it **actually has six**: 'h', 'e', 'l', 'l', 'o', and '\0' (a character with **ASCII code 0**). This trailing '\0' character is a special character called a null terminator, and it is used to indicate the end of the string. A string that ends with a null terminator is called a null-terminated string.

The **reason for the null-terminator is also historical**: it can be used to **determine where the string ends**.

Unlike most other literals (**which are values, not objects**), **C-style string literals are const objects** that are created at the start of the program and are **guaranteed to exist for the entirety of the program**. This fact will become important in a few lessons, when we discuss **std::string_view**.

Unlike C-style string literals, **std::string** and **std::string_view** literals create temporary objects. These temporary objects **must be used immediately**, as they are destroyed at the end of the full expression in which they are created.

7.2.4 *Magic numbers*

A magic number is a **literal** (usually a number) **that either has an unclear meaning or may need to be changed later**.

Best practice: **Avoid magic numbers** in your code (use **constexpr variables** instead).

7.3 Numeral systems (decimal, binary, hexadecimal, and octal)

In everyday life, we count using **decimal numbers**, where each numerical digit can be 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9. Decimal is also called “**base 10**”, because there are 10 possible digits (0 through 9). In this system, we count like this: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ... By default, numbers in C++ programs are assumed to be decimal.

In **binary**, there are only 2 digits: 0 and 1, so it is called “**base 2**”. In binary, we count like this: 0, 1, 10, 11, 100, 101, 110, 111, ...

Decimal and binary are two examples of **numeral systems**, which is a **fancy name for a collection of symbols** (e.g. digits) used to represent numbers. There are **4 main numeral systems** available in C++. In order of popularity, these are: decimal (**base 10**), binary (**base 2**), hexadecimal (**base 16**), and octal (**base 8**).

Numeral systems **prefixes**:

0 **Octal**

0x **Hexadecimal**

0b **Binary**

```
int main()
{
    int hex = 0x0008;
    std::cout << hex << "\n";

    int oct = 010;
    std::cout << oct << "\n";

    int bin = 0b1000;
    std::cout << bin << "\n";

    return 0;
}
```

All of the following cout statements print the number 8.

7.3.1 Digit separators

Because long literals can be hard to read, C++14 also adds the ability to use a **quotation mark (')** as a **digit separator**.

```
int bin { 0b1011'0010 }; // assign binary 1011 0010 to the variable
long value { 2'132'673'462 }; // much easier to read than 2132673462
```

7.3.2 Outputting values in decimal, octal, or hexadecimal (and binary)

By default, C++ outputs values in decimal. However, you can change the output format via use of the **std::dec**, **std::oct**, and **std::hex** I/O manipulators. Note that once applied, the I/O manipulator remains set for future output until it is changed again.

7.3.2.1 Outputting values in binary

Outputting values in binary is a little harder, as **std::cout** doesn't come with this capability built-in. Fortunately, the **C++ standard library includes a type called `std::bitset`** that will do this for us (in the **<bitset>** header).

To use **std::bitset**, we can **define a `std::bitset` variable** and tell **std::bitset** how many bits we want to store. The **number of bits must be a compile-time constant**. **std::bitset** can be initialized with an integral value (in any format, including decimal, octal, hex, or binary).

```
1 #include <bitset> // for std::bitset
2 #include <iostream>
3
4 int main()
5 {
6     // std::bitset<8> means we want to store 8 bits
7     std::bitset<8> bin1{ 0b1100'0101 }; // binary literal for binary 1100 0101
8     std::bitset<8> bin2{ 0xC5 }; // hexadecimal literal for binary 1100 0101
9
10    std::cout << bin1 << '\n' << bin2 << '\n';
11    std::cout << std::bitset<4>{ 0b1010 } << '\n'; // create a temporary std::bitset and print it
12
13    return 0;
14 }
```

Like the code above has shown. **Std::bitset<n>** creates a **temporary (unnamed) std::bitset object with n amount of bits**, initializes it with the inserted binary literal, prints the value in binary, and then discards the temporary object.

7.4 Constant expressions and compile-time optimization (run or compile-time)

The as-if rule:

In C++, compilers are given a lot of leeway to optimize programs. The as-if rule says that the compiler can modify a program however it likes in order to produce more optimized code, so long as those modifications do not affect a program's “**observable behavior**”.

For advanced readers: There is **one exception to the as-if rule: unnecessary calls to a copy constructor can be elided (omitted) even if those copy constructors have observable behavior**. We cover this topic in lesson: Class initialization and **copy elision**.

Exactly how a compiler optimizes a given program is up to the compiler itself. **However, there are things we can do to help the compiler optimize better.**

7.4.1 An optimization opportunity

Consider the following program with output 7:

```
1  #include <iostream>
2
3  int main()
4  {
5      int x { 3 + 4 };
6      std::cout << x << '\n';
7
8      return 0;
9  }
```

There's an interesting optimization possibility hidden within this program.

If this program were compiled exactly as it was written (with no optimizations), the compiler would generate an executable that **calculates the result of $3 + 4$ at runtime** (when the program is run). **If the program were executed a million times, $3 + 4$ would be evaluated a million times, and the resulting value of 7 produced a million times.**

Note: Some compilers use **constant folding**, an optimization technique where the **compiler evaluates constant expressions at compile-time** and replaces them with their results. This reduces the need for calculations during run-time, thereby improving performance.

7.4.2 *Compile-time evaluation of expressions*

Modern C++ compilers are able to evaluate some expressions at compile-time (constant folding). When this occurs, the compiler can replace the expression with the result of the expression. The resulting executable **no longer needs to spend CPU cycles calculating** at runtime! Even better, we don't need to do anything to enable this behavior (**besides have optimizations turned on**).

7.4.3 *Constant expressions*

One kind of **expression that can always be evaluated at compile time** is called a “**constant expression**”. The precise definition of a constant expression is complicated, so we'll take a simplified view:

A constant expression is an **expression that contains only compile-time constants and operators/functions that support compile-time evaluation**.

A **compile-time constant** is a constant whose value must be known at compile time. This includes:

- **Literals**
- **Const integral variables with a constant expression initializer** (historical exception)
- **Constexpr variables and functions** (preferred in C++)
- **Preprocessor macros**
- **Non-type template parameters**
- **Enumerators**

Const variables that are not compile-time constants are sometimes called **runtime constants**. Runtime constants **cannot be used in a constant expression**.

Const non-integral variables are always runtime constants (even if they have a constant expression initializer). If you need such variables to be compile-time constants, **define them as constexpr variables instead**.

The most common type of **operators and functions that support compile-time evaluation** include:

- **Arithmetic operators** with operands that are compile-time constants (e.g. $1 + 2$)
- **Constexpr and consteval functions** (we'll discuss these later in the chapter)

In the following example, we **identify the constant expressions and non-constant expressions**. We also identify which variables are non-constant, runtime constant, or compile-time constant.

```
1  #include <iostream>
2
3  int getNumber()
4  {
5      std::cout << "Enter a number: ";
6      int y{};
7      std::cin >> y;
8
9      return y;
10 }
11
12 int main()
13 {
14     // Non-const variables:
15     int a { 5 };           // 5 is a constant expression
16     double b { 1.2 + 3.4 }; // 1.2 + 3.4 is a constant expression
17
18     // Const integral variables with a constant expression initializer
19     // are compile-time constants:
20     const int c { 5 };     // 5 is a constant expression
21     const int d { c };     // c is a constant expression
22     const long e { c + 2 }; // c + 2 is a constant expression
23
24     // Other const variables are runtime constants:
25     const int f { a };     // a is not a constant expression
26     const int g { a + 1 }; // a + 1 is not a constant expression
27     const long h { a + c }; // a + c is not a constant expression
28     const int i { getNumber() }; // getNumber() is not a constant expression
29
30     const double j { b };  // b is not a constant expression
31     const double k { 1.2 }; // 1.2 is a constant expression
32
33     return 0;
34 }
```

An expression that is **not a constant expression** is sometimes called a **runtime expression**. For example, `std::cout << x << '\n'` is a runtime expression, both because `x` is not a compile-time constant, and because `operator<<` doesn't support compile-time evaluation when used for output (since **output can't be done at compile-time**).

7.4.4 *Why we care about constant expressions*

Constant expressions are **useful for** (at least) **three reasons**:

- **Constant expressions are always eligible for compile-time evaluation**, meaning they are more likely to be **optimized at compile-time**. This produces **faster and smaller code**.
- With runtime expressions, only the type of the expression is known at compile-time. **With constant expression, both the type AND the value of the expression is known at compile-time**. This **allows us to do compile-time sanity checking** of those values. If such a value does not meet our requirements, we can fail the build, **allowing us to identify and fix the issue immediately**. The result is **code that is safer, easier to test, and more difficult to misuse**.
- **Certain C++ features** that we'll cover in future lessons **require constant expressions** (see below)

For advanced readers: There are a few common cases where a constant expression is required:

- The initializer of a constexpr variable.
- A non-type template argument.
- The defined length of a std::array

Key insight:

Constant expressions (including constexpr variables and constexpr functions, which we'll cover shortly) can **help produce code that is faster, smaller, and safer**. **Some C++ features require constant expressions**.

7.4.5 When are constant expressions evaluated?

The compiler is only required to evaluate constant expressions at compile-time in contexts that **require a constant expression (such as the initializer of a compile-time constant)**. In contexts that do not require a constant expression, the **compiler may choose** whether to evaluate a constant expression at compile-time or at runtime.

In short: **Using the const modifier under the conditions that the compiler can evaluate the expression at compile-time enforces compile-time evaluation.**

Even when not required to do so, **modern compilers will usually evaluate a constant expression at compile-time** because it is an easy optimization and more performant to do so.

7.4.5.1 Partial optimization of constant subexpressions (constant folding)

Now consider the following example:

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << 3 + 4 << '\n';
6
7     return 0;
8 }
```

The **full expression** `std::cout << 3 + 4 << '\n';` is a runtime expression because **output can only be done at runtime**. But notice that the **full expression contains constant subexpression 3 + 4**.

Compilers have long been **able to optimize constant subexpressions**, even when the full expression is a runtime expression. This optimization process is called “**constant folding**”, and is **allowed under the as-if rule**.

7.4.5.2 Optimization of non-constant expressions

Compilers are even capable of optimizing non-constant expressions or subexpressions in certain cases. Let's revisit a prior example:

```
1 #include <iostream>
2
3 int main()
4 {
5     int x { 7 };           // x is non-const
6     std::cout << x << '\n'; // x is a non-constant subexpression
7
8     return 0;
9 }
```

When `x` is initialized, the value 7 will be stored in the memory allocated for `x`. Then on the next line, the program will go out to memory again to fetch the value 7 so it can be printed. **Even though `x` is non-const, a smart compiler might realize that `x` will always evaluate to 7 in this particular program, and under the as-if rule, optimize the program to this:**

```
1 #include <iostream>
2
3 int main()
4 {
5     int x { 7 };
6     std::cout << 7 << '\n';
7
8     return 0;
9 }
```

Since `x` is no longer used in the program, the compiler could go one step further and optimize the program to this:

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << 7 << '\n';
6
7     return 0;
8 }
```

In this version, the **variable `x` was removed completely** (because it was not used, and thus not needed). When a variable is removed from a program, **we say the variable has been optimized out (or optimized away)**.

However, since `x` is non-const, such optimizations require the compiler to realize that the value of `x` actually doesn't change (even though it could). **Whether the compiler realizes this comes down to how complex the program is and how sophisticated the compiler's optimization routines are.**

7.4.5.3 Const variables are easier to optimize

Now let's consider this similar example:

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     const int x { 7 }; // x is now const
6 |     std::cout << x << '\n';
7 |
8 |     return 0;
9 | }
```

In this version, the **compiler no longer has to infer that x won't change**. Because **x is now const**, the compiler now has a guarantee that x can't be changed after initialization. This makes it easier for the compiler to understand that it can safely optimize x out of this program, and therefore it is more likely to do so.

Ranking variables by the likelihood of the compiler being able to optimize them:

- Compile-time constant variables (always eligible to be optimized)
- Runtime constant variables
- Non-const variables (likely optimized in simple cases only)

7.4.6 *Optimization can make programs harder to debug*

When the compiler optimizes a program, variables, expressions, statements, and function calls may be rearranged, altered, replaced with a value, or even removed entirely. Such changes can make it hard to debug a program effectively.

At runtime, it can be hard to debug compiled code that no longer correlates very well with the original source code. For example, if you try to watch a variable that has been optimized out, the debugger won't be able to locate the variable. If you try to step into a function that has been optimized away, the debugger will simply skip over it. **So, if you are debugging your code and the debugger is behaving strangely, this is the most likely reason.**

At compile-time, we have little visibility and few tools to help us understand what the compiler is even doing. If a variable or expression is replaced with a value, and that value is wrong, how do we even go about debugging it? This is an ongoing challenge.

To help minimize such issues, **debug builds will typically turn down (or turn off) optimizations, so that the compiled code will more closely match the source code.**

7.5 Constexpr variables (compile-time)

7.5.1 *The compile-time const challenge*

We discussed how **one way to make a compile-time constant variable is to use the `const` keyword**. If the `const` variable has an integral type and a constant expression initializer, it is a compile-time constant. **All other `const` variables are treated as runtime constants**.

However, **this method has two challenges**:

First, when using `const`, **our integral variables could end up as either a compile-time const or a runtime const**, depending on whether the initializer is a constant expression or not. **In some cases, this can make it hard to tell whether the `const` variable is actually a compile-time constant or not**. Consider a scenario where you're reading code and **see a `const` variable initialized through another variable that you don't have access to**. This might or might not be `const` depending on the referenced variable.

Second, this use of `const` to create compile-time constant variables **does not extend to non-integral variables**. And there are many cases where we would like non-integral variables to be compile-time constants too.

7.5.2 *The `constexpr` keyword*

Fortunately, we can **enlist the compiler's help to ensure we get a compile-time constant variable where we desire one**. To do so, we **use the `constexpr` keyword** (which is **shorthand for "constant expression"**) instead of `const` in a variable's declaration. A `constexpr` variable is always a compile-time constant. **As a result, a `constexpr` variable must be initialized with a constant expression, otherwise a compilation error will result**.

7.5.3 The meaning of `const` vs `constexpr` for variables

For variables:

- **`const`** means that the **value of an object cannot be changed after initialization**. The value of the initializer **may be known at compile-time or runtime**. The `const` object can be evaluated at runtime.
- **`constexpr`** means that the object can be used in a **constant expression**. The value of the initializer **must be known at compile-time**. The `constexpr` object can be evaluated at runtime or compile-time.

Constexpr variables are implicitly `const`. `Const` variables are **not implicitly `constexpr`** (except for `const` integral variables with a constant expression initializer).

Although a variable **can be defined as both `constexpr` and `const`**, in most cases this is **redundant**, and we only need to use either `const` or `constexpr`.

Best practice:

- Any constant **variable whose initializer is a constant expression should be declared as `constexpr`**.
- Any constant **variable whose initializer is not a constant expression** (making it a runtime constant) **should be declared as `const`**.

Caveat: In the future we will discuss some **types that are not fully compatible with `constexpr`** (including `std::string`, `std::vector`, and other types that use dynamic memory allocation). For constant objects of these types, either **use `const` instead of `constexpr`**, or pick a different type that is `constexpr` compatible (e.g. `std::string_view` or `std::array`).

7.5.4 `Const` and `constexpr` function parameters

Normal function calls are evaluated at runtime, with the supplied arguments being used to **initialize the function's parameters**. Because the initialization of function parameters happens at runtime, this leads to **two consequences**:

- **`const` function parameters are treated as runtime constants** (even when the supplied argument is a compile-time constant).
- Function parameters **cannot be declared as `constexpr`**, since their **initialization value isn't determined until runtime**.

7.5.5 *Nomenclature recap*

Compile-time constant

- A value or **non-modifiable object whose value must be known at compile time** (e.g. literals and constexpr variables).

Constexpr

- **Keyword** that **declares variables as compile-time constants** (and functions that can be evaluated at compile-time). Informally, shorthand for “constant expression”.

Constant expression

- An **expression that contains only compile-time constants** and operators/functions that support compile-time evaluation.

Runtime expression

- An expression that is **not a constant expression**.

Runtime constant

- A value or **non-modifiable object that is not a compile-time constant**.

7.6 The conditional operator

Operator	Symbol	Form	Meaning
Conditional	?:	<code>c ? x : y</code>	If conditional <code>c</code> is <code>true</code> then evaluate <code>x</code> , otherwise evaluate <code>y</code>

The **conditional operator** (`?:`) (also sometimes called the **arithmetic if** operator) is a **ternary operator** (an **operator that takes 3 operands**). Because it has historically been C++'s only ternary operator, it's also sometimes referred to as "**the ternary operator**".

The `?:` operator provides a **shorthand method** for doing a particular type of **if-else statement**.

Syntax: **`condition ? expression_if_true : expression_if_false`**

7.6.1 The conditional operator evaluates as an expression

Because the operands of the conditional operator are **expressions rather than statements**, the **conditional operator can be used in places where an expression is required**.

For example, when initializing a variable:

```
constexpr bool inBigClassroom { false };
constexpr int classSize { inBigClassroom ? 30 : 20 };
std::cout << "The class size is: " << classSize << '\n';
```

Best practice:

- Parenthesize the entire conditional operator when used in a compound expression.
- For readability, consider parenthesizing the condition if it contains any operators (other than the function call operator).

7.6.2 *Parenthesizing the conditional operator*

Because C++ prioritizes the evaluation of most operators above the evaluation of the conditional operator, it's quite easy to write expressions using the conditional operator that don't evaluate as expected.

For this reason, the conditional operator should be parenthesized as follows:

- **Parenthesize the entire conditional operator when used in a compound expression** (an expression with other operators).
- For readability, **consider parenthesizing the condition** if it contains any operators (other than the function call operator).
- The **operands** of the conditional operator **do not need to be parenthesized**.

7.6.3 *The type of the expressions must match or be convertible*

To comply with C++'s type checking rules, **one of the following must be true**:

- The **type of the second and third operand must match**.
- The **compiler must be able to find a way to convert one or both of the second and third operands to matching types**. The conversion rules the compiler uses are fairly complex and may yield surprising results in some cases.

```
1  #include <iostream>
2
3  int main()
4  {
5      std::cout << (true ? 1 : 2) << '\n';    // okay: both operands have matching type int
6
7      std::cout << (false ? 1 : 2.2) << '\n'; // okay: int value 1 converted to double
8
9      std::cout << (true ? -1 : 2u) << '\n';  // surprising result: -1 converted to unsigned int, result out of range
10
11     return 0;
12 }
```

In general, it's **okay to mix operands with fundamental types** (excluding mixing signed and unsigned values). If either operand is not a fundamental type, it's **generally best to explicitly convert one or both operands to a matching type yourself** so you know exactly what you'll get.

The conditional operator is **most useful when** doing one of the following:

- **Initializing** an object **with one of two values**.
- **Assigning one of two values** to an object.
- **Passing one of two values** to a function.
- **Returning one of two values** from a function.
- **Printing one of two values**.

Best practice:

Prefer to **avoid the conditional operator in complicated expressions**.

7.7 Inline functions and variables (may or may not inline)

Consider the case where you need to write some code to perform some discrete task, like reading input from the user, or outputting something to a file, or calculating a particular value. When implementing this code, you essentially have two options:

- **Write the code as part of an existing function** (called writing code “**in-place**” or “**inline**”).
- **Create a function** (and possibly sub-functions) to handle the task.

Writing **functions provides many potential benefits**, as code in a function:

- Is **easier to read** and understand in the context of the overall program.
- Is **easier to use**, as you can call the function without understanding how it is implemented.
- Is **easier to update**, as the code in a function can be updated in one place.
- Is **easier to reuse**, as functions are naturally modular.

However, **one downside** of using a function is that **every time a function is called, there is a certain amount of performance overhead (“overhead cost”)** that occurs.

7.7.1 *Function overhead*

When a call to a function is encountered the CPU has to:

1. Store the address of the current instruction (bookmark **to return to later**)
2. Store values of various CPU registers (to **restore later**)
3. Function **parameters get instantiated and then initialized**
4. Execution path makes a **jump into the function**
5. Jump back to location of the call instruction
6. Return value has to be copied for use

For **functions that are large and/or perform complex tasks**, the **overhead** of the function call is **typically insignificant** compared to the amount of time the function takes to run.

However, for small functions, the overhead costs can be larger than the time needed to actually execute the function's code! In cases where a small function is called often, **using a function can result in a significant performance penalty over writing the same code in-place.**

7.7.2 *Inline expansion*

Fortunately, the C++ compiler has a trick that it can use to avoid such overhead cost: **Inline expansion** is a process where a **function call is replaced by the code from the called function's definition.**

Function calls will be replaced by the code in the body of the called function (with the **value of the arguments substituted for the parameters**). This **allows us to avoid the overhead** of those calls, **while preserving the results** of the code.

7.7.3 *The performance of inline code*

Beyond **removing the cost of function call**, inline expansion can also allow the compiler to optimize the resulting code more efficiently. For example: the code that gets inlined could **potentially be optimized into a constant expression**.

However, **inline expansion has its own potential cost**:

if the body of the function being expanded takes more instructions than the function call being replaced, then each inline expansion will **cause the executable to grow larger**. Larger executables tend to be slower (due to **not fitting as well in memory caches**).

The decision about whether a function would benefit from being made inline (because removal of the function call overhead outweighs the cost of a larger executable) **is not straightforward**. Inline expansion **could result in performance improvements, performance reductions, or no change to performance at all**, depending on the relative cost of a function call, the size of the function, and what other optimizations can be performed.

Inline expansion is best suited to simple, short functions (e.g. **no more than a few statements**), especially cases where a single function call can be executed more than once (e.g. function calls inside a loop).

7.7.4 *When inline expansion occurs*

Every function falls into one of two categories, where **calls to the function**:

- **May be expanded** (most functions are in this category).
- **Can't be expanded**.

Most functions fall into the “**may**” category: their function calls **can be expanded if and when it is beneficial to do so**. For functions in this category, a modern **compiler will assess each function and each function call** to make a determination about whether that particular function call would benefit from inline expansion. A **compiler might decide to expand none, some, or all of the function calls** to a given function.

In short: **Modern optimizing compilers make the decision about when functions should be expanded inline**.

7.7.5 *The inline keyword, historically*

Historically, **compilers either didn't have the capability to determine whether inline expansion would be beneficial, or were not very good at it.** For this reason, **C++ provided the keyword inline**, which was **originally intended to be used as a hint to the compiler** that a function would (probably) benefit from being expanded inline.

A function that is declared using the inline keyword is called an inline function.

However, in modern C++, the inline keyword is no longer used to request that a function be expanded inline. There are quite a few reasons for this:

- **Using inline to request inline expansion is a form of premature optimization, and misuse could actually harm performance.**
- **The inline keyword is just a hint -- the compiler is completely free to ignore a request to inline a function.** This is likely to be the result if you try to inline a lengthy function! The compiler is also free to perform inline expansion of functions that do not use the inline keyword as part of its normal set of optimizations.
- **The inline keyword is defined at the wrong level of granularity. We use the inline keyword on a function definition, but inline expansion is actually determined per function call.** It may be beneficial to expand some function calls and detrimental to expand others, and there is no syntax to influence this.

Best practice:

Do not use the inline keyword to request inline expansion for your functions. Modern C++ compilers will do the work for you either way.

7.7.6 *The inline keyword, modernly*

In previous chapters, we mentioned that you should not implement functions (with external linkage) in header files, because when those headers are included into multiple .cpp files, the function definition will be copied into multiple .cpp files. These files will then be compiled, and the linker will throw an error because it will note that you've defined the same function more than once, which is a violation of the one-definition rule.

In modern C++, the term inline has evolved to mean “multiple definitions are allowed”. Thus, an inline function is one that is **allowed to be defined in multiple translation units (without violating the ODR)**.

Inline functions have **two primary requirements**:

- The compiler needs to be able to see the full definition of an inline function in each translation unit where the function is used (a forward declaration will not suffice on its own). The definition can occur after the point of use if a forward declaration is also provided. **Only one such definition can occur per translation unit, otherwise a compilation error will occur.**
- Every **definition for an inline function must be identical, otherwise undefined behavior will result.**

Important rule: The **compiler needs to** be able to **see the full definition** of an inline function **wherever it is used**, and **all such definitions must be identical** (or undefined behavior will result).

The **linker will consolidate all inline function definitions for an identifier into a single definition** (thus still meeting the requirements of the one definition rule).

main.cpp:

```
1 | #include <iostream>
2 |
3 | double circumference(double radius); // forward declaration
4 |
5 | inline double pi() { return 3.14159; }
6 |
7 | int main()
8 | {
9 |     std::cout << pi() << '\n';
10 |    std::cout << circumference(2.0) << '\n';
11 |
12 |    return 0;
13 | }
```

math.cpp

```
1 | inline double pi() { return 3.14159; }
2 |
3 | double circumference(double radius)
4 | {
5 |     return 2.0 * pi() * radius;
6 | }
```

Notice that both files have a definition for function `pi()` -- however, **because this function has been marked as inline, this is acceptable**, and the linker will **de-duplicate** them. If you remove the `inline` keyword from both definitions of `pi()`, you'll get an ODR violation (as duplicate definitions for non-inline functions are disallowed).

Inline functions are **typically defined in header files**, where they can be **#included into the top of any code file that needs to see the full definition of the identifier**. This ensures that all inline definitions for an identifier are identical.

pi.h:

```
1 | #ifndef PI_H
2 | #define PI_H
3 |
4 | inline double pi() { return 3.14159; }
5 |
6 | #endif
```

main.cpp:

```
1 | #include "pi.h" // will include a copy of pi() here
2 | #include <iostream>
3 |
4 | double circumference(double radius); // forward declaration
5 |
6 | int main()
7 | {
8 |     std::cout << pi() << '\n';
9 |     std::cout << circumference(2.0) << '\n';
10 |
11 |     return 0;
12 | }
```

Math.cpp

```
1 | #include "pi.h" // will include a copy of pi() here
2 |
3 | double circumference(double radius)
4 | {
5 |     return 2.0 * pi() * radius;
6 | }
```

This is particularly useful for **header-only libraries**, which are one or more header files that implement some capability (no .cpp files are included). Header-only libraries are popular because there are no source files that need to be added to a project to use them and nothing that needs to be linked. You simply #include the header-only library and then can use it.

Best practice:

Avoid the use of the inline keyword unless you have a specific, compelling reason to do so (e.g. you're defining those functions or variables in a header file).

7.7.7 *Why not make all functions inline and defined in a header file?*

There are a few good reasons:

- **It can increase your compile times.** When a function in a code file changes, only that code file needs to be recompiled. **When an inline function in a header file changes, every code file that includes that header (either directly or via another header) needs to be recompiled.** On large projects, this can have a drastic impact.

- It can lead to more naming collisions since you'll end up with more code in a single code file.

7.7.8 *Inline variables*

In the above example, `pi()` was written as a function that returns a constant value. **It would be more straightforward if `pi` were implemented as a `(const)` variable instead.** However, prior to C++17, there were some obstacles and inefficiencies in doing so.

C++17 introduces inline variables, which are **variables that are allowed to be defined in multiple files**. Inline variables work similarly to inline functions, and **have the same requirements** (the compiler must be able to see an identical full definition everywhere the variable is used).

7.7.9 *Summary and implicit inlines*

Functions calls contain **overhead** and this increases execution time. Only consider inline if you have a small function (**small enough to outweigh the overhead cost**) that you know will be called many times.

The inline keyword is **no more than the suggestion** (for the compiler) to inline something. The **compiler can still inline or not inline the function**.

Also note that **inline expansions have their own potential cost**:

- If not done correctly it **could result in either performance improvements, reductions or no changes to the performance at all**.
- It can **increase compile times**
- Once an inline function **has changes in a header file, every translation unit** using that file **has to be recompiled**
- In the long run **you could end up with naming collisions** because the **amount of code across files increases**

Historically compilers didn't know how to pick, choose or detect functions the could be expanded with inline. Hence why **historically inline was used to hint the compiler to inline a function**.

However, **as of today** inline is **used to allow multiple definitions**. So, inline functions are actually **allowed to be defined in multiple translation units without violating ODR** (the linker will actually de-duplicate them).

There's still **rules** to how many times it can be included:

- Once per translation unit
- Every definition of the inline function has to be identical

In general, it is **best practice to avoid the inline keyword** unless you have a specific reason to do so.

Inline variables work similarly to inline functions, and have the same requirements (the compiler must be able to see an identical full definition everywhere the variable is used).

Advanced:

The following are **implicitly inlined functions**:

- Functions defined inside a class, struct, or union type definition
- **constexpr / consteval functions**
- Functions implicitly instantiated from function templates

The following are **implicitly inlined variables**:

- Static **constexpr data members**

Unlike constexpr functions, constexpr variables are not inline by default (excepting those noted above)!

7.8 constexpr functions (run or compile-time) and consteval functions (compile-time)

Earlier we introduced the **constexpr** keyword, which we used to create **compile-time (symbolic) constants**. We also introduced **constant expressions**, which are **expressions that can be evaluated at compile-time rather than runtime**.

One challenge with constant expressions is that a function call to a normal function is allowed in constant expressions. This means we cannot use such function calls anywhere a constant expression is required.

Consider the following program:

```
1  #include <iostream>
2
3  int main()
4  {
5      constexpr double radius { 3.0 };
6      constexpr double pi { 3.14159265359 };
7      constexpr double circumference { 2.0 * radius * pi };
8
9      std::cout << "Our circle has circumference " << circumference << "\n";
10
11     return 0;
12 }
```

Having a complex initializer for circumference isn't great (and requires us to instantiate two supporting variables, radius and pi). Making it into a function instead results in much cleaner code.

```
constexpr double circumference { calcCircumference(3.0) }; // compile error
```

Unfortunately, it also **doesn't compile**. Cpp variable circumference **requires that its initializer is a constant expression**, and the call **calcCircumference()** **isn't a constant expression**. In this particular case, **we could make circumference non-constexpr**, and the program would compile. While we'd lose the benefits of constant expressions, at least the program would run.

However, there are other cases in C++ (which we'll introduce in the future) where we do not have alternate options available, and only a constant expression will do. **In those cases, we'd really like to be able to use functions**, but calls to normal functions just won't work.

So, what are we to do? → Cpp or consteval functions

7.8.1 *Constexpr functions can be used in constant expressions*

Here's the same example as above, but using a constexpr function:

```
1  #include <iostream>
2
3  constexpr double calcCircumference(double radius) // now a constexpr function
4  {
5      constexpr double pi { 3.14159265359 };
6      return 2.0 * pi * radius;
7  }
8
9  int main()
10 {
11     constexpr double circumference { calcCircumference(3.0) }; // now compiles
12
13     std::cout << "Our circle has circumference " << circumference << "\n";
14
15     return 0;
16 }
```

Because calcCircumference() is now a constexpr function, it can be used in a constant expression, such as the initializer of circumference.

7.8.2 *Constexpr functions can be evaluated at compile time*

In Constant expressions and compile-time optimization, we noted that in contexts that require a constant expression (such as the initialization of a constexpr variable), a constant expression is required to evaluate at compile-time. **If a required constant expression contains a constexpr function call, that constexpr function call must evaluate at compile-time.**

In our example above, variable circumference is constexpr and thus requires a constant expression initializer. Since calcCircumference() is part of this required constant expression, calcCircumference() must be evaluated at compile-time.

When a function call is evaluated at compile-time, the compiler will calculate the return value of the function call at compile-time, and then replace the function call with the return value.

To evaluate at compile-time, two other things must also be true:

- The call to the constexpr function must have **arguments that are known at compile time** (e.g. are **constant expressions**).
- **All statements and expressions within the constexpr function must be evaluable at compile-time.**

When a constexpr (or consteval) function is being evaluated at compile-time, **any other functions it calls are required to be evaluated at compile-time (otherwise the initial function would not be able to return a result at compile-time).**

For advanced readers: There are some other lesser encountered criteria as well. These can be found on the cppreference website.

7.8.3 *Constexpr functions can also be evaluated at runtime*

Constexpr functions can also be evaluated at runtime, in which case they will return a non-constexpr result.

Key insight: When a constexpr function evaluates at runtime, it evaluates just like a normal (non-constexpr) function would. In other words, the **constexpr has no effect in this case.**

Allowing functions with a constexpr return type to be evaluated at either compile-time or runtime was allowed so that a single function can serve both cases.

Otherwise, you'd need to have separate functions (a function with a constexpr return type, and a function with a non-constexpr return type). **This would not only require duplicate code, the two functions would also need to have different names!**

Note that even **non-constexpr functions could be evaluated at compile-time** under the as-if rule!

Key insight:

Put another way, we can categorize the **likelihood that a function will actually be evaluated at compile-time** as follows:

Always (required by the standard):

- Constexpr function is called **where constant expression is required**.
- Constexpr function is called **from other function being evaluated at compile-time**.

Probably (there's little reason not to):

- Constexpr function is called **where constant expression isn't required, all arguments are constant expressions**.

Possibly (if optimized under the as-if rule):

- Constexpr function is called **where constant expression isn't required, some arguments are not constant expressions but their values are known at compile-time**.
- **Non-constexpr function** capable of being evaluated at compile-time, **all arguments are constant expressions**.

Never (not possible):

- Constexpr function is called **where constant expression isn't required, some arguments have values that are not known at compile-time**.

Note that **your compiler's optimization level setting may have an impact on whether it decides to evaluate a function at compile-time or runtime**. This also means that your compiler **may make different choices for debug vs. release builds** (as debug builds typically have optimizations turned off).

For example, both gcc and Clang will not compile-time evaluate a constexpr function called where a constant expression isn't required unless the compiler told to optimize the code (e.g. using the -O2 compiler option).

For advanced readers: The **compiler might also choose to inline a function call, or even optimize a function call away entirely**. Both of these can affect when (or if) the content of the function call are evaluated.

7.8.4 What about `std::is_constant_evaluated` or `if constexpr`? (Advanced)

C++ does not currently provide any reliable mechanisms to determine if a constexpr function call is evaluating at compile-time or runtime.

Neither of these capabilities tell you whether a function call is evaluating at compile-time or runtime.

`std::is_constant_evaluated()` (defined in the `<type_traits>` header) **returns a bool indicating whether the current function is executing in a constant-evaluated context**. A constant-evaluated context (also called a **constant context**) is **defined as one in which a constant expression is required** (such as the initialization of a constexpr variable). So, in cases where the compiler is required to evaluate a constant expression at compile-time `std::is_constant_evaluated()` will true as expected.

However, the compiler may also choose to evaluate a constexpr function at compile-time in a context that does not require a constant expression. In such cases, `std::is_constant_evaluated()` will return false even though the function did evaluate at compile-time. So `std::is_constant_evaluated()` really means “the compiler is being forced to evaluate this at compile-time”, not “this is evaluating at compile-time”.

While this may seem strange, there are several reasons for this:

- As the paper that proposed this feature indicates, the standard doesn’t actually make a distinction between “compile time” and “runtime”. Defining behavior involving that distinction would have been a larger change.
- If `std::is_constant_evaluated()` returned true when a function was evaluated at compile-time for any reason, the optimizer (acting under the as-if rule) would potentially be able to change the behavior of a program by optionally evaluating a function at compile time (which would be a violation of the as-if rule). While this could be addressed in various ways, those involve adding additional complexity to the optimizer and/or limiting its ability to optimize certain cases.

Introduced in C++23, `if constexpr` is a replacement for `if (std::is_constant_evaluated())` that provides a nicer syntax and fixes some other issues. However, it evaluates the same way.

7.8.5 *Forcing a constexpr function to be evaluated at compile-time*

There is no way to tell the compiler that a constexpr function should prefer to evaluate at **compile-time whenever it can** (e.g. in cases where the return value of a constexpr function is used in a non-constant expression).

However, we can force a constexpr function that is eligible to be evaluated at compile-time to **actually evaluate at compile-time by ensuring the return value is used where a constant expression is required**. This needs to be done on a per-call basis.

The most common way to do this is to **use the return value to initialize a constexpr variable** (this is why we've been using variable 'g' in prior examples). **Unfortunately, this requires introducing a new variable into our program just to ensure compile-time evaluation, which is ugly and reduces code readability.**

7.8.6 *Consteval functions (compile-time)*

C++20 introduces the keyword **constexpr**, which is used to indicate that a function must evaluate at compile-time, otherwise a compile error will result. Such functions are called **immediate functions**.

Best practice: Use **constexpr** if you have a function that must evaluate at compile-time for some reason (e.g. because it does something that can only be done at compile time).

Perhaps **surprisingly, the parameters of a constexpr function are not constexpr** (even though constexpr functions can only be evaluated at compile-time). This decision was made for the sake of consistency.

7.8.7 Using consteval to make constexpr execute at compile-time

The **downside** of consteval functions is that **such functions can't evaluate at runtime**, making them **less flexible than constexpr functions**, which can do either. Therefore, it would still be useful to have a convenient way to force constexpr functions to evaluate at compile-time (even when the return value is being used where a constant expression is not required), so that we could have compile-time evaluation when possible, and runtime evaluation when we can't.

Consteval functions provides a way to make this happen, using a neat helper function:

```

1  #include <iostream>
2
3  // Uses abbreviated function template (C++20) and 'auto' return type to make this function work with any type of value
4  // See 'related content' box below for more info (you don't need to know how these work to use this function)
5  consteval auto compileTimeEval(auto value)
6  {
7      return value;
8  }
9
10 constexpr int greater(int x, int y) // function is constexpr
11 {
12     return (x > y ? x : y);
13 }
14
15 int main()
16 {
17     std::cout << greater(5, 6) << '\n'; // may or may not execute at compile-time
18     std::cout << compileTimeEval(greater(5, 6)) << '\n'; // will execute at compile-time
19
20     int x { 5 };
21     std::cout << greater(x, 6) << '\n'; // we can still call the constexpr version at runtime if we wish
22
23     return 0;
24 }
```

This works because consteval functions require constant expressions as arguments -- therefore, if we use the return value of a constexpr function as an argument to a consteval function, the constexpr function must be evaluated at compile-time! The consteval function just returns this argument as its own return value, so the caller can still use it.

Note that the **consteval function returns by value**. While this might be inefficient to do at runtime (if the value was some type that is expensive to copy, e.g. `std::string`), in a compile-time context, it doesn't matter because the entire call to the consteval function will simply be replaced with the calculated return value.

7.8.8 *Constexpr/constexpr functions can use non-const local variables*

Within a constexpr or constexpr function, we can use local variables that are not constexpr, and the value of these variables can be changed.

```

2
3  constexpr int doSomething(int x, int y) // function is constexpr
4  {
5      x = x + 2;      // we can modify the value of non-const function parameters
6
7      int z { x + y }; // we can instantiate non-const local variables
8      if (x > y)
9          z = z - 1;  // and then modify their values
10
11      return z;
12  }
13

```

When such functions are evaluated at compile-time, the compiler will essentially “execute” the function and return the calculated value.

7.8.9 *Constexpr/constexpr functions can use function parameters and local variables as arguments in constexpr function calls*

Above, we noted, “When a constexpr (or constexpr) function is being evaluated at compile-time, any other functions it calls are required to be evaluated at compile-time.”

Perhaps surprisingly, a constexpr or constexpr function can use its function parameters (which aren’t constexpr) or even local variables (which may not be const at all) as arguments in a constexpr function call. When a constexpr or constexpr function is being evaluated at compile-time, the value of all function parameters and local variables **must be known to the compiler** (otherwise it couldn’t evaluate them at compile-time). Therefore, in this specific context, C++ allows these values to be used as arguments in a call to a constexpr function, and that constexpr function call can still be evaluated at compile-time.

```

1  #include <iostream>
2
3  constexpr int goo(int c) // goo() is now constexpr
4  {
5      return c;
6  }
7
8  constexpr int foo(int b) // b is not a constant expression within foo()
9  {
10     return goo(b);      // if foo() is resolved at compile-time, then 'goo(b)' can also be resolved at compile-time
11 }
12
13 int main()
14 {
15     std::cout << foo(5);
16
17     return 0;
18 }

```

In the above example, `foo(5)` may or may not be evaluated at compile time. If it is, then the compiler knows that `b` is 5. And even though `b` is not `constexpr`, the compiler can treat the call to `goo(b)` as if it were `goo(5)` and evaluate that function call at compile-time. If `foo(5)` is instead resolved at runtime, then `goo(b)` will also be resolved at runtime.

7.8.10 Can a `constexpr` function call a non-`constexpr` function?

The answer is yes, but **only when the `constexpr` function is being evaluated in a non-constant context**. A non-`constexpr` function may not be called when a `constexpr` function is evaluating in a constant context (because then the `constexpr` function wouldn't be able to produce a compile-time constant value), and doing so will produce a compilation error.

Calling a non-`constexpr` function is allowed so that a `constexpr` function can do something like this:

```
1 #include <type_traits> // for std::is_constant_evaluated
2
3 constexpr int someFunction()
4 {
5     if (std::is_constant_evaluated()) // if evaluating in constant context
6         return someConstexprFcn();
7     else
8         return someNonConstexprFcn();
9 }
```

Now consider this variant:

```
1 constexpr int someFunction(bool b)
2 {
3     if (b)
4         return someConstexprFcn();
5     else
6         return someNonConstexprFcn();
7 }
```

This is **legal as long as `someFunction(false)` is never called in a constant expression**.

For best results, we'd advise the following:

- **Avoid calling non-`constexpr` functions from within a `constexpr` function** if possible.
- If your `constexpr` function requires **different behavior for constant and non-constant contexts**, **conditionalize the behavior with `if (std::is_constant_evaluated())`** (in C++20) or **`if constexpr`** (C++23 onward).
- **Always test your `constexpr` functions in a constant context**, as they may work when called in a non-constant context but fail in a constant context.

7.8.11 *Why not constexpr every function?*

There are a few reasons you may not want to constexpr a function:

- constexpr signals that a function can be used in a constant expression. **If your function cannot be evaluated as part of a constant expression, it should not be marked as constexpr.**
- **constexpr is part of the interface of a function.** Once a function is made constexpr, it can be called by other constexpr functions or used in contexts that require constant expressions. **Removing the constexpr later will break such code.**
- **constexpr makes functions harder to debug** because you **can't inspect them at runtime.**

7.8.12 *Why constexpr a function when it is not actually evaluated at compile-time?*

New programmers sometimes ask, “why should I constexpr a function when it is only evaluated at runtime in my program (e.g. **because the arguments in the function call are non-const**)”?

There are a few reasons:

- There's **little downside to using constexpr**, and it **may help the compiler optimize your program** to be smaller and faster.
- Just because you're not calling the function in a compile-time evaluable context right now **doesn't mean you won't call it in such a context when you modify or extend your program.** And if you haven't constexpr'd the function already, you may not think to when you do start to call it in such a context, and then you'll miss out on the performance benefits. Or you may be forced to constexpr it later when you need to use the return value in a context that requires a constant expression somewhere.
- **Repetition helps ingrain best practices.**

Best practice:

Unless you have a specific reason not to, a **function that can evaluate as part of a constant expression should be made constexpr.**

A function that cannot be evaluated as part of a constant expression should not be marked as constexpr.

7.9 Introduction to `std::string`

“This is a C-style string literal!”

While **C-style string literals** are fine to use, **C-style string variables behave oddly**, are **hard to work with** (e.g. you **can’t use assignment to assign a C-style string variable a new value**), and are **dangerous** (e.g. **if you copy a larger C-style string into the space allocated for a shorter C-style string, undefined behavior will result**). In modern C++, **C-style string variables are best avoided**.

Fortunately, **C++ has introduced two additional string types** into the language that are much easier and safer to work with: **`std::string` and `std::string_view`** (C++17). Unlike the types we’ve introduced previously, **`std::string` and `std::string_view` aren’t fundamental types (they’re class types**, which we’ll cover in the future). However, **basic usage of each is straightforward and useful enough** that we’ll introduce them here.

7.9.1 *Introducing `std::string`*

The easiest way to work with strings and string objects in C++ is via the **`std::string` type**, which lives in the **`<string>` header**.

We can **create objects of type `std::string` just like other objects**.

Assignment, output behave just like string types would in most other languages.

Key insight:

If `std::string` doesn’t have enough memory to store a string, it will request additional memory (at runtime) using a form of memory allocation known as dynamic memory allocation. This ability to acquire additional memory is part of what **makes `std::string` so flexible, but also comparatively slow**.

We cover dynamic memory allocation in a future chapter.

7.9.2 String input with `std::cin`

Using `std::string` with `std::cin` may yield some surprises! Consider the following example:

```
4 int main()
5 {
6     std::cout << "Enter your full name: ";
7     std::string name{};
8     std::cin >> name; // this won't work as expected since std::cin breaks on whitespace
9
10    std::cout << "Enter your favorite color: ";
11    std::string color{};
12    std::cin >> color;
13
14    std::cout << "Your name is " << name << " and your favorite color is " << color << '\n';
15
16    return 0;
17 }
```

Output:

Enter your full name: John Doe

Enter your favorite color: Your name is John and your favorite color is Doe

It turns out that when using operator<> to extract a string from `std::cin`, **operator<> only returns characters up to the first whitespace it encounters**. Any **other characters are left inside `std::cin`**, waiting for the next extraction.

So, when we used operator<> to extract input into variable `name`, only "John" was extracted, leaving " Doe" inside `std::cin`. When we then used operator<> to get extract input into variable `color`, it extracted "Doe" instead of waiting for us to input a color. Then the program ends.

7.9.3 Use `std::getline()` to input text

To read a full line of input into a string, you're better off using the `std::getline()` function instead. `std::getline()` requires two arguments: the first is `std::cin`, and the second is your string variable.

```
1 #include <iostream>
2 #include <string> // For std::string and std::getline
3
4 int main()
5 {
6     std::cout << "Enter your full name: ";
7     std::string name{};
8     std::getline(std::cin >> std::ws, name); // read a full line of text into name
9 }
```


7.9.4 *What is std::ws? (input manipulator)*

In Floating point numbers, **we discussed output manipulators**, which allow us to **alter the way output is displayed**. In that lesson, we used the output manipulator function **std::setprecision()** to change the number of digits of precision that **std::cout** displayed.

C++ also supports input manipulators, which **alter the way that input is accepted**. The **std::ws** input manipulator tells **std::cin** to **ignore any leading whitespace before extraction**. Leading whitespace is any whitespace character (spaces, tabs, newlines) that occur at the start of the string.

Best practice:

If using **std::getline()** to read strings, **use std::cin >> std::ws** input manipulator to **ignore leading whitespace**. This **needs to be done for each std::getline() call**, as **std::ws** is not preserved across calls.

Key insight:

When extracting to a variable, the **extraction operator (>>)** **ignores leading whitespace**. It **stops extracting when encountering non-leading whitespace**.

std::getline() **does not ignore leading whitespace**. If you want it to ignore leading whitespace, **pass std::cin >> std::ws as the first argument**. It stops extracting when encountering a newline.

7.9.5 The length of a `std::string`

If we want to know how many characters are in a `std::string`, we can ask a `std::string` object for its length. The syntax for doing this is different than you've seen before, but is pretty straightforward:

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
6     std::string name{ "Alex" };
7     std::cout << name << " has " << name.length() << " characters\n";
8
9     return 0;
10 }
```

Although `std::string` is required to be null-terminated (as of C++11), the **returned length of a `std::string` does not include the implicit null-terminator** character.

Note that **instead of asking** for the string length as `length(name)`, **we say `name.length()`**. The `length()` function isn't a normal standalone function -- it's a **special type of function that is nested within `std::string` called a member function**. Because the `length()` member function is declared inside of `std::string`, it is sometimes written as `std::string::length()` in documentation.

We'll cover member functions, including how to write your own, in more detail later.

Key insight:

With **normal functions**, we call `function(object)`. With **member functions**, we call `object.function()`.

Also note that `std::string::length()` returns an unsigned integral value (most likely of type `size_t`). **If you want to assign the length to an int variable**, you should `static_cast` it to avoid compiler warnings about signed/unsigned conversions:

```
1 | int length { static_cast<int>(name.length()) };
```

In C++20, you can also use the `std::ssize()` function to get the length of a `std::string` as a large **signed integral type** (usually `std::ptrdiff_t`). Since a `ptrdiff_t` may be larger than an `int`, if you want to store the result of `std::ssize()` in an `int` variable, **you should `static_cast` the result to an `int`**.

7.9.6 *Initializing a std::string is expensive*

Whenever a `std::string` is initialized, a copy of the string used to initialize it is made. Making copies of strings is expensive, so **care should be taken to minimize the number of copies** made.

7.9.7 *Do not pass std::string by value*

When a `std::string` is passed to a function by value, the `std::string` function parameter must be instantiated and initialized with the argument. This **results in an expensive copy**.

Best practice:

Do not pass `std::string` by value, as it makes an expensive copy. **In most cases, use a `std::string_view` parameter instead.**

7.9.8 *Returning a std::string*

In C++, when a function returns a value, you might be concerned that returning a `std::string` by value could be expensive because it would involve copying the string. However, modern C++ compilers optimize this process using techniques like **Return Value Optimization (RVO)** and **move semantics**, making it efficient to return `std::string` by value in many **common scenarios**.

As a **rule of thumb**, it is okay to return a `std::string` by value in following scenarios:

- A **local (to the function)** variable of type `std::string`. (**RVO**)
- A `std::string` returned by another function call or operator (**move semantics**).
- A `std::string` temporary that is created as part of the return statement. (**RVO or move semantics**)

For advanced readers:

`std::string` supports a capability called **move semantics**, which allows an object that will be destroyed at the end of the function to instead be returned by value without making a copy.

How move semantics works is beyond the scope of this introductory article, but is something we introduce in lesson: Returning `std::vector`, and an introduction to move semantics. **In most other cases, prefer to avoid returning a `std::string` by value**, as doing so will make an expensive copy.

7.9.9 Literals for `std::string`

Double-quoted string literals (like “Hello, world!”) are C-style strings by default (and thus, have a strange type).

We can create string literals with type `std::string` by using a **s suffix** after the double-quoted string literal. The s must be lower case.

```
1  #include <iostream>
2  #include <string> // for std::string
3
4  int main()
5  {
6      using namespace std::string_literals; // easy access to the s suffix
7
8      std::cout << "foo\n"; // no suffix is a C-style string literal
9      std::cout << "goo\n"s; // s suffix is a std::string literal
10
11     return 0;
12 }
```

Tip: The “s” suffix lives in the `namespace std::literals::string_literals`.

The most concise way to access the literal suffixes is via using-directive using namespace `std::literals`. However, this imports all of the standard library literals into the scope of the using-directive, which brings in a bunch of stuff you probably aren’t going to use.

We recommend using namespace `std::string_literals`, which imports only the literals for `std::string`.

We discuss using-directives in the future. This is one of the exception cases where using an entire namespace is generally okay, because the suffixes defined within are unlikely to collide with any of your code. **Avoid such using-directives outside of functions in header files.**

You probably won’t need to use `std::string` literals very often (**as it’s fine to initialize a `std::string` object with a C-style string literal**), but we’ll see a few cases in future lessons (involving type deduction) where using `std::string` literals instead of C-style string literals makes things easier.

For advanced readers:

`"Hello"s` resolves to `std::string { "Hello", 5 }` which creates a temporary `std::string` initialized with C-style string literal “Hello” (which has a length of 5, excluding the implicit null-terminator).

7.9.10 *Constexpr strings*

If you try to define a constexpr std::string, your compiler will probably generate an error. This happens because **constexpr std::string isn't supported at all in C++17 or earlier, and only works in very limited cases in C++20/23**. If you need constexpr strings, **use std::string_view instead**.

7.9.11 *Conclusion*

std::string is complex, leveraging many language features that we haven't covered yet. Fortunately, you don't need to understand these complexities to use std::string for simple tasks, like basic string input and output. **We encourage you to start experimenting with strings now, and we'll cover additional string capabilities later.**

7.10 Introduction to std::string_view

Consider the following program:

```
1 #include <iostream>
2
3 int main()
4 {
5     int x { 5 }; // x makes a copy of its initializer
6     std::cout << x << '\n';
7
8     return 0;
9 }
```

When the definition for `x` is executed, the initialization value 5 is copied into the memory allocated for variable `int x`. **For fundamental types, initializing and copying a variable is fast.**

Now consider this similar program:

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
6     std::string s{ "Hello, world!" }; // s makes a copy of its initializer
7     std::cout << s << '\n';
8
9     return 0;
10 }
```

When `s` is initialized, the C-style string literal "Hello, world!" is copied into memory allocated for `std::string s`. **Unlike fundamental types, initializing and copying a `std::string` is slow.**

In the above program, all we do with `s` is print the value to the console, and then `s` is destroyed. **We've essentially made a copy of "Hello, world!" just to print and then destroy that copy. That's inefficient.**

We see something similar in this example:

```
1  #include <iostream>
2  #include <string>
3
4  void printString(std::string str) // str makes a copy of its initializer
5  {
6      std::cout << str << '\n';
7  }
8
9  int main()
10 {
11     std::string s{ "Hello, world!" }; // s makes a copy of its initializer
12     printString(s);
13
14     return 0;
15 }
```

This example makes two copies of the C-style string “Hello, world!”: one when we initialize `s` in `main()`, and another when we initialize parameter `str` in `printString()`. That’s a lot of needless copying just to print a string!

7.10.1 `std::string_view`

To address the issue with `std::string` being expensive to initialize (or copy), C++17 introduced `std::string_view` (which lives in the `<string_view>` header). **`std::string_view` provides read-only access to an existing string (C-style, `std::string`, or `std::string_view`) without making a copy.** Read-only means that we can access and use the value being viewed, but we cannot modify it.

The following example is identical to the prior one, except we’ve replaced `std::string` with `std::string_view`. **Read-only** means that we can access and use the value being viewed, but we cannot modify it.

The following example is identical to the prior one, except we've replaced `std::string` with `std::string_view`:

```
1 #include <iostream>
2 #include <string_view> // C++17
3
4 // str provides read-only access to whatever argument is passed in
5 void printSV(std::string_view str) // now a std::string_view
6 {
7     std::cout << str << '\n';
8 }
9
10 int main()
11 {
12     std::string_view s{ "Hello, world!" }; // now a std::string_view
13     printSV(s);
14
15     return 0;
16 }
```

This program produces the same output as the prior one, but no copies of the string “Hello, world!” are made.

When we initialize `std::string_view s` with C-style string literal “Hello, world!”, `s` provides read-only access to “Hello, world!” without making a copy of the string. **When we pass `s` to `printSV()`, parameter `str` is initialized from `s`. This allows us to access “Hello, world!” through `str`, again without making a copy of the string.**

Best practice:

Prefer **`std::string_view`** over `std::string` when you need a read-only string, especially for function parameters.

7.10.2 *`std::string_view` can be initialized with many different types of strings*

One of the neat things about a `std::string_view` is how flexible it is. A `std::string_view` object can be initialized with:

- a C-style string
- a `std::string`
- a `std::string_view`

std::string_view parameters will accept many different types of string arguments

Both a C-style string and a std::string will implicitly convert to a std::string_view. Therefore, a std::string_view parameter will accept arguments of type C-style string, a std::string, or std::string_view.

7.10.3 std::string_view will not implicitly convert to std::string

Because std::string makes a copy of its initializer (which is expensive), C++ won't allow implicit conversion of a std::string_view to a std::string. This is to prevent accidentally passing a std::string_view argument to a std::string parameter, and inadvertently making an expensive copy where such a copy may not be required.

However, if this is desired, we have two options:

- Explicitly create a std::string with a std::string_view initializer (which is allowed, since this will rarely be done unintentionally)
- Convert an existing std::string_view to a std::string using static_cast

7.10.4 Assignment changes what the std::string_view is viewing

Assigning a new string to a std::string_view causes the std::string_view to view the new string. It does not modify the prior string being viewed in any way.

7.10.5 Literals for std::string_view

Double-quoted string literals are C-style string literals by default. We can create string literals with type std::string_view by using a **sv suffix** after the double-quoted string literal. The sv must be lower case.

```
std::cout << "foo\n"; // no suffix is a C-style string literal
std::cout << "goo\n"s; // s suffix is a std::string literal
std::cout << "moo\n"sv; // sv suffix is a std::string_view literal
```

7.10.6 *constexpr std::string_view*

Unlike `std::string`, **`std::string_view` has full support for `constexpr`**. This makes `constexpr std::string_view` the preferred choice when string symbolic constants are needed.

7.11 Safety guidelines: `std::string` & `std::string_view`

Because `std::string_view` is our first encounter with a **view type**, we're going to spend some additional time discussing it further. We will **focus on how to use `std::string_view` safely, and provide some examples illustrating how it can be used incorrectly**. We'll conclude with some **guidelines on when to use `std::string` vs `std::string_view`**.

7.11.1 *An introduction to owners and viewers*

Let's sidebar into an analogy for a moment. Say you've decided that you're going to paint a picture of a bicycle. But you don't have a bicycle! What are you to do?

Well, you could go to the local cycle shop and buy one. You would own that bike. This has some benefits: you now have a bike that you can ride. You can guarantee the bike will always be available when you want it. You can decorate it, or move it. There are also some downsides to this choice. Bicycles are expensive. And if you buy one, you are now responsible for it. You have to periodically maintain it. And when you eventually decide you don't want it any more, you have to properly dispose of it.

Ownership can be expensive. As an owner, **it is your responsibility to acquire, manage, and properly dispose of the objects you own.**

On your way out of the house, you glance out your window front. You notice that your neighbor has parked their bike across from your window. You could just paint a picture of your neighbor's bike (as seen from your window) instead. Sparing you from the expenses, maintenance and disposing of it. When you are done viewing, you can just shut your curtains and move on with your life. This ends your view of the object, but the object itself is not affected by this. There are also some potential downsides to this choice. You can't paint or customize your neighbor's bike. And while you are viewing the bike, your neighbor may decide to change the way the bike looks, or move it out of your view altogether. **You may end up with a view of something unexpected instead.**

Viewing is inexpensive. As a viewer, **you have no responsibility for the objects you are viewing, but you also have no control over those objects.**

7.11.2 Owner `std::string`

`std::string` is an owner, you might be wondering why `std::string` makes an expensive copy of its initializer. When an object is instantiated, memory is allocated for that object to store whatever data it needs to use throughout its lifetime. This memory is reserved for the object, and guaranteed to exist for as long as the object does. It is a safe space. `std::string` (and most other objects) copies the initialization value they are given into this memory so that they can have their own independent value to access and manipulate later. Once the initialization value has been copied, the object is no longer reliant on the initializer in any way.

And **that's a good thing, because the initializer generally can't be trusted after initialization is complete**. If you imagine the initialization process as a function call that initializes the object, who is passing in the initializer? The caller. When initialization is done, control returns back to the caller. At this point, the initialization statement is complete, and one of two things will typically happen:

- If the **initializer was a temporary** value or object, that temporary **will be destroyed immediately**.
- If the **initializer was a variable**, the caller still has access to that object. The **caller can then do whatever it wants with the object, including modify or destroy it**.

Key insight: An initialized object has no control over what happens to the initialization value after initialization is finished.

Because `std::string` makes its own copy of the value, it doesn't have to worry about what happens after initialization is finished. The initializer can be destroyed, or modified, and it doesn't affect the `std::string`. The downside is that this independence comes with the cost of an expensive copy.

In the context of our analogy, `std::string` is an owner. It has its own data that it manages. And when it is destroyed, it cleans up after itself.

7.11.3 *We don't always need a copy*

```
void printString(std::string str) // str makes a copy of its initializer
```

This is expensive, there's essentially three simple rules:

- Could the string variable be destroyed while the function variable is using it?
- Could the string variable be modified while the function variable is using it?
- Does the string variable modify the string in some way that the caller won't expect ?

Essentially when all three of the above criteria are false, there wouldn't be any reason to make a new (local) copy of the string variable.

7.11.4 *std::string_view is a viewer*

std::string_view takes a different approach to initialization. Instead of making an expensive copy of the initialization string, **std::string_view creates an inexpensive view of the initialization string.** The `std::string_view` can then be used whenever access to the string is required.

In the context of our analogy, `std::string_view` is a viewer. It views an object that already exists elsewhere, and cannot modify that object. When the view is destroyed, the object being viewed is not affected.

It is important to note that a `std::string_view` remains dependent on the initializer through its lifetime. If the string being viewed is modified or destroyed while the view is still being used, unexpected or undefined behavior will result.

Whenever we use a view, it is up to us to ensure these possibilities do not occur.

Warning & Key insight: A view is dependent on the object being viewed. If the object being viewed is modified or destroyed while the view is still being used, unexpected or undefined behavior will result.

A `std::string_view` that is viewing a string that has been destroyed is sometimes called a **dangling view**.

7.11.5 `std::string_view` is best used as a read-only function parameter

The **best use for `std::string_view` is as a read-only function parameter**. This allows us to pass in a C-style string, `std::string`, or `std::string_view` argument without making a copy, as the `std::string_view` will create a view to the argument.

7.11.6 Should I prefer `std::string_view` or `const std::string&` function parameters?

Prefer `std::string_view` in most cases. We cover this topic further in lesson: Pass by const lvalue reference.

7.11.7 Improperly using `std::string_view`

Let's take a look at a few cases where misusing `std::string_view` gets us into trouble.

```
int main()
{
    std::string_view sv{};

    { // create a nested block
        std::string s{ "Hello, world!" }; // create a std::string local to this nested block
        sv = s; // sv is now viewing s
    } // s is destroyed here, so sv is now viewing an invalid string

    std::cout << sv << '\n'; // undefined behavior

    return 0;
}
```

In this example, we're creating `std::string s` inside a nested block (don't worry about what a nested block is yet). Then we set `sv` to view `s`. `s` is then destroyed at the end of the nested block. **`sv` doesn't know that `s` has been destroyed.** When we then use `sv`, we are accessing an invalid object, and undefined behavior results.

```
std::string getName()
{
    std::string s { "Alex" };
    return s;
}

int main()
{
    std::string_view name { getName() }; // name initialized with return value of function
    std::cout << name << '\n'; // undefined behavior

    return 0;
}
```

```
int main()
{
    using namespace std::string_literals;
    std::string_view name { "Alex"s }; // "Alex"s creates a temporary std::string
    std::cout << name << '\n'; // undefined behavior

    return 0;
}
```

Warning:

Do not initialize a `std::string_view` with a `std::string` literal, as this will leave the `std::string_view` dangling.

It is okay to initialize a `std::string_view` with a C-style string literal or a `std::string_view` literal. It's also okay to initialize a `std::string_view` with a C-style string object, a `std::string` object, or a `std::string_view` object, as long as that string object outlives the view.

```
std::string s { "Hello, world!" };
std::string_view sv { s }; // sv is now viewing s

s = "Hello, universe!";    // modifies s, which invalidates sv (s is still valid)
std::cout << sv << '\n';  // undefined behavior
```

In this example, `sv` is again set to view `s`. `s` is then modified. **When a `std::string` is modified, all views into that `std::string` are invalidated**, meaning those views are now invalid. Using an invalidated view will result in undefined behavior. Thus, when we print `sv`, undefined behavior results.

Key insight: Modifying a `std::string` invalidates all views into that `std::string`.

7.11.8 Revalidating an invalid `std::string_view`

Invalidated objects can often be revalidated (made valid again) **by setting them back to a known good state**. For an invalidated `std::string_view`, we can do this **by assigning the invalidated `std::string_view` object a valid string to view**.

Here's the same example as prior, but we'll revalidate `sv`:

```
std::string s { "Hello, world!" };
std::string_view sv { s }; // sv is now viewing s

s = "Hello, universe!";    // modifies s, which invalidates sv (s is still valid)
std::cout << sv << '\n';  // undefined behavior

sv = s;                    // revalidate sv: sv is now viewing s again
std::cout << sv << '\n';  // prints "Hello, universe!"
```

After `sv` is invalidated by the modification of `s`, we revalidate `sv` **via the statement `sv = s`**, which **causes `sv` to become a valid view of `s` again**. When we print `sv` the second time, it prints "Hello, universe!".

7.11.9 *Be careful returning a `std::string_view`*

`std::string_view` can be used as the return value of a function. **However, this is often dangerous.**

Because local variables are destroyed at the end of the function, **returning a `std::string_view` to a local variable will result in the returned `std::string_view` being invalid**, and further use of that `std::string_view` will result in undefined behavior. For example:

```
std::string_view getBoolName(bool b)
{
    std::string t { "true" }; // local variable
    std::string f { "false" }; // local variable

    if (b)
        return t; // return a std::string_view viewing t

    return f; // return a std::string_view viewing f
} // t and f are destroyed at the end of the function
```

In the above example, when `getBoolName(true)` is called, the function returns a `std::string_view` that is viewing `t`. **However, `t` is destroyed at the end of the function.** This means **the returned `std::string_view` is viewing an object that has been destroyed**. So, when the returned `std::string_view` is printed, undefined behavior results.

Your compiler may or may not warn you about such cases.

There are **two main cases where a `std::string_view` can be returned safely**.

- First, because C-style string literals exist for the entire program, it's fine (and useful) to **return C-style string literals from a function that has a return type of `std::string_view`**.
- Second, it is generally okay to **return a function parameter of type `std::string_view`**.

There is **one important subtlety here**. If the argument is a **temporary object** (that will be destroyed at the end of the full expression containing the function call), the `std::string_view` return value **must be used in the same expression**. After that point, the temporary is destroyed and the `std::string_view` is left dangling.

Warning: If an argument is a temporary that is destroyed at the end of the full expression containing the function call, the returned `std::string_view` must be used immediately, as it will be left dangling after the temporary is destroyed.

7.11.10 View modification functions

Consider a window in your house, looking at an electric car sitting on the street. You can look through the window and see the car, but you can't touch or move the car. Your window just provides a view to the car, which is a completely separate object.

Many windows have curtains, which allow us to modify our view. We can close either the left or right curtain to reduce what we can see. **We don't change what's outside, we just reduce the visible area.**

Because `std::string_view` is a view, it contains functions that let us modify our view by "closing the curtains". **This does not modify the string being viewed in any way, just the view itself.**

- The `remove_prefix()` member function **removes characters from the left side of the view.**
- The `remove_suffix()` member function **removes characters from the right side of the view.**

Unlike real curtains, once `remove_prefix()` and `remove_suffix()` have been called, **the only way to reset the view is by reassigning the source string to it again.**

7.11.11 `std::string_view` can view a substring

This brings up an important use of `std::string_view`. While **`std::string_view` can be used to view an entire string** without making a copy, **they are also useful when we want to view a substring** without making a copy. A **substring** is a contiguous sequence of characters within an existing string. For example, given the string "snowball", some substrings are "snow", "all", and "now". **"owl" is not a substring of "snowball" because these characters do not appear contiguously in "snowball".**

7.11.12 `std::string_view` may or may not be null-terminated

The ability to view just a substring of a larger string **comes with one consequence** of note: a `std::string_view` may or may not be null-terminated.

Consider the string “snowball”, which is null-terminated (because it is a **C-style string literal, which are always null-terminated**). If a `std::string_view` views the whole string, then it is viewing a null-terminated string. However, **if `std::string_view` is only viewing the substring “now”, then that substring is not null-terminated (the next character is a ‘b’)**.

Key insight:

- A C-style string literal and a `std::string` are always null-terminated.
- A `std::string_view` may or may not be null-terminated.

In almost all cases, this doesn’t matter -- a `std::string_view` keeps track of the length of the string or substring it is viewing, so it doesn’t need the null-terminator. Converting a `std::string_view` to a `std::string` will work regardless of whether or not the `std::string_view` is null-terminated.

Warning: Take care not to write any code that assumes a `std::string_view` is null terminated.

Tip: If you have a non-null-terminated `std::string_view` and you need a null-terminated string for some reason, **convert the `std::string_view` into a `std::string`**.

7.11.13 A quick guide on when to use `std::string` vs `std::string_view`

7.11.13.1 Variables

`Std::string`:

- You need a string that you can **modify**.
- You need to **store user-inputted** text.
- You need to **store the return value of a function** that returns a `std::string`.

`Std::string_view`:

- You need **read-only access** to part or all of a string that already exists elsewhere and **will not be modified or destroyed** before use of the `std::string_view` is complete.
- You need a **symbolic constant** for a C-style string.
- You need to **continue viewing** the return value of a function that returns a C-style string or a non-dangling `std::string_view`.

7.11.13.2 Function parameters

Std::string:

- The **function needs to modify the string passed in** as an argument without affecting the caller. This is rare.
- (You are using language standard C++14 or older and aren't comfortable using references yet.)

Std::string_view:

- The function needs a **read-only string**.
- The **function needs to work with non-null-terminated strings**.

Advanced: const std::string&:

- You are using language standard C++14 or older, and the function needs a **read-only** string to work with (as std::string_view is not available until C++17).
- You are **calling other functions that require a const std::string, const std::string&, or const C-style string** (as std::string_view may not be null-terminated).

Advanced: std::string&:

- You **using a std::string as an out-parameter**.
- You are **calling other functions that require a std::string&, or non-const C-style string**.

7.11.13.3 Return types

Std::string:

- The **return value** is a `std::string` local variable.
- The **return value** is a function call or operator that returns a `std::string` by value.

Std::string_view:

- The **function** returns a C-style string literal or local `std::string_view` that has been **initialized with a C-style string literal**.
- Returning a **`std::string_view` parameter**.
- **Advanced:** Writing an accessor for a `std::string_view` member.

Advanced: `const std::string&`:

- Returning a **`const std::string&` parameter**.
- Writing an accessor for a `std::string` or `const std::string` member.
- **Returning a static (local or global) `const std::string`**.

Advanced: `std::string&`:

- Returning a **`std::string&` parameter**.

7.11.13.4 Insights

Things to remember about `std::string`:

- Initializing and **copying `std::string` is expensive**, so avoid this as much as possible.
- **Avoid passing `std::string` by value**, as this makes a copy.
- If possible, **avoid creating short-lived `std::string` objects**.
- **Modifying a `std::string` will invalidate any views** to that string.
- **It is okay to return a local `std::string` by value**.

Things to remember about `std::string_view`:

- `std::string_view` is **typically used for passing string function parameters and returning string literals**.
- Because **C-style string literals exist for the entire program**, it is always okay to set a `std::string_view` to a C-style string literal.
- **When a string is destroyed, all views to that string are invalidated**.
- Using an invalidated view (other than using assignment to revalidate the view) will cause undefined behavior.
- A `std::string_view` **may or may not be null-terminated**.

7.12 Constants and Strings summary

A **constant** is a **value that may not be changed** during the program's execution. C++ supports two types of constants: **named constants, and literals**.

A **named constant** is a constant value that is **associated with an identifier**. A Literal constant is a constant value not associated with an identifier.

A variable whose value cannot be changed is called a **constant variable**. The `const` keyword can be used to make a variable constant. Constant variables must be initialized. Avoid using `const` when passing by value or returning by value.

A **type qualifier** is a keyword that is applied to a type that **modifies how that type behaves**. As of C++23, C++ only supports **`const` and `volatile`** as type qualifiers.

A **constant expression** is an **expression that can be evaluated at compile-time**.

An expression that is not a constant expression is sometimes called a **runtime expression**.

A **compile-time constant** is a **constant whose value is known at compile-time**.

A **runtime constant** is a **constant whose initialization value isn't known until runtime**.

A **`constexpr`** variable **must be a compile-time constant**, and initialized with a constant expression. Function parameters cannot be `constexpr`.

Literals are values inserted directly into the code. Literals **have types, and literal suffixes** can be used to change the type of a literal from the default type.

A **magic number** is a **literal (usually a number) that either has an unclear meaning or may need to be changed later**. **Don't use magic numbers in your code. Instead, use symbolic constants**.

In everyday life, we count using **decimal** numbers, which have 10 digits. Computers use **binary**, which only has 2 digits. C++ also supports **octal** (base 8) and **hexadecimal** (base 16). These are all **examples of numeral systems**, which are collections of symbols (digits) used to represent numbers.

The **conditional operator** (`?:`) (also called the **arithmetic if** operator) is a ternary operator (an operator that takes 3 operands). Given a conditional operation of the form `c ? x : y`, if conditional `c` evaluates to true then `x` will be evaluated, otherwise `y` will be evaluated. The conditional operator typically needs to be **parenthesized as follows**:

- Parenthesize the **entire conditional operator when used in a compound expression** (an expression with other operators).
- For readability, **parenthesize the condition if it contains any operators** (other than the function call operator).

Inline expansion is a process where a **function call is replaced by the code from the called function's definition**. A function that is declared using the inline keyword is called an **inline function**.

Inline functions and variables have **two primary requirements**:

- The **compiler needs to be able to see the full definition** of an inline function or variable in each translation unit where the function is used (a forward declaration will not suffice on its own). **The definition can occur after the point of use if a forward declaration is also provided.**
- **Every definition for an inline function or variable must be identical**, otherwise undefined behavior will result.

In modern C++, the term inline has evolved to mean “**multiple definitions are allowed**”. Thus, an inline function is one that is allowed to be defined in multiple files.

C++17 introduced **inline variables**, which are variables that are allowed to be defined in multiple files.

Inline functions and variables are particularly useful for **header-only libraries**, which are one or more header files that implement some capability (no .cpp files are included).

A **constexpr function** is a function whose return value **may be computed at compile-time**. To make a function a constexpr function, we simply use the constexpr keyword in front of the return type. Constexpr functions are **only guaranteed to be evaluated at compile-time when used in a context that requires a constant expression**. Otherwise, they may be evaluated at compile-time (if eligible) or runtime.

A **consteval function** is a function that **must evaluate at compile-time**.

Constexpr functions and consteval functions are implicitly inline.

A **string** is a **collection of sequential characters** that is used to represent text (such as names, words, and sentences). String literals are always placed between double quotes. **String literals in C++ are C-style strings**, which have a strange type that is hard to work with.

std::string offers an **easy and safe way to deal with text strings**. **std::string** lives in the `<string>` header. **std::string** is **expensive to initialize (or assign to) and copy**.

std::string_view provides **read-only access to an existing string** (a C-style string literal, a **std::string**, or a char array) without making a copy. A **std::string_view** that is viewing a string that has been destroyed is sometimes called a **dangling** view. When a **std::string** is modified, all views into that **std::string** are **invalidated**, meaning those views are now invalid. Using an invalidated view (other than to revalidate it) will produce undefined behavior.

Because C-style string literals exist for the entire program, **it is okay to set a std::string_view to a C-style string literal, and even return such a std::string_view from a function**.

A **substring** is a **contiguous sequence** of characters within an existing string.

7.12.1 Quiz

1. Why are named constants often a better choice than literal constants?

Using literal constants (a.k.a. **magic numbers**) in your program makes your program harder to understand and harder to modify. Symbolic constants help document what the numbers actually represent, and changing a symbolic constant at its declaration changes the value everywhere it is used.

2. Why are const/constexpr variables usually a better choice than #defined symbolic constants?

#define constants do not show up in the debugger and are more likely to have naming conflicts.

3. What are the primary differences between std::string and std::string_view?

std::string provides a **modifiable string**. It is expensive to initialize and copy.

std::string_view provide a **read-only view** of a string that exists elsewhere. It is inexpensive to initialize and copy.

4. What can go wrong when using a std::string_view?

std::string_view can be **dangerous when the string being viewed is destroyed** before the std::string_view that is viewing it.

8 Operators

8.1 Operator precedence and associativity

This chapter builds on top of the concepts from lesson: Introduction to literals and operators.

A quick review follows:

An **operation** is a mathematical process involving zero or more input values (called **operands**) that produces a new value (called an output value).

The specific operation to be performed is denoted by a construct (typically a symbol or pair of symbols) called an **operator**.

8.1.1 Evaluation of compound expressions

Now, let's consider a compound expression, such as $4 + 2 * 3$. Should this be grouped as $(4 + 2) * 3$ which evaluates to 18, or $4 + (2 * 3)$ which evaluates to 10? Using normal mathematical precedence rules (which state that multiplication is resolved before addition), we know that the above expression should be grouped as $4 + (2 * 3)$ to produce the value 10. But how does the compiler know?

In order to evaluate an expression, **the compiler must do two things:**

- **At compile time**, the compiler must **parse the expression and determine how operands are grouped with operators**. This is done via the **precedence and associativity rules**, which we'll discuss momentarily.
- **At compile time or runtime**, the **operands are evaluated and operations executed to produce a result**.

8.1.2 Operator precedence

To assist with parsing a compound expression, all operators are assigned a level of precedence. **Operators with a higher precedence level are grouped with operands first.**

You can see in the **table below** that multiplication and division (precedence level 5) have a higher precedence level than addition and subtraction (precedence level 6). Thus, multiplication and division will be grouped with operands before addition and subtraction. In other words, $4 + 2 * 3$ will be grouped as $4 + (2 * 3)$.

8.1.3 Operator associativity

Consider a compound expression like $7 - 4 - 1$. Should this be grouped as $(7 - 4) - 1$ which evaluates to 2, or $7 - (4 - 1)$, which evaluates to 4? **Since both subtraction operators have the same precedence level, the compiler cannot use precedence alone to determine how this should be grouped.**

If two operators with the same precedence level are adjacent to each other in an expression, the operator's **associativity** tells the compiler whether to evaluate the operators from left to right or from right to left. Subtraction has precedence level 6, and the operators in precedence level 6 have an **associativity of left to right**. So, this expression is grouped from left to right: $(7 - 4) - 1$.

8.1.4 Table of operator precedence and associativity

The below table is primarily meant to be a reference chart that you can refer back to in the future to resolve any precedence or associativity questions you have.

Notes:

- Precedence level 1 is the highest precedence level, and level 17 is the lowest. Operators with a higher precedence level have their operands grouped first.
- L->R means left to right associativity.
- R->L means right to left associativity.

Prec/Ass	Operator	Description	Pattern
1 L->R	::	Global scope (unary)	::name
	::	Namespace scope (binary)	class_name::member_name
2 L->R	()	Parentheses	(expression)
	()	Function call	function_name(arguments)
	type()	Functional cast	type(expression)
	type{}	List init temporary object (C++11)	type{expression}
	[]	Array subscript	pointer[expression]
	.	Member access from object	object.member_name
	->	Member access from object ptr	object_pointer->member_name
	++	Post-increment	lvalue++
	--	Post-decrement	lvalue--
	typeid	Run-time type information	typeid(type) or typeid(expression)
	const_cast	Cast away const	const_cast<type>(expression)
	dynamic_cast	Run-time type-checked cast	dynamic_cast<type>(expression)
	reinterpret_cast	Cast one type to another	reinterpret_cast<type>(expression)
	static_cast	Compile-time type-checked cast	static_cast<type>(expression)
	sizeof...	Get parameter pack size	sizeof...(expression)
	noexcept	Compile-time exception check	noexcept(expression)
	alignof	Get type alignment	alignof(type)
3 R->L	+	Unary plus	+expression
	-	Unary minus	-expression
	++	Pre-increment	++lvalue
	--	Pre-decrement	--lvalue
	!	Logical NOT	!expression
	not	Logical NOT	not expression
	~	Bitwise NOT	~expression
	(type)	C-style cast	(new_type)expression
	sizeof	Size in bytes	sizeof(type) or sizeof(expression)
	co_await	Await asynchronous call	co_await expression (C++20)
	&	Address of	&lvalue
	*	Dereference	*expression
	new	Dynamic memory allocation	new type
	new[]	Dynamic array allocation	new type[expression]
	delete	Dynamic memory deletion	delete pointer
	delete[]	Dynamic array deletion	delete[] pointer
4 L->R	->*	Member pointer selector	object_pointer->*pointer_to_member
	.*	Member object selector	object.*pointer_to_member
5 L->R	*	Multiplication	expression * expression
	/	Division	expression / expression
	%	Remainder	expression % expression
6 L->R	+	Addition	expression + expression
	-	Subtraction	expression - expression
7 L->R	<<	Bitwise shift left / Insertion	expression << expression
	>>	Bitwise shift right / Extraction	expression >> expression
8 L->R	<=>	Three-way comparison (C++20)	expression <=> expression
9 L->R	<	Comparison less than	expression < expression
	<=	Comparison less than or equals	expression <= expression
	>	Comparison greater than	expression > expression
	>=	Comparison greater than or equals	expression >= expression
10 L->R	==	Equality	expression == expression
	!=	Inequality	expression != expression
11 L->R	&	Bitwise AND	expression & expression
12 L->R	^	Bitwise XOR	expression ^ expression
13 L->R		Bitwise OR	expression expression
14 L->R	&&	Logical AND	expression && expression
	and	Logical AND	expression and expression
15 L->R		Logical OR	expression expression
	or	Logical OR	expression or expression
16 R->L	throw	Throw expression	throw expression
	co_yield	Yield expression (C++20)	co_yield expression
	?:	Conditional	expression ? expression : expression
	=	Assignment	lvalue = expression
	*=	Multiplication assignment	lvalue *= expression
	/=	Division assignment	lvalue /= expression
	%=	Remainder assignment	lvalue %= expression
	+=	Addition assignment	lvalue += expression
	-=	Subtraction assignment	lvalue -= expression
	<<=	Bitwise shift left assignment	lvalue <<= expression
	>>=	Bitwise shift right assignment	lvalue >>= expression
	&=	Bitwise AND assignment	lvalue &= expression
	=	Bitwise OR assignment	lvalue = expression
	^=	Bitwise XOR assignment	lvalue ^= expression
17 L->R	,	Comma operator	expression, expression

You should already recognize a few of these operators, such as `+`, `-`, `*`, `/`, `()`, and `sizeof`. However, unless you have experience with another programming language, the majority of the operators in this table will probably be incomprehensible to you right now. That's expected at this point. We'll cover many of them in this chapter, and the **rest will be introduced as there is a need for them**.

Note:

- **C++ doesn't include an operator to do exponentiation** (`operator^` has a different function in C++).
- Note that **`operator<<` handles both bitwise left shift and insertion**, and **`operator>>` handles both bitwise right shift and extraction**. The compiler can determine which operation to perform **based on the types of the operands**.

8.1.5 Parenthesizing

Due to the precedence rules, `4 + 2 * 3` will be grouped as `4 + (2 * 3)`. But what if we actually meant `(4 + 2) * 3`? Just like in normal mathematics, **in C++ we can explicitly use parentheses to set the grouping of operands as we desire**. This works **because parentheses have one of the highest precedence levels**, so **parentheses generally evaluate before whatever is inside them**.

Best practice:

- Use parentheses to **make it clear how a non-trivial compound expression should evaluate** (even if they are technically unnecessary).
- Rule of thumb:
 - **Parenthesize everything, except addition, subtraction, multiplication, and division.**
 - Expressions with a **single assignment operator do not need to have the right operand of the assignment wrapped in parenthesis.**

8.1.6 Value computation

The C++ standard uses the term **value computation** to mean the execution of operators in an expression to produce a value. The **precedence and association rules determine the order in which value computation happens**.

8.1.7 *Order of evaluation (of operands)*

The C++ standard (mostly) uses the term **evaluation** to refer to the **evaluation of operands** (not the evaluation of operators or expressions!). For example, **given expression $a + b$, a will be evaluated to produce some value, and b will be evaluated to produce some value.** These values can be then used as operands to operator+ for value computation.

However **informally**, we typically use the term “evaluates” to mean the evaluation of an entire expression (value computation), not just the operands of an expression.

Note: The order of evaluation of operands and function arguments is mostly unspecified

Key insight: Operands, function arguments, and subexpressions may be evaluated in any order.

It is a **common mistake to believe that operator precedence and associativity affects order of evaluation.** Precedence and associativity is used only to determine how operands are grouped with operators, and the order of value computation.

Warning: Ensure that the expressions (or function calls) you write are not dependent on operand (or argument) evaluation order.

8.2 *Arithmetic operators*

This summary going to go deep into detail here like other chapters did. Most of arithmetic operators speaks for itself.

8.2.1 *Unary arithmetic operators*

- Unary plus +
- Unary minus -

8.2.2 *Binary arithmetic operators*

- Addition +
- Subtraction −
- Multiplication *
- Division /
- Remainder %

8.2.3 *Integer and floating-point division*

It is easiest to think of the division operator as having **two different “modes”**.

- **Floating point division:** If any of the operands are floating-point type the fraction is preserved.
- **Integer division:** If both operands are integers types the fraction is lost.

8.2.4 *Dividing by 0 and 0.0*

Integer division with a divisor of 0 will cause undefined behavior, as the results are mathematically undefined!

The result of **dividing by floating point value 0.0 is implementation-defined** (meaning the behavior is determined by the compiler/architecture). On architectures that support IEEE754 floating point format, the **result will be NaN or Inf. On other architectures, the result will likely be undefined behavior.**

8.2.5 *Arithmetic assignment operators*

- Assignment =
- Addition assignment +=
- Subtraction assignment -=
- Multiplication assignment *=
- Division assignment /=
- Remainder assignment %=

8.3 Remainder and exponentiation

8.3.1 *The remainder operator (operator%)*

The remainder operator (also commonly called the **modulo operator** or **modulus operator**) is an operator that **returns the remainder after doing an integer division**. For example, **7 / 4 = 1 remainder 3**. Therefore, **7 % 4 = 3**. As another example, 25 / 7 = 3 remainder 4, thus 25 % 7 = 4. The remainder operator only works with integer operands.

This is **most useful for testing whether a number is evenly divisible by another number** (meaning that after division, there is no remainder): if $x \% y$ evaluates to 0, then we know that x is evenly divisible by y .

8.3.2 *Remainder with negative numbers*

The remainder operator can also work with negative operands. $x \% y$ always returns results with the sign of x .

8.3.3 *Nomenclature*

The C++ standard does not actually give a name to `operator%`. However, the C++20 standard does say, “the binary `%` operator yields the remainder from the division of the first expression by the second”.

Although `operator%` is often called the “modulo” or “modulus” operator, this can be confusing, because modulo in mathematics is often defined in a way that yields a different result to what `operator%` in C++ produces when one (and only one) of the operands is negative.

For this reason, we believe **the name “remainder” is a more accurate name for `operator%` than “modulo”**.

Best practice: Prefer to **compare** (don’t divide by 0) the result of the remainder operator (`operator%`) against 0 if possible.

8.3.4 *Where's the exponent operator?*

You'll note that the **^ operator** (commonly used to denote exponentiation in mathematics) **is a Bitwise XOR operation** in C++ (covered in lesson: Bit manipulation with bitwise operators and bit masks). C++ does not include an exponent operator.

To do exponents in C++, #include the `<cmath>` header, and use the `pow()` function:

```
1 | #include <cmath>
2 |
3 | double x{ std::pow(3.0, 4.0) }; // 3 to the 4th power
```

Note that the parameters (and return value) of function `pow()` are of type `double`. Due to rounding errors in floating point numbers, the results of `pow()` may not be precise (even if you pass it integers or whole numbers).

If you want to do integer exponentiation, you're best off using your own function to do so.

Warning: In the vast majority of cases, integer exponentiation will overflow the integral type. This is likely why such a function wasn't included in the standard library in the first place.

8.4 Increment/decrement operators, and side effects

8.4.1 Incrementing and decrementing variables

Incrementing (adding 1 to) and decrementing (subtracting 1 from) a variable are both so common that they have their own operators.

- Prefix increment ++
 - Increment x, then return x
- Prefix decrement --
 - Decrement x, then return x
- Postfix increment ++
 - Copy x, then increment x, then return the copy
- Postfix decrement --
 - Copy x, then decrement x, then return the copy

Note that there are two versions of each operator -- a prefix version (where the operator comes before the operand) and a postfix version (where the operator comes after the operand).

8.4.2 Prefix increment and decrement

The prefix increment/decrement operators are very straightforward. First, the operand is incremented or decremented, and then expression evaluates to the value of the operand.

```
1  #include <iostream>
2
3  int main()
4  {
5      int x { 5 };
6      int y { ++x }; // x is incremented to 6, x is evaluated to the value 6, and 6 is assigned to y
7
8      std::cout << x << ' ' << y << '\n';
9      return 0;
10 }
```

This prints:

```
6 6
```

8.4.3 Postfix increment and decrement

The postfix increment/decrement operators are trickier. First, **a copy of the operand is made**. Then the **operand (not the copy) is incremented or decremented**. Finally, **the copy (not the original) is evaluated**. For example:

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     int x { 5 };
6 |     int y { x++ }; // x is incremented to 6, copy of original x is evaluated to the value 5, and 5 is assigned to y
7 |
8 |     std::cout << x << ' ' << y << '\n';
9 |     return 0;
10| }
```

This prints:

```
6 5
```

Let's examine how this line 6 works in more detail. First, a temporary copy of *x* is made that starts with the same value as *x* (5). Then the actual *x* is incremented from 5 to 6. Then the copy of *x* (which still has value 5) is returned and assigned to *y*. Then the temporary copy is discarded.

Consequently, *y* ends up with the value of 5 (the pre-incremented value), and *x* ends up with the value 6 (the post-incremented value).

Note that the postfix version takes a lot more steps, and thus may not be as performant as the prefix version.

8.4.4 When to use prefix vs postfix

In many cases, the prefix and postfix operators produce the same behavior:

```
1  int main()
2  {
3      int x { 0 };
4      ++x; // increments x to 1
5      x++; // increments x to 2
6
7      return 0;
8  }
```

Best Practice: In cases where code can be written to use either prefix or postfix, **prefer the prefix versions**, as they are generally more performant, and less likely to cause surprises.

Use the postfix versions when doing so produces significantly more concise or understandable code than the equivalent code written using the prefix versions.

8.4.5 Side effects

A function or expression is said to have a **side effect** if it has some **observable effect beyond producing a return value**.

Common examples of side effects include changing the value of objects, doing input or output, or updating a graphical user interface (e.g. enabling or disabling a button).

Most of the time, side effects are useful:

```
1  x = 5; // the assignment operator has side effect of changing value of x
2  ++x; // operator++ has side effect of incrementing x
3  std::cout << x; // operator<< has side effect of modifying the state of the console
```

8.4.6 Side effects can cause order of evaluation issues

In some cases, side effects can lead to order of evaluation problems. For example:

```
3  int add(int x, int y)
4  {
5      return x + y;
6  }
7
8  int main()
9  {
10     int x { 5 };
11     int value{ add(x, ++x) }; // undefined behavior: is this 5 + 6, or 6 + 6?
12     // It depends on what order your compiler evaluates the function arguments in
```

The C++ standard does not define the order in which function arguments are evaluated. If the left argument is evaluated first, this becomes a call to `add(5, 6)`, which equals 11. If the right argument is evaluated first, this becomes a call to `add(6, 6)`, which equals 12! **Note that this is only a problem because one of the arguments to function `add()` has a side effect.**

The C++ standard intentionally does not define these things so that compilers can do whatever is most natural (and thus most performant) for a given architecture.

8.4.7 The sequencing of side effects

In many cases, C++ also **does not specify when** the **side effects** of operators **must be applied**. This can lead to undefined behavior in cases where an object with a side effect applied is used more than once in the same statement.

For example, the expression `x + ++x` is unspecified behavior. When `x` is initialized to 1, Visual Studio and GCC evaluate this as `2 + 2`, and Clang evaluates it as `1 + 2`! This is due to differences in when the compilers apply the side effect of incrementing `x`.

Even when the C++ standard does make it clear how things should be evaluated, historically this has been an area where there have been many compiler bugs. These problems can generally all be avoided by ensuring that any variable that has a side-effect applied is used no more than once in a given statement.

Warning:

- C++ does not define the order of evaluation for function arguments or the operands of operators.
- Don't use a variable that has a side effect applied to it more than once in a given statement. If you do, the result may be undefined.
- One exception is for simple assignment expression statements like `x = x + y;` (which could also be rewritten as `x += y;`).

8.5 The comma operator

- Comma ,

The **comma operator (,)** allows you to **evaluate multiple expressions** wherever a single expression is allowed. The comma operator **evaluates the left operand, then the right operand, and then returns the result of the right operand.**

Note that **comma has the lowest precedence of all the operators**, even lower than assignment. Because of this, the following two lines of code do different things:

```
1 | z = (a, b); // evaluate (a, b) first to get result of b, then assign that value to variable z.
2 | z = a, b; // evaluates as "(z = a), b", so z gets assigned the value of a, and b is evaluated and discarded.
```

This **makes the comma operator somewhat dangerous to use**. In almost every case, a **statement written using the comma operator would be better written as separate statements.**

Best practice: Avoid using the comma operator, except within *for loops*.

8.5.1 Comma as a separator

In C++, the comma symbol is often used as a separator, and these uses do not invoke the comma operator.

- Between parameters or arguments
- Between statements

There is no need to avoid separator commas (**except when declaring multiple variables, which you should not do**).

8.6 Relational operators and floating-point comparisons

Relational operators are operators that let you **compare two values**. There are 6 relational operators:

- Greater than >
- Less than <
- Greater or equals >=
- Less or equals <=
- Equality ==
- Inequality !=

You have already seen how most of these work, and they are pretty intuitive. Each of these operators **evaluates to the boolean value true (1), or false (0)**.

Safety Warning:

Avoid using operator== and operator!= to compare floating point values if there is any chance those values have been calculated.

It is okay to compare two floating point literals of the same type when the number of significant digits in each literal does not exceed the minimum precision for that type. Float has a minimum precision of 6 significant digits, and double has a minimum precision of 15 significant digits.

It is generally not safe to compare floating point literals of different types.

8.6.1 Boolean conditional values

By default, conditions in an *if statement* or *conditional operator* (and a few other places) evaluate as Boolean values.

Many new programmers will write statements like this one: **if (b1 == true) or if (b1 == false)**

This is redundant, as the == true doesn't actually add any value to the condition. Instead, we should write: **if (b1) or if (!b1)**

Best practice: Don't add unnecessary == or != to conditions. It makes them harder to read without offering any additional value.

8.7 Logical operators

While **relational (comparison) operators** can be used to test whether a particular condition is true or false, **they can only test one condition at a time**. Often we need to know whether multiple conditions are true simultaneously. For example, to check whether we've won the lottery, we have to compare whether all of the multiple numbers we picked match the winning numbers. In a lottery with 6 numbers, this would involve 6 comparisons, all of which have to be true. In other cases, we need to know whether any one of multiple conditions is true. For example, we may decide to skip work today if we're sick, or if we're too tired, or if we won the lottery in our previous example. This would involve checking whether any of 3 comparisons is true.

Logical operators provide us with the capability to test multiple conditions.

- Logical NOT !
- Logical AND &&
- Logical OR ||

Best practice:

If logical NOT is intended to operate on the result of other operators, **the other operators and their operands need to be enclosed in parentheses**. Simple uses of logical NOT, such as `if (!value)` do not need parentheses because precedence does not come into play.

Warning:

New programmers sometimes try this:

```
1 | if (value == 0 || 1) // incorrect: if value is 0, or if 1
```

When 1 is evaluated, it will implicitly convert to `bool true`. Thus, this conditional will always evaluate to true.

If you want to compare a variable against multiple values, **you need to compare the variable multiple times**:

```
1 | if (value == 0 || value == 1) // correct: if value is 0, or if value is 1
```

8.7.1 Short circuit evaluation

In order for logical AND to return true, **both operands must evaluate to true**. If the left operand evaluates to false, logical AND knows it must return false regardless of whether the right operand evaluates to true or false. **In this case, the logical AND operator will go ahead and return false immediately without even evaluating the right operand!** This is known as **short circuit evaluation**, and it is done primarily for optimization purposes.

Similarly, if the left operand for logical OR is true, then the entire OR condition has to evaluate to true, and the right operand won't be evaluated.

Short circuit evaluation presents another opportunity to show why operators that cause side effects should not be used in compound expressions. Consider the following snippet:

```
if (x == 1 && ++y == 2)
    // do something
```

if x does not equal 1, the whole condition must be false, so **$++y$ never gets evaluated!** Thus, **y will only be incremented if x evaluates to 1**, which is probably not what the programmer intended!

Warning: Short circuit evaluation may cause Logical OR and Logical AND to not evaluate the right operand. Avoid using expressions with side effects in conjunction with these operators.

Key insight: The Logical OR and logical AND operators are an exception to the rule that the operands may evaluate in any order, as the **standard explicitly states that the left operand must evaluate first**.

8.7.2 Mixing ANDs and ORs

Mixing logical AND and logical OR operators in the same expression often **cannot be avoided, but it is an area full of potential dangers**.

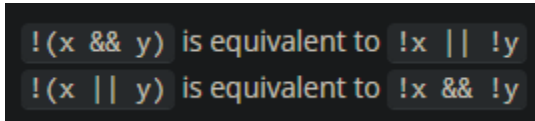
Because logical AND and logical OR seem like a pair, **many programmers assume they have the same precedence** (just like addition/subtraction and multiplication/division). **However, logical AND has higher precedence than logical OR**, thus logical **AND operators will be evaluated ahead of logical OR operators (unless they have been parenthesized)**.

Best practice: When mixing logical AND and logical OR in a single expression, **explicitly parenthesize each operation** to ensure they evaluate how you intend.

8.7.3 De Morgan's laws

Many programmers also make the mistake of thinking that **!(x && y) is the same thing as !x && !y**. Unfortunately, **you cannot “distribute” the logical NOT in that manner**.

De Morgan's laws tell us how the logical NOT should be distributed in these cases:



```
!(x && y) is equivalent to !x || !y
!(x || y) is equivalent to !x && !y
```

In other words, when you distribute the logical NOT, you also need to flip logical AND to logical OR, and vice-versa!

This can sometimes be useful when trying to make complex expressions easier to read.

8.7.4 Where's the logical exclusive or (XOR) operator?

Logical XOR is a logical operator provided in some languages that is used to test whether an odd number of conditions is true.

C++ doesn't provide an explicit logical XOR operator (operator^ is a **bitwise XOR, not a logical XOR**). Unlike logical OR or logical AND, logical XOR cannot be short circuit evaluated. Because of this, making a logical XOR operator out of logical OR and logical AND operators is challenging.

However, operator!= produces the same result as a logical XOR when given bool operands.

8.7.5 *Alternative operator representations*

Many operators in C++ (such as operator `||`) have names that are just symbols. Historically, not all keyboards and language standards have supported all of the symbols needed to type these operators. As such, C++ supports an alternative set of keywords for the operators that use words instead of symbols. For example, **instead of `||`, you can use the keyword `or`**.

Primary	Alternative
<code>&&</code>	<code>and</code>
<code>&=</code>	<code>and_eq</code>
<code>&</code>	<code>bitand</code>
<code> </code>	<code>bitor</code>
<code>~</code>	<code>compl</code>
<code>!</code>	<code>not</code>
<code>!=</code>	<code>not_eq</code>
<code> </code>	<code>or</code>
<code> =</code>	<code>or_eq</code>
<code>^</code>	<code>xor</code>
<code>^=</code>	<code>xor_eq</code>
<code>{</code>	<code><%</code>
<code>}</code>	<code>%></code>
<code>[</code>	<code><:</code>
<code>]</code>	<code>::></code>
<code>#</code>	<code>:%:</code>
<code>##</code>	<code>:%: %:</code>

8.8 Summary

Always use parentheses to disambiguate the precedence of operators if there is any question or opportunity for confusion.

The **arithmetic operators all work like they do in normal mathematics**. The remainder (%) operator returns the remainder from an integer division.

The **increment and decrement operators** can be used to easily increment or decrement numbers. **Avoid the postfix versions of these operators** whenever possible.

Beware of side effects, particularly **when it comes to the order that function parameters are evaluated**. Do not use a variable that has a side effect applied more than once in a given statement.

The **comma operator can compress multiple statements into one**. **Writing the statements separately is usually better**.

The **conditional operator is a nice short version of an if-statement**, but don't use it as an alternative to an if-statement. **Only use the conditional operator if you use its result**.

Relational operators can be used to compare floating point numbers. **Beware using equality and inequality on floating point numbers**.

Logical operators allow us to form compound conditional statements.

8.8.1 Quiz

Evaluate the following:

1. $(5 > 3 \ \&\& \ 4 < 8)$

true

2. $(4 > 6 \ \&\& \ \text{true})$

false

3. $(3 \geq 3 \ || \ \text{false})$

true

4. $(\text{true} \ || \ \text{false}) ? 4 : 5$

4

5. $7 / 4$

1

6. $14 \% 5$

4

Why should you never do the following?:

7. `int y{ foo(++x, x) };`

Operator++ applies a side effect to x, we should not use x again in the same expression. In this case, the parameters to function foo() can be evaluated in any order, so it's indeterminate whether x or ++x gets evaluated first. Because ++x changes the value of x, it's unclear what values will be passed into the function.

8. `double x{ 0.1 + 0.1 + 0.1 }; return (x == 0.3);`

Comparison with floating point numbers like this often leads to unspecified behavior due to floating-point precision errors. In this case it will evaluate to false, even though it should evaluate to true.

9. `int x{ 3 / 0 };`

Dividing by 0 or 0.0 should be avoided due to undefined behavior.

9 Optional chapter: Bit Manipulation

9.1 Bit flags and bit manipulation via `std::bitset`

On modern computer architectures, the smallest addressable unit of memory is a byte. **Since all objects need to have unique memory addresses, this means objects must be at least one byte in size.** For most variable types, this is fine. However, **for Boolean values, this is a bit wasteful** (pun intended). Boolean types only have two states: true (1), or false (0). **This set of states only requires one bit to store.** However, if a variable must be at least a byte, and a byte is 8 bits, that means **a Boolean is using 1 bit and leaving the other 7 unused.**

In the majority of cases, this is fine -- we're usually not so hard-up for memory that we need to care about 7 wasted bits (we're better off optimizing for understandability and maintainability). However, **in some storage-intensive cases, it can be useful to "pack" 8 individual Boolean values into a single byte for storage efficiency purposes.**

Doing these things requires that we can manipulate objects at the bit level. Fortunately, C++ gives us tools to do precisely this. Modifying individual bits within an object is called **bit manipulation**.

Note: Bit manipulation is used a lot in certain programming contexts (e.g. **graphics, encryption, compression, optimization**), but **not as much in general programming**. Because of that, this entire chapter is optional reading. Feel free to skip or skim it and come back later.

9.1.1 *Bit flags*

Up to this point, we've used variables to hold single values. However, instead of viewing objects as holding a single value, **we can instead view them as a collection of individual bits.** When individual bits of an object are used as Boolean values, the bits are called **bit flags**.

In computing, a **flag** is a value that signals when some condition exists in a program. **With a bit flag, a value of true means the condition exists.**

Analogously, in the United States, many mailboxes have small (usually red) physical flags attached to the side. When outgoing mail is waiting to be picked up by the carrier, the flag is raised to signal that there is outgoing mail.

To define a set of bit flags, **we'll typically use an unsigned integer of the appropriate size** (8 bits, 16 bits, 32 bits, etc... depending on how many flags we have), **or `std::bitset`.**

```
1 #include <bitset> // for std::bitset
2
3 std::bitset<8> mybitset {}; // 8 bits in size means room for 8 flags
```

Best practice: Bit manipulation is **one of the few times when you should unambiguously use unsigned integers** (or `std::bitset`).

In this lesson, we'll show how to do bit manipulation the easy way, via `std::bitset`. In the next set of lessons, we'll explore how to do it the more difficult but versatile way.

9.1.2 Bit numbering and bit positions

Given a sequence of bits, we typically number the bits from right to left, starting with 0 (not 1). Each number denotes a **bit position**.

```
76543210 Bit position
00000101 Bit sequence
```

Given the bit sequence 0000 0101, the bits that are in position 0 and 2 have value 1, and the other bits have value 0.

9.1.3 Manipulation bits via `std::bitset`

`std::bitset` provides 4 key member functions that are useful for doing bit manipulation:

- **test()** allows us to query whether a bit is a 0 or 1.
- **set()** allows us to turn a bit on (this will do nothing if the bit is already on).
- **reset()** allows us to turn a bit off (this will do nothing if the bit is already off).
- **flip()** allows us to flip a bit value from a 0 to a 1 or vice versa.

Each of these functions takes the position of the bit we want to operate on as their only argument.

```
std::bitset<8> bits{ 0b0000'0101 }; // we need 8 bits, start with bit pattern 0000 0101
bits.set(3); // set bit position 3 to 1 (now we have 0000 1101)
bits.flip(4); // flip bit 4 (now we have 0001 1101)
bits.reset(4); // set bit 4 back to 0 (now we have 0000 1101)
```

9.1.4 *What if we want to get or set multiple bits at once*

`std::bitset` doesn't make this easy. In order to do this, or if we want to use unsigned integer bit flags instead of `std::bitset`, we need to turn to more traditional methods. We'll cover these in the next couple of lessons.

The size of `std::bitset`

One potential surprise is that `std::bitset` is optimized for speed, not memory savings. The size of a `std::bitset` is typically the number of bytes needed to hold the bits, rounded *up* to the nearest `sizeof(size_t)`, which is 4 bytes on 32-bit machines, and 8-bytes on 64-bit machines.

Thus, a **`std::bitset<8>` will typically use either 4 or 8 bytes of memory**, even though it technically only needs 1 byte to store 8 bits. Thus, `std::bitset` is most useful when we desire convenience, not memory savings.

9.1.5 *Querying `std::bitset`*

There are a few other member functions that are often useful:

- **`size()`** returns the number of bits in the `bitset`.
- **`count()`** returns the number of bits in the `bitset` that are set to true.
- **`all()`** returns a Boolean indicating whether all bits are set to true.
- **`any()`** returns a Boolean indicating whether any bits are set to true.
- **`none()`** returns a Boolean indicating whether no bits are set to true.

9.2 Bitwise operators

C++ provides 6 bit-manipulation operators, often called bitwise operators:

- Left shift <<
- Right shift >>
- Bitwise NOT ~
- Bitwise AND &
- Bitwise OR |
- Bitwise XOR ^

Note:

In the following examples, **we will largely be working with 4-bit binary values**. This is for the sake of convenience and keeping the examples simple. In actual programs, the number of bits used is based on the size of the object (e.g. a 2-byte object would store 16 bits). For readability, **we'll also omit the 0b prefix outside of code examples** (e.g. instead of 0b0101, we'll just use 0101).

The **bitwise operators are defined for integral types and std::bitset**. We'll use std::bitset in our examples because it's easier to print the output in binary.

Best practice: Avoid using the bitwise operators with signed operands, as many operators will return implementation-defined results prior to C++20 or have other potential gotchas that are easily avoided by using unsigned operands (or std::bitset).

Advanced: Bit-shifting in C++ is **endian-agnostic**. Left-shift is always **towards the most significant bit**, and right-shift towards the least significant bit.

9.2.1 Bitwise left shift (<<) and bitwise right shift (>>) operators

The **bitwise left shift** (<<) operator **shifts bits to the left**. The left operand is the expression to shift the bits of, and the **right operand is an integer number of bits to shift left by**.

```
0011 << 1 is 0110  
0011 << 2 is 1100  
0011 << 3 is 1000
```

The **bitwise right shift** (>>) does exactly the same but in the other direction.

```
1100 >> 1 is 0110  
1100 >> 2 is 0011  
1100 >> 3 is 0001
```

Note that in the third case we shifted a bit off the right end of the number, so it is lost.

9.2.2 Bitwise NOT

The bitwise NOT operator (~) is perhaps the easiest to understand of all the bitwise operators. It simply flips each bit from a 0 to a 1, or vice versa. Note that the result of a bitwise NOT is dependent on what size your data type is.

- Flipping 4 bits:
 - **~0100 is 1011**
- Flipping 8 bits:
 - ~0000 0100 is 1111 1011

In both the 4-bit and 8-bit cases, we start with the same number (binary 0100 is the same as 0000 0100 in the same way that decimal 7 is the same as 07), but we end up with a different result.

9.2.3 Bitwise OR

Bitwise OR (`|`) works much like its *logical OR* counterpart. However, instead of applying the *OR* to the operands to produce a single result, *bitwise OR* applies to each bit! For example, consider the expression `0b0101 | 0b0110`.

```
0 1 0 1 OR
0 1 1 0
-----
0 1 1 1
```

9.2.4 Bitwise AND

Bitwise AND (`&`) works similarly to the above. **Logical AND** evaluates to true if both the left and right operand evaluate to true. Bitwise AND evaluates to true (1) **if both bits in the column are 1**. Consider the expression `0b0101 & 0b0110`. Lining each of the bits up and applying an AND operation to each column of bits.

```
0 1 0 1 AND
0 1 1 0
-----
0 1 0 0
```

9.2.5 Bitwise XOR

The last operator is the **bitwise XOR** (`^`), also **known as exclusive or**.

When evaluating two operands, XOR **evaluates to true (1) if one and only one of its operands is true (1)**. If neither or both are true, it evaluates to 0. Consider the expression `0b0110 ^ 0b0011`:

```
0 1 1 0 XOR
0 0 1 1
-----
0 1 0 1
```

Bitwise assignment operators

Similar to the arithmetic assignment operators, C++ provides bitwise assignment operators in order to facilitate easy modification of variables.

- Left shift assignment `<<=`
- Right shift assignment `>>=`
- Bitwise OR assignment `|=`
- Bitwise AND assignment `&=`
- Bitwise XOR assignment `^=`

Advanced:

- Bitwise operators perform integral promotion on smaller integral types.
- Bitwise operators will promote operands with narrower integral types to `int` or unsigned `int`.
- `operator~` and `operator<<` are width-sensitive and may produce different results depending on the width of the operand.
- `static_cast` the result of such bitwise operations back to the narrower integral type before using to ensure correct results.

9.2.6 Summary

- When evaluating bitwise OR, if any bit in a column is 1, the result for that column is 1.
- When evaluating bitwise AND, if all bits in a column are 1, the result for that column is 1.
- When evaluating bitwise XOR, if there are an odd number of 1 bits in a column, the result for that column is 1.

9.3 Bit manipulation with bitwise operators and bit masks

9.3.1 *Bit masks*

In order to manipulate individual bits (e.g. turn them on or off), we need some way to identify the specific bits we want to manipulate. **Unfortunately, the bitwise operators don't know how to work with bit positions.** Instead, **they work with bit masks.**

A bit mask is a predefined set of bits that is used to select which specific bits will be modified by subsequent operations.

Consider a real-life case where you want to paint a window frame. If you're not careful, you risk painting not only the window frame, but also the glass itself. You might buy some masking tape and apply it to the glass and any other parts you don't want painted. Then when you paint, the masking tape blocks the paint from reaching anything you don't want painted. In the end, only the non-masked parts (the parts you want painted) get painted.

A bit mask essentially performs the same function for bits -- the bit mask blocks the bitwise operators from touching bits we don't want modified, and allows access to the ones we do want modified.

Let's first explore how to define some simple bit masks, and then we'll show you how to use them.

9.3.2 Defining bit masks in C++14

The simplest set of bit masks is to **define one bit mask for each bit position**. We use 0s to mask out the bits we don't care about, and 1s to denote the bits we want modified.

Although bit masks can be literals, they're often defined as symbolic constants so they can be given a meaningful name and easily reused.

Because C++14 supports binary literals, defining these bit masks is easy:

```
constexpr std::uint8_t mask0{ 0b0000'0001 }; // represents bit 0
constexpr std::uint8_t mask1{ 0b0000'0010 }; // represents bit 1
constexpr std::uint8_t mask2{ 0b0000'0100 }; // represents bit 2
constexpr std::uint8_t mask3{ 0b0000'1000 }; // represents bit 3
constexpr std::uint8_t mask4{ 0b0001'0000 }; // represents bit 4
constexpr std::uint8_t mask5{ 0b0010'0000 }; // represents bit 5
constexpr std::uint8_t mask6{ 0b0100'0000 }; // represents bit 6
constexpr std::uint8_t mask7{ 0b1000'0000 }; // represents bit 7
```

Now we have a set of symbolic constants that represents each bit position. We can use these to manipulate the bits (which we'll show how to do in just a moment).

9.3.3 Defining bit masks in C++11 or earlier

```
constexpr std::uint8_t mask0{ 0x01 }; // hex for 0000 0001
constexpr std::uint8_t mask1{ 0x02 }; // hex for 0000 0010
constexpr std::uint8_t mask2{ 0x04 }; // hex for 0000 0100
constexpr std::uint8_t mask3{ 0x08 }; // hex for 0000 1000
constexpr std::uint8_t mask4{ 0x10 }; // hex for 0001 0000
constexpr std::uint8_t mask5{ 0x20 }; // hex for 0010 0000
constexpr std::uint8_t mask6{ 0x40 }; // hex for 0100 0000
constexpr std::uint8_t mask7{ 0x80 }; // hex for 1000 0000
```

Or

```
constexpr std::uint8_t mask0{ 1 << 0 }; // 0000 0001
constexpr std::uint8_t mask1{ 1 << 1 }; // 0000 0010
constexpr std::uint8_t mask2{ 1 << 2 }; // 0000 0100
constexpr std::uint8_t mask3{ 1 << 3 }; // 0000 1000
constexpr std::uint8_t mask4{ 1 << 4 }; // 0001 0000
constexpr std::uint8_t mask5{ 1 << 5 }; // 0010 0000
constexpr std::uint8_t mask6{ 1 << 6 }; // 0100 0000
constexpr std::uint8_t mask7{ 1 << 7 }; // 1000 0000
```

9.3.4 Testing a bit (to see if it is on or off)

```
std::cout << "bit 0 is " << (static_cast<bool>(flags & mask0) ? "on\n" : "off\n");
std::cout << "bit 1 is " << (static_cast<bool>(flags & mask1) ? "on\n" : "off\n");
```

9.3.5 Setting a bit

```
std::uint8_t flags{ 0b0000'0101 }; // 8 bits in size means room for 8 flags

std::cout << "bit 1 is " << (static_cast<bool>(flags & mask1) ? "on\n" : "off\n");

flags |= mask1; // turn on bit 1
```

9.3.6 Resetting a bit

```
flags &= ~mask2; // turn off bit 2
```

We can turn off multiple bits at the same time:

```
1 | flags &= ~(mask4 | mask5); // turn bits 4 and 5 off at the same time
```

Key insight:

Some compilers may complain about a sign conversion with this line:

```
flags &= ~mask2;
```

Because the type of mask2 is smaller than int, operator~ **causes operand mask2 to undergo integral promotion to type int**. Then the compiler complains that we're trying to use operator&= where the left operand is unsigned and the right operand is signed.

If this is the case, **try the following**:

```
flags &= static_cast<std::uint8_t>(~mask2);
```

9.3.7 Flipping a bit

To toggle (flip) a bit state (from 0 to 1 or from 1 to 0), we use Bitwise XOR:

```
flags ^= mask2; // flip bit 2
```

9.3.8 Bit masks and std::bitset

std::bitset supports the full set of bitwise operators. So even though it's easier to use the functions (test, set, reset, and flip) to modify individual bits, you can use bitwise operators and bit masks if you want.

Why would you want to? **The functions only allow you to modify individual bits.** The **bitwise operators allow you to modify multiple bits at once.**

9.3.9 *When are bit flags most useful?*

Astute readers may note that the above examples don't actually save any memory. 8 separate booleans values would normally take 8 bytes. But the examples above (using `std::uint8_t`) use 9 bytes -- 8 bytes to define the bit masks, and 1 byte for the flag variable!

NOTE: If the variable is a LOCAL compile-time constexpr the variable would get optimized at compile-time and wouldn't take up any memory.

Bit flags make the most sense when you have many identical flag variables. For example, in the example above, imagine that instead of having one person (me), you had 100. If you used 8 Booleans per person (one for each possible state), you'd use 800 bytes of memory. With bit flags, you'd use 8 bytes for the bit masks, and 100 bytes for the bit flag variables, for a total of 108 bytes of memory -- approximately 8 times less memory.

For most programs, the amount of memory saved using bit flags is not worth the added complexity. But **in programs where there are tens of thousands or even millions of similar objects, using bit flags can reduce memory use substantially**. It's a useful optimization to have in your toolkit if you need it.

9.3.10 Summary

Summary

Summarizing how to set, clear, toggle, and query bit flags:

To query bit states, we use *bitwise AND*:

```
1 | if (flags & option4) ... // if option4 is set, do something
```

To set bits (turn on), we use *bitwise OR*:

```
1 | flags |= option4; // turn option 4 on.  
2 | flags |= (option4 | option5); // turn options 4 and 5 on.
```

To clear bits (turn off), we use *bitwise AND* with *bitwise NOT*:

```
1 | flags &= ~option4; // turn option 4 off  
2 | flags &= ~(option4 | option5); // turn options 4 and 5 off
```

To flip bit states, we use *bitwise XOR*:

```
1 | flags ^= option4; // flip option4 from on to off, or vice versa  
2 | flags ^= (option4 | option5); // flip options 4 and 5
```

10 Scope, Duration and Linkage

10.1 Compound statements (blocks)

A **compound statement** (also called a **block**, or **block statement**) is a **group of zero or more statements** that is **treated by the compiler as if it were a single statement**.

Blocks begin with a { symbol, end with a } symbol, with the statements to be executed being placed in between. Blocks can be used anywhere a single statement is allowed. No semicolon is needed at the end of a block.

```
int main()
{ // start block

    // multiple statements
    int value {}; // this is initialization, not a block
    add(3, 4);

    return 0;

} // end block (no semicolon)
```

10.1.1 Blocks inside other blocks

Although functions can't be nested inside other functions, **blocks can be nested** inside other blocks. When blocks are nested, the enclosing block is typically called the **outer block** and the enclosed block is called the **inner block** or **nested block**.

10.1.2 Using blocks to execute multiple statements conditionally

One of the most common use cases for blocks is in conjunction with if statements. By default, an if statement executes a single statement if the condition evaluates to true. However, we can replace this single statement with a block of statements if we want multiple statements to execute when the condition evaluates to true.

```
if (value >= 0)
{ // start of nested block
    std::cout << value << " is a positive integer (or zero)\n";
    std::cout << "Double this number is " << value * 2 << '\n';
} // end of nested block
```

10.1.3 Block nesting levels / depth

It is even possible to put **blocks inside of blocks inside of blocks**:

```
int main()
{ // block 1, nesting level 1
  std::cout << "Enter an integer: ";
  int value {};
  std::cin >> value;

  if (value > 0)
  { // block 2, nesting level 2
    if ((value % 2) == 0)
    { // block 3, nesting level 3
      std::cout << value << " is positive and even\n";
    }
    else
    { // block 4, also nesting level 3
      std::cout << value << " is positive and odd\n";
    }
  }

  return 0;
}
```

The **nesting level** (also called the **nesting depth**) of a function is the maximum number of nested blocks you can be inside at any point in the function (including the outer block). In the above function, there are 4 blocks, but the nesting level is 3 since in this program you can never be inside more than 3 blocks at any point.

The C++ standard says that C++ compilers should support 256 levels of nesting -- however not all do (e.g. as of the time of writing, Visual Studio supports fewer).

It's a good idea to keep your nesting level to 3 or less. Just as overly-long functions are good candidates for refactoring (breaking into smaller functions), overly-nested blocks are hard to read and are good candidates for refactoring (with the most-nested blocks becoming separate functions).

Best practice: Keep the **nesting level of your functions to 3 or less**. If your function has a need for more nested levels, consider refactoring your function into sub-functions.

10.2 User-defined namespaces and the scope resolution operator

In lesson: Naming collisions and an introduction to namespaces, we introduced the concept of naming collisions and namespaces. As a reminder, **a naming collision occurs when two identical identifiers are introduced into the same scope**, and the compiler can't disambiguate which one to use. When this happens, compiler or linker will produce an error because they do not have enough information to resolve the ambiguity.

Key insight: **As programs become larger**, the number of identifiers increases, which in turn **causes the probability of a naming collision occurring to increase significantly**. Because every name in a given scope can potentially collide with every other name in the same scope, a linear increase in identifiers will result in an exponential increase in potential collisions! **This is one of the key reasons for defining identifiers in the smallest scope possible.**

Note: naming collision **errors happen at the point of redefinition**, so it doesn't matter whether a function is ever called.

One way to resolve this would be to **rename one of the functions**, so the names no longer collide. But this would also require changing the names of all the function calls, **which can be a pain, and is subject to error**. A better way to avoid collisions is to **put your functions into your own namespaces**. For this reason, the standard library was moved into the std namespace.

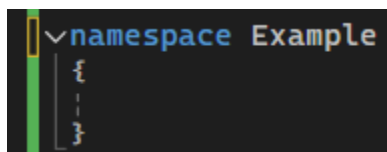
10.2.1 Defining your own namespaces

C++ allows us to define our own namespaces via the namespace keyword. Namespaces that you create in your own programs are casually called **user-defined namespaces** (though it would be more accurate to call them **program-defined namespaces**).

We start with the namespace keyword, followed by an identifier for the namespace, and then curly braces with the content of the namespace inside.

Historically, namespace names have not been capitalized, and many style guides still recommend this convention.

The syntax for a namespace is as follows:

A screenshot of a code editor showing the syntax for defining a namespace. The text 'namespace Example' is highlighted in blue, followed by an opening curly brace '{' and a closing curly brace '}' on the next line. The code is set against a dark background with a light blue line number indicator on the left.

Advanced:

- It is convention to name program-defined types starting with a capital letter. Using the same convention for program-defined namespaces is consistent (especially when using a qualified name such as `Foo::x`, where `Foo` could be a namespace or a class type).
- It helps prevent naming collisions with other system-provided or library-provided lower-cased names.
- The C++20 standards document uses this style.
- The C++ Core guidelines document uses this style.

We recommend starting namespace names with a capital letter. However, **either style should be seen as acceptable.**

A namespace must be defined either in the global scope, or inside another namespace. Much like the content of a function, the content of a namespace is conventionally indented one level. You may occasionally see an optional semicolon placed after the closing brace of a namespace.

There are two different ways to tell the compiler which version of a function to use, via the **scope resolution operator**, or via **using statements**.

10.2.2 Accessing a namespace with the scope resolution operator (::)

The best way to tell the compiler to look in a particular namespace for an identifier is to use the **scope resolution operator (::)**. The scope resolution operator **tells the compiler that the identifier specified by the right-hand operand should be looked for in the scope of the left-hand operand**.

Here is an example of using the scope resolution operator to tell the compiler that we explicitly want to use the version of doSomething() that lives in the Foo namespace:

```
1  #include <iostream>
2
3  namespace Foo // define a namespace named Foo
4  {
5      // This doSomething() belongs to namespace Foo
6      int doSomething(int x, int y)
7      {
8          return x + y;
9      }
10 }
11
12 namespace Goo // define a namespace named Goo
13 {
14     // This doSomething() belongs to namespace Goo
15     int doSomething(int x, int y)
16     {
17         return x - y;
18     }
19 }
20
21 int main()
22 {
23     std::cout << Foo::doSomething(4, 3) << '\n'; // use the doSomething() that exists in namespace Foo
24     return 0;
25 }
```

10.2.3 Using the scope resolution operator with no name prefix

The scope resolution operator can also be used in front of an identifier without providing a namespace name (e.g. ::doSomething). In such a case, the identifier (e.g. doSomething) is looked for in the **global namespace**.

10.2.4 Identifier resolution from within a namespace

If an identifier inside a namespace is used and no scope resolution is provided, the compiler will first try to find a matching declaration in that same namespace. If no matching identifier is found, the compiler will then check each containing namespace in sequence to see if a match is found, with the global namespace being checked last.

```
namespace Foo
{
    void print() // this print() lives in the Foo namespace
    {
        std::cout << "Hello";
    }

    void printHelloThere()
    {
        print(); // calls print() in Foo namespace
        ::print(); // calls print() in global namespace
    }
}
```

In the above example, `print()` is called with no scope resolution provided. Because this use of `print()` is inside the `Foo` namespace, the compiler will first see if a declaration for `Foo::print()` can be found. Since one exists, `Foo::print()` is called.

If `Foo::print()` had not been found, the compiler would have checked the containing namespace (in this case, the global namespace) to see if it could match a `print()` there.

10.2.5 Forward declaration of content in namespaces

In lesson: Header files we discussed how we can use header files to propagate forward declarations. For identifiers inside a namespace, those forward declarations also need to be inside the same namespace:

```
1  #ifndef ADD_H
2  #define ADD_H
3
4  namespace BasicMath
5  {
6      // function add() is part of namespace BasicMath
7      int add(int x, int y);
8  }
9
10 #endif
```

If the forward declaration for `add()` wasn't placed inside namespace `BasicMath`, then `add()` would be declared in the global namespace instead, and the compiler would complain that it hadn't seen a declaration for the call to `BasicMath::add(4, 3)`. If the definition of function `add()` wasn't inside namespace `BasicMath`, the linker would complain that it couldn't find a matching definition for the call to `BasicMath::add(4, 3)`.

10.2.6 Multiple namespace blocks are allowed

It's **legal to declare namespace blocks in multiple locations** (either across multiple files, or multiple places within the same file). All declarations within the namespace are considered part of the namespace.

The **standard library makes extensive use of this feature**, as each standard library header file contains its declarations inside a namespace `std` block contained within that header file.

Otherwise, the entire standard library would have to be defined in a single header file!

Note that **this capability also means you could add your own functionality to the `std` namespace**. Doing so **causes undefined behavior** most of the time, **because the `std` namespace has a special rule prohibiting extension from user code**.

Warning: Do not add custom functionality to the `std` namespace.

10.2.7 Nested namespaces

Namespaces can be nested inside other namespaces.

```
1
2
3 namespace Foo
4 {
5     namespace Goo // Goo is a namespace inside the Foo namespace
6     {
7         int add(int x, int y)
8         {
9             return x + y;
10        }
11    }
12 }
13
14 int main()
15 {
16     std::cout << Foo::Goo::add(1, 2) << '\n';
17     return 0;
18 }
```

Note that because namespace Goo is inside of namespace Foo, we access add as `Foo::Goo::add`.

Since C++17, nested namespaces can also be declared this way:

```
namespace Foo::Goo // Goo is a namespace inside the Foo namespace (C++17 style)
{
    int add(int x, int y)
    {
        return x + y;
    }
}
```

This is equivalent to the prior example.

If you later need to add declarations to the Foo namespace (only), you can define a separate Foo namespace to do so:

```
namespace Foo::Goo // Goo is a namespace inside the Foo namespace (C++17 style)
{
    int add(int x, int y)
    {
        return x + y;
    }
}

namespace Foo
{
    void someFcn() {} // This function is in Foo only
}
```

10.2.8 *Namespace aliases*

Because typing the qualified name of a variable or function inside a nested namespace can be painful, C++ allows you to create **namespace aliases**, which allow us to temporarily shorten a long sequence of namespaces into something shorter:

namespace Active = Foo::Goo; active now refers to Foo::Goo.

One nice advantage of namespace aliases: **If you ever want to move the functionality within Foo::Goo to a different place, you can just update the Active alias** to reflect the new destination, rather than having to find/replace every instance of Foo::Goo.

10.2.9 How to use namespaces

It's worth noting that **namespaces in C++ were not originally designed as a way to implement an information hierarchy** -- they were **designed primarily as a mechanism for preventing naming collisions**. As **evidence of this, note that the entirety of the standard library lives under the single top-level namespace `std`**. Newer standard library features that introduce lots of names have started using nested namespaces (e.g. `std::ranges`) to avoid naming collisions within the `std` namespace.

- **Small applications** developed for your own use **typically do not need to be placed in namespaces**. However, **for larger personal projects that include lots of third-party libraries, namespaces your code can help prevent naming collisions** with libraries that aren't properly namespaced.
- **Any code that will be distributed to others should definitely be namespaced** to prevent conflicts with the code it is integrated into. Often a single top-level namespace will suffice (e.g. `Foologger`). As an additional advantage, placing library code inside a namespace also allows the user to see the contents of your library by using their editor's auto-complete and suggestion feature (e.g. if you type `Foologger`, autocomplete will show you all of the names inside `Foologger`).
- **In multi-team organizations, two-level or even three-level namespaces are often used to prevent naming conflicts between code generated by different teams**. These often take the form of one of the following:
 - Project or library :: module (e.g. `Foologger::Lang`)
 - Company or org :: project or library (e.g. `Foosoft::Foologger`)
 - Company or org :: project or library :: module (e.g. `Foosoft::Foologger::Lang`)

Use of module-level namespaces can help separate code that might be reusable later from application-specific code that will not be reusable. For example, physics and math functions could go into one namespace (e.g. `Math::`). Language and localization functions in another (e.g. `Lang::`). However, directory structures can also be used for this (with app-specific code in the project directory tree, and reusable code in a separate shared directory tree).

In general, **you should avoid deeply nested namespaces (more than 3 levels)**.

10.3 Local variables

Local variables, which are **variables that are defined inside a function (including function parameters)**.

It turns out that **C++ actually doesn't have a single attribute that defines a variable as being a local variable. Instead, local variables have several different properties that differentiate how these variables behave from other kinds of (non-local) variables.** We'll explore these properties in this and upcoming lessons.

In lesson: Introduction to local scope, we also introduced the **concept of scope**. An identifier's scope determines where an identifier can be accessed within the source code. When an identifier can be accessed, we say it is **in scope**. When an identifier cannot be accessed, we say it is **out of scope**. Scope is a compile-time property, and trying to use an identifier when it is out of scope will result in a compile error.

10.3.1 Local variables have block scope

Local variables have **block scope**, which means they are *in scope* from their **point of definition to the end of the block** they are defined within.

Although **function parameters** are not defined inside the function body, for typical functions they can be **considered to be part of the scope of the function body block**.

10.3.2 All variable names within a scope must be unique

Variable names must be unique within a given scope, otherwise any reference to the name will be ambiguous. Consider the following program:

```
void someFunction(int x)
{
    int x{}; // compilation failure due to name collision with function parameter
}
```

10.3.3 *Local variables have automatic storage duration*

A variable's **storage duration** (usually just called **duration**) determines what rules govern when and **how a variable will be created (instantiated) and destroyed**. In most cases, a variable's storage duration directly determines its **lifetime**.

For example, local variables have **automatic storage duration**, which means they are created at the point of definition and destroyed at the end of the block they are defined in. For this reason, local variables are sometimes called **automatic variables**.

10.3.4 *Local variables in nested blocks*

Local variables can be defined inside nested blocks. This works identically to local variables in function body blocks:

```
int main() // outer block
{
    int x { 5 }; // x enters scope and is created here

    { // nested block
        int y { 7 }; // y enters scope and is created here
    } // y goes out of scope and is destroyed here

    // y can not be used here because it is out of scope in this block

    return 0;
} // x goes out of scope and is destroyed here
```

10.3.5 Local variables have no linkage

Identifiers have another property named *linkage*. An identifier's **linkage determines whether a declaration of that same identifier in a different scope refers to the same object (or function).**

Local variables have no linkage. Each declaration of an identifier with no linkage refers to a unique object or function.

For example:

```
1  int main()
2  {
3      int x { 2 }; // local variable, no linkage
4
5      {
6          int x { 3 }; // this declaration of x refers to a different object than the previous x
7      }
8
9      return 0;
10 }
```

Scope and linkage may seem somewhat similar. However,

- Scope, determines: **where declaration of a single identifier can be seen and used** in the code.
- Linkage determines: whether multiple **declarations of the same identifier refer to the same object or not.**

Linkage isn't very interesting in the context of local variables, but we'll talk about it more in the next few lessons.

10.3.6 Variables should be defined in the most limited scope

If a variable is only used within a nested block, it should be defined inside that nested block:

```
int main()
{
    // do not define y here

    {
        // y is only used inside this block, so define it here
        int y { 5 };
        std::cout << y << '\n';
    }

    // otherwise y could still be used here, where it's not needed

    return 0;
}
```

By limiting the scope of a variable, you reduce the complexity of the program because the number of active variables is reduced. Further, it **makes it easier to see where variables are used (or aren't used)**. A variable defined inside a block can only be used within that block (or nested blocks). This can make the program easier to understand.

If a variable is needed in an outer block, it needs to be declared in the outer block:

```
int y { 5 }; // we're declaring y here because we need it in this outer block later

{
    int x{};
    std::cin >> x;

    // if we declared y here, immediately before its actual first use...
    if (x == 4)
        y = 4;
} // ... it would be destroyed here

std::cout << y; // and we need y to exist here
```

New developers sometimes wonder whether it's worth creating a nested block just to intentionally limit a variable's scope (and force it to go out of scope / be destroyed early). Doing so makes that variable simpler, but the overall function becomes longer and more complex as a result. **The tradeoff generally isn't worth it.** If creating a nested block seems useful to intentionally limit the scope of a chunk of code, that code might be better to put in a separate function instead.

Best practice: Define variables in the most limited existing scope. Avoid creating new blocks whose only purpose is to limit the scope of variables.

10.4 Introduction to global variables

We covered that local variables are variables defined inside a function body. Local variables have block scope (are only visible within the block they are declared in), and have automatic duration (they are created at the point of definition and destroyed when the block is exited).

In C++, variables can also be declared *outside* of a function. Such variables are called **global variables**.

10.4.1 Declaring global variables

By convention, global variables are declared at the top of a file, below the includes, in the global namespace. Here's an example of a global variable being defined:

```
1  #include <iostream>
2
3  // Variables declared outside of a function are global variables
4  int g_x {}; // global variable g_x
5
6  void doSomething()
7  {
8      // global variables can be seen and used everywhere in the file
9      g_x = 3;
10     std::cout << g_x << '\n';
11 }
12
13 int main()
14 {
15     doSomething();
16     std::cout << g_x << '\n';
17
18     // global variables can be seen and used everywhere in the file
19     g_x = 5;
20     std::cout << g_x << '\n';
21
22     return 0;
23 }
24 // g_x goes out of scope here
```

10.4.2 The scope of global variables

Identifiers declared in the global namespace have **global namespace scope** (commonly called **global scope**, and sometimes informally called **file scope**), which means they are visible from the **point of declaration until the end of the *file*** in which they are declared.

Once declared, a global variable **can be used anywhere in the file from that point onward!** In the above example, global variable `g_x` is used in both functions `doSomething()` and `main()`.

Global variables **can also be defined inside a user-defined namespace**. Here is the same example as above, but `g_x` has been moved from the global scope into user-defined namespace `Foo`:

```
namespace Foo // Foo is defined in the global scope
{
    int g_x {}; // g_x is now inside the Foo namespace, but is still a global variable
}
```

Although the identifier `g_x` is now limited to the scope of namespace `Foo`, that name is still globally accessible (via `Foo::g_x`), and `g_x` is still a global variable.

Key insight: Variables declared inside a namespace are also global variables.

Best practice: Prefer defining global variables inside a namespace rather than in the global namespace.

10.4.3 Global variables have static duration

Global variables are created when the program starts (before `main()` begins execution), and **destroyed when it ends**. This is called **static duration**. Variables with *static duration* are sometimes called **static variables**.

10.4.4 Naming global variables

By convention, **some developers prefix global variable identifiers with “g” or “g_”** to indicate that they are global. This prefix serves several purposes:

- It helps **avoid naming collisions** with other identifiers in the global namespace.
- It helps **prevent inadvertent name shadowing**
- It helps **indicate that the prefixed variables persist beyond the scope of the function**, and thus any changes we make to them will also persist.

Global variables defined inside a user-defined namespace often omit the prefix (since the first two points in the list above are not an issue in this case, and we can infer that a variable is a global when we see a prepended namespace name). However, it doesn't hurt if you want to keep the prefix as a more visible reminder of the third point.

Best practice: Consider using a “g” or “g_” prefix when naming global variables (**especially those defined in the global namespace**), to help differentiate them from local variables and function parameters.

10.4.5 Global variable initialization

Unlike local variables, which are uninitialized by default, **variables with static duration are zero-initialized by default**.

Non-constant global variables can be optionally initialized:

```
1 int g_x;      // no explicit initializer (zero-initialized by default)
2 int g_y {};   // value initialized (resulting in zero-initialization)
3 int g_z { 1 }; // list initialized with specific value
```

10.4.6 Constant global variables

Just like local variables, global variables can be constant. **As with all constants, constant global variables must be initialized.**

```
const int g_x;      // error: constant variables must be initialized
constexpr int g_w;  // error: constexpr variables must be initialized

const int g_y { 1 }; // const global variable g_y, initialized with a value
constexpr int g_z { 2 }; // constexpr global variable g_z, initialized with a value
```

10.4.7 A word of caution about (non-constant) global variables

New programmers are often tempted to use lots of global variables, because they can be used without having to explicitly pass them to every function that needs them. However, **use of non-constant global variables should generally be avoided altogether!** We'll discuss why in upcoming lesson: Why (non-const) global variables are evil.

10.4.8 Quick Summary

```
1 // Non-constant global variables
2 int g_x;           // defines non-initialized global variable (zero initialized by default)
3 int g_x {};        // defines explicitly value-initialized global variable
4 int g_x { 1 };     // defines explicitly initialized global variable
5
6 // Const global variables
7 const int g_y;      // error: const variables must be initialized
8 const int g_y { 2 }; // defines initialized global const
9
10 // Constexpr global variables
11 constexpr int g_y;  // error: constexpr variables must be initialized
12 constexpr int g_y { 3 }; // defines initialized global constexpr
```


10.5 Variable shadowing (name hiding)

Each block defines its own scope region. So, **what happens when we have a variable inside a nested block that has the same name as a variable in an outer block?** When this happens, the nested variable “hides” the outer variable in areas where they are both in scope. This is called **name hiding** or **shadowing**.

10.5.1 Shadowing of local variables

```
1  #include <iostream>
2
3  int main()
4  { // outer block
5      int apples { 5 }; // here's the outer block apples
6
7      { // nested block
8          // apples refers to outer block apples here
9          std::cout << apples << '\n'; // print value of outer block apples
10
11         int apples{ 0 }; // define apples in the scope of the nested block
12
13         // apples now refers to the nested block apples
14         // the outer block apples is temporarily hidden
15
16         apples = 10; // this assigns value 10 to nested block apples, not outer block apples
17
18         std::cout << apples << '\n'; // print value of nested block apples
19     } // nested block apples destroyed
20
21     std::cout << apples << '\n'; // prints value of outer block apples
22
23     return 0;
24 } // outer block apples destroyed
```

If you run this program, it prints:

- 5
- 10
- 5

Note that **if the nested block apples had not been defined, the name apples in the nested block would still refer to the outer block apples**, so the assignment of value 10 to apples would have applied to the outer block apples.

10.5.2 Shadowing of global variables

Similar to how variables in a nested block can shadow variables in an outer block, **local variables with the same name as a global variable will shadow the global variable** wherever the local variable is in scope.

However, because global variables are part of the global namespace, **we can use the scope operator (::) with no prefix to tell the compiler we mean the global variable instead of the local variable.**

10.5.3 Avoid variable shadowing

Shadowing of local variables should generally be avoided, as it can lead to inadvertent errors where the wrong variable is used or modified. Some compilers will issue a warning when a variable is shadowed.

For the same reason that we recommend avoiding shadowing local variables, we recommend avoiding shadowing global variables as well. This is trivially avoidable if all of your global names use a “g_” prefix.

Best practice: Avoid variable shadowing.

10.6 Internal linkage

In lesson: Local variables, we said, “**An identifier’s linkage determines whether other declarations of that name refer to the same object or not**”, and we discussed how local variables have no linkage.

Global variable and functions identifiers can have either internal linkage or external linkage.

We’ll cover the internal linkage case in this lesson, and the external linkage case in lesson: External linkage and variable forward declarations.

An identifier with internal linkage can be seen and used within a single translation unit, but it is not accessible from other translation units. **This means that if two source files have identically named identifiers with internal linkage, those identifiers will be treated as independent (and do not result in an ODR violation for having duplicate definitions).**

Key insight: Identifiers with internal linkage may not be visible to the linker at all. Alternatively, they may be visible to the linker, but marked for use in a specific translation unit only.

10.6.1 Global variables with internal linkage

Global variables with internal linkage are sometimes called internal variables.

To make a **non-constant global variable internal**, we use the **static** keyword.

Const and constexpr global variables have internal linkage by default (and thus don't need the **static** keyword -- if it is used, it will be ignored).

Internally linked globals recap:

- Static
- Const
- Cconstexpr

10.6.2 Functions with internal linkage

As noted above, function identifiers also have linkage. Functions default to external linkage (which we'll cover in the next lesson), but can be set to internal linkage via the **static** keyword:

```
1 // This function is declared as static, and can now be used only within this file
2 // Attempts to access it from another file via a function forward declaration will fail
3 [[maybe_unused]] static int add(int x, int y)
4 {
5     return x + y;
6 }
```

10.6.3 The one-definition rule and internal linkage

The one-definition rule says that an object or function can't have more than one definition, either within a file or a program.

However, it's worth noting that internal objects (and functions) that are defined in different files **are considered to be independent entities** (even if their names and types are identical), so **there is no violation of the one-definition rule**. Each internal object only has one definition.

10.6.4 *static vs unnamed namespaces*

In modern C++, use of the **static keyword for giving identifiers internal linkage is falling out of favor.**

Unnamed namespaces can give internal linkage to a wider range of identifiers (e.g. type identifiers), and they are better suited for giving many identifiers internal linkage.

10.6.5 *Why bother giving identifiers internal linkage?*

There are typically two reasons to give identifiers internal linkage:

- There is **an identifier we want to make sure isn't accessible to other files**. This could be a global variable we don't want messed with, or a helper function we don't want called.
- **To be pedantic about avoiding naming collisions**. Because identifiers with internal linkage aren't exposed to the linker, they can only collide with names in the same translation unit, not across the entire program.

Many modern development guides recommend giving every variable and function that isn't meant to be used from another file internal linkage. If you have the discipline, this is a good recommendation.

For now, we'll recommend a lighter-touch approach as a minimum: **give internal linkage to any identifier that you have an explicit reason to disallow access from other files.**

Best practice:

- Give identifiers internal linkage when you have an explicit reason to disallow access from other files.
- Consider giving all identifiers you don't want accessible to other files internal linkage (use an unnamed namespace for this).

10.6.6 Summary

```
1 // Internal global variables definitions:
2 static int g_x;           // defines non-initialized internal global variable (zero initialized by default)
3 static int g_x{ 1 };      // defines initialized internal global variable
4
5 const int g_y { 2 };      // defines initialized internal global const variable
6 constexpr int g_y { 3 }; // defines initialized internal global constexpr variable
7
8 // Internal function definitions:
9 static int foo() {};      // defines internal function
```

10.7 External linkage and variable forward declarations

In the prior lesson, we discussed how **internal linkage limits the use of an identifier to a single file**. In this lesson, we'll explore the concept of external linkage.

An identifier with **external linkage can be seen and used both from the file in which it is defined, and from other code files** (via a forward declaration). In this sense, **identifiers with external linkage are truly “global”** in that they can be used anywhere in your program!

Key insight:

Identifiers with **external linkage are visible to the linker**. This allows the linker to do two things:

- **Connect an identifier used in one translation unit with the appropriate definition in another translation unit.**
- **Deduplicate inline identifiers** so one canonical definition remains.

10.7.1 Functions have external linkage by default

In: Programs with multiple code files, you learned that you can call a function defined in one file from another file. This is because functions have external linkage by default.

In order to call a function defined in another file, you must place a forward declaration for the function in any other files wishing to use the function. The **forward declaration tells the compiler about the existence of the function, and the linker connects the function calls to the actual function definition**.

10.7.2 Global variables with external linkage

Non-const global variables are external by default (if used, the `extern` keyword will be ignored).

Global variables with external linkage are sometimes called **external variables**. To make a global (**internal**) variable external (and thus accessible by other files), we can **use the `extern` keyword to do so**:

```
1 int g_x { 2 }; // non-constant globals are external by default
2
3 extern const int g_y { 3 }; // const globals can be defined as extern, making them external
4 extern constexpr int g_z { 3 }; // constexpr globals can be defined as extern, making them external (but this is pretty useless, see the
  warning in the next section)
```

10.7.3 Variable forward declarations via the `extern` keyword

To actually use an external global variable that has been defined in another file, you also must place a **forward declaration for the global variable in any other files wishing to use the variable**. For variables, **creating a forward declaration is also done via the `extern` keyword (with no initialization value)**.

Note that the **`extern` keyword has different meanings in different contexts**.

- In some contexts, `extern` means “give this variable external linkage”.
- In other contexts, `extern` means “this is a forward declaration for an external variable that is defined somewhere else”.

Warning:

If you want to define an uninitialized non-const global variable, do not use the `extern` keyword, otherwise C++ will think you’re trying to make a forward declaration for the variable.

Although `constexpr` variables can be given external linkage via the `extern` keyword, **they cannot be forward declared as `constexpr`**. This is because the compiler needs to know the value of the `constexpr` variable (at compile time). If that value is defined in some other file, the compiler has no visibility on what value was defined in that other file.

However, **you can forward declare a `constexpr` variable as `const`**, which the compiler will treat as a runtime `const`. **This isn’t particularly useful**. In this case modern C++ developers make use of inline globals, this keeps them compile-time and external.

Note that function forward declarations don't need the `extern` keyword -- the compiler is able to tell whether you're defining a new function or making a forward declaration based on whether you supply a function body or not. Variables forward declarations *do* need the `extern` keyword **to help differentiate uninitialized variables definitions from variable forward declarations** (they look otherwise identical):

```
1 // non-constant
2 int g_x; // variable definition (can have initializer if desired)
3 extern int g_x; // forward declaration (no initializer)
4
5 // constant
6 extern const int g_y { 1 }; // variable definition (const requires initializers)
7 extern const int g_y; // forward declaration (no initializer)
```

10.7.4 Quick summary

```
1 // Forward declarations:
2 extern int g_y;           // forward declaration for non-constant global variable
3 extern const int g_y;     // forward declaration for const global variable
4 extern constexpr int g_y; // not allowed: constexpr variables can't be forward declared
5
6 // External global variable definitions:
7 int g_x;                 // defines non-initialized external global variable (zero initialized by default)
8 extern const int g_x{ 1 }; // defines initialized const external global variable
9 extern constexpr int g_x{ 2 }; // defines initialized constexpr external global variable
```

10.8 Why (non-const) global variables are evil

If you were to ask a veteran programmer for one piece of advice on good programming practices, after some thought, the most likely answer would be, “**Avoid global variables!**”. And with good reason: global variables are one of the most historically abused concepts in the language. Although they may seem harmless in small academic programs, they are often problematic in larger ones.

New programmers are often tempted to use lots of global variables, because they are easy to work with, especially when many calls to different functions are involved (passing data through function parameters is a pain). However, this is generally a bad idea. **Many developers believe non-const global variables should be avoided completely!**

But before we go into why, we should make a clarification. When developers tell you that global variables are evil, **they’re usually not talking about all global variables**. They’re **mostly talking about non-const global variables**.

10.8.1 Why (non-const) global variables are evil

By far the biggest reason non-const global variables are dangerous is because **their values can be changed by any function that is called**, and there is no easy way for the programmer to know that this will happen.

In simple words: Globals can be accessed from anywhere in the code, by any function. Generally, we can sometimes forget what every function does, thus not remembering what the value of a global is at any given time. It’s more of a management issue than anything else.

Consider the following program:

```
1 #include <iostream>
2
3 int g_mode; // declare global variable (will be zero-initialized by default)
4
5 void doSomething()
6 {
7     g_mode = 2; // set the global g_mode variable to 2
8 }
9
10 int main()
11 {
12     g_mode = 1; // note: this sets the global g_mode variable to 1. It does not declare a local g_mode variable!
13
14     doSomething();
15
16     // Programmer still expects g_mode to be 1
17     // But doSomething changed it to 2!
18
19     if (g_mode == 1)
20     {
21         std::cout << "No threat detected.\n";
22     }
23     else
24     {
25         std::cout << "Launching nuclear missiles...\n";
26     }
27
28     return 0;
29 }
```

In short, global variables make the program's state unpredictable. Every function call becomes potentially dangerous, and the programmer has no easy way of knowing which ones are dangerous and which ones aren't! Local variables are much safer because other functions cannot affect them directly.

There are plenty of other good reasons not to use non-const globals.

With global variables, it's not uncommon to find a piece of code that looks like this:

```
1 void someFunction()
2 {
3     // useful code
4
5     if (g_mode == 4)
6     {
7         // do something good
8     }
9 }
```

Best practice: Use local variables instead of global variables whenever possible.

10.8.2 The initialization order problem of global variables

Initialization of static variables (which includes global variables) happens as part of program startup, before execution of the main function. This proceeds in two phases.

The first phase is called **static initialization**. In the static initialization phase, global variables with **constexpr initializers (including literals) are initialized to those values**. Also, global variables without initializers are zero-initialized.

The second phase is called **dynamic initialization**. This phase is more complex and nuanced, but the gist of it is that **global variables with non-constexpr initializers are initialized**.

Within a single file, for each phase, global variables are generally initialized in order of definition (there are a few exceptions to this rule for the dynamic initialization phase). Given this, you need to be careful not to have variables dependent on the initialization value of other variables that won't be initialized until later.

```
3  int initX(); // forward declaration
4  int initY(); // forward declaration
5
6  int g_x{ initX() }; // g_x is initialized first
7  int g_y{ initY() };
8
9  int initX()
10 {
11     return g_y; // g_y isn't initialized when this is called
12 }
13
14 int initY()
15 {
16     return 5;
17 }
```

Warning: Avoid initializing objects with static duration using other objects with static duration from a different translation unit.

Dynamic initialization of global variables is also susceptible to initialization order issues and should be avoided whenever possible.

10.8.3 *So, what are very good reasons to use non-const global variables?*

There aren't many. In most cases, using local variables and passing them as arguments to other functions is preferable. But in some cases, **judicious use of non-const global variables can actually reduce program complexity, and in these rare cases, their use may be better than the alternatives.**

A good example is a log file, where you can dump error or debug information. It probably makes sense to define this as a global, **because you're likely to only have one such log in a program and it will likely be used everywhere in your program.** Another good example would be a random number generator.

For what it's worth, **the `std::cout` and `std::cin` objects are implemented as global variables** (inside the `std` namespace).

As a rule of thumb, any use of a global variable should meet at least the following **two criteria**:

- **There should only ever be one of the thing the variable represents** in your program
- **and its use should be ubiquitous** throughout your program.

Many new programmers make the mistake of thinking that something can be implemented as a global because only one is needed right now. For example, you might think that because you're implementing a single player game, you only need one player. But what happens later when you want to add a multiplayer mode (versus or hotseat)?

10.8.4 Protecting yourself from global destruction

If you do find a good use for a non-const global variable, a few useful bits of advice will minimize the amount of trouble you can get into. This advice isn't only for non-const global variables, but can help with all global variables.

First, **prefix** all non-namespaced global variables with "g" or "g_", **or better yet, put them in a namespace**, to reduce the chance of naming collisions.

Second, instead of allowing direct access to the global variable, it's a better practice to **"encapsulate"** the variable. Make sure the variable can only be accessed from within the file it's declared in, e.g. by making the variable static or const, then provide external global "access functions" to work with the variable. These functions can ensure proper usage is maintained (e.g. do input validation, range checking, etc...). Also, if you ever decide to change the underlying implementation (e.g. move from one database to another), you only have to update the access functions instead of every piece of code that uses the global variable directly.

```
1 namespace constants
2 {
3     constexpr double gravity { 9.8 }; // has internal linkage, is accessible only within this file
4 }
5
6 double getGravity() // has external linkage, can be accessed by other files
7 {
8     // We could add logic here if needed later
9     // or change the implementation transparently to the callers
10    return constants::gravity;
11 }
```

Third, when writing an otherwise standalone function that uses the global variable, **don't use the variable directly in your function body. Pass it in as an argument instead.** That way, if your function ever needs to use a different value for some circumstance, you can simply vary the argument. **This helps maintain modularity.**

10.9 Sharing global constants across multiple files (using inline variables)

In some applications, certain symbolic constants may need to be used throughout your code (not just in one location). These can include physics or mathematical constants that don't change (e.g. π or Avogadro's number), or application-specific "tuning" values (e.g. friction or gravity coefficients). **Instead of redefining these constants in every file that needs them (a violation of the "Don't Repeat Yourself" rule), it's better to declare them once in a central location and use them wherever needed.** That way, if you ever need to change them, **you only need to change them in one place**, and those changes can be propagated out. This lesson discusses the most common ways to do this.

10.9.1 Global constants as internal variables

Prior to C++17, the following is the easiest and most common solution:

1. **Create a header file** to hold these constants
2. Inside this header file, **define a namespace**
3. **Add all your constants** inside the namespace (**make sure they're constexpr**)
4. **#include the header file** wherever you need it

```
1  #ifndef CONSTANTS_H
2  #define CONSTANTS_H
3
4  // define your own namespace to hold constants
5  namespace constants
6  {
7      // all constants have internal linkage by default
8      constexpr double pi { 3.14159 };
9      constexpr double avogadro { 6.0221413e23 };
10     constexpr double myGravity { 9.2 }; // m/s^2 -- gravity is light on this planet
11     // ... other related constants
12 }
13 #endif
```

10.9.2 Global constants as external variables

The above method has a few potential **downsides**.

While this is simple (and fine for smaller programs), every time `constants.h` gets `#included` into a different code file, each of these variables is copied into the including code file. Therefore, **if `constants.h` gets included into 20 different code files, each of these variables is duplicated 20 times**. **Header guards won't stop this from happening, as they only prevent a header from being included more than once into a single including file**, not from being included one-time into multiple different code files. This introduces **two challenges**:

1. **Changing a single constant value would require recompiling every file** that includes the constants header, which can lead to lengthy rebuild times for larger projects.
2. **If the constants are large in size and can't be optimized away, this can use a lot of memory.**

One way to avoid these problems is by turning these constants into external variables, since we can then have a single variable (initialized once) that is shared across all files. In this method, we'll **define the constants in a `.cpp` file (to ensure the definitions only exist in one place)**, and **put forward declarations in the header (which will be included by other files)**.

`constants.cpp:`

```
1 | #include "constants.h"
2 |
3 | namespace constants
4 | {
5 |     // actual global variables
6 |     extern constexpr double pi { 3.14159 };
7 |     extern constexpr double avogadro { 6.0221413e23 };
8 |     extern constexpr double myGravity { 9.2 }; // m/s^2 -- gravity is light on this planet
9 | }
```

`constants.h:`

```
1 | #ifndef CONSTANTS_H
2 | #define CONSTANTS_H
3 |
4 | namespace constants
5 | {
6 |     // since the actual variables are inside a namespace, the forward declarations need to be inside a namespace as well
7 |     // we can't forward declare variables as constexpr, but we can forward declare them as (runtime) const
8 |     extern const double pi;
9 |     extern const double avogadro;
10 |    extern const double myGravity;
11 | }
12 |
13 | #endif
```

Because global symbolic constants should be namespaced (to avoid naming conflicts with other identifiers in the global namespace), the use of a “g_” naming prefix is not necessary.

Now **the symbolic constants will get instantiated only once** (in constants.cpp) instead of in each code file where constants.h is #included, and all uses of these constants will be linked to the version instantiated in constants.cpp. **Any changes made to constants.cpp will require recompiling only constants.cpp.**

However, there are a **couple of downsides to this method**. First, these constants are now **considered compile-time constants only within the file they are actually defined in** (constants.cpp). **In other files, the compiler will only see the forward declaration, which doesn't define a constexpr value** (and must be resolved by the linker). This means **in other files, these are treated as runtime constant values**, not compile-time constants. Thus, outside of constants.cpp, these variables can't be used anywhere that requires a compile-time constant. **Second, because compile-time constants can typically be optimized more than runtime constants, the compiler may not be able to optimize these as much.**

Key insight: In order for variables to be usable in compile-time contexts, such as array sizes, the **compiler has to see the variable's definition (not just a forward declaration).**

Because the compiler compiles each source file individually, it can only see variable definitions that appear in the source file being compiled (which includes any included headers). **For example, variable definitions in constants.cpp are not visible when the compiler compiles main.cpp. For this reason, constexpr variables cannot be separated into header and source file, they have to be defined in the header file.**

Given the above downsides, **prefer defining your constants in a header file** (either per the prior section, or per the next section). If you find that the values for your constants are changing a lot (e.g. because you are tuning the program) and this is leading to long compilation times, you can temporarily move just the offending constants into a .cpp file as needed.

10.9.3 Global constants as inline variables

Global constants as internal variables after C++17:

In lesson: Inline functions and variables, we covered inline variables, which are **variables that can have more than one definition, so long as those definitions are identical**. By making our constexpr variables inline, **we can define them in a header file and then #include them into any .cpp file that requires them**. This avoids both ODR violations and the downside of duplicated variables.

Reminder: **Constexpr functions are implicitly inline, but constexpr variables are not implicitly inline**. If you want an inline constexpr variable, you must explicitly mark it as inline.

Key insight:

Inline variables have external linkage by default, so that they are visible to the linker. This is necessary so the linker can de-duplicate the definitions.

Non-inline constexpr variables have internal linkage. If included into multiple translation units, each translation unit will get its own copy of the variable. **This is not an ODR violation because they are not exposed to the linker**.

```
constants.h:
1  #ifndef CONSTANTS_H
2  #define CONSTANTS_H
3
4  // define your own namespace to hold constants
5  namespace constants
6  {
7      inline constexpr double pi { 3.14159 }; // note: now inline constexpr
8      inline constexpr double avogadro { 6.0221413e23 };
9      inline constexpr double myGravity { 9.2 }; // m/s^2 -- gravity is light on this planet
10     // ... other related constants
11 }
12 #endif
```

We can include constants.h into as many code files as we want, but these variables will only be **instantiated once and shared across all code files**.

This method **does retain the downside of requiring every file that includes the constants header be recompiled if any constant value is changed**.

Best practice :If you need global constants and your compiler is C++17 capable, prefer defining inline constexpr global variables in a header file.

10.10 Static local variables

The term **static** is one of the most confusing terms in the C++ language, in large part because **static has different meanings in different contexts**.

In prior lessons, we covered that global variables have **static duration**, which means they are created when the program starts and destroyed when the program ends.

We also discussed how the **static keyword gives a global identifier internal linkage**, which means the identifier can only be used in the file in which it is defined.

In this lesson, we'll explore the use of the static keyword when applied to a local variable.

10.10.1 Static local variables

In lesson: Introduction to local scope, you learned that **local variables have automatic duration by default**, which means they are created at the point of definition, and destroyed when the block is exited.

Using the static keyword on a local variable changes its duration from automatic duration to static duration. This means **the variable is now created at the start of the program, and destroyed at the end of the program** (just like a global variable). As a result, **the static variable will retain its value even after it goes out of scope!**

The easiest way to show the difference between automatic duration and static duration local variables is by example.

Automatic duration:

```
void incrementAndPrint()
{
    int value{ 1 }; // automatic duration by default
    ++value;
    std::cout << value << '\n';
} // value is destroyed here
```

Static duration:

```
void incrementAndPrint()
{
    static int s_value{ 1 }; // static duration via static keyword. This initializer is only executed once.
    ++s_value;
    std::cout << s_value << '\n';
} // s_value is not destroyed here, but becomes inaccessible because it goes out of scope
```

In this program, because `s_value` has been declared as static, it is created at the program start.

Static local variables that are zero initialized or have a constexpr initializer can be initialized at program start.

Static local variables that have no initializer or a non-constexpr initializer are zero-initialized at program start. Static local variables with a non-constexpr initializer are reinitialized the first time the variable definition is encountered. The definition is skipped on subsequent calls, so no further reinitialization happens. Because they have static duration, **static local variables that are not explicitly initialized will be zero-initialized by default.**

Best practice: Initialize your static local variables. Static local variables are only initialized the first time the code is executed, not on subsequent calls.

10.10.2 ID generation

One of the **most common uses** for static duration local variables **is for unique ID generators**. Imagine a program where you have many similar objects (e.g. a game where you're being attacked by many zombies, or a simulation where you're displaying many triangles). If you notice a defect, it can be near impossible to distinguish which object is having problems. However, if each object is given a unique identifier upon creation, then it can be easier to differentiate the objects for further debugging.

Generating a unique ID number is very easy to do with a static duration local variable:

```
1 int generateID()
2 {
3     static int s_itemID{ 0 };
4     return s_itemID++; // makes copy of s_itemID, increments the real s_itemID, then returns the value in the copy
5 }
```

Static variables offer some of the benefit of global variables (they don't get destroyed until the end of the program) while limiting their visibility to block scope. **This makes them easier to understand and safer to use.**

Key insight:

A static local variable has **block scope** like a local variable, but its **lifetime is until the end of the program** like a global variable.

Static local variables **can be used when a function needs a persistent object** that is not directly accessible outside of the function.

10.10.3 Static local constants

Static local variables can be made `const` (or `constexpr`). **One good use for a `const` static local variable is when you have a function that needs to use a `const` value, but creating or initializing the object is expensive (e.g. you need to read the value from a database).** If you used a normal local variable, the variable would be created and initialized every time the function was executed. **With a `const/constexpr` static local variable, you can create and initialize the expensive object once, and then reuse it whenever the function is called.**

Key insight: Static local variables are **best used to avoid expensive local object initialization each time a function is called.**

10.10.4 Don't use static local variables to alter flow

```
int getInteger()
{
    static bool s_isFirstCall{ true };

    if (s_isFirstCall)
    {
        std::cout << "Enter an integer: ";
        s_isFirstCall = false;
    }
    else
    {
        std::cout << "Enter another integer: ";
    }

    int i{};
    std::cin >> i;
    return i;
}
```

This code does what it's supposed to do, but because we used a static local variable, **we made the code harder to understand**. If someone reads the code in `main()` without reading the implementation of `getInteger()`, they'd have no reason to assume that the two calls to `getInteger()` do something different. But the two calls do something different, which can be very confusing if the difference is more than a changed prompt.

Best practice: `Const` static local variables are generally okay to use.

Non-const static local variables should generally be avoided. If you do use them, ensure the variable never needs to be reset, and isn't used to alter program flow.

10.11 Summary Scope, Linkage, Duration, etc.

The concepts of scope, duration, and linkage cause a lot of confusion, so we're going to take an extra lesson to summarize everything. Some of these things we haven't covered yet, and they're here just for completeness / reference later.

10.11.1 Scope

An identifier's *scope* determines where the identifier can be accessed within the source code.

- Variables with **block (local) scope** can only be accessed from the point of declaration until the end of the block in which they are declared (including nested blocks). This includes:
 - Local variables
 - Function parameters
 - Program-defined type definitions (such as enums and classes) declared inside a block
- Variables and functions with **global scope** can be accessed from the point of declaration until the end of the file. This includes:
 - Global variables
 - Functions
 - Program-defined type definitions (such as enums and classes) declared inside a namespace or in the global scope

10.11.2 Duration

A variable's *duration* determines when it is created and destroyed.

- Variables with **automatic duration** are created at the point of definition, and destroyed when the block they are part of is exited. This includes:
 - Local variables
 - Function parameters
- Variables with **static duration** are created when the program begins and destroyed when the program ends. This includes:
 - Global variables
 - Static local variables
- Variables with **dynamic duration** are created and destroyed by programmer request. This includes:
 - Dynamically allocated variables

10.11.3 Linkage

An identifier's **linkage** determines whether a declaration of that same identifier in a different scope refers to the same entity (object, function, reference, etc...) or not.

Local variables have no linkage. Each declaration of an identifier with no linkage refers to a unique object or function.

- An identifier with **no linkage** means another declaration of the same identifier refers to a unique entity. Entities whose identifiers have no linkage include:
 - Local variables
 - Program-defined type identifiers (such as enums and classes) declared inside a block
- An identifier with **internal linkage** means a declaration of the same identifier within the same translation unit refers to the same object or function. Entities whose identifiers have internal linkage include:
 - Static global variables (initialized or uninitialized)
 - Static functions
 - Const global variables
 - Unnamed namespaces and anything defined within them
- An identifier with **external linkage** means a declaration of the same identifier within the entire program refers to the same object or function. Entities whose identifiers have external linkage include:
 - Non-static functions
 - Non-const global variables (initialized or uninitialized)
 - Extern const global variables
 - Inline const global variables
 - Namespaces

Identifiers with external linkage will generally cause a duplicate definition linker error if the definitions are compiled into more than one .cpp file (due to violating the one-definition rule). There are some exceptions to this rule (for types, templates, and inline functions and variables) - we'll cover these further in future lessons when we talk about those topics.

Also note that functions have external linkage by default. They can be made internal by using the static keyword.

10.11.4 Variable scope, duration, and linkage summary

Type	Example	Scope	Duration	Linkage	Notes
Local variable	<code>int x;</code>	Block	Automatic	None	
Static local variable	<code>static int s_x;</code>	Block	Static	None	
Dynamic local variable	<code>int* x { new int{} };</code>	Block	Dynamic	None	
Function parameter	<code>void foo(int x)</code>	Block	Automatic	None	
Internal non-const global variable	<code>static int g_x;</code>	Global	Static	Internal	Initialized or uninitialized
External non-const global variable	<code>int g_x;</code>	Global	Static	External	Initialized or uninitialized
Inline non-const global variable (C++17)	<code>inline int g_x;</code>	Global	Static	External	Initialized or uninitialized
Internal constant global variable	<code>constexpr int g_x { 1 };</code>	Global	Static	Internal	Must be initialized
External constant global variable	<code>extern const int g_x { 1 };</code>	Global	Static	External	Must be initialized
Inline constant global variable (C++17)	<code>inline constexpr int g_x { 1 };</code>	Global	Static	External	Must be initialized

10.11.5 Forward declaration summary

Type	Example	Notes
Function forward declaration	<code>void foo(int x);</code>	Prototype only, no function body
Non-constant variable forward declaration	<code>extern int g_x;</code>	Must be uninitialized
Const variable forward declaration	<code>extern const int g_x;</code>	Must be uninitialized
Constexpr variable forward declaration	<code>extern constexpr int g_x;</code>	Not allowed, constexpr cannot be forward declared

A constexpr variable (which is implicitly const) can be forward declared using a const variable forward declaration. **When accessed through the forward declaration, the variable will be considered const (not constexpr).**

10.11.6 What the heck is a storage class specifier?

When used as part of an identifier declaration, the **static** and **extern** keywords are called **storage class specifiers**. In this context, they set the storage duration and linkage of the identifier.

C++ supports **4 active storage class specifiers**:

Specifier	Meaning	Note
extern	static (or thread_local) storage duration and external linkage	
static	static (or thread_local) storage duration and internal linkage	
thread_local	thread storage duration	
mutable	object allowed to be modified even if containing class is const	
auto	automatic storage duration	Deprecated in C++11
register	automatic storage duration and hint to the compiler to place in a register	Deprecated in C++17

10.12 Using declarations and using directives

You've probably seen this program in a lot of textbooks and tutorials:

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      cout << "Hello world!\n";
8
9      return 0;
10 }
```

If you see this, run. Your textbook or tutorial are probably out of date. In this lesson, we'll explore why.

10.12.1 A short history lesson

Back before C++ had support for namespaces, all of the names that are now in the `std` namespace were in the global namespace. This caused naming collisions between program identifiers and standard library identifiers. Programs that worked under one version of C++ might have a naming conflict with a newer version of C++.

In 1995, namespaces were standardized, and all of the functionality from the standard library was moved out of the global namespace and into namespace `std`. This change broke older code that was still using names without `std::`.

As anyone who has worked on a large codebase knows, any change to a codebase (no matter how trivial) risks breaking the program. Updating every name that was now moved into the `std` namespace to use the `std::` prefix was a massive risk. A solution was requested.

Fast forward to today -- if you're using the standard library a lot, typing `std::` before everything you use from the standard library can become repetitive, and in some cases, can make your code harder to read.

C++ provides some solutions to both of these problems, in the form of using-statements.

But first, let's define two terms.

10.12.2 Qualified and unqualified names

A name can be either qualified or unqualified.

A qualified name is a name that includes an associated scope. Most often, names are qualified with a namespace using the scope resolution operator (`::`). For example:

```
1 | std::cout // identifier cout is qualified by namespace std
2 | ::foo    // identifier foo is qualified by the global namespace
```

Advanced: A name can also be qualified by a class name using the scope resolution operator (`::`), or by a class object using the member selection operators.

An **unqualified name** is a name that does not include a scoping qualifier. For example, `cout` and `x` are unqualified names, as they do not include an associated scope.

10.12.3 Using-declarations

One way to reduce the repetition of typing `std::` over and over is to utilize a using-declaration statement. A **using declaration** allows us to use an unqualified name (with no scope) as an alias for a qualified name.

```
int main()
{
    using std::cout; // this using declaration tells the compiler that cout should resolve to std::cout
    cout << "Hello world!\n"; // so no std:: prefix is needed here!

    return 0;
} // the using declaration expires at the end of the current scope
```

The using-declaration `using std::cout;` tells the compiler that we're going to be using the object `cout` from the `std` namespace. So, whenever it sees `cout`, it will assume that we mean `std::cout`. If there's a naming conflict between `std::cout` and some other use of `cout`, `std::cout` will be preferred. Therefore, on line 6, we can type `cout` instead of `std::cout`.

This doesn't save much effort in this trivial example, but if you are using `cout` many times inside of a function, a using-declaration can make your code more readable. Note that you will need a separate using-declaration for each name (e.g. one for `std::cout`, one for `std::cin`, etc...).

The using-declaration is active from the point of declaration to the end of the scope in which it is declared.

Although using-declarations are less explicit than using the `std::` prefix, they are generally considered safe and acceptable to use in source (.cpp) files, after the #includes (particularly inside functions).

Warning: Avoid using-declarations in header files.

10.12.4 Using-directives

Another way to simplify things is to use a using-directive. A **using directive** allows *all* identifiers in a given namespace to be referenced without qualification from the scope of the using-directive.

```
int main()
{
    using namespace std; // all names from std namespace now accessible without qualification
    cout << "Hello world!\n"; // so no std:: prefix is needed here

    return 0;
} // the using-directive ends at the end of the current scope
```

The using-directive `using namespace std;` tells the compiler that all of the names in the `std` namespace should be accessible without qualification in the current scope (in this case, of function `main()`). When we then use unqualified identifier `cout`, it will resolve to `std::cout`.

Using-directives are the solution that was provided for old pre-namespace codebases that used unqualified names for standard library functionality. Rather than having to manually update every unqualified name to a qualified name (which was risky), a single using-directive (`using namespace std;`) could be placed at the top of each file, and all of the names that had been moved to the `std` namespace could still be used unqualified.

10.12.5 Problems with using-directives (a.k.a. why you should avoid “using namespace std;”)

In modern C++, using-directives generally offer little benefit (saving some typing) compared to the risk. This is due to two factors:

1. Using-directives **allow unqualified access to *all* of the names** from a namespace (potentially including lots of names you’ll never use).
2. Using-directives **do not prefer names from the namespace of the using-directive over other names**.

The end result is that the **possibility for naming collisions to occur increases significantly** (especially if you import the `std` namespace).

10.12.6 The scope of using-declarations and using-directives

If a using-declaration or using-directive is used within a block, the names are applicable to just that block (it follows normal block scoping rules). This is a good thing, as it reduces the chances for naming collisions to occur to just within that block.

If a using-declaration or using-directive is **used in the global namespace**, the names are **applicable to the entire rest of the file (they have file scope)**.

10.12.7 Cancelling or replacing a using-statement

Once a using-statement has been declared, there's no way to cancel or replace it with a different using-statement within the scope in which it was declared.

```
1 int main()
2 {
3     using namespace Foo;
4
5     // there's no way to cancel the "using namespace Foo" here!
6     // there's also no way to replace "using namespace Foo" with a different using statement
7
8     return 0;
9 } // using namespace Foo ends here
```

The best you can do is intentionally limit the scope of the using-statement from the outset using the block scoping rules.

```
1 int main()
2 {
3     {
4         using namespace Foo;
5         // calls to Foo:: stuff here
6     } // using namespace Foo expires
7
8     {
9         using namespace Goo;
10        // calls to Goo:: stuff here
11    } // using namespace Goo expires
12
13    return 0;
14 }
```

Of course, all of this headache can be avoided by explicitly using the scope resolution operator (::) in the first place.

10.12.8 Best practices for using-statements

Avoid using-directives (particularly using namespace std;) altogether, except in specific circumstances (such as using **namespace std::literals** to access the s and sv literal suffixes).

Using-declarations are generally considered safe to use inside functions. Avoid using them in namespaces, especially the global namespace of a header file.

10.13 Unnamed and inline namespaces

C++ supports two variants of namespaces that are worth at least knowing about. We won't build on these, so **consider this lesson optional for now.**

10.13.1 Unnamed (anonymous) namespaces (internal linkage)

An **unnamed namespace** (also called an **anonymous namespace**) is a namespace that is defined without a name, like so:

```
namespace // unnamed namespace
{
    void doSomething() // can only be accessed in this file
    {
        std::cout << "v1\n";
    }
}

int main()
{
    doSomething(); // we can call doSomething() without a namespace prefix

    return 0;
}
```

All content declared in an unnamed namespace is treated as if it is part of the parent namespace. So even though function doSomething() is defined in the unnamed namespace, the function itself is accessible from the parent namespace (which in this case is the global namespace), which is why we can call doSomething() from main() without any qualifiers.

This might make unnamed namespaces seem useless. But the other effect of unnamed namespaces is that **all identifiers inside an unnamed namespace are treated as if they have internal linkage**, which means that the **content of an unnamed namespace can't be seen outside of the file in which the unnamed namespace is defined**.

For functions, **this is effectively the same as defining all functions in the unnamed namespace as static functions**. The following program is effectively identical to the one above:

```
static void doSomething() // can only be accessed in this file
{
    std::cout << "v1\n";
}

int main()
{
    doSomething(); // we can call doSomething() without a namespace prefix

    return 0;
}
```

Unnamed namespaces are **typically used when you have a lot of content that you want to ensure stays local to a given file**, as it's easier to cluster such content in a single unnamed namespace than individually mark all declarations as static. Unnamed namespaces will also keep program-defined types (something we'll discuss in a later lesson) local to the file, something for which there is no alternative equivalent mechanism to do.

10.13.2 Inline namespaces

Let's say you're not happy with `foo()`, and you want to improve it in some way that changes how it behaves. But if you do this, **you risk breaking existing programs using the older version**. How do you handle this?

One way would be to create a new version of the function with a different name. But over the course of many changes, **you could end up with a whole set of almost-identically named functions** (`foo`, `foo_v2`, `foo_v3`, etc...).

An alternative is to use an inline namespace. An inline namespace is a namespace that is **typically used to version content**. Much like an unnamed namespace, **anything declared inside an inline namespace is considered part of the parent namespace**. However, unlike unnamed namespaces, **inline namespaces don't affect linkage**.

```
inline namespace V1 // declare an inline namespace named V1
{
    void doSomething()
    {
        std::cout << "V1\n";
    }
}

namespace V2 // declare a normal namespace named V2
{
    void doSomething()
    {
        std::cout << "V2\n";
    }
}

int main()
{
    V1::doSomething(); // calls the V1 version of doSomething()
    V2::doSomething(); // calls the V2 version of doSomething()

    doSomething(); // calls the inline version of doSomething() (which is V1)

    return 0;
}
```

In the above example, callers to `doSomething()` will get the V1 (the inline version) of `doSomething()`. Callers who want to use the newer version can explicitly call `V2::doSomething()`. This preserves the function of existing programs while allowing newer programs to take advantage of newer/better variations.

Alternatively, if you want to push the newer version:

```
1  #include <iostream>
2
3  namespace V1 // declare a normal namespace named V1
4  {
5      void doSomething()
6      {
7          std::cout << "V1\n";
8      }
9  }
10
11 inline namespace V2 // declare an inline namespace named V2
12 {
13     void doSomething()
14     {
15         std::cout << "V2\n";
16     }
17 }
18
19 int main()
20 {
21     V1::doSomething(); // calls the V1 version of doSomething()
22     V2::doSomething(); // calls the V2 version of doSomething()
23
24     doSomething(); // calls the inline version of doSomething() (which is V2)
25
26     return 0;
27 }
```

10.13.3 Mixing inline and unnamed namespaces

A namespace can be both inline and unnamed.

```
inline namespace // declare an inline unnamed namespace
{
    void doSomething() // has internal linkage
    {
        std::cout << "V2\n";
    }
}
```

10.14 Summary

A compound statement or block is a group of zero or more statements that is treated by the compiler as if it were a single statement. Blocks begin with a { symbol, end with a } symbol, with the statements to be executed placed in between. Blocks can be used anywhere a single statement is allowed. No semicolon is needed at the end of a block. Blocks are often used in conjunction with if statements to execute multiple statements.

User-defined namespaces are namespaces that are defined by you for your own declarations. Namespaces provided by C++ (such as the global namespace) or by libraries (such as namespace std) are not considered user-defined namespaces.

You can **access a declaration in a namespace via the scope resolution operator (::)**. The scope resolution operator tells the compiler that the identifier specified by the right-hand operand should be looked for in the scope of the left-hand operand. If no left-hand operand is provided, the global namespace is assumed.

Local variables are variables defined within a function (including function parameters). **Local variables have block scope**, meaning they are in-scope from their point of definition to the end of the block they are defined within. **Local variables have automatic storage duration**, meaning they are created at the point of definition and destroyed at the end of the block they are defined in.

A name declared in a nested block can shadow or name hide an identically named variable in an outer block. This should be avoided.

Global variables are variables defined outside of a function. **Global variables have file scope**, which means they are visible from the point of declaration until the end of the file in which they are declared. **Global variables have static duration**, which means they are created when the program starts, and destroyed when it ends. Avoid dynamic initialization of static variables whenever possible.

An identifier's linkage determines whether other declarations of that name refer to the same object or not. **Local variables have no linkage**. Identifiers with internal linkage can be seen and used within a single file, but are not accessible from other files. Identifiers with external linkage can be seen and used both from the file in which they are defined, and from other code files (via a forward declaration).

Avoid non-const global variables whenever possible. Const globals are generally seen as acceptable. **Use inline variables to share global constants if your compiler is C++17 capable.**

Local variables can be given static duration via the static keyword.

A qualified name is a name that includes an associated scope (e.g. `std::string`). An unqualified name is a name that does not include a scoping qualifier (e.g. `string`).

Using statements (including **using declarations** and **using directives**) can be used to avoid having to qualify identifiers with an explicit namespace. A using declaration allows us to use an unqualified name (with no scope) as an alias for a qualified name. **A using directive imports all of the identifiers from a namespace into the scope of the using directive.** Both of these should generally be avoided.

Inline functions were originally designed as a way to request that the compiler replace your function call with inline expansion of the function code. You should not need to use the `inline` keyword for this purpose because the compiler will generally determine this for you. **In modern C++, the `inline` keyword is used to exempt a function from the one-definition rule**, allowing its definition to be imported into multiple code files. Inline functions are typically defined in header files so they can be `#included` into any code files that needs them.

Finally, C++ supports **unnamed namespaces**, which **implicitly treat all contents of the namespace as if it had internal linkage**. C++ also supports **inline namespaces**, which provide some primitive versioning capabilities for namespaces.

11 Control Flow

11.1 Control flow introduction

When a program is run, the CPU begins execution at the top of `main()`, executes some number of statements (in sequential order by default), and then the program terminates at the end of `main()`. The specific sequence of statements that the CPU executes is called the program's **execution path** (or **path**, for short).

Straight-line programs take the same path (execute the same statements in the same order) every time they are run.

However, often this is not what we desire. For example, if we ask the user for input, and the user enters something invalid, ideally we'd like to ask the user to make another choice. This is not possible in a straight-line program. In fact, the user may repeatedly enter invalid input, so the number of times we might need to ask them to make another selection isn't knowable until runtime.

Fortunately, C++ provides a number of different **control flow statements** (also called **flow control statements**), which are statements that allow the programmer to change the normal path of execution through the program.

When a control flow statement causes point of execution to change to a non-sequential statement, this is called **branching**.

11.1.1 Categories of flow control statements

Category	Meaning	Implemented in C++ by
Conditional statements	Causes a sequence of code to execute only if some condition is met.	if, else, switch
Jumps	Tells the CPU to start executing the statements at some other location.	goto, break, continue
Function calls	Jump to some other location and back.	function calls, return
Loops	Repeatedly execute some sequence of code zero or more times, until some condition is met.	while, do-while, for, ranged-for
Halts	Terminate the program.	<code>std::exit()</code> , <code>std::abort()</code>
Exceptions	A special kind of flow control structure designed for error handling.	try, throw, catch

Prior to this chapter, the number of things you could have a program do was fairly limited. Being able to control the flow of your program (particularly using loops) makes any number of interesting things possible! No longer will you be restricted to toy programs -- you will be able to write programs that have real utility.

This is where the real fun begins. So, let's get to it!

11.2 If statements and blocks

The first category of control flow statements we'll talk about is **conditional statements**. A conditional statement is a **statement that specifies whether some associated statement(s) should be executed or not**.

C++ supports **two basic kinds of conditionals: if statements and switch statements**.

11.2.1 If-statement recap

Syntax:

```
if (condition)
    true_statement;
else
    false_statement;
```

If the condition evaluates to true, the true_statement executes. If the condition evaluates to false and the optional else statement exists, the false_statement executes.

11.2.2 If or else with multiple conditional statements

New programmers often try something like this:

```
if (x >= constants::minRideHeightCM)
    std::cout << "You are tall enough to ride.\n";
else
    std::cout << "You are not tall enough to ride.\n";
    std::cout << "Too bad!\n"; // focus on this line
```

However, consider the following run of the program:

Enter your height (in cm): 180

You are tall enough to ride.

Too bad!

This program doesn't work as expected because the `true_statement` and `false_statement` can only be a single statement. **The indentation is deceiving us here** -- the above program executes as if it had been written as follows:

```
if (x >= constants::minRideHeightCM)
    std::cout << "You are tall enough to ride.\n";
else
    std::cout << "You are not tall enough to ride.\n";

std::cout << "Too bad!\n"; // focus on this line
```

However, it's common to want to execute multiple statements based on some condition. To do so, **we can use a compound statement (block)**:

```
if (x >= constants::minRideHeightCM)
    std::cout << "You are tall enough to ride.\n";
else
{ // note addition of block here
    std::cout << "You are not tall enough to ride.\n";
    std::cout << "Too bad!\n";
}
```

11.2.3 To block or not to block single statements

There is debate within the programmer community as to whether single statements following an `if` or `else` should be explicitly enclosed in blocks or not.

The best argument for not using blocks around single statements is that adding blocks makes you able to see less of your code at one time by spacing it out vertically, which makes your code less readable and can lead to other, more serious mistakes.

The **community seems to be more in favor of always using blocks** than not, though this recommendation certainly isn't ubiquitous.

Best practice: Consider putting single statements associated with an `if` or `else` in blocks (particularly while you are learning). More experienced C++ developers sometimes disregard this practice in favor of tighter vertical spacing.

A middle-ground alternative is to put single-lines on the same line as the if or else:

```
1 | if (age >= minDrinkingAge) purchaseBeer();  
2 | else std::cout << "No drinky for you\n".
```

One fair criticism of the single-line method is that it produces code that is harder to debug:

- Because the conditional and associated statement will execute as part of the same step, it is more difficult to tell whether the statement actually executed or was skipped.
- Because the conditional and associated statement are on the same line, you can't breakpoint just the associated statement (to stop execution only when the statement actually executes).

However, if either of the above points are hindering when debugging, you can insert a newline between the conditional and the statement (so they are on separate lines), do your debugging, then remove the newline afterward.

11.2.4 *Implicit blocks*

If the programmer does not declare a block in the statement portion of an if statement or else statement, the **compiler will implicitly declare one**. Thus:

```
if (condition)  
    true_statement;  
else  
    false_statement;
```

Is actually the equivalent of:

```
if (condition)  
{  
    true_statement;  
}  
else  
{  
    false_statement;  
}
```

Most of the time, this doesn't matter. However, **new programmers sometimes try to define variables in the implicit block**. This won't compile.

11.3 Common if statement problems

11.3.1 Nested if statements and the dangling else problem

It is possible to nest if statements within other if statements:

```
if (x >= 0) // outer if statement
    // it is bad coding style to nest if statements this way
    if (x <= 20) // inner if statement
        std::cout << x << " is between 0 and 20\n";
```

Now consider the following program:

```
if (x >= 0) // outer if statement
    // it is bad coding style to nest if statements this way
    if (x <= 20) // inner if statement
        std::cout << x << " is between 0 and 20\n";

// which if statement does this else belong to?
else
    std::cout << x << " is negative\n";
```

The above program introduces a source of potential ambiguity called a **dangling else** problem. Is the else statement in the above program matched up with the outer or inner if statement?

The answer is that an else statement is paired up with the last unmatched if statement in the same block. Thus, in the program above, the else is matched up with the inner if statement, as if the program had been written like this:

```
if (x >= 0) // outer if statement
{
    if (x <= 20) // inner if statement
        std::cout << x << " is between 0 and 20\n";
    else // attached to inner if statement
        std::cout << x << " is negative\n";
}
```

To avoid such ambiguities when nesting if statements, it is a good idea to explicitly enclose the inner if statement within a block. This allows us to attach an else to either if statement without ambiguity.

11.3.2 Flattening nested if statements

Nested if statements **can often be flattened by either restructuring the logic or by using logical operators**. Code that is **less nested is less error prone**.

For example, the above example can be flattened as follows:

```
if (x < 0)
    std::cout << x << " is negative\n";
else if (x <= 20) // only executes if x >= 0
    std::cout << x << " is between 0 and 20\n";
else // only executes if x > 20
    std::cout << x << " is greater than 20\n";
```

Here's another example that uses logical operators to check multiple conditions within a single if statement:

```
if (x > 0 && y > 0) // && is logical and -- checks if both conditions are true
    std::cout << "Both numbers are positive\n";
else if (x > 0 || y > 0) // || is logical or -- checks if either condition is true
    std::cout << "One of the numbers is positive\n";
else
    std::cout << "Neither number is positive\n";
```

11.3.3 Null statements

A **null statement** is an expression statement that consists of just a semicolon:

```
1 | if (x > 10)
2 |     ; // this is a null statement
```

Null statements do nothing. They are **typically used when the language requires a statement to exist but the programmer doesn't need one**. For readability, null statements are typically placed on their own lines.

We'll see examples of intentional null statements later in this chapter, when we cover loops. Null statements are rarely intentionally used with if statements. However, they can unintentionally cause problems for new (or careless) programmers. Consider the following snippet:

```
1 | if (nuclearCodesActivated());
2 |     blowUpTheWorld();
```

In the above snippet, the programmer accidentally put a semicolon on the end of the if statement (**a common mistake since semicolons end many statements**). This unassuming error compiles fine, and causes the snippet to execute as if it had been written like this:

```
1 | if (nuclearCodesActivated())
2 |   ; // the semicolon acts as a null statement
3 |   blowUpTheWorld(); // and this line always gets executed!
```

Warning: Be careful not to “terminate” your if statement with a semicolon, otherwise your conditional statement(s) will execute unconditionally (even if they are inside a block).

11.3.4 *Operator== vs Operator= inside the conditional*

Inside your conditional, you should be using operator== when testing for equality, not operator= (which is assignment). Consider the following program:

```
if (x = 0) // oops, we used an assignment here instead of a test for equality
    std::cout << "You entered 0\n";
else
    std::cout << "You entered 1\n";
```

This program will compile and run, but will produce the wrong result in some cases:

Enter 0 or 1: 0

You entered 1

In fact, this program will always produce the result You entered 1. This happens because `x = 0` first assigns the value 0 to `x`, then evaluates to the value of `x`, which is now 0, which is Boolean false. Since the conditional is always false, the else statement always executes.

11.4 Constexpr if statements

Normally, the conditional of an if-statement is evaluated at runtime.

However, consider the case where the conditional is a constant expression, such as in the following example:

```
constexpr double gravity{ 9.8 };  
  
// reminder: low-precision floating point literals of the same type can be tested for equality  
if (gravity == 9.8) // constant expression, always true  
    std::cout << "Gravity is normal.\n"; // will always be executed  
else  
    std::cout << "We are not on Earth.\n"; // will never be executed
```

Because gravity is constexpr and initialized with value 9.8, the conditional gravity == 9.8 must evaluate to true. As a result, the else-statement will never be executed.

Evaluating a constexpr conditional at runtime is wasteful (since the result will never vary). **It is also wasteful to compile code into the executable that can never be executed.**

11.4.1 Constexpr if statements

C++17 introduces the **constexpr if statement**, which requires the conditional to be a constant expression. The conditional of a constexpr-if-statement will be evaluated at **compile-time**.

If the constexpr conditional evaluates to true, the entire if-else will be replaced by the true-statement. If the constexpr conditional evaluates to false, the entire if-else will be replaced by the false-statement (if it exists) or nothing (if there is no else).

To use a constexpr-if-statement, we add the constexpr keyword after the if:

```
if constexpr (gravity == 9.8) // now using constexpr if  
    std::cout << "Gravity is normal.\n";  
else  
    std::cout << "We are not on Earth.\n";
```

When the above code is compiled, the compiler will evaluate the conditional at compile time, see that it is always true, and keep only the single statement `std::cout << "Gravity is normal.\n";`.

In other words, it will compile this:

```
1  int main()
2  {
3      constexpr double gravity{ 9.8 };
4
5      std::cout << "Gravity is normal.\n";
6
7      return 0;
8  }
```

Best practice: Favor `constexpr` if statements over non-`constexpr` if statements when the conditional is a constant expression.

11.4.2 Modern compilers and if statements with `constexpr` conditionals

For optimization purposes, **modern compilers will generally treat non-`constexpr` if-statements that have `constexpr` conditionals as if they were `constexpr`-if-statements. However, they are not required to do so.**

A compiler that encounters a non-`constexpr` if-statement with a `constexpr` conditional **may issue a warning advising you to use `if constexpr` instead. This will ensure that compile-time evaluation will occur (even if optimizations are disabled).**

11.5 Switch statement basics

Although it is possible to chain many if-else statements together, this is both difficult to read and inefficient. Consider the following program:

```
if (x == 1)
    std::cout << "One";
else if (x == 2)
    std::cout << "Two";
else if (x == 3)
    std::cout << "Three";
else
    std::cout << "Unknown";
```

Variable `x` in `printDigitName()` **will be evaluated up to three times** depending on the value passed in (which is inefficient), and the reader has to be sure that it is `x` being evaluated each time (not some other variable).

Because testing a variable or expression for equality against a set of different values is common, C++ provides an alternative conditional statement called a **switch statement** that is specialized for this purpose. Here is the same program as above using a switch:

```
switch (x)
{
    case 1:
        std::cout << "One";
        return;
    case 2:
        std::cout << "Two";
        return;
    case 3:
        std::cout << "Three";
        return;
    default:
        std::cout << "Unknown";
        return;
}
```

The idea behind a **switch statement** is simple: an expression (sometimes called the condition) is evaluated to produce a value. If the expression's value is equal to the value after any of the case labels, the statements after the matching case label are executed. **If no matching value can be found and a default label exists, the statements after the default label are executed instead.**

Best practice: Prefer switch statements over if-else chains when there is a choice.

11.5.1 *Starting a switch*

We start a switch statement by using the **switch keyword**, followed by parentheses with the conditional expression that we would like to evaluate inside. Often the expression is just a single variable, but it can be any valid expression.

The one restriction is that the **condition must evaluate to an integral type or an enumerated type** (covered in future lesson: Unscoped enumerations), **or be convertible to one. Expressions that evaluate to floating point types, strings, and most other non-integral types may not be used here.**

Following the conditional expression, we declare a block. Inside the block, we use **labels** to define all of the values we want to test for equality. There are two kinds of labels used with switch statements, which we'll discuss subsequently.

11.5.2 *Case labels*

The first kind of label is the case label, which is declared using the **case keyword** and **followed by a constant expression**. The constant **expression must either match the type of the condition or must be convertible to that type**.

If the value of the conditional expression equals the expression after a case label, execution begins at the first statement after that case label and then continues sequentially.

Here's an example of the condition matching a case label:

```
3 void printDigitName(int x)
4 {
5     switch (x) // x is evaluated to produce value 2
6     {
7         case 1:
8             std::cout << "One";
9             return;
10        case 2: // which matches the case statement here
11            std::cout << "Two"; // so execution starts here
12            return; // and then we return to the caller
13        case 3:
14            std::cout << "Three";
15            return;
16        default:
17            std::cout << "Unknown";
18            return;
19    }
20 }
21
22 int main()
23 {
24     printDigitName(2);
25     std::cout << '\n';
26
27     return 0;
28 }
```

This code prints: **Two**

In the above program, *x* is evaluated to produce value 2. Because there is a case label with value 2, execution jumps to the statement underneath that matching case label. The program prints Two, and then the return statement is executed, which returns back to the caller.

There is no practical limit to the number of case labels you can have, but all case labels in a switch must be unique.

If the conditional expression does not match any of the case labels, no cases are executed. We'll show an example of this shortly.

11.5.3 The default label

The second kind of label is the **default label** (often called the **default case**), which is declared using the default keyword. If the conditional expression does not match any case label and a default label exists, execution begins at the first statement after the default label.

The default label is optional, and there can only be one default label per switch statement. By convention, **the default case is placed last in the switch block.**

Best practice: Place the default case last in the switch block.

11.5.4 No matching case label and no default case

If the value of the conditional expression does not match any of the case labels, and no default case has been provided, then no cases inside the switch are executed. **Execution continues after the end of the switch block.**

11.5.5 Taking a break

In the above examples, we used **return statements to stop execution** of the statements after our labels. However, **this also exits the entire function.**

A break statement (declared using the break keyword) tells the compiler that we are done executing statements within the switch, and that execution should continue with the statement after the end of the switch block. This allows us to exit a switch statement without exiting the entire function.

Best practice: Each set of **statements underneath a label should end in a break statement or a return statement.** This includes the statements underneath the last label in the switch.

So, what happens if you don't end a set of statements under a label with a break or return? We'll explore that topic, and others, in the next lesson.

11.5.6 Labels are conventionally not indented

In lesson: Naming collisions and an introduction to namespaces, **we noted that code is typically indented one level to help identify that it's part of a nested scope region**. Since the curly braces of the switch defines a new scope region, we'd normally indent everything inside the curly braces one level.

A label, on the other hand, does not define a nested scope. Therefore, the code following a label is conventionally not indented.

However, if we indent both the labels and the subsequent statements to the same level, we end up with something like this:

```
switch (x)
{
    case 1:
        std::cout << "One";
        return;
    case 2:
        std::cout << "Two";
        return;
    case 3:
        std::cout << "Three";
        return;
    default:
        std::cout << "Unknown";
        return;
}
```

This makes it really hard to determine where each case starts and ends.

Conventionally, labels are simply not indented:

```
1 // Preferred version
2 void printDigitName(int x)
3 {
4     switch (x)
5     {
6         case 1: // not indented from switch statement
7             std::cout << "One";
8             return;
9         case 2:
10            std::cout << "Two";
11            return;
12        case 3:
13            std::cout << "Three";
14            return;
15        default:
16            std::cout << "Unknown";
17            return;
18    }
19 }
```

This makes it easy to identify each label. And because the statements are only indented one level from the switch block, it correctly implies that the statements are all part of the scope of the switch block.

In future lessons, we'll encounter other types of labels -- these are also conventionally not indented for the same reason.

Best practice: Prefer not to indent labels. This allows them to stand out from the surrounding code without implying that they are defining a nested scope region.

11.6 Switch fallthrough and scoping

This lesson continues our exploration of switch statements that we started in the prior lesson. In the prior lesson, we mentioned that **each set of statements underneath a label should end in a break statement or a return statement.**

In this lesson, we'll explore why, and talk about some switch scoping issues that sometimes trip up new programmers.

11.6.1 Fallthrough

When a switch expression matches a case label or optional default label, execution begins at the first statement following the matching label. **Execution will then continue sequentially until one of the following termination conditions happens:**

- The **end of the switch block is reached**.
- Another **control flow statement** (typically a **break** or **return**) causes the switch block or function to exit.
- **Something else interrupts the normal flow of the program** (e.g. the OS shuts the program down, the universe implodes, etc...)

Note that **the presence of another case label is *not* one of these terminating conditions** -- thus, **without a break or return, execution will overflow into subsequent cases**.

Here is a program that exhibits this behavior:

```
switch (2)
{
case 1: // Does not match
    std::cout << 1 << '\n'; // Skipped
case 2: // Match!
    std::cout << 2 << '\n'; // Execution begins here
case 3:
    std::cout << 3 << '\n'; // This is also executed
case 4:
    std::cout << 4 << '\n'; // This is also executed
default:
    std::cout << 5 << '\n'; // This is also executed
}
```

This is probably not what we wanted! When execution flows from a statement underneath a label into statements underneath a subsequent label, this is called **fallthrough**.

Since fallthrough is rarely desired or intentional, **many compilers and code analysis tools will flag fallthrough as a warning**.

11.6.2 The `[[fallthrough]]` attribute

Commenting intentional fallthrough is a common convention to tell other developers that fallthrough is intended. While this works for other developers, the compiler and code analysis tools don't know how to interpret comments, so it won't get rid of the warnings.

To help address this, C++17 adds a new attribute called `[[fallthrough]]`.

Attributes are a modern C++ feature that allows the programmer to provide the compiler with some additional data about the code. To specify an attribute, the attribute name is placed between double brackets. Attributes are not statements -- rather, they can be used almost anywhere where they are contextually relevant.

The `[[fallthrough]]` attribute modifies a null statement to indicate that fallthrough is intentional (and no warnings should be triggered):

```
switch (2)
{
case 1:
    std::cout << 1 << '\n';
    break;
case 2:
    std::cout << 2 << '\n'; // Execution begins here
    [[fallthrough]]; // intentional fallthrough -- note the semicolon to indicate the null statement
case 3:
    std::cout << 3 << '\n'; // This is also executed
    break;
}
```

Best practice: Use the `[[fallthrough]]` attribute (along with a null statement) to indicate intentional fallthrough.

11.6.3 Sequential case labels

You can use the logical OR operator to combine multiple tests into a single statement:

```
bool isVowel(char c)
{
    return (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
            c=='A' || c=='E' || c=='I' || c=='O' || c=='U');
```

This suffers from the same challenges that we presented in the introduction to switch statements: `c` gets evaluated multiple times and the reader has to make sure it is `c` that is being evaluated each time.

You can do something similar using switch statements by placing multiple case labels in sequence:

```
bool isVowel(char c)
{
    switch (c)
    {
        case 'a': // if c is 'a'
        case 'e': // or if c is 'e'
        case 'i': // or if c is 'i'
        case 'o': // or if c is 'o'
        case 'u': // or if c is 'u'
        case 'A': // or if c is 'A'
        case 'E': // or if c is 'E'
        case 'I': // or if c is 'I'
        case 'O': // or if c is 'O'
        case 'U': // or if c is 'U'
            return true;
        default:
            return false;
    }
}
```

Remember, execution begins at the first statement after a matching case label. Case labels aren't statements (they're labels), so they don't count.

The **first statement after all of the case statements** in the above program is **return true**, so if any case labels match, the function will return true.

Thus, we can “stack” case labels to make all of those case labels **share the same set of statements** afterward. **This is not considered fallthrough behavior, so use of comments or `[[fallthrough]]` is not needed here.**

11.6.4 Labels do not define a new scope

With if statements, you can only have a single statement after the if-condition, and that statement is considered to be implicitly inside a block. However, with switch statements, the statements after labels are all scoped to the switch block. No implicit blocks are created.

```
1  switch (1)
2  {
3  case 1: // does not create an implicit block
4      foo(); // this is part of the switch scope, not an implicit block to case 1
5      break; // this is part of the switch scope, not an implicit block to case 1
6  default:
7      std::cout << "default case\n";
8      break;
9  }
```

11.6.5 Variable declaration and initialization inside case statements

You can declare or define (but not initialize) variables inside the switch, both before and after the case labels:

```
1  switch (1)
2  {
3      int a; // okay: definition is allowed before the case labels
4      int b{ 5 }; // illegal: initialization is not allowed before the case labels
5
6  case 1:
7      int y; // okay but bad practice: definition is allowed within a case
8      y = 4; // okay: assignment is allowed
9      break;
10
11 case 2:
12     int z{ 4 }; // illegal: initialization is not allowed if subsequent cases exist
13     y = 5; // okay: y was declared above, so we can use it here too
14     break;
15
16 case 3:
17     break;
18 }
```

Although variable y was defined in case 1, it was used in case 2 as well. All statements inside the switch are considered to be part of the same scope. Thus, a variable declared or defined in one case can be used in a later case, even if the case in which the variable is defined is never executed (because the switch jumped over it)!

However, initialization of variables *does* require the definition to execute at runtime (since the value of the initializer must be determined at that point). Initialization of variables is disallowed

in any case that is not the last case (because the switch could jump over the initializer if there is a subsequent case defined, in which case the variable would be undefined, and accessing it would result in undefined behavior). Initialization is also disallowed before the first case, as those statements will never be executed, as there is no way for the switch to reach them.

If a case needs to define and/or initialize a new variable, the best practice is to do so inside an explicit block underneath the case statement:

```
1  switch (1)
2  {
3  case 1:
4  { // note addition of explicit block here
5    int x{ 4 }; // okay, variables can be initialized inside a block inside a case
6    std::cout << x;
7    break;
8  }
9
10 default:
11     std::cout << "default case\n";
12     break;
13 }
```

Best practice: If defining variables used in a case statement, do so in a block inside the case.

11.7 Goto statements

The next kind of control flow statement we'll cover is the unconditional jump. An unconditional jump causes execution to jump to another spot in the code. The term "unconditional" means the jump always happens (unlike an if-statement or switch-statement, where the jump only happens conditionally based on the result of an expression).

In C++, unconditional jumps are implemented via a **goto statement**, and the spot to jump to is identified through use of a **statement label**. Just like with switch case labels, **statement labels are conventionally not indented**.

The following is an example of a goto statement and statement label:

```
int main()
{
    double x{};
tryAgain: // this is a statement label
    std::cout << "Enter a non-negative number: ";
    std::cin >> x;

    if (x < 0.0)
        goto tryAgain; // this is the goto statement

    std::cout << "The square root of " << x << " is " << std::sqrt(x) << '\n';
    return 0;
}
```

In this program, the user is asked to enter a non-negative number. However, if a negative number is entered, the program utilizes a goto statement to **jump back to the tryAgain label**. The user is then asked again to enter a new number. In this way, we can continually ask the user for input until he or she enters something valid.

11.7.1 Statement labels have function scope

In the chapter on object scope, we covered two kinds of scope: **local (block) scope**, and **file (global) scope**. Statement labels utilize a third kind of scope: **function scope**, which means the **label is visible throughout the function even before its point of declaration**. The goto statement and its corresponding statement label must appear in the same function.

While the above example shows a goto statement that jumps backwards (to a preceding point in the function), goto statements can also jump forward:

```
1  #include <iostream>
2
3  void printCats(bool skip)
4  {
5      if (skip)
6          goto end; // jump forward; statement label 'end' is visible here due to it having function scope
7
8      std::cout << "cats\n";
9  end:
10     ; // statement labels must be associated with a statement
11 }
12
13 int main()
14 {
15     printCats(true); // jumps over the print statement and doesn't print anything
16     printCats(false); // prints "cats"
17
18     return 0;
19 }
```

Beyond the jumping forward, there are a couple of interesting things worth mentioning in the program above.

First, note that **statement labels must be associated with a statement** (hence their name: they label a statement). Because the end of the function had no statement, we had to use a null statement so we had a statement to label. **Second, we were able to jump to the statement labeled by end even though we hadn't declared end yet due to statement labels having function scope.** No forward declaration of statement labels is necessary. Third, **it's worth explicitly mentioning that the above program is poor form -- it would have been better to use an if-statement to skip the print statement than a goto statement to jump over it.**

There are **two primary limitations** to jumping:

- You can **jump only within the bounds of a single function** (you can't jump out of one function and into another).
- And if you jump forward, you **can't jump forward over the initialization of any variable that is still in scope at the location being jumped to.**

Note that you **can jump backwards over a variable initialization, and the variable will be re-initialized when the initialization is executed.**

11.7.2 *Avoid using goto*

Use of goto is shunned in C++ (and other modern high-level languages as well). Edsger W. **Dijkstra**, a noted computer scientist, **laid out the case for avoiding goto in a famous but difficult to read paper called Go To Statement Considered Harmful.** The primary problem with goto is that **it allows a programmer to jump around the code arbitrarily.** This creates what is not-so-affectionately known as **spaghetti code.** Spaghetti code is code that has a path of execution that resembles a bowl of spaghetti (all tangled and twisted), making it extremely difficult to follow the logic of such code.

As Dijkstra says somewhat humorously, "the quality of programmers is a decreasing function of the density of go to statements in the programs they produce".

Almost any code written using a goto statement can be more clearly written using other constructs in C++, such as if-statements and loops. **One notable exception is when you need to exit a nested loop but not the entire function --** in such a case, a goto to just beyond the loops is probably the cleanest solution.

Best practice: Avoid goto statements (unless the alternatives are significantly worse for code readability).

11.7.3 Introduction to loops and while statements

11.7.4 Introduction to loops

And now the real fun begins -- in the next set of lessons, we'll cover loops. **Loops are control flow constructs that allow a piece of code to execute repeatedly until some condition is met. Loops add a significant amount of flexibility into your programming toolkit**, allowing you to do many things that would otherwise be difficult.

For example, let's say you wanted to print all the numbers between 1 and 10. Without loops, you might try something like this:

```
3 int main()
4 {
5     std::cout << "1 2 3 4 5 6 7 8 9 10";
6     std::cout << " done!\n";
7     return 0;
8 }
```

While that's doable, it becomes increasingly less so as you want to print more numbers: what if you wanted to print all the numbers between 1 and 1000? **That would be quite a bit of typing!** But **such a program is writable in this way because we know at compile time how many numbers we want to print.**

Now, let's change the parameters a bit. What if we wanted to ask the user to enter a number and then print all the numbers between 1 and the number the user entered? The number the user will enter isn't knowable at compile-time. So how might we go about solving this?

11.7.5 While statements

The **while statement** (also called a **while loop**) is the simplest of the three loop types that C++ provides, and it has a definition very similar to that of an if-statement:

```
while (condition)
    statement;
```

A while statement is declared using the **while** keyword. When a while-statement is executed, the expression *condition* is evaluated. If the condition evaluates to true, the associated *statement* executes.

However, **unlike an if-statement, once the statement has finished executing, control returns to the top of the while-statement and the process is repeated.** This means a while-statement will keep looping as long as the condition continues to evaluate to true.

11.7.6 *While-statements that evaluate to false initially*

Note that if the condition initially evaluates to false, the associated **statement will not execute** at all.

11.7.7 *Infinite loops*

On the other hand, if the expression always evaluates to true, the while loop will execute forever. This is called an **infinite loop**. Here is an example of an infinite loop:

```
int main()
{
    int count{ 1 };
    while (count <= 10) // this condition will never be false
    {
        std::cout << count << ' '; // so this line will repeatedly execute
    }

    std::cout << '\n'; // this line will never execute

    return 0; // this line will never execute
}
```

Because count is never incremented in this program, count <= 10 will always be true. Consequently, the loop will never terminate, and the program will print 1 1 1 1 1... forever.

11.7.8 Intentional infinite loops

We can declare an intentional infinite loop like this:

```
1 while (true)
2 {
3     // this loop will execute forever
4 }
```

The only way to exit an infinite loop is through a return-statement, a break-statement, an exit-statement, a goto-statement, an exception being thrown, or the user killing the program.

Best practice: Favor while(true) for intentional infinite loops.

11.7.9 Loop variables and naming

A **loop variable** is a variable that is used to control how many times a loop executes. For example, given while (count <= 10), count is a loop variable. While **most loop variables have type int, you will occasionally see other types** (e.g. char).

Loop variables are often given **simple names, with i, j, and k being the most common**.

As an aside: The use of i, j, and k for loop variable names arose because **these are the first three shortest names for integral variables in the Fortran** programming language. The convention has persisted since.

However, if you want to know where in your program a loop variable is used, and you use the search function on i, j, or k, the search function will return half of the lines in your program! For this reason, some developers prefer loop variable names like iii, jjj, or kkk. Because these names are more unique, this makes searching for loop variables much easier, and helps them stand out as loop variables. An even better idea is to use “real” variable names, such as count, index, or a name that gives more detail about what you’re counting (e.g. userCount).

The most common type of loop variable is called a **counter**, which is a loop variable that counts how many times a loop has executed. In the examples above, the **variable count is a counter**.

11.7.10 Integral loop variables should be signed

Integral loop variables should almost always be signed, as **unsigned integers can lead to unexpected issues**. Consider the following code:

```
unsigned int count{ 10 }; // note: unsigned

// count from 10 down to 0
while (count >= 0)
{
    if (count == 0)
    {
        std::cout << "blastoff!";
    }
    else
    {
        std::cout << count << ' ';
    }
    --count;
}
```

Take a look at the above example and see if you can spot the error. It's not very obvious if you haven't seen this before.

It turns out, this program is an infinite loop. It starts out by printing 10 9 8 7 6 5 4 3 2 1 blastoff! as desired, but then loop variable `count` overflows, and starts counting down from 4294967295 (assuming 32-bit integers). Why? Because the loop condition `count >= 0` will never be false! When `count` is 0, `0 >= 0` is true. Then `--count` is executed, and `count` wraps around back to 4294967295. And since `4294967295 >= 0` is true, the program continues. Because `count` is unsigned, it can never be negative, and because it can never be negative, the loop won't terminate.

Best practice: Integral loop variables should generally be a signed integral type.

11.7.11 Doing something every N iterations

Each time a loop executes, it is called an iteration.

Often, we want to do something every 2nd, 3rd, or 4th iteration, such as print a newline. This can easily be done by **using the remainder operator on our counter**:

```
1  #include <iostream>
2
3  // Iterate through every number between 1 and 50
4  int main()
5  {
6      int count{ 1 };
7      while (count <= 50)
8      {
9          // print the number (pad numbers under 10 with a leading 0 for formatting purposes)
10         if (count < 10)
11         {
12             std::cout << '0';
13         }
14
15         std::cout << count << ' ';
16
17         // if the loop variable is divisible by 10, print a newline
18         if (count % 10 == 0)
19         {
20             std::cout << '\n';
21         }
22
23         // increment the loop counter
24         ++count;
25     }
26
27     return 0;
28 }
```

This program produces the result:

```
01 02 03 04 05 06 07 08 09 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
```

11.7.12 Nested loops

It is also possible to nest loops inside of other loops. Nested loops tend to be a bit confusing to new programmers, so let's start with a slightly easier example:

```
void printUpto(int outer)
{
    // loop between 1 and outer
    // note: inner will be created and destroyed at the end of the block
    int inner{ 1 };
    while (inner <= outer)
    {
        std::cout << inner << ' ';
        ++inner;
    }
} // inner destroyed here

int main()
{
    // outer loops between 1 and 5
    int outer{ 1 };
    while (outer <= 5)
    {
        // For each iteration of the outer loop, the code in the body of the loop executes once

        // This function prints numbers between 1 and outer
        printUpto(outer);

        // print a newline at the end of each row
        std::cout << '\n';
        ++outer;
    }

    return 0;
}
```

In this example, we have an outer loop with a counter named `outer` that counts from 1 to 5. For each iteration of the loop, the body of the loop calls `printUpto()` using the outer loop variable as an argument, prints a newline, and increments `outer`. The `printUpto()` function also has a loop which prints all the numbers between 1 and the value passed in.

Therefore, when `outer` is 1, the loop body calls `printUpto(1)`, which prints the number 1. The loop body then prints a newline and increments `outer`. Now `outer` is 2. The loop body executes again, calling `printUpto(2)`, which prints 1 2. The loop body prints another newline and increments `outer`. Successive iterations call `printUpto(3)`, `printUpto(4)`, and `printUpto(5)`.

Therefore, this program prints:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Note that this is a form of a nested loop -- in the body of the outer loop, **we're calling a function that has a loop itself**. In other words, **for each iteration of the outer loop, the loop in the function executes**.

If you're still finding this confusing, stepping through this program in a debugger line-by-line and watching the values of inner and outer is a good way to get a better understanding of what's happening.

11.8 Do while statements

Consider the case where we want to show the user a menu and ask them to make a selection -- and if the user chooses an invalid selection, to ask them again. Clearly the menu and selection should go inside a loop of some kind (so we can keep asking the user until they enter valid input), but what kind of loop should we choose?

Since a while loop evaluates the condition up front, it's an awkward choice. We could instead add a new variable to track validity, like so:

```
3 int main()
4 {
5     int selection { 0 };
6     bool invalid { true }; // new variable just to gate the loop
7
8     while (invalid)
9     {
10         std::cout << "Please make a selection: \n";
11         std::cout << "1) Addition\n";
12         std::cout << "2) Subtraction\n";
13         std::cout << "3) Multiplication\n";
14         std::cout << "4) Division\n";
15
16         std::cin >> selection;
17         invalid = (selection < 1 || selection > 4);
18     }
19
20     // do something with selection here
21     // such as a switch statement
22
23     std::cout << "You selected option #" << selection << '\n';
24
25     return 0;
26 }
```

While this avoids the magic number, it introduces a new variable just to ensure the loop runs once, and that adds complexity and the possibility of additional errors.

11.8.1 Do while statements

To help solve problems like the above, C++ offers the do-while statement:

```
do
    statement; // can be a single statement or a compound statement
while (condition);
```

A **do while statement** is a looping construct that works just like a while loop, **except the statement always executes at least once**. After the statement has been executed, the do-while loop checks the condition. **If the condition evaluates to true, the path of execution jumps back to the top of the do-while loop and executes it again.**

Here is our example above using a do-while loop instead of a while loop:

```
do
{
    std::cout << "Please make a selection: \n";
    std::cout << "1) Addition\n";
    std::cout << "2) Subtraction\n";
    std::cout << "3) Multiplication\n";
    std::cout << "4) Division\n";
    std::cin >> selection;
}
while (selection < 1 || selection > 4);
```

In this way, we've avoided both magic numbers and additional variables.

One thing worth discussing in the above example is that the selection variable must be declared outside of the do block. If the selection variable were to be declared inside the do block, it would be destroyed when the do block terminates, which happens before the conditional is evaluated. But we need the variable in the while conditional -- consequently, the selection variable must be declared outside the do block (even if it wasn't used later in the body of the function).

In practice, do-while loops aren't commonly used. Having the condition at the bottom of the loop obscures the loop condition, which can lead to errors. Many developers recommend avoiding do-while loops altogether as a result. We'll take a softer stance and advocate for preferring while loops over do-while when given an equal choice.

Best practice: Favor while loops over do-while when given an equal choice.

11.9 For statements

By far, the most utilized loop statement in C++ is the for-statement. The for statement (also called a **for loop**) is **preferred when we have an obvious loop variable** because it lets us easily and concisely define, initialize, test, and change the value of loop variables.

As of C++11, there are two different kinds of for-loops. We'll cover the **classic for-statement** in this lesson, **and the newer range-based for-statement** in a future lesson (Range-based for loops (for-each)) once we've covered some other prerequisite topics.

The for-statement looks pretty simple in abstract:

```
for (init-statement; condition; end-expression)
    statement;
```

The easiest way to initially understand how a for-statement works is to convert it into an equivalent while-statement:

```
{ // note the block here
    init-statement; // used to define variables used in the loop
    while (condition)
    {
        statement;
        end-expression; // used to modify the loop variable prior to reassessment of the condition
    }
} // variables defined inside the loop go out of scope here
```


11.9.1 Evaluation of for-statements

A for-statement is **evaluated in 3 parts**:

First, the init-statement is executed. This only happens once when the loop is initiated. The init-statement is typically used for variable definition and initialization. These **variables have “loop scope”**, which really **just is a form of block scope** where these variables exist from the point of definition through the end of the loop statement. In our while-loop equivalent, you can see that the init-statement is inside a block that contains the loop, so the variables defined in the init-statement go out of scope when the block containing the loop ends.

Second, with each loop iteration, the condition is evaluated. If this evaluates to true, the statement is executed. If this evaluates to false, the loop terminates and execution continues with the next statement beyond the loop.

Finally, after the statement is executed, the end-expression is evaluated. Typically, this expression is used to increment or decrement the loop variables defined in the init-statement. **After the end-expression has been evaluated, execution returns to the second step (and the condition is evaluated again).**

Tip: The **end-expression** of a for-loop **executes AFTER the loop statement**.

For-loops can be hard for new programmers to read -- however, experienced programmers love them because they are a very compact way to do loops with a counter, with all of the necessary information about the loop variables, loop conditions, and loop variable modifiers presented up front. **This helps reduce errors.**

11.9.2 The perils of operator!= in for-loop conditions

When writing a loop condition involving a value, we can often write the condition in many different ways. The following two loops execute identically:

```
for (int i { 0 }; i < 10; ++i) // uses <
    std::cout << i;

for (int i { 0 }; i != 10; ++i) // uses !=
    std::cout << i;
```

So, which should we prefer? **The former is the better choice, as it will terminate even if i jumps over the value 10, whereas the latter will not.**

Best practice: Avoid operator!= when doing numeric comparisons in the for-loop condition. Prefer operator< or operator<= where possible.

11.9.3 Off-by-one errors

One of the biggest problems that new programmers have with for-loops (and other loops that utilize counters) are **off-by-one errors**. Off-by-one errors **occur when the loop iterates one too many or one too few times** to produce the desired result.

Although the most common cause for these errors is using the **wrong relational operator**, they **can sometimes occur by using pre-increment or pre-decrement instead of post-increment or post-decrement, or vice-versa.**

11.9.4 Omitted expressions

It is possible to write *for loops* that omit any or all of the statements or expressions. For example, in the following example, we'll omit the init-statement and end-expression, leaving only the condition:

```
for ( ; i < 10; ) // no init-statement or end-expression
{
    std::cout << i << ' ';
    ++i;
}
```

Rather than having the *for loop* do the initialization and incrementing, we've done it manually. We have done so purely for academic purposes in this example, but there are cases where not defining a loop variable (because you already have one) or not incrementing it in the end-expression (because you're incrementing it some other way) is desired.

Although you do not see it very often, it is worth noting that the following example produces an infinite loop:

```
1 | for (;;)
2 |     statement;
```

This might be a little unexpected, as you'd probably expect an omitted condition-expression to be treated as false. However, the C++ standard explicitly (and inconsistently) defines that an omitted condition-expression in a for-loop should be treated as true.

Best practice: avoid this form of the for loop altogether and **use while (true) instead.**

11.9.5 For-loops with multiple counters

Although for-loops typically iterate over only one variable, **sometimes for-loops need to work with multiple variables.** To assist with this, **the programmer can define multiple variables in the init-statement, and can make use of the comma operator to change the value of multiple variables in the end-expression:**

```
for (int x{ 0 }, y{ 9 }; x < 10; ++x, --y)
    std::cout << x << ' ' << y << '\n';
```

This loop defines and initializes two new variables: *x* and *y*. It iterates *x* over the range 0 to 9, and after each iteration *x* is incremented and *y* is decremented.

This is about the only place in C++ where defining multiple variables in the same statement, and use of the comma operator is considered an acceptable practice.

Best practice: Defining multiple variables (in the init-statement) and using the comma operator (in the end-expression) is acceptable inside a for-statement.

11.9.6 Nested for-loops

Like other types of loops, for-loops can be nested inside other loops.

11.9.7 Variables used only inside a loop should be defined inside the loop.

New programmers often think that creating variables is expensive, so that it is better to create a variable once (and then assign values to it) than create it many times (and use initialization). That leads to loops that look like some variation of the following:

```
int i {}; // i defined outside loop
for (i = 0; i < 10; ++i) // i assigned value
{
    std::cout << i << ' ';
}
// i can still be accessed here
```

However, creating a variable has no cost -- it is the initialization that has a cost, and **there is typically no cost difference between initialization and assignment**. The above example makes `i` available beyond the loop. Unless using a variable beyond the loop is required, **defining a variable outside the loop is likely to have two consequences**:

1. It **makes our program more complex**, as we have to read more code to see where the variable is used.
2. It **may actually be slower because the compiler may not be able to optimize a variable with a larger scope as effectively**.

Consistent with our best practice to define variables in the smallest reasonable scope possible, a variable that is only used within a loop should be defined inside the loop rather than outside.

Best practice: Variables used only inside a loop should be defined inside the scope of the loop.

11.9.8 Conclusion

For-statements are the most commonly used loop in the C++ language. Even though its syntax is typically a bit confusing to new programmers, you will see for-loops so often that you will understand them in no time at all!

For-statements excel when you have a counter variable. If you do not have a counter, a while-statement is probably a better choice.

Best practice

- Prefer for-loops over while-loops when there is an obvious loop variable.
- Prefer while-loops over for-loops when there is no obvious loop variable.

11.10 Break and continue

Although you have already seen the break statement in the context of switch statements, it deserves a fuller treatment since **it can be used with other types of control flow statements as well**. The **break statement causes a while loop, do-while loop, for loop, or switch statement to end**, with execution continuing with the next statement after the loop or switch being broken out of.

11.10.1 Breaking a switch

In the context of a switch statement, a break is **typically used at the end of each case to signify the case is finished** (which prevents fallthrough into subsequent cases):

```
switch (ch)
{
case '+':
    std::cout << x << " + " << y << " = " << x + y << '\n';
    break; // don't fall-through to next case
case '-':
    std::cout << x << " - " << y << " = " << x - y << '\n';
    break; // don't fall-through to next case
case '*':
    std::cout << x << " * " << y << " = " << x * y << '\n';
    break; // don't fall-through to next case
case '/':
    std::cout << x << " / " << y << " = " << x / y << '\n';
    break;
}
```

11.10.2 Breaking a loop

In the context of a loop, a break statement can be used to end the loop early. Execution continues with the next statement after the end of the loop. For example:

```
int sum{ 0 };

// Allow the user to enter up to 10 numbers
for (int count{ 0 }; count < 10; ++count)
{
    std::cout << "Enter a number to add, or 0 to exit: ";
    int num{};
    std::cin >> num;

    // exit loop if user enters 0
    if (num == 0)
        break; // exit the loop now

    // otherwise add number to our sum
    sum += num;
}

// execution will continue here after the break
std::cout << "The sum of all the numbers you entered is: " << sum << '\n';
```

This program allows the user to type up to 10 numbers, and displays the sum of all the numbers entered at the end. If the user enters 0, the break causes the loop to terminate early (before 10 numbers have been entered).

A break is also a common way to get out of an intentional infinite loop.

11.10.3 Break vs return

New programmers sometimes have trouble understanding the difference between break and return. A **break statement terminates** the switch or loop, and execution **continues at the first statement beyond the switch or loop**. A **return statement terminates the entire function** that the loop is within, and **execution continues at point where the function was called**.

```
void Foo()
{
    for (int i{ 0 }; i <= 10; i++)
    {
        std::cout << "hello world!" << '\n';

        //break exits the loop when i == 3
        if (i == 3)
        {
            break;
        }
    }

    //first statement after the loop still executes
    std::cout << "I still get printed!" << '\n';
}

void Goo()
{
    for (int i{ 0 }; i <= 10; i++)
    {
        std::cout << "hello world!" << '\n';

        //return exits the function entirely
        if (i == 3)
        {
            return;
        }
    }

    //other statements inside the function after return get skipped
    std::cout << "I don't get printed!" << '\n';
}
```

11.10.4 Continue

The **continue statement** provides a convenient way to **end the current iteration of a loop without terminating the entire loop**.

Here's an example of using continue:

```
void Boo()
{
    for (int i{ 0 }; i <= 10; i++)
    {
        std::cout << "hello world!" << '\n';

        //continue terminates the current ITERATION
        if (i == 3)
        {
            continue;
        }
        // The continue statement jumps to here
    }

    std::cout << "I still get printed after the for loop is done!" << '\n';
}
```

A continue statement works by **causing the current point of execution to jump to the bottom of the current loop**.

Be careful when using a continue statement with while or do-while loops. These loops typically change the value of variables used in the condition inside the loop body. If use of a continue statement causes these lines to be skipped, **then the loop can become infinite!**

Of course, you already know that **if you have an obvious counter variable, you should be using a for loop, not a while or do while loop.**

11.10.5 The debate over use of break and continue

Many **textbooks caution readers not to use break and continue in loops**, both because it **causes the execution flow to jump around**, and because it can make the **flow of logic harder to follow**. For example, a break in the middle of a complicated piece of logic could either be missed, or it may not be obvious under what conditions it should be triggered.

However, **used judiciously, break and continue can help make loops more readable by keeping the number of nested blocks down and reducing the need for complicated looping logic.**

The **continue statement is most effectively used at the top of a for-loop to skip loop iterations when some condition is met.** This can allow us to avoid a nested block.

Minimizing the number of variables used and keeping the number of nested blocks down both improve code comprehensibility more than a break or continue harms it. For that reason, we believe judicious use of break or continue is acceptable.

Best practice: Use break and continue when they simplify your loop logic.

11.10.6 The debate over use of early returns

There's a similar argument to be made for return statements. **A return statement that is not the last statement in a function is called an early return.** Many programmers believe early returns should be avoided. A function that only has one return statement at the bottom of the function has a simplicity to it -- you can assume the function will take its arguments, do whatever logic it has implemented, and return a result without deviation. Having extra returns complicates the logic.

The **counter-argument is that using early returns allows your function to exit as soon as it is done, which reduces having to read through unnecessary logic and minimizes the need for conditional nested blocks, which makes your code more readable.**

Some developers take a middle ground, and only use early returns at the top of a function to do parameter validation (catch bad arguments passed in), and then a single return thereafter.

Our stance is that early returns are more helpful than harmful, but we recognize that there is a bit of art to the practice.

Best practice: Use early returns when they simplify your function's logic.

11.11 Halts (exiting your program early)

The last category of control flow statement we'll cover in this chapter is the **halt**. **A halt is a flow control statement that terminates the program.** In C++, **halts are implemented as functions** (rather than keywords), so our halt statements will be function calls.

11.11.1 When a program exits normally

Let's take a brief detour, and recap what happens when a program exits normally. When the `main()` function returns (either by reaching the end of the function, or via a `return` statement), a number of different things happen.

First, because we're leaving the function, **all local variables and function parameters are destroyed (as per usual).**

Next, a special function called `std::exit()` is called, with the return value from `main()` (the status code) passed in as an argument. So, what is `std::exit()`?

11.11.2 The `std::exit()` function

`std::exit()` is a function that causes the program to terminate normally. **Normal termination** means the program has exited in an expected way. Note that **the term normal termination does not imply anything about whether the program was successful** (that's what the status code is for). For example, let's say you were writing a program where you expected the user to type in a filename to process. If the user typed in an invalid filename, your program would probably return a non-zero status code to indicate the failure state, but it would still have a normal termination.

`std::exit()` performs a number of cleanup functions. First, **objects with static storage duration are destroyed.** Then some **other miscellaneous file cleanup is done if any files were used.** Finally, **control is returned back to the OS, with the argument passed to `std::exit()` used as the status code.**

11.11.3 Calling `std::exit()` explicitly

Although `std::exit()` is called implicitly after function `main()` returns, **`std::exit()` can also be called explicitly to halt the program before it would normally terminate.** When `std::exit()` is called this way, you will need to include the `cstdlib` header.

Key insight: `std::exit` is called implicitly when `main()` returns.

Here is an example of using `std::exit()` explicitly:

```
10 int main()
11 {
12     std::cout << 1 << '\n';
13     cleanup();
14
15     std::exit(0); // terminate and return status code 0 to operating system
16
17     // The following statements never execute
18     std::cout << 2 << '\n';
19
20     return 0;
21 }
```

Note that the **statements after the call to `std::exit()` never execute because the program has already terminated.**

Although in the program above we call `std::exit()` from function `main()`, **`std::exit()` can be called from any function to terminate the program at that point.**

`std::exit()` does not clean up local variables

One important note about calling `std::exit()` **explicitly: `std::exit()` does not clean up any local variables (either in the current function, or in functions up the call stack).** This means calling `std::exit()` can be dangerous if your program relies on any local variables cleaning themselves up.

11.11.4 `std::atexit`

Because `std::exit()` terminates the program immediately, **you may want to manually do some cleanup before terminating**. In this context, **cleanup means things like closing database or network connections, deallocating any memory you have allocated, writing information to a log file, etc...**

In the above example, we called function `cleanup()` to handle our cleanup tasks. However, **remembering to manually call a cleanup function before calling every call to `exit()` adds burden to the programmer and is a recipe for errors**.

To assist with this, **C++ offers the `std::atexit()` function, which allows you to specify a function that will automatically be called on program termination via `std::exit()`**.

As an aside:

When an application exits, modern OSes will generally clean up any memory that the application does not properly clean up itself. This leads to the question, “so why bother doing cleanup on `exit()`?”.

1. Cleaning up allocated memory is a “good habit” that you will need to use to avoid memory leaks while the application is running. Cleaning up in some cases and not others is inconsistent and can lead to errors. Not cleaning up memory properly can also impact the way certain tools like memory profilers behave (they may be unable to distinguish memory that you inadvertently aren’t intentionally cleaning up from memory that you intentionally aren’t cleaning up because you don’t have to).
2. There are other kinds of cleanup that may be necessary for your program to behave predictably. For example, if you write data to a file and then unexpectedly exit, that data may not have been flushed to the file yet, and may be lost when the program exits. Closing the file before shutting down helps ensure that all cached data will be written first. Or you may want to send data about the user’s session, or why the program is shutting down to server before the shutdown actually happens.

Related content: We discuss passing functions as arguments in lesson: Function pointers

11.11.5 *std::abort* and *std::terminate*

C++ contains **two other halt-related functions**:

- The **`std::abort()`** function causes your program to terminate abnormally. Abnormal termination means **the program had some kind of unusual runtime error and the program couldn't continue to run**. For example, trying to divide by 0 will result in an abnormal termination. `std::abort()` does not do any cleanup.
- The **`std::terminate()`** function is typically used in conjunction with exceptions (we'll cover exceptions in a later chapter). Although `std::terminate` **can be called explicitly**, it **is more often called implicitly when an exception isn't handled** (and in a few other exception-related cases). By default, `std::terminate()` calls `std::abort()`.

11.11.6 *When should you use a halt?*

The short answer is **"almost never"**. Destroying local objects is an important part of C++ (particularly when we get into classes), and **none of the above-mentioned functions clean up local variables**. Exceptions are a better and safer mechanism for handling error cases.

Best practice: Only use a halt if there is no safe or reasonable way to return normally from the main function. **If you haven't disabled exceptions, prefer using exceptions for handling errors safely.**

11.12 Summary

The specific sequence of statements that the CPU executes in a program is called the program's **execution path**. A **straight-line program** takes the same path every time it is run.

Control flow statements (also called **Flow control statements**) allow the programmer to change the normal path of execution. When a control flow statement causes the program to begin executing some non-sequential instruction sequence, this is called a **branch**.

A **conditional statement** is a statement that specifies whether some associated statement(s) should be executed or not.

If statements allow us to execute an associated statement based on whether some condition is true. **Else statements** execute if the associated condition is false. You can chain together multiple if and else statements.

A **dangling else** occurs **when it is ambiguous which if statement an else statement is connected to**. Dangling else statements are **matched up with the last unmatched if statement** in the same block. Thus, we trivially avoid dangling else statements by ensuring the body of an if statement is placed in a block.

A **null statement** is a statement that consists of just a semicolon. It does nothing, and is used when the language requires a statement to exist but the programmer does not need the statement to do anything.

Switch statements provide a cleaner and faster method for selecting between a number of matching items. Switch statements only work with integral types. **Case labels** are used to identify the values for the evaluated condition to match. The statements beneath a **default label** are executed if no matching case label can be found.

When execution flows from a statement underneath a label into statements underneath a subsequent label, this is called **fallthrough**. A **break statement (or return statement)** can be used to prevent fallthrough. The **[[fallthrough]] attribute** can be used to **document intentional fallthrough**.

Goto statements allow the program to jump to somewhere else in the code, either forward or backwards. These should generally be avoided, as they can create **spaghetti code**, which occurs when a program has a path of execution that resembles a bowl of spaghetti.

While loops allow the program to loop as long as a given condition evaluates to true. The condition is evaluated before the loop executes.

An **infinite loop** is a loop that has a condition that always evaluates to true. These loops will loop forever unless another control flow statement is used to stop them.

A **loop variable** (also called a **counter**) is an integer variable used to count how many times a loop has executed. Each execution of a loop is called an **iteration**.

Do while loops are similar to while loops, but the condition is evaluated after the loop executes instead of before.

For loops are the most used loop, and are ideal when you need to loop a specific number of times. An off-by-one error occurs when the loop iterates one too many or one too few times.

Break statements allow us to break out of a switch, while, do while, or for loop (also range-based for loops, which we haven't covered yet). **Continue statements** allow us to move immediately to the next loop iteration.

Halts allow us to terminate our program. **Normal termination** means the program has exited in an expected way (and the status code will indicate whether it succeeded or not). **std::exit()** is automatically called at the end of main, or it can be called explicitly to terminate the program. It does some cleanup, but does not cleanup any local variables, or unwind the call stack.

Abnormal termination occurs when the program encountered some kind of unexpected error and had to be shut down. **std::abort** can be called for an abnormal termination.

11.12.1 Optional (RNG-related)

An **algorithm** is a finite sequence of instructions that can be followed to solve some problem or produce some useful result. An algorithm is considered to be **stateful** if it retains some information across calls. Conversely, a **stateless** algorithm does not store any information (and must be given all the information it needs to work with when it is called). When applied to algorithms, the term **state** refers to the current values held in stateful variables.

An algorithm is considered **deterministic** if for a given input (the value provided for start) it will always produce the same output sequence.

12 Error detection and handling

12.1 Introduction to testing your code

So, you've written a program, it compiles, and it even appears to work! What now?

Well, it depends. If you've written your program to be run once and discarded, then you're done. In this case, it may not matter that your program doesn't work for every case -- if it works for the one case you needed it for, and you're only going to run it once, then you're done.

If your program is entirely linear (has no conditionals, such as if statements or switch statements), takes no inputs, and produces the correct answer, then you're probably done. In this case, you've already tested the entire program by running it and validating the output. You may want to compile and run the program on several different systems to ensure it behaves consistently (if it doesn't, you've probably done something that produces undefined behavior that just happens to work anyway on your initial system).

But most likely you've written a program you intend to run many times, that uses loops and conditional logic, and accepts user input of some kind. You've possibly written functions that may be reusable in other future programs. You may have experienced a bit of scope creep, where you added some new capabilities that were originally not planned for. **Maybe you're even intending to distribute this program to other people (who are likely to try things you haven't thought of). In this case, you really should be validating that your program works like you think it does under a wide variety of conditions -- and that requires some proactive testing.**

Just because your program worked for one set of inputs doesn't mean it's going to work correctly in all cases.

Software testing (also called **software validation**) is the process of determining whether or not the software actually works as expected.

12.1.1 The testing challenge

Before we talk about some practical ways to test your code, let's talk about why testing your program comprehensively is difficult.

Consider this simple program:

```
1  #include <iostream>
2
3  void compare(int x, int y)
4  {
5      if (x > y)
6          std::cout << x << " is greater than " << y << '\n'; // case 1
7      else if (x < y)
8          std::cout << x << " is less than " << y << '\n'; // case 2
9      else
10         std::cout << x << " is equal to " << y << '\n'; // case 3
11 }
12
13 int main()
14 {
15     std::cout << "Enter a number: ";
16     int x{};
17     std::cin >> x;
18
19     std::cout << "Enter another number: ";
20     int y{};
21     std::cin >> y;
22
23     compare(x, y);
24
25     return 0;
26 }
```

Assuming a 4-byte integer, explicitly testing this program with every possible combination of inputs would require that you run the program 18,446,744,073,709,551,616 (~18 quintillion) times. **Clearly that's not a feasible task!**

Every time we ask for user input, or have a conditional in our code, we increase the number of possible ways our program can execute by some multiplicative factor. For all but the simplest programs, explicitly testing every combination of inputs becomes impossible almost immediately.

Now, your intuition should be telling you that you really shouldn't need to run the above program 18 quintillion times to ensure it works. **You may reasonably conclude that if case 1 works for one pair of x and y values where $x > y$, it should work for any pair of x and y where $x > y$.** Given that, it becomes apparent that **we really only need to run this program about three times (once to exercise each of the three cases in function compare()) to have a high degree of confidence it works as desired.** There are other similar tricks we can use to dramatically reduce the number of times we have to test something, in order to make testing manageable.

There's a lot that can be written about testing methodologies -- in fact, we could write a whole chapter on it. But since it's not a C++ specific topic, we'll stick to a brief and informal introduction, covered from the point of view of you (as the developer) testing your own code. In the next few subsections, we'll talk about some **practical** things you should be thinking about as you test your code.

12.1.2 *Test your programs in small pieces*

Consider an auto manufacturer that is building a custom concept car. Which of the following do you think they do?

- a) Build (or buy) and test each car component individually before installing it. Once the component has been proven to work, integrate it into the car and retest it to make sure the integration worked. At the end, test the whole car, as a final validation that everything seems good.
- b) Build a car out of all of the components all in one go, then test the whole thing for the first time right at the end.

It probably seems obvious that **option a) is a better choice**. And yet, **many new programmers write code like option b)!**

In case b), **if any of the car parts were to not work as expected, the mechanic would have to diagnose the entire car to determine what was wrong** -- the issue could be anywhere. A symptom might have many causes -- for example, is the car not starting due to a faulty spark plug, battery, fuel pump, or something else? This leads to lots of wasted time trying to identify exactly where the problems are, and what to do about them. And if a problem is found, the consequences can be disastrous -- **a change in one area might cause "ripple effects"** (changes) in multiple other places. For example, a fuel pump that is too small might lead to an engine redesign, which leads to a redesign of the car frame. In the worst case, you might end up redesigning a huge part of the car, just to accommodate what was initially a small issue!

In case a), the company tests as they go. If any component is bad right out of the box, they'll know immediately and can fix/replace it. Nothing is integrated into the car until it's proven working by itself, and then that part is retested again as soon as it's been integrated into the car. This way any unexpected issues are discovered as early as possible, while they are still small problems that can be easily fixed.

By the time they get around to having the whole car assembled, they should have reasonable confidence that the car will work -- after all, all the parts have been tested in isolation and when

initially integrated. It's still possible that unexpected issues will be found at this point, but that risk is minimized by all the prior testing.

The above analogy holds true for programs as well, though for some reason, new programmers often don't realize it. You're much better off writing small functions (or classes), and then **compiling and testing them immediately**. That way, if you make a mistake, you'll know it has to be in the small amount of code that you changed since the last time you compiled/tested. That means fewer places to look, and far less time spent debugging.

Testing a small part of your code in isolation to ensure that “**unit**” of code is correct is called **unit testing**. Each unit test is designed to ensure that a particular behavior of the unit is correct.

Best practice: Write your program in small, well-defined units (functions or classes), compile often, and test your code as you go.

If the program is short and accepts user input, trying a variety of user inputs might be sufficient. But as programs get longer and longer, this becomes less sufficient, and there is more value in testing individual functions or classes before integrating them into the rest of the program.

So how can we test our code in units?

12.1.3 Informal testing

One way you can test code is to do informal testing as you write the program. After writing a unit of code (a function, a class, or some other discrete “package” of code), you can **write some code to test the unit that was just added**, and then **erase the test once the test passes**. As an example, for the following `isLowerVowel()` function, you might write the following code:

```
1  #include <iostream>
2
3  // We want to test the following function
4  // For simplicity, we'll ignore that 'y' is sometimes counted as a vowel
5  bool isLowerVowel(char c)
6  {
7      switch (c)
8      {
9          case 'a':
10         case 'e':
11         case 'i':
12         case 'o':
13         case 'u':
14             return true;
15         default:
16             return false;
17     }
18 }
19
20 int main()
21 {
22     // So here's our temporary tests to validate it works
23     std::cout << isLowerVowel('a') << '\n'; // temporary test code, should produce 1
24     std::cout << isLowerVowel('q') << '\n'; // temporary test code, should produce 0
25
26     return 0;
27 }
```

If the results come back as 1 and 0, then you’re good to go. You know your function works for some basic cases, and you can reasonably infer by looking at the code that it will work for the cases you didn’t test (‘e’, ‘i’, ‘o’, and ‘u’). So, you can erase that temporary test code, and continue programming.

12.1.4 Preserving your tests

Although writing temporary tests is a quick and easy way to test some code, **it doesn't account for the fact that at some point, you may want to test that same code again later**. Perhaps you modified a function to add a new capability, and want to make sure you didn't break anything that was already working. For that reason, it can make more sense to preserve your tests so they can be run again in the future. For example, instead of erasing your temporary test code, you could move the tests into a `testVowel()` function:

```
// Not called from anywhere right now
// But here if you want to retest things later
void testVowel()
{
    std::cout << isLowerVowel('a') << '\n'; // temporary test code, should produce 1
    std::cout << isLowerVowel('q') << '\n'; // temporary test code, should produce 0
}
```

As you create more tests, you can simply add them to the `testVowel()` function.

12.1.5 Automating your test functions

One **problem with the above test function is that it relies on you to manually verify the results when you run it**. This requires you to remember what the expected answer was at worst (assuming you didn't document it), and manually compare the actual results to the expected results.

We can do better by writing a test function that contains both the tests AND the expected answers and compares them so we don't have to.

```
// returns the number of the test that failed, or 0 if all tests passed
int testVowel()
{
    if (!isLowerVowel('a')) return 1;
    if (isLowerVowel('q')) return 2;

    return 0;
}
```

Now, you can call `testVowel()` at any time to re-prove that you haven't broken anything, and the **test routine will do all the work for you**, returning either an "all good" signal (return value 0), or the test number that didn't pass, so you can investigate why it broke. **This is particularly useful when going back and modifying old code, to ensure you haven't accidentally broken anything!**

12.1.6 Unit testing frameworks

Because writing functions to exercise other functions is so common and useful, there are entire frameworks (called **unit testing frameworks**) that are designed to help simplify the process of writing, maintaining, and executing unit tests. Since these involve third party software, we won't cover them here, but **you should be aware they exist**.

12.1.7 Integration testing

Once each of your units has been tested in isolation, they can be integrated into your program and retested to make sure they were integrated properly. This is called an **integration test**. Integration testing tends to be more complicated -- for now, running your program a few times and spot checking the behavior of the integrated unit will suffice.

12.2 Code coverage

In this lesson, we'll talk about what kind of tests are useful to write to ensure your code is correct.

12.2.1 Code coverage

The term **code coverage** is used to describe **how much of the source code of a program is executed while testing**. There are many different metrics used for code coverage. We'll cover a few of the more useful and popular ones in the following sections.

12.2.1.1 Statement coverage

The term **statement coverage** refers to the percentage of statements in your code that have been exercised by your testing routines. While aiming for **100% statement coverage is good, it's often not enough to ensure correctness**.

12.2.1.2 Branch coverage

Branch coverage refers to the percentage of branches that have been executed, **each possible branch counted separately**. An **if statement has two branches** -- a branch that executes when the condition is true, and a branch that executes when the condition is false (even if there is no corresponding else statement to execute). **A switch statement can have many branches**.

Best practice: aim for 100% branch coverage of your code.

12.2.1.3 Loop coverage (*"the 0, 1, 2 test"*)

Loop coverage (informally called **the 0, 1, 2 test**) says that if you have a loop in your code, you should **ensure it works properly when it iterates 0 times, 1 time, and 2 times**. If it works correctly for the 2-iteration case, it should work correctly for all iterations greater than 2. These three tests therefore cover all possibilities (since a loop can't execute a negative number of times).

Best practice: Use the 0, 1, 2 test to ensure your loops work correctly with different number of iterations.

12.2.2 *Testing different categories of input*

When writing functions that accept parameters, or when accepting user input, **consider what happens with different categories of input**. In this context, we're using the term "category" to mean a set of inputs that have similar characteristics.

For example, if I wrote a function to produce the square root of an integer, what values would it make sense to test it with? You'd probably start with some **normal value, like 4**. **But it would also be a good idea to test with 0, and a negative number**.

Here are some basic guidelines for category testing:

For integers, make sure you've considered how your function handles **negative values, zero, and positive values**. You should also check for overflow if that's relevant.

For floating point numbers, make sure you've considered how your function handles values that have precision issues (values that are slightly larger or smaller than expected). Good double type values to test with are **0.1 and -0.1 (to test numbers that are slightly larger than expected)** and **0.6 and -0.6 (to test numbers that are slightly smaller than expected)**.

For strings, make sure you've considered how your function handles an **empty string, an alphanumeric string, strings that have whitespace (leading, trailing, and inner), and strings that are all whitespace**.

If your function takes a **pointer**, don't forget to **test nullptr as well** (don't worry if this doesn't make sense, we haven't covered it yet).

Best practice: Test different categories of input values to make sure your unit handles them properly.

12.3 Common semantic errors in C++

In lesson: Syntax and semantic errors, we covered **syntax errors**, which occur when you write **code that is not valid according to the grammar of the C++ language**. The compiler will notify you of such errors, so they are trivial to catch, and usually straightforward to fix.

We also covered **semantic errors**, which occur when you write **code that does not do what you intended**. The **compiler generally will not catch semantic errors** (though in some cases, smart compilers may be able to generate a warning).

Semantic errors can cause most of the same symptoms of undefined behavior, such as causing the program to produce the wrong results, **causing erratic behavior, corrupting program data, causing the program to crash -- or they may not have any impact at all**.

When writing programs, it is almost **inevitable that you will make semantic errors**. You will probably notice some of these just by using the program: for example, if you were writing a maze game, and your character was able to walk through walls. Testing your program (Introduction to testing your code) can also help surface semantic errors.

But there's **one other thing that can help -- and that's knowing which type of semantic errors are most common**, so you can spend a little more time ensuring things are right in those cases.

In this lesson, we'll cover a bunch of the most common types of semantic errors that occur in C++ (most of which have to do with flow control in some way).

12.3.1 Conditional logic errors

One of the most common types of semantic errors is a conditional logic error. A **conditional logic error** occurs when the programmer incorrectly codes the logic of a conditional statement or loop condition.

12.3.2 Infinite loops

We covered the unsigned type problem in a previous lesson.

12.3.3 Off-by-one errors

An **off-by-one** error is an error that occurs when a loop executes one too many or one too few times.

12.3.4 Incorrect operator precedence

This can also happen when mixing Logical OR and Logical AND in the same expression (Logical AND takes precedence over Logical OR). **Use explicit parenthesization** to avoid these kinds of errors.

12.3.5 Precision issues with floating point types

The following floating-point variable doesn't have enough precision to store the entire number:

```
float f{ 0.123456789f };
```

Because of this lack of precision, the number is rounded slightly: **0.123457**

we talked about how **using operator== and operator!= can be problematic with floating point numbers** due to small rounding errors (as well as what to do about it).

The more arithmetic you do with a floating-point number, the more it will accumulate small rounding errors.

12.3.6 Integer division

In the following example, we mean to do a floating-point division, but because both operands are integers, we end up doing an integer division instead:

```
int x{ 5 };  
int y{ 3 };  
  
std::cout << x << " divided by " << y << " is: " << x / y << '\n'; // integer division
```

This prints: 5 divided by 3 is: 1

In lesson: Arithmetic operators, we showed that we can **use static_cast to convert one of the integral operands to a floating-point value in order to do floating point division.**

12.3.7 *Accidental null statements*

In lesson: Common if statement problems, we covered null statements, which are **statements that do nothing**.

12.3.8 *Not using a compound statement when one is required*

This is similar to the dangling else issue:

```
if (c == 'y')
    std::cout << "Okay, here we go...\n";
    blowUpWorld(); // Will always execute. Should be inside compound statement.
```

12.3.9 *Using assignment instead of equality inside a conditional*

Because the assignment operator (=) and equality operator (==) are similar, we may intend to use equality but accidentally use assignment instead.

Because assignment inside a conditional is almost never intended, modern compilers will often warn when they encounter this. However, if you aren't in the habit of resolving all your warnings, such warnings can easily get lost.

12.3.10 *What else?*

The above represents a good sample of the most common type of semantic errors new C++ programmers tend to make, but **there are plenty more**. Readers, if you have any additional ones that you think are common pitfalls, leave a note in the comments.

12.4 Detecting and handling errors

In lesson: Common semantic errors in C++, we covered many types of common C++ semantic errors that new C++ programmers run into with the language. If an error is the result of a misused language feature or logic error, the error can simply be corrected.

But most errors in a program don't occur as the result of inadvertently misusing language features -- rather, **most errors occur due to faulty assumptions made by the programmer and/or a lack of proper error detection/handling.**

For example, in a function designed to look up a grade for a student, you might have assumed:

- The student being looked up will exist.
- All student names will be unique.
- The class uses letter grading (instead of pass/fail).

What if any of these assumptions aren't true? **If the programmer didn't anticipate these cases, the program will likely malfunction or crash** when such cases arise (usually at some point in the future, well after the function has been written).

There are **three key places where assumption errors typically occur:**

- **When a function returns**, the programmer may have assumed the called function was successful when it was not.
- When a program receives input (either from the user, or a file), the **programmer may have assumed the input was in the correct format and semantically valid** when it was not.
- When a function has been called, the **programmer may have assumed the arguments would be semantically valid when they were not.**

Many new programmers write code and then only test the happy path: only the cases where there are no errors. But **you should also be planning for and testing your sad paths**, where things can and will go wrong. In lesson: Finding issues before they become problems, **we defined defensive programming as the practice of trying to anticipate all of the ways software can be misused, either by end-users, or by developers** (either the programmer themselves, or others). Once you've anticipated (or discovered) some misuse, the next thing to do is handle it.

In this lesson, we'll talk about error handling strategies (**what to do when things go wrong**) **inside a function**. In the subsequent lessons, we'll talk about validating user input, and then introduce a useful tool to help document and validate assumptions.

12.4.1 Handling errors in functions

Functions may fail for any number of reasons -- the caller may have passed in an **argument with an invalid value**, or something may fail within the body of the function. For example, a function that opens a file for reading might fail if the file cannot be found.

When this happens, you have quite a few options at your disposal. **There is no best way to handle an error** -- it really depends on the nature of the problem and whether the problem can be fixed or not.

There are **4 general strategies** that can be used:

- **Handle the error within the function**
- **Pass the error back to the caller** to deal with
- **Halt the program**
- **Throw an exception**

12.4.2 Handling the error within the function

If possible, **the best strategy is to recover from the error in the same function in which the error occurred**, so that **the error can be contained and corrected without impacting any code outside the function**. There are two options here:

- Retry until successful
- Cancel the operation being executed.

If the error has occurred due to something outside of the program's control, the program can retry until success is achieved. For example, if the program requires an internet connection, and the user has lost their connection, the program may be able to display a warning and then use a loop to periodically recheck for internet connectivity. Alternatively, if the user has entered invalid input, the program can ask the user to try again, and loop until the user is successful at entering valid input. We'll show examples of handling invalid input and using loops to retry in the next lesson (std::cin and handling invalid input).

An alternate strategy is just to ignore the error and/or cancel the operation. For example:

```
// Silent failure if y=0
void printIntDivision(int x, int y)
{
    if (y != 0)
        std::cout << x / y;
}
```

12.4.3 *Passing errors back to the caller*

In many cases, the error can't reasonably be handled in the function that detects the error. For example, consider the following function:

```
1 int doIntDivision(int x, int y)
2 {
3     return x / y;
4 }
```

If y is 0, what should we do? We can't just skip the program logic, because the function needs to return some value. We shouldn't ask the user to enter a new value for y because this is a calculation function, and introducing input routines into it may or may not be appropriate for the program calling this function.

In such cases, the best option can be to pass the error back to the caller in hopes that the caller will be able to deal with it.

How might we do that?

If the function has a void return type, it can be changed to return a bool that indicates success or failure. For example, instead of:

```
1 bool printIntDivision(int x, int y)
2 {
3     if (y == 0)
4     {
5         std::cout << "Error: could not divide by zero\n";
6         return false;
7     }
8
9     std::cout << x / y;
10
11     return true;
12 }
```

That way, the caller can check the return value to see if the function failed for some reason.

If the function returns a normal value, things are a little more complicated. In some cases, the full range of return values isn't used. In such cases, we can use a return value that wouldn't otherwise be possible to occur normally to indicate an error.

However, what happens if the user calls this function as `reciprocal(0.0)`? We get a divide by zero error and a program crash, so clearly we should protect against this case. But this function must return a double value, so what value should we return? It turns out that this function will never produce 0.0 as a legitimate result, so we can return 0.0 to indicate an error case.

```
1 // The reciprocal of x is 1/x, returns 0.0 if x=0
2 double reciprocal(double x)
3 {
4     if (x == 0.0)
5         return 0.0;
6
7     return 1.0 / x;
8 }
```

A **sentinel value** is a value that has some special meaning in the context of a function or algorithm. In our `reciprocal()` function above, **0.0 is a sentinel value indicating that the function failed**. The caller can test the return value to see if it matches the sentinel value -- if so, then the caller knows the function failed.

However, if the full range of return values can be produced by the function, then using a sentinel value to indicate an error is problematic (because the caller would not be able to tell whether the return value is a valid value or an error value).

12.4.4 Fatal errors

If the error is so bad that the program cannot continue to operate properly, this is called a **non-recoverable** error (also called a **fatal error**). In such cases, the **best thing to do is terminate the program**.

If your code is in `main()` or a function called directly from `main()`, the best thing to do is let `main()` return a non-zero status code. However, **if you're deep in some nested subfunction, it may not be convenient or possible to propagate the error all the way back to `main()`.** In such a case, a **halt statement** (such as `std::exit()`) can be used.

12.4.5 Exceptions

Because returning an error from a function back to the caller is complicated (and the many different ways to do so leads to inconsistency, and inconsistency leads to mistakes), **C++ offers an entirely separate way to pass errors back to the caller: exceptions.**

The basic idea is that when an error occurs, **an exception is “thrown”**. **If the current function does not “catch” the error, the caller of the function has a chance to catch the error.** If the caller does not catch the error, the caller’s caller has a chance to catch the error. **The error progressively moves up the call stack until it is either caught and handled** (at which point execution continues normally), **or until main() fails to handle the error** (at which point the program is terminated with an exception error).

12.4.6 When to use `std::cout` vs `std::cerr` vs logging

You may be wondering when you should be using `std::cerr` vs `std::cout` vs logging to a text file.

By default, both `std::cout` and `std::cerr` print text to the console. However, modern OSes provide a way to redirect output streams to files, so that the output can be captured for review or automated processing later.

For this discussion, it is useful to differentiate two types of applications:

- **Interactive applications** are those that the user will interact with after running. Most standalone applications, like games and music apps, fall into this category.
- **Non-interactive applications** are applications that do not require user interaction to operate. The output of these programs may be used as input for another application

Within non-interactive applications, there are two types:

- **Tools** are non-interactive applications that are typically launched in order to produce some immediate result, and then terminate after producing such a result. An example of this is Unix's `grep` command, which is a utility that searches text for lines that match some pattern.
- **Services** are non-interactive applications that typically run silently in the background to perform some ongoing function. An example of this would be a virus scanner.

Here's some rules of thumb:

- **Use `std::cout` for all conventional, user-facing text.**
- For an interactive program, use **`std::cout` for normal user-facing error messages** (e.g. "Your input was invalid"). Use **`std::cerr` or a logfile for status and diagnostic information that may be helpful for diagnosing issues** but probably isn't interesting for normal users. **This can include technical warnings and errors** (e.g. bad input to function x), **status updates** (e.g. successfully opened file x, failed to connect to internet service x), **percentage completion of long tasks** (e.g. encoding 50% complete), etc...
- For a non-interactive program (tool or service), **use `std::cerr` for error output only** (e.g. could not open file x). This allows errors to be displayed or parsed separately from normal output.
- **For any application type that is transactional in nature** (e.g. one that processes specific events, such as an interactive web browser or non-interactive web server), **use a logfile to produce a transactional log of events that can be reviewed later.** This can include outputting to the logfile which file is being processed, updates on the percentage complete, timestamps of when it started certain stages of computing, warning and error messages, etc...

12.5 std::cin and handling invalid input

Most programs that have a user interface of some kind need to handle user input. In the programs that you have been writing, you have been using `std::cin` to ask the user to enter text input. Because text input is so free-form (the user can enter anything), it's very easy for the user to enter input that is not expected.

As you write programs, **you should always consider how users will (unintentionally or otherwise) misuse your programs**. A well-written program will anticipate how users will misuse it, and either handle those cases gracefully or prevent them from happening in the first place (if possible). **A program that handles error cases well is said to be robust.**

In this lesson, we'll take a look specifically at ways the user can enter invalid text input via `std::cin`, and show you some different ways to handle those cases.

Before we discuss how `std::cin` and `operator>>` can fail, let's recap how they work. We discussed this material in lesson: Introduction to `iostream`: `cout`, `cin`, and `endl`.

Here's a **simplified view of how `operator>>` works** for input:

1. First, **leading whitespace** (spaces, tabs, and newlines at the front of the buffer) is **discarded** from the input buffer. This **will discard any unextracted newline character remaining from a prior line of input**.
2. If the input buffer is now empty, `operator>>` will **wait for the user to enter more data**. **Leading whitespace is again discarded**.
3. `operator>>` then **extracts as many consecutive characters as it can, until it encounters either a newline character** (representing the end of the line of input) **or a character that is not valid for the variable being extracted to**.

The result of the extraction is as follows:

- If any characters were extracted in step 3 above, **extraction is a success**. The extracted **characters are converted into a value that is then assigned to the variable**.
- If no characters could be extracted in step 3 above, **extraction has failed**. The **object being extracted to is assigned the value 0** (as of C++11), and **any future extractions will immediately fail (until `std::cin` is cleared)**.

12.5.1 Validating input

The process of checking whether user input conforms to what the program is expecting is called **input validation**.

There are **three basic ways** to do input validation:

Inline (as the user types):

1. **Prevent the user from typing invalid input** in the first place.

Post-entry (after the user types):

2. Let the user enter whatever they want into a string, then **validate whether the string is correct, and if so, convert the string to the final variable format**.
3. Let the user enter whatever they want, let `std::cin` and `operator>>` try to extract it, **and handle the error cases**.

Some graphical user interfaces and advanced text interfaces will let you **validate input as the user enters it (character by character)**. Generally speaking, the programmer provides a validation function that accepts the input the user has entered so far, and returns true if the input is valid, and false otherwise. **This function is called every time the user presses a key**. If the validation function returns true, the key the user just pressed is accepted. If the validation function returns false, the character the user just input is discarded (and not shown on the screen). Using this method, you can ensure that any input the user enters is guaranteed to be valid, because any invalid keystrokes are discovered and discarded immediately. **Unfortunately, `std::cin` does not support this style of validation**.

Since strings do not have any restrictions on what characters can be entered, extraction is **guaranteed to succeed (though remember that `std::cin` stops extracting at the first non-leading whitespace character)**. Once a string is entered, **the program can then parse the string to see if it is valid or not**. However, parsing strings and converting string input to other types (e.g. numbers) can be challenging, so this is **only done in rare cases**.

Most often, we let `std::cin` and the extraction operator do the hard work. Under this method, we let the user enter whatever they want, have `std::cin` and `operator>>` try to extract it, and deal with the fallout if it fails. **This is the easiest method, and the one we'll talk more about below**.

12.5.2 Types of invalid text input

We can generally separate input text errors into **four types**:

- **Input extraction succeeds** but the **input is meaningless to the program** (e.g. entering 'k' as your mathematical operator).
- **Input extraction succeeds** but the user enters **additional input** (e.g. entering '*q hello' as your mathematical operator).
- **Input extraction fails** (e.g. trying to enter 'q' into a numeric input).
- **Input extraction succeeds** but the **user overflows a numeric value**.

Thus, to make our programs robust, whenever we ask the user for input, we ideally should determine whether each of the above can possibly occur, and if so, **write code to handle those cases**.

Let's dig into each of these cases, and how to handle them using `std::cin`.

12.5.2.1 Error case 1: Extraction succeeds but input is meaningless

This is the simplest case.

```
Enter one of the following: +, -, *, or /: k
```

In this case, **we asked the user to enter one of four symbols, but they entered 'k' instead**. 'k' is a valid character, so `std::cin` happily extracts it to variable `op`, and this gets returned to `main`. But **our program wasn't expecting this to happen**, so it doesn't properly deal with this case (and thus never outputs anything).

The solution here is simple: do input validation. This usually consists of 3 steps:

1. **Check whether the user's input was what you were expecting.**
2. **If so, return the value** to the caller.
3. **If not, tell the user something went wrong and have them try again.**

Here's an updated `getOperator()` function that does input validation.

```
while (true) // Loop until user enters a valid input
{
    std::cout << "Enter one of the following: +, -, *, or /: ";
    char operation{};
    std::cin >> operation;

    // Check whether the user entered meaningful input
    switch (operation)
    {
        case '+':
        case '-':
        case '*':
        case '/':
            return operation; // return it to the caller
        default: // otherwise tell the user what went wrong
            std::cout << "Oops, that input is invalid. Please try again.\n";
    }
} // and try again
```

As you can see, we're using a while loop to continuously loop until the user provides valid input. If they don't, we ask them to try again until they either give us valid input, shutdown the program, or destroy their computer.

12.5.2.2 Error case 2: Extraction succeeds but with extraneous input

```
Enter a decimal number: 5*7
```

The program prints the right answer, but the formatting is all messed up. Let's take a closer look at why.

When the user enters `5*7` as input, that input goes into the buffer. Then `operator>>` extracts the 5 to variable `x`, leaving `*7\n` in the buffer. Next, the program prints "Enter one of the following: +, -, *, or /:". However, when the extraction operator was called, it sees `*7\n` waiting in the buffer to be extracted, so it uses that instead of asking the user for more input. Consequently, it extracts the `*` character, leaving `7\n` in the buffer.

After asking the user to enter another decimal number, the 7 in the buffer gets extracted without asking the user. Since the user never had an opportunity to enter additional data and hit enter (causing a newline), the output prompts all run together on the same line.

Although the above program works, the execution is messy. It would be better if any extraneous characters entered were simply ignored. Fortunately, it's easy to ignore characters:

```
std::cin.ignore(100, '\n'); // clear up to 100 characters out of the buffer, or until a '\n' character is removed
```

This call would remove up to 100 characters, but if the user entered more than 100 characters we'll get messy output again. To ignore all characters up to the next '\n', we can pass `std::numeric_limits<std::streamsize>::max()` to `std::cin.ignore()`. `std::numeric_limits<std::streamsize>::max()` returns the largest value that can be stored in a variable of type `std::streamsize`. **Passing this value to `std::cin.ignore()` causes it to disable the count check.**

```
void ignoreLine()
{
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
```

```
double getDouble()
{
    std::cout << "Enter a decimal number: ";
    double x{};
    std::cin >> x;

    ignoreLine();
    return x;
}
```

Now our program will work as expected, even if we enter "5*7" for the first input -- the 5 will be extracted, and the rest of the characters will be removed from the input buffer. Since the input buffer is now empty, the user will be properly asked for input the next time an extraction operation is performed!

12.5.2.3 Error case 3: Extraction fails

Extraction fails when no input can be extracted to the specified variable.

Now consider the following execution of our updated calculator program:

```
Enter a decimal number: a
```

You shouldn't be surprised that the program doesn't perform as expected, but how it fails is interesting:

```
Enter a decimal number: a
Enter one of the following: +, -, *, or /: Oops, that input is invalid. Please try again.
Enter one of the following: +, -, *, or /: Oops, that input is invalid. Please try again.
Enter one of the following: +, -, *, or /: Oops, that input is invalid. Please try again.
```

and that last line keeps printing until the program is closed.

This looks pretty similar to the extraneous input case, but it's a little different. Let's take a closer look.

When the user enters 'a', that character is placed in the buffer. Then `operator>>` tries to extract 'a' to variable `x`, which is of type `double`. Since 'a' can't be converted to a `double`, `operator>>` can't do the extraction. Two things happen at this point: 'a' is left in the buffer, and `std::cin` goes into "failure mode".

Once in "failure mode", future requests for input extraction will silently fail. Thus, in our calculator program, the output prompts still print, but any requests for further extraction are ignored. This means that instead waiting for us to enter an operation, the input prompt is skipped, and we get stuck in an infinite loop because there is no way to reach one of the valid cases.

Fortunately, we can detect whether an extraction has failed:

```
if (std::cin.fail()) // If the previous extraction failed
{
    // Let's handle the failure
    std::cin.clear(); // Put us back in 'normal' operation mode
    ignoreLine();    // And remove the bad input
}
```

For fundamental types, a **failed extraction due to invalid input will cause the variable to be assigned the value 0** (or whatever value 0 converts to in the variable's type).

Key insight: Once an extraction has failed, **future requests for input extraction** (including calls to `ignore()`) **will silently fail until the `clear()` function is called**. Thus, after detecting a failed extraction, calling `clear()` is usually the first thing you should do.

On Unix systems, entering an **end-of-file (EOF) character** (via ctrl-D) **closes the input stream**. This is something that `std::cin.clear()` can't fix, so `std::cin` never leaves failure mode, which causes all subsequent input operations to fail. When this happens inside an infinite loop, your program will then loop endlessly until killed. To handle this case more elegantly, you can explicitly test for EOF using **`std::cin.eof()`**.

Because clearing a failed input stream is something we're likely to be checking for a lot, this is a good candidate for a reusable function:

```
// returns true if extraction failed, false otherwise
bool clearFailedExtraction()
{
    // Check for failed extraction
    if (!std::cin) // If the previous extraction failed
    {
        if (std::cin.eof()) // If the stream was closed
        {
            exit(0); // Shut down the program now
        }

        // Let's handle the failure
        std::cin.clear(); // Put us back in 'normal' operation mode
        ignoreLine();    // And remove the bad input

        return true;
    }

    return false;
}
```


12.5.2.4 Error case 4: Extraction succeeds but the user overflows a numeric value

What happens if the **user enters a number that is too large** (e.g. 40000)?

In the above case, `std::cin` goes immediately into “failure mode”, but also assigns the closest in-range value to the variable. When the entered value is larger than the largest possible value for the type, the closest in-range value is the largest possible value for the type. Consequently, `x` is left with the assigned value of 32767. Additional inputs are skipped, leaving `y` with the initialized value of 0. We can handle this kind of error in the same way as a failed extraction.

12.5.3 Putting it all together

```
1  #include <cstdlib> // for std::exit
2  #include <iostream>
3  #include <limits> // for std::numeric_limits
4
5  void ignoreLine()
6  {
7      std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
8  }
9
10 // returns true if extraction failed, false otherwise
11 bool clearFailedExtraction()
12 {
13     // Check for failed extraction
14     if (!std::cin) // If the previous extraction failed
15     {
16         if (std::cin.eof()) // If the stream was closed
17         {
18             exit(0); // Shut down the program now
19         }
20
21         // Let's handle the failure
22         std::cin.clear(); // Put us back in 'normal' operation mode
23         ignoreLine();    // And remove the bad input
24
25         return true;
26     }
27
28     return false;
29 }
30
31 double getDouble()
32 {
33     while (true) // Loop until user enters a valid input
34     {
35         std::cout << "Enter a decimal number: ";
36         double x{};
37         std::cin >> x;
38
39         if (clearFailedExtraction())
40         {
41             std::cout << "Oops, that input is invalid. Please try again.\n";
42             continue;
43         }
44
45         ignoreLine(); // Remove any extraneous input
46         return x;    // Return the value we extracted
47     }
48 }
49
50 char getOperator()
51 {
52     while (true) // Loop until user enters a valid input
53     {
54         std::cout << "Enter one of the following: +, -, *, or /: ";
55         char operation{};
56         std::cin >> operation;
57
58         if (!clearFailedExtraction()) // we'll handle error messaging if extraction failed below
59             ignoreLine(); // remove any extraneous input (only if extraction succeeded)
60
61         // Check whether the user entered meaningful input
62         switch (operation)
63         {
64             case '+':
65             case '-':
66             case '*':
67             case '/':
68                 return operation; // Return the entered char to the caller
69             default: // Otherwise tell the user what went wrong
70                 std::cout << "Oops, that input is invalid. Please try again.\n";
71         }
72     }
73 }
```

```
75 void printResult(double x, char operation, double y)
76 {
77     std::cout << x << " " << operation << " " << y << " is ";
78
79     switch (operation)
80     {
81     case '+':
82         std::cout << x + y << '\n';
83         return;
84     case '-':
85         std::cout << x - y << '\n';
86         return;
87     case '*':
88         std::cout << x * y << '\n';
89         return;
90     case '/':
91         if (y == 0.0)
92             break;
93
94         std::cout << x / y << '\n';
95         return;
96     }
97
98     std::cout << "???"; // Being robust means handling unexpected parameters as well, even though getOperator() guarantees operation is valid
99     // in this particular program
100 }
101
102 int main()
103 {
104     double x{ getDouble() };
105     char operation{ getOperator() };
106     double y{ getDouble() };
107
108     // Handle division by 0
109     while (operation == '/' && y == 0.0)
110     {
111         std::cout << "The denominator cannot be zero. Try again.\n";
112         y = getDouble();
113     }
114
115     printResult(x, operation, y);
116
117     return 0;
118 }
```

12.5.4 Conclusion

As you write your programs, consider how users will misuse your program, especially around text input. For each point of text input, consider:

- Could extraction fail?
- Could the user enter more input than expected?
- Could the user enter meaningless input?
- Could the user overflow an input?

You can use if statements and boolean logic to test whether input is expected and meaningful.

12.6 Assert and static_assert

In a function that takes parameters, the caller may be able to pass in arguments that are **syntactically valid but semantically meaningless**.

If the program terminates (via `std::exit`) then we will have lost our call stack and any debugging information that might help us isolate the problem. `std::abort` is a better option for such cases, as typically the developer will be given the option to start debugging at the point where the program aborted.

12.6.1 Preconditions, invariants, and postconditions

In programming, a **precondition** is any **condition that must be true prior to the execution of some section of code** (typically the body of a function).

Preconditions for a function are **best placed at the top of a function**, using an **early return to return back to the caller if the precondition isn't met**. For example:

```
void printDivision(int x, int y)
{
    if (y == 0) // handle
    {
        std::cerr << "Error: Could not divide by zero\n";
        return; // bounce the user back to the caller
    }

    // We now know that y != 0
    std::cout << static_cast<double>(x) / y;
}
```

This is sometimes known as the “**bouncer pattern**”.

An **invariant** is a condition that must be true **while some section of code is executing**. This is often used with loops, where the loop body will only execute so long as the invariant is true.

Similarly, a **postcondition** is something that must be true **after the execution of some section of code**. Our function doesn't have any postconditions.

12.6.2 Assertions

Using a conditional statement to detect an invalid parameter (or to validate some other kind of assumption), along with printing an error message and terminating the program, is **such a common method of detecting problems that C++ provides a shortcut method for doing this.**

An **assertion is an expression that will be true unless there is a bug in the program.** If the expression evaluates to true, the assertion statement does nothing. **If the conditional expression evaluates to false, an error message is displayed and the program is terminated (via `std::abort`).** This error message **typically contains the expression that failed as text, along with the name of the code file and the line number of the assertion.** This makes it very easy to tell not only what the problem was, but where in the code the problem occurred. **This can help with debugging efforts immensely.**

Key insight:

When an assertion evaluates to false, your program is immediately stopped. This gives you an opportunity to use debugging tools to examine the state of your program and determine why the assertion failed. Working backwards, you can then find and fix the issue.

Without an assertion to detect an error and fail, such an error would likely cause your program to malfunction later. In such cases, it can be very difficult to determine where things are going wrong, or what the root cause of the issue actually is.

In C++, **runtime assertions** are implemented via the **`assert`** preprocessor macro, which lives in the **`<cassert>`** header.

```
double calculateTimeUntilObjectHitsGround(double initialHeight, double gravity)
{
    assert(gravity > 0.0); // The object won't reach the ground unless there is positive gravity.

    if (initialHeight <= 0.0)
    {
        // The object is already on the ground. Or buried.
        return 0.0;
    }

    return std::sqrt((2.0 * initialHeight) / gravity);
}
```

When the program calls **`calculateTimeUntilObjectHitsGround(100.0, -9.8)`**, **`assert(gravity > 0.0)`** will evaluate to false, which will trigger the assert. That will print a message similar to this:

`dropsimulator: src/main.cpp:6: double calculateTimeUntilObjectHitsGround(double, double): Assertion 'gravity > 0.0' failed.`

The actual message varies depending on which compiler you use.

Although asserts are most often used to validate function parameters, they can be used anywhere you would like to validate that something is true.

Although we told you previously to avoid preprocessor macros, asserts are one of the few preprocessor macros that are considered acceptable to use. We encourage you to use assert statements liberally throughout your code.

The actual message varies depending on which compiler you use.

Although asserts are most often used to validate function parameters, they can be used anywhere you would like to validate that something is true.

Although we told you previously to avoid preprocessor macros, **asserts are one of the few preprocessor macros that are considered acceptable to use.** We encourage you to use **assert statements liberally throughout your code.**

12.6.3 Making your assert statements more descriptive

Sometimes assert expressions aren't very descriptive. Consider the following statement:

```
1 | assert(found);
```

 If this assert is triggered, the assert will say: **Assertion failed: found, file C:\\VCProjects\\Test.cpp, line 34**

What does this even mean? Clearly **found was false** (since the assert triggered), **but what wasn't found?** You'd have to go look at the code to determine that.

Fortunately, there's a little trick you can use to make your assert statements more descriptive. Simply add a string literal joined by a logical AND:

```
assert(found && "Car could not be found in database");
```

Here's why this works: A **string literal always evaluates to Boolean true**. So, **if found is false, false && true is false**. If found is true, true && true is true. Thus, logical AND-ing a string literal doesn't impact the evaluation of the assert.

However, when the assert triggers, the string literal will be included in the assert message:

Assertion failed: found && "Car could not be found in database", file C:\\VCProjects\\Test.cpp, line 34

12.6.4 Asserts vs error handling

Assertions and error handling are similar enough that their purposes can be confused, so let's clarify:

The **goal of an assertion is to catch programming errors by documenting something that should never happen**. If that thing does happen, then the programmer made an error somewhere, and that error can be identified and fixed. **Assertions do not allow recovery from errors** (after all, if something should never happen, there's no need to recover from it), and the program will not produce a friendly error message.

On the other hand, **error handling is designed to gracefully handle cases that could happen (however rarely)** in release configurations. These **may or may not be recoverable**, but one should always assume a user of the program may encounter them.

Best practice: Use assertions to document cases that should be logically impossible.

Assertions are also sometimes used to document cases that were not implemented because they were not needed at the time the programmer wrote the code:

```
assert(moved && "Need to handle case where student was just moved to another classroom");
```

That way, if a future user of the code does encounter a situation where this case is needed, the code will fail with a useful error message, and the programmer can then determine how to implement that case.

12.6.5 NDEBUG

The **assert** macro comes with a small performance cost that is incurred each time the assert condition is checked. Furthermore, **asserts should (ideally) never be encountered in production code (because your code should already be thoroughly tested)**. Consequently, many **developers prefer that asserts are only active in debug builds**. C++ comes with a way to turn off asserts in production code. If the macro **NDEBUG** is defined, the assert macro gets disabled.

Some IDEs **set NDEBUG by default** as part of the project settings **for release configurations**. For example, in Visual Studio, the following preprocessor definitions are set at the project level: **WIN32;NDEBUG;_CONSOLE**. If you're using Visual Studio and want your asserts to trigger in release builds, you'll need to remove **NDEBUG** from this setting.

If you're using an IDE or build system that doesn't automatically define **NDEBUG** in release configuration, you will need to add it in the project or compilation settings manually.

12.6.6 *Some assert limitations and warnings*

There are a **few pitfalls and limitations** to asserts. First, the **assert itself can be improperly written**. If this happens, the assert will either **report an error where none exists, or fail to report a bug where one does exist**.

Second, your **asserts should have no side effects** -- that is, the program should run the same with and without the assert. Otherwise, what you are testing in a debug configuration will not be the same as in a release configuration (assuming you ship with NDEBUG).

Also note that the **abort()** function **terminates the program immediately, without a chance to do any further cleanup** (e.g. close a file or database). Because of this, asserts should be used only in cases where corruption isn't likely to occur if the program terminates unexpectedly.

12.6.7 *Static_assert*

C++ also has another type of assert called `static_assert`. A **static_assert** is an assertion that is **checked at compile-time rather than at runtime**, with a failing `static_assert` causing a compile error. **Unlike `assert`, which is declared in the `<cassert>` header, `static_assert` is a keyword, so no header needs to be included to use it.**

```
static_assert(condition, diagnostic_message)
```

If the condition is not true, the diagnostic message is printed.

A few **useful notes about `static_assert`**:

- Because `static_assert` is evaluated by the compiler, the **condition must be a constant expression**.
- `static_assert` **can be placed anywhere in the code file** (even in the global namespace).
- **`static_assert` is not deactivated in release builds** (like normal `assert` is).

Prior to C++17, the diagnostic message must be supplied as the second parameter. **Since C++17, providing a diagnostic message is optional.**

12.7 Summary

Scope creep occurs when a project's capabilities grow beyond what was originally intended at the start of the project or project phase.

Software verification is the process of testing whether or not the software works as expected in all cases. A **unit test** is a test designed to test a small portion of the code (typically a function or call) in isolation to ensure a particular behavior occurs as expected. **Unit test frameworks** can help you organize your unit tests. **Integration testing** tests the integration of a bunch of units together to ensure they work properly.

Code coverage refers to how much of the source code is executed while testing. **Statement coverage** refers to the percentage of statements in a program that have been exercised by testing routines. **Branch coverage** refers to the percentage of branches that have been executed by testing routines. Loop coverage (also called the **0, 1, 2 test**) means that if you have a loop, you should ensure it works properly when it iterates 0 times, 1 time, and 2 times.

The **happy path** is the path of execution that occurs when there are no errors encountered. A **sad path** is one where an error or failure state occurs. A **non-recoverable error** (also called a **fatal error**) is an error that is severe enough that the program can't continue running. A program that handles error cases well is **robust**.

A **buffer** is a piece of memory set aside for storing data temporarily while it is moved from one place to another.

The process of checking whether user input conforms to what the program is expecting is called **input validation**.

std::cerr is an output stream (like **std::cout**) designed to be used for error messages.

A **precondition** is any condition that must always be true prior to the execution of some segment of code. An **invariant** is a condition that must be true while some component is executing. A **postcondition** is any condition that must always be true after the execution of some code.

An **assertion** is an expression that will be true unless there is a bug in the program. In C++, runtime assertions are typically implemented using the **assert** preprocessor macro. Assertions are usually turned off in non-debug code. A **static_assert** is an assertion that is evaluated at compile-time.

Assertions should be used to document cases that should be logically impossible. Error handling should be used to handle cases that are possible.

13 Type Conversion, Type Aliases, and Type Deduction

13.1 Implicit type conversion

The value of an object is stored as a sequence of bits, and the data type of the object tells the compiler how to interpret those bits into meaningful values. **Different data types may represent the “same” number differently.** For example, the integer value 3 might be stored as binary 0000 0000 0000 0000 0000 0000 0011, whereas floating point value 3.0 might be stored as binary 0100 0000 0100 0000 0000 0000 0000 0000.

So, what happens when we do something like this?

```
1 float f{ 3 }; // initialize floating point variable with int 3
```

In such a case, the compiler can't just copy the bits representing the int value 3 into the memory allocated for float variable f. Instead, it needs to convert the integer value 3 to the equivalent floating-point value 3.0, which can then be stored in the memory allocated for f.

The process of producing a new value of some type from a value of a different type is called a **conversion**.

Key insight: Conversions do not change the value or type being converted. Instead, **a new value with the desired type is created as a result of the conversion.**

Type conversion can be **invoked in one of two ways**:

- **implicitly** (as needed by the **compiler**)
- **explicitly** (when **requested** by the programmer).

13.1.1 Implicit type conversion

Implicit type conversion (also called **automatic type conversion** or **coercion**) is performed automatically by the compiler when one data type is required, but a different data type is supplied. The vast majority of type conversions in C++ are implicit type conversions:

- When **initializing** or **assigning** a value to a variable of a different data type.
- When the type of a **return value is different from the function's declared return type**.
- When using certain binary **operators with operands of different types**.
- When using a **non-Boolean value in an if-statement**.
- When an **argument passed to a function is a different type than the function parameter**.

13.1.2 What happens when a type conversion is invoked

When a type conversion is invoked (whether implicitly or explicitly), the compiler will determine whether it can convert the value from the current type to the desired type. If a valid conversion can be found, then the **compiler will produce a new value of the desired type**. Note that type conversions **don't change the value or type of the value or object being converted**.

If the compiler can't find an acceptable conversion, then the compilation will fail with a compile error. Type conversions can fail for any number of reasons. For example, the compiler might not know how to convert a value between the original type and the desired type. In other cases, statements may disallow certain types of conversions. For example:

```
int x { 3.5 }; // brace-initialization disallows conversions that result in data loss
```

Even though the compiler knows how to convert a double value to an int value, such conversions are disallowed when using brace-initialization.

There are also cases where the compiler may not be able to figure out which of several possible type conversions is unambiguously the best one to use.

So how does the compiler actually determine whether it can convert a value from one type to another?

13.1.3 The standard conversions

The C++ language standard defines how different fundamental types (and in some cases, compound types) can be converted to other types. These conversion rules are called the **standard conversions**.

The standard conversions can be broadly divided into **4 categories**, each covering different types of conversions:

- Numeric promotions
- Numeric conversions
- Arithmetic conversions
- Other conversions (which includes various pointer and reference conversions)

When a type conversion is needed, the compiler will see if there are standard conversions that it can use to convert the value to the desired type. **The compiler may apply zero, one, or (in certain cases) two standard conversions in the conversion process.**

As an aside...

How do you have a type conversion with zero conversions? As an example, on architectures where `int` and `long` both have the **same size and range**, the same sequence of bits is used to represent values of both types. Therefore, no actual conversion is needed to convert a value between those types -- the **value can simply be copied**.

The full set of rules describing how type conversions work is both lengthy and complicated, and for the most part, type conversion “just works”. In the next set of lessons, we’ll cover the most important things you need to know about type conversions. If finer detail is required for some uncommon case, the full rules are detailed in [technical reference documentation for implicit conversions](#).

13.2 Numeric promotions (For processing efficiency)

In lesson: sizes and the `sizeof` operator, we noted that **C++ has minimum size guarantees for each of the fundamental types**. However, the **actual size of these types can vary based on the compiler and architecture**.

This **variability was allowed so that the `int` and `double` data types could be set to the size that maximizes performance on a given architecture**. For example, a 32-bit computer will typically be able to process 32-bits of data at a time. In such cases, an `int` would likely be set to a width of 32-bits, since this is the “natural” size of the data that the CPU operates on (and likely to be the most performant).

A reminder: The **number of bits a data type uses is called its width**. A wider data type is one that uses more bits, and a narrower data type is one that uses less bits.

But what happens when we want our 32-bit CPU to modify an 8-bit value (such as a `char`) or a 16-bit value? **Some 32-bit processors (such as 32-bit x86 CPUs) can manipulate 8-bit or 16-bit values directly**. However, **doing so is often slower** than manipulating 32-bit values! **Other 32-bit CPUs (like 32-bit PowerPC CPUs), can only operate on 32-bit values**, and **additional tricks must be employed to manipulate narrower values**.

13.2.1 Numeric promotion

Because C++ is designed to be portable and performant across a wide range of architectures, the language designers did not want to assume a given CPU would be able to efficiently manipulate values that were narrower than the natural data size for that CPU.

To help address this challenge, C++ defines a category of type conversions informally called the numeric promotions. A **numeric promotion is the type conversion of certain narrower numeric types (such as a char) to certain wider numeric types (typically int or double) that can be processed efficiently.**

All numeric promotions are value-preserving. A **value-preserving conversion** (also called a **safe conversion**) is one where every possible source value can be converted into an equal value of the destination type.

Because promotions are safe, the compiler will freely use numeric promotion as needed, and will not issue a warning when doing so.

13.2.2 Numeric promotion reduces redundancy

Numeric promotion **solves another problem as well.**

Consider the case where you wanted to write a function to print a value of type int:

```
void printInt(int x)
{
    std::cout << x << '\n';
}
```

While this is straightforward, what happens if we want to also be able to print a value of type short, or type char? **If type conversions did not exist, we'd have to write a different print function for short and another one for char** (redundancy). And don't forget another version for unsigned char, signed char, unsigned short, wchar_t, char8_t, char16_t, and char32_t! **You can see how this quickly becomes unmanageable.**

Numeric promotion comes to the rescue here: we can write functions that have int and/or double parameters (such as the printInt() function above). That **same code can then be called with arguments of types that can be numerically promoted** to match the types of the function parameters.

13.2.3 Numeric promotion categories

The numeric promotion rules are divided into **two subcategories**:

- integral promotions
- floating-point promotions

Only the conversions listed in these categories are considered to be numeric promotions.

13.2.4 Floating point promotions

We'll start with the easier one.

Using the floating-point promotion rules, a value of type **float can be converted to a value of type double**. This means we can write a function that takes a double and then call it with either a double or a float value.

13.2.5 Integral promotions

The integral promotion rules are more complicated.

Using the **integral promotion** rules, the following conversions can be made:

- **signed char or signed short** can be converted **to int**.
- **unsigned char, char8_t, and unsigned short** can be converted **to int if int can hold the entire range of the type, or unsigned int otherwise**.
- **bool** can be converted **to int**, with false becoming 0 and true becoming 1.

Assuming an 8-bit byte and an int size of 4 bytes or larger (which is typical these days), the above basically means that bool, char, signed char, unsigned char, signed short, and unsigned short all get promoted to int.

In most cases, this lets us write a function taking an int parameter, and then use it with a wide variety of other integral types.

```
short s{ 3 }; // there is no short literal suffix, so we'll use a variable for this one
printInt(s); // numeric promotion of short to int

printInt('a'); // numeric promotion of char to int
printInt(true); // numeric promotion of bool to int
```

There are two things worth noting here. First, on some architectures (e.g. with 2-byte ints) **it is possible for some of the unsigned integral types to be promoted to unsigned int rather than int.**

Second, some **narrower unsigned types (such as unsigned char) may be promoted to larger signed types (such as int).** So, **while integral promotion is value-preserving, it does not necessarily preserve the signedness** (signed/unsigned) of the type.

13.2.6 Not all widening conversions are numeric promotions

Some widening type conversions (such as char to short, or int to long) are **not considered to be numeric promotions** in C++ (**they are numeric conversions**, which we'll cover soon). This is **because such conversions do not assist in the goal of converting smaller types to larger types that can be processed more efficiently.**

The **distinction is mostly academic.** However, in certain cases, the compiler will favor numeric promotions over numeric conversions. We'll see examples where this makes a difference when we cover function overload resolution.

13.3 Numeric conversions

In the previous lesson, we covered numeric promotions, which are conversions of specific **narrower numeric types to wider numeric types (typically int or double) that can be processed efficiently.**

C++ supports another category of numeric type conversions, called numeric conversions. These numeric conversions cover additional type conversions between fundamental types.

Key insight: Any type **conversion covered by the numeric promotion rules** is called a **numeric promotion, not a numeric conversion.**

There are **five basic types** of numeric conversions:

1. Converting an **integral type** to any other **integral type** (excluding integral **promotions**)

```
1 short s = 3; // convert int to short
2 long l = 3; // convert int to long
3 char ch = s; // convert short to char
4 unsigned int u = 3; // convert int to unsigned int
```

2. Converting a **floating-point type** to any other **floating-point type** (excluding floating point **promotions**)

```
1 float f = 3.0; // convert double to float
2 long double ld = 3.0; // convert double to long double
```

3. Converting a **floating-point type** to any **integral type**

```
1 int i = 3.5; // convert double to int
```

4. Converting an **integral type** to any **floating-point type**

```
1 double d = 3; // convert int to double
```

5. Converting an **integral type** or a **floating-point type** to a **bool**

```
1 bool b1 = 3; // convert int to bool
2 bool b2 = 3.0; // convert double to bool
```

Key insight: Because **brace initialization** strictly disallows some types of numeric conversions (more on this next lesson), we use copy initialization in this lesson (which does not have any such limitations) in order to keep the examples simple.

13.3.1 Safe and unsafe conversions

Unlike numeric promotions (which are always value-preserving and thus “safe”), many numeric conversions are unsafe. An **unsafe conversion** is one where at least one value of the source type cannot be converted into an equal value of the destination type.

Numeric conversions fall into **three general safety categories**:

- Value-preserving conversions
 - **safe** numeric conversions where the destination type can exactly represent all possible values in the source type.
 - E.g.: `int` → `long` and `short` → `double`
 - can always be converted back to the source type
- Reinterpretive conversions
 - **unsafe** numeric conversions where the converted value may be different than the source value, but **no data is lost**. Signed/unsigned conversions fall into this category.
 - E.g.: `Signed int` → `unsigned int`
 - can always be converted back to the source type
- Lossy conversions
 - **unsafe** numeric conversions where **data may be lost** during the conversion.
 - E.g.: `double` → `int` and `double` → `float`
 - Converting back to original type results in data loss.

Warning: Some conversions may fall into different categories depending on the platform.

For example, `int` to `double` is typically a safe conversion, because `int` is usually 4 bytes and `double` is usually 8 bytes, and on such systems, all possible `int` values can be represented as a `double`. However, there are architectures where both `int` and `double` are 8 bytes. On such architectures, `int` to `double` is a lossy conversion!

Unsafe conversions should be avoided as much as possible. However, this is not always possible. When unsafe conversions are used, it is most often when:

- We can constrain the values to be converted to just those that can be converted to equal values. For example, an `int` can be safely converted to an `unsigned int` when we can guarantee that the `int` is non-negative.
- We don't mind that some data is lost because it is not relevant. For example, converting an `int` to a `bool` results in the loss of data, but we're typically okay with this because we're just checking if the `int` has value 0 or not.

13.3.2 *More on numeric conversions*

The specific rules for numeric conversions are complicated and numerous, so here are the most **important things to remember**:

- In all cases, converting a value into a type whose range doesn't support that value will lead to results that are probably **unexpected**.
- Remember that **overflow** is well-defined for unsigned values and **produces undefined behavior for signed values**.
- Converting from a larger integral or floating-point type to a smaller type from the same family **will generally work so long as the value fits in the range of the smaller type**.
- In the case of floating-point values, some **rounding may occur due to a loss of precision in the smaller type**. Float only supports 7 significant digits.
- Converting from an integer to a floating-point number generally works **as long as the value fits within the range of the floating-point type**.
- Converting from a floating point to an integer works as long **as the value fits within the range of the integer, but any fractional values are lost**.

While the numeric conversion rules might seem scary, **in reality the compiler will generally warn you if you try to do something dangerous** (excluding some signed/unsigned conversions).

13.4 Narrowing conversions, list initialization, and constexpr initializers

13.4.1 *Narrowing conversions*

In C++, a **narrowing conversion** is a **potentially unsafe numeric conversion** where the **destination type may not be able to hold all the values** of the source type.

The following conversions are defined to be narrowing:

- Floating-point type → integral type
- Floating-point type → narrower floating-point type
 - unless the value being converted is constexpr and is in range of the destination type (even if the destination type doesn't have the precision to store all the significant digits of the number).
- Integral type → floating-point type
 - unless the value being converted is constexpr and whose value can be stored exactly in the destination type.
- Integral type → integral type that cannot represent all values of the original type
 - unless the value being converted is constexpr and whose value can be stored exactly in the destination type. This covers both wider to narrower integral conversions, as well as integral sign conversions (signed to unsigned, or vice-versa).

In most cases, implicit narrowing conversions will result in **compiler warnings**, with the **exception of signed/unsigned conversions** (which may or may not produce warnings, depending on how your compiler is configured).

Narrowing conversions should be avoided as much as possible, because they are potentially unsafe, and thus a source of potential errors.

Best practice: Because they can be unsafe and are a source of errors, avoid narrowing conversions whenever possible.

13.4.2 *Make intentional narrowing conversions explicit*

Narrowing conversions are not always avoidable -- this is particularly true for function calls, where the function parameter and argument may have mismatched types and require a narrowing conversion.

In such cases, it is a good idea to convert an implicit narrowing conversion into an explicit narrowing conversion using `static_cast`. Doing so helps document that the narrowing conversion is intentional, and will suppress any compiler warnings or errors that would otherwise result.

Best practice: If you need to perform a narrowing conversion, use `static_cast` to convert it into an explicit conversion.

13.4.3 *Brace initialization disallows narrowing conversions*

Narrowing conversions are disallowed when using brace initialization (which is one of the primary reasons this initialization form is preferred), and attempting to do so will produce a compile error.

```
int i { 3.5 }; won't compile.
```

If you actually want to do a narrowing conversion inside a brace initialization, **use `static_cast` to convert the narrowing conversion into an explicit conversion.**

13.4.4 *Some constexpr conversions aren't considered narrowing*

When the source value of a narrowing conversion isn't known until runtime, the result of the conversion also can't be determined until runtime. In such cases, whether the narrowing conversion preserves the value or not also can't be determined until runtime.

However, you may have noticed that most of the narrowing conversions definitions have an exception clause that begins with **"unless the value being converted is constexpr and ..."**. For example, a conversion is narrowing when it is "From an integral type to another integral type that cannot represent all values of the original type, unless the value being converted is constexpr and whose value can be stored exactly in the destination type."

When the source value of a narrowing conversion is constexpr, the specific value to be converted must be known to the compiler. **In such cases, the compiler can perform the conversion itself, and then check whether the value was preserved.** If the value was not preserved, the **compiler can halt compilation with an error. If the value is preserved, the conversion is not considered to be narrowing** (and the compiler can replace the entire conversion with the converted result, knowing that doing so is safe).

Strangely, **conversions from a floating-point type to an integral type do not have a constexpr exclusion clause**, so these are always considered narrowing conversions even when the value to be converted is constexpr and fits in the range of the destination type.

Even more strangely, **conversions from a constexpr floating point type to a narrower floating-point type are not considered narrowing** even when there is a loss of precision!

13.4.5 List initialization with constexpr initializers

These **constexpr** exception clauses are incredibly useful when list initializing non-int/non-double objects, as we can use an int or double literal (or a constexpr object) initialization value.

This allows us to avoid:

- Having to use literal suffixes in most cases
- Having to clutter our initializations with a static_cast

For example:

```
// We can avoid literals with suffixes
unsigned int u { 5 }; // okay (we don't need to use `5u`)
float f { 1.5 };      // okay (we don't need to use `1.5f`)

// We can avoid static_casts
constexpr int n{ 5 };
double d { n };        // okay (we don't need a static_cast here)
short s { 5 };         // okay (there is no suffix for short, we don't need a static_cast here)
```

This also works with copy and direct initialization.

One caveat worth mentioning: initializing a narrower or lesser ranked floating-point type with a constexpr value is allowed as long as the value is in range of the destination type, even if the destination type doesn't have enough precision to precisely store the value!

Key insight: Floating point types are ranked in this order (**greater to lesser**):

- Long double
- Double
- Float

Therefore, something like this is legal and will not emit an error:

```
float f { 1.23456789 }; // not a narrowing conversion, even though precision lost!
```

13.5 Arithmetic conversions

Consider the following expression:

```
1 | int x { 2 + 3 };
```

When binary operator+ is invoked, it is given two operands, both of type int. **Because both operands are of the same type, that type will be used to perform the calculation and to return the result.** Thus, 2 + 3 will evaluate to int value 5.

But what happens when the operands of a binary operator are of different types?

```
1 | ??? y { 2 + 3.5 };
```

In this case, operator+ is being given one operand of type int and another of type double. Should the result of the operator be returned as an int, a double, or possibly something else altogether? When defining a variable, we can choose what type it has. In other cases, for example when using `std::cout <<`, the type the calculation evaluates to changes the behavior of what is output.

In C++, certain operators require that their operands be of the same type. If one of these operators is invoked with operands of different types, **one or both of the operands will be implicitly converted to matching types using a set of rules** called the **usual arithmetic conversions**.

13.5.1 The operators that require operands of the same type

The following operators require their operands to be of the same type:

- The **binary** arithmetic operators: +, -, *, /, %
- The **binary** relational operators: <, >, <=, >=, ==, !=
- The **binary** bitwise arithmetic operators: &, ^, |
- The **conditional operator** ?: (excluding the condition, which is expected to be of type bool)

13.5.2 *The usual arithmetic conversion rules*

The usual arithmetic conversion rules are somewhat complex, so we'll simplify a bit. The **compiler has a ranked list of types** that looks something like this:

- long double (**highest rank**)
- double
- float
- long long
- long
- int (**lowest rank**)

The following rules are applied to find a matching type:

- **If one operand is an integral type and the other a floating-point type, the integral operand is converted to the type of the floating-point operand** (no integral promotion takes place).

Otherwise, any integral operands are numerically promoted:

- If one operand is signed and the other unsigned, **special rules apply (see below)**
- **Otherwise, the operand with lower rank is converted to the type of the operand with higher rank.**

For advanced readers: The special matching rules for integral operands with **different signs**:

- If the rank of the unsigned operand is greater than the rank of the signed operand, the signed operand is converted to the type of the unsigned operand.
- If the type of the signed operand can represent all the values of the type of the unsigned operand, the type of the unsigned operand is converted to the type of the signed operand.
- Otherwise, both operands are converted to the corresponding unsigned type of the signed operand.

13.5.3 Signed and unsigned issues

This prioritization hierarchy and conversion rules can cause some problematic issues when mixing signed and unsigned values. For example, take a look at the following code:

```
1 #include <iostream>
2 #include <typeinfo> // for typeid()
3
4 int main()
5 {
6     std::cout << typeid(5u-10).name() << ' ' << 5u - 10 << '\n'; // 5u means treat 5 as an unsigned integer
7
8     return 0;
9 }
```

You might expect the expression `5u - 10` to evaluate to `-5` since $5 - 10 = -5$. But here's what actually results: unsigned int **4294967291**

Due to the conversion rules, the `int` operand is converted to an unsigned `int`. And since the value `-5` is out of range of an unsigned `int`, we get a result we don't expect.

This is **one of the primary reasons to avoid unsigned integers** -- when you mix them with signed integers in arithmetic expressions, **you're at risk for unexpected results**. And the **compiler probably won't even issue a warning**.

13.6 Explicit type conversion (casting) and static_cast

Many new C++ programmers try something like this:

```
1 double d = 10 / 4; // does integer division, initializes d with value 2.0
```

Because 10 and 4 are both of type `int`, integer division is performed, and the expression evaluates to `int` value 2. This value then undergoes numeric conversion to double value 2.0 before being used to initialize variable `d`. Most likely, this isn't what was intended.

In the case where you are using literal operands, replacing one or both of the integer literals with double literals will cause floating point division to happen instead:

```
1 double d = 10.0 / 4.0; // does floating point division, initializes d with value 2.5
```

But what if you are using variables instead of literals? Consider this case:

```
1 int x { 10 };  
2 int y { 4 };  
3 double d = x / y; // does integer division, initializes d with value 2.0
```

Because integer division is used here, variable `d` will end up with the value of 2.0. How do we tell the compiler that we want to use floating point division instead of integer division in this case? Literal suffixes can't be used with variables. We need some way to convert one (or both) of the variable operands to a floating-point type, so that floating point division will be used instead.

Fortunately, C++ comes with a number of different **type casting operators** (more commonly called **casts**) that can be used by the programmer to request that the compiler perform a type conversion. Because casts are explicit requests by the programmer, this form of type conversion is often called an **explicit type conversion** (as opposed to implicit type conversion, where the compiler performs a type conversion automatically).

13.6.1 Type casting

C++ supports 5 different types of casts:

- C-style casts
- Static casts
- Const casts
- Dynamic casts
- Reinterpret casts

The latter four are sometimes referred to as **named casts**.

Const casts and reinterpret casts should generally be avoided because they are only useful in rare cases and can be harmful if used incorrectly.

13.6.2 C-style casts

In standard C programming, casts are done via the () operator, with the name of the type to convert the value placed inside the parentheses. You may still see these used in code (or by programmers) that have been converted from C.

```
double d { (double)x / y }; // convert x to a double so we get floating point division
std::cout << d << '\n'; // prints 2.5
```

In the above program, we use a C-style cast to tell the compiler to convert x to a double. Because the left operand of operator/ now evaluates to a floating-point value, the right operand will be converted to a floating-point value as well, and the division will be done using floating point division instead of integer division!

C++ will also let you use a C-style cast with a more function-call like syntax:

```
1 double d { double(x) / y }; // convert x to a double so we get floating point division
```

This performs identically to the prior example, but has the benefit of parenthesizing the value being converted (making it easier to tell what is being converted).

Although a C-style cast appears to be a single cast, it can actually perform a variety of different conversions depending on context. This can include a static cast, a const cast or a reinterpret cast (the latter two of which we mentioned above you should avoid). As a result, C-style casts are at risk for being inadvertently misused and not producing the expected behavior, something which is easily avoidable by using the C++ casts instead.

Also, because C-style casts are just a type name, parenthesis, and variable or value, they are both difficult to identify (making your code harder to read) and even more difficult to search for.

Best practice: Avoid using C-style casts.

13.6.3 *static_cast*

C++ introduces a casting operator called **static_cast**, which can be used to convert a value of one type to a value of another type.

You've previously seen `static_cast` used to convert a `char` into an `int` so that `std::cout` prints it as an integer instead of a `char`:

```
char c { 'a' };  
std::cout << c << ' ' << static_cast<int>(c) << '\n'; // prints a 97
```

The `static_cast` operator takes an expression as input, and returns the evaluated value converted to the type specified inside the angled brackets. **static_cast is best used to convert one fundamental type into another.**

The main advantage of `static_cast` is that it provides **compile-time type checking**, making it harder to make an inadvertent error.

`static_cast` is also (intentionally) less powerful than C-style casts, so you can't inadvertently remove `const` or do other things you may not have intended to do.

```
const int x{ 5 };  
int& ref{ static_cast<int&>(x) }; // invalid: will produce compilation error
```

Best practice: Favor `static_cast` when you need to convert a value from one type to another type.

13.6.4 Using `static_cast` to make narrowing conversions explicit

Compilers will often issue warnings when a potentially unsafe (narrowing) implicit type conversion is performed. For example, consider the following program:

```
int i { 48 };  
char ch = i; // implicit narrowing conversion
```

Casting an `int` (2 or 4 bytes) to a `char` (1 byte) is potentially unsafe (as the compiler can't tell whether the integer value will overflow the range of the `char` or not), and so the compiler will typically print a warning. If we used list initialization, the compiler would yield an error.

To get around this, **we can use a static cast to explicitly convert our integer to a char:**

```
1 int i { 48 };  
2  
3 // explicit conversion from int to char, so that a char is assigned to variable ch  
4 char ch { static_cast<char>(i) };
```

When we do this, we're explicitly telling the compiler that this conversion is intended, and we accept responsibility for the consequences (e.g. overflowing the range of a `char` if that happens). Since the output of this `static_cast` is of type `char`, the initialization of variable `ch` doesn't generate any type mismatches, and hence no warnings or errors.

13.7 Typedefs and type aliases

In C++, **using** is a keyword that creates an alias for an existing data type. To create such a type alias, we use the `using` keyword, followed by a name for the type alias, followed by an equals sign and an existing data type. For example:

```
1 using Distance = double; // define Distance as an alias for type double
```

Once defined, a type alias can be used anywhere a type is needed. For example, we can create a variable with the type alias name as the type:

```
1 Distance milesToDestination{ 3.4 }; // defines a variable of type double
```

When the compiler encounters a type alias name, it will substitute in the aliased type.

For advanced readers: Type aliases can also be templated.

13.7.1 Naming type aliases

Historically, there hasn't been a lot of consistency in how type aliases have been named. There are three common naming conventions (and you will run across all of them):

- Type aliases that end in a “_t” suffix (the “_t” is short for “type”). This convention is often used by the **standard library** for **globally scoped type names** (like `size_t` and `nullptr_t`).

This **convention was inherited from C**, and used to be the most popular when defining your own type aliases (and sometimes other types), but has **fallen out of favor in modern C++**. Note that POSIX reserves the “_t” suffix for globally scoped type names, so using this convention may cause type naming conflicts on POSIX systems.

- Type aliases that end in a “_type” suffix. This convention is **used by some standard library types** (like `std::string`) to name **nested type aliases** (e.g. `std::string::size_type`).

But many such nested type aliases do not use a suffix at all (e.g. `std::string::iterator`), so this usage is **inconsistent at best**.

- Type aliases that use no suffix.

In modern C++, the convention is to name type aliases (or any other type) that you define yourself **starting with a capital letter**, and using no suffix. The capital letter **helps differentiate the names of types from the names of variables and functions** (which start with a lower-case letter), and **prevents naming collisions** between them.

When using this naming convention, it is **common to see this usage**:

```
void printDistance(Distance distance); // Distance is some defined type
```

In this case, `Distance` is the type, and `distance` is the parameter name. **C++ is case-sensitive, so this is fine.**

Best practice: Name your type aliases starting with a capital letter and do not use a suffix (unless you have a specific reason to do otherwise).

13.7.2 Type aliases are not distinct types

An alias **does not actually define a new, distinct type** (one that is considered separate from other types) -- **it just introduces a new identifier for an existing type**. A type alias is completely interchangeable with the aliased type.

This allows us to do things that are syntactically valid but semantically meaningless. For example:

```
using Miles = long; // define Miles as an alias for type long
using Speed = long; // define Speed as an alias for type long

Miles distance { 5 }; // distance is actually just a long
Speed mhz { 3200 }; // mhz is actually just a long

// The following is syntactically valid (but semantically meaningless)
distance = mhz;
```

Although conceptually we intend Miles and Speed to have distinct meanings, both are just aliases for type long. This effectively means Miles, Speed, and long can all be used interchangeably. And indeed, when we assign a value of type Speed to a variable of type Miles, the compiler only sees that we're assigning a value of type long to a variable of type long, and it will not complain.

Because the compiler does not prevent these kinds of semantic errors for type aliases, we say that **aliases are not type safe**. In spite of that, they are still useful.

Warning: Care must be taken not to mix values of aliases that are intended to be semantically distinct.

As an aside...

Some languages support the concept of a **strong typedef** (or strong type alias). A strong typedef actually creates a new type that has all the original properties of the original type, but the compiler will throw an error if you try to mix values of the aliased type and the strong typedef. As of C++20, C++ does not directly support strong typedefs, but there are quite a few 3rd party C++ libraries that implement strong typedef-like behavior.

13.7.3 The scope of a type alias

Because scope is a property of an identifier, type alias identifiers follow the same scoping rules as variable identifiers: a type alias defined inside a block has block scope and is usable only within that block, whereas a type alias defined in the global namespace has global scope and is usable to the end of the file. In the above example, `Miles` and `Speed` are only usable in the `main()` function.

If you need to use one or more type aliases across multiple files, they can be defined in a header file and #included into any code files that needs to use the definition:

```
mytypes.h:
1  #ifndef MYTYPES_H
2  #define MYTYPES_H
3
4      using Miles = long;
5      using Speed = long;
6
7  #endif
```

Type aliases #included this way will be imported into the global namespace and thus have global scope.

13.7.4 Typedefs

A **typedef** (which is short for “type definition”) is an older way of creating an alias for a type. To create a typedef alias, we use the typedef keyword:

```
1  // The following aliases are identical
2  typedef long Miles;
3  using Miles = long;
```

Typedefs are still in C++ for backwards compatibility reasons, but they have been **largely replaced by type aliases in modern C++**.

Typedefs have a few syntactical issues. First, it's **easy to forget whether the name of the typedef or the name of the type to alias comes first**. Which is correct?

```
1  typedef Distance double; // incorrect (typedef name first)
2  typedef double Distance; // correct (aliased type name first)
```


It's easy to get backwards. Fortunately, in such cases, the compiler will complain.

Second, the syntax for typedefs can get ugly with more complex types. For example, here is a hard-to-read typedef, along with an equivalent (and slightly easier to read) type alias:

```
1 typedef int (*FcnType)(double, char); // FcnType hard to find
2 using FcnType = int(*)(double, char); // FcnType easier to find
```

In the above typedef definition, the **name of the new type (FcnType) is buried in the middle of the definition**, whereas in the type alias, the name of the new type and the rest of the definition are separated by an equals sign.

Third, the name “typedef” suggests that a new type is being defined, but that’s not true. A typedef is just an alias.

Best practice: Prefer type aliases over typedefs.

Nomenclature: The C++ standard uses the term “typedef names” for the names of both typedefs and type aliases. In conventional language, the term “typedef” is often used to mean “either a typedef or a type alias” since they effectively do the same thing.

13.7.5 Using type aliases for platform independent coding

Now that we’ve covered what type aliases are, let’s talk about what they are useful for.

One of the primary uses for type aliases is to **hide platform specific details**. On some platforms, an int is 2 bytes, and on others, it is 4 bytes. Thus, **using int to store more than 2 bytes of information can be potentially dangerous when writing platform independent code.**

Because **char, short, int, and long give no indication of their size**, it is fairly **common for cross-platform programs to use type aliases to define aliases that include the type’s size in bits**. For example, int8_t would be an 8-bit signed integer, int16_t a 16-bit signed integer, and int32_t a 32-bit signed integer. Using type aliases in this manner helps prevent mistakes and makes it more clear about what kind of assumptions have been made about the size of the variable.

In order to make sure each aliased type resolves to a type of the right size, **type aliases of this kind are typically used in conjunction with preprocessor directives:**

```
1  #ifdef INT_2_BYTES
2  using int8_t = char;
3  using int16_t = int;
4  using int32_t = long;
5  #else
6  using int8_t = char;
7  using int16_t = short;
8  using int32_t = int;
9  #endif
```

On machines where integers are only 2 bytes, `INT_2_BYTES` can be `#defined` (as a compiler/preprocessor setting), and the program will be compiled with the top set of type aliases. On machines where integers are 4 bytes, leaving `INT_2_BYTES` undefined will cause the bottom set of type aliases to be used. In this way, as long as `INT_2_BYTES` is `#defined` correctly, `int8_t` will resolve to a 1-byte integer, `int16_t` will resolve to a 2 bytes integer, and `int32_t` will resolve to a 4-byte integer (using the combination of `char`, `short`, `int`, and `long` that is appropriate for the machine the program is being compiled on).

The fixed-width integer types (such as `std::int16_t` and `std::uint32_t`) and the `size_t` type are actually just type aliases to various fundamental types.

This is also why when you print an 8-bit fixed-width integer using `std::cout`, you're likely to get a character value.

```
std::int8_t x{ 97 }; // int8_t is usually a typedef for signed char
std::cout << x << '\n';
```

This program prints: **a**

Because `std::int8_t` is typically a typedef for signed char, variable `x` will likely be defined as a signed char. And char types print their values as ASCII characters rather than as integer values.

13.7.6 Using type aliases to make complex types easier to read

Although we have only dealt with simple data types so far, in advanced C++, types can be complicated and lengthy to manually enter on your keyboard. For example, you might see a function and variable defined like this:

```
std::vector<std::pair<std::string, int>> pairlist;
```

Typing `std::vector<std::pair<std::string, int>>` everywhere you need to use that type is cumbersome, and it is easy to make a typing mistake. It's much easier to use a type alias:

```
using VectPairSI = std::vector<std::pair<std::string, int>>; // make VectPairSI an alias for this crazy type
```

Much better! Now we only have to type `VectPairSI` instead of `std::vector<std::pair<std::string, int>>`.

Don't worry if you don't know what `std::vector`, `std::pair`, or all these crazy angle brackets are yet. The only thing you really need to understand here is that type aliases allow you to take complex types and give them a simpler name, which makes your code easier to read and saves typing.

This is probably the best use for type aliases.

13.7.7 Using type aliases to document the meaning of a value

Type aliases can also help with code documentation and comprehension.

With variables, we have the variable's identifier to help document the purpose of the variable. But consider the case of a function's return value. Data types such as `char`, `int`, `long`, `double`, and `bool` describe what type of value a function returns, but more often we want to know what the meaning of a return value is.

In our experience, **creating a type alias just to document the return type of a single function isn't worth it (use a comment instead)**. But if you have multiple functions passing or returning such a type, creating a type alias might be worthwhile.

13.7.8 Using type aliases for easier code maintenance

Type aliases also allow you to change the underlying type of an object without having to update lots of hardcoded types. For example, if you were using a short to hold a student's ID number, but then later decided you needed a long instead, you'd have to comb through lots of code and replace short with long. It would probably be difficult to figure out which objects of type short were being used to hold ID numbers and which were being used for other purposes.

However, if you use type aliases, then changing types becomes as simple as updating the type alias (e.g. from using StudentId = short; to using StudentId = long;).

While this seems like a nice benefit, **caution is necessary whenever a type is changed, as the behavior of the program may also change.** This is especially true when changing the type of a type alias to a type in a different type family (e.g. an integer to a floating-point value, or a signed to unsigned value)! The new type may have comparison or integer/floating point division issues, or other issues that the old type did not. If you change an existing type to some other type, your code should be thoroughly retested.

13.7.9 Downsides and conclusion

While type aliases offer some benefits, they also introduce yet another identifier into your code that needs to be understood. If this isn't offset by some benefit to readability or comprehension, then the type alias is doing more harm than good.

A poorly utilized type alias can take a familiar type (such as std::string) and hide it behind a custom name that needs to be looked up. In some cases (such as with smart pointers, which we'll cover in a future chapter), obscuring the type information can also be harmful to understanding how the type should be expected to work.

For this reason, type aliases should be used primarily in cases where there is a clear benefit to code readability or code maintenance. This is as much of an art as a science. Type aliases are most useful when they can be used in many places throughout your code, rather than in fewer places.

Best practice: Use type aliases judiciously, when they provide a clear benefit to code readability or code maintenance.

13.8 Type deduction for objects using the auto keyword

There's a subtle redundancy lurking in this simple variable definition:

```
1 | double d{ 5.0 };
```

In C++, we are required to provide an explicit type for all objects. Thus, we've specified that variable `d` is of type `double`.

However, the literal value `5.0` used to initialize `d` also has type `double` (**implicitly determined via the format of the literal**).

In cases where we want a variable and its initializer to have the same type, **we're effectively providing the same type information twice**.

13.8.1 Type deduction for initialized variables

Type deduction (also sometimes called **type inference**) is a feature that **allows the compiler to deduce the type of an object from the object's initializer**. To use type deduction with variables, the `auto` keyword is used in place of the variable's type:

```
auto d{ 5.0 }; // 5.0 is a double literal, so d will be type double
auto i{ 1 + 2 }; // 1 + 2 evaluates to an int, so i will be type int
auto x { i }; // i is an int, so x will be type int too
```

In the first case, because `5.0` is a double literal, the compiler will deduce that variable `d` should be of type `double`. In the second case, the expression `1 + 2` yields an `int` result, so variable `i` will be of type `int`. In the third case, `i` was previously deduced to be of type `int`, so `x` will also be deduced to be of type `int`.

Warning:

Prior to C++17, `auto d{ 5.0 };` would deduce `d` to be of type `std::initializer_list<double>` rather than `double`. This was fixed in C++17, and many compilers (such as `gcc` and `Clang`) have backported this change to previous language standards.

If you are using C++14 or older, and the above example doesn't compile on your compiler, use copy initialization with `auto` instead (`auto d = 5.0`).

Because function calls are valid expressions, we can even use type deduction when our initializer is a non-void function call:

```
1 int add(int x, int y)
2 {
3     return x + y;
4 }
5
6 int main()
7 {
8     auto sum { add(5, 6) }; // add() returns an int, so sum's type will be deduced to int
9
10    return 0;
11 }
```

The add() function returns an int value, so the compiler will deduce that variable sum should have type int.

Literal suffixes can be used in combination with type deduction to specify a particular type:

```
auto a { 1.23f }; // f suffix causes a to be deduced to float
auto b { 5u };    // u suffix causes b to be deduced to unsigned int
```

Type deduction will not work for objects that do not have initializers or have empty initializers. It also will not work when the initializer has type void (or any other incomplete type). Thus, the following is not valid:

```
1 #include <iostream>
2
3 void foo()
4 {
5 }
6
7 int main()
8 {
9     auto x;           // The compiler is unable to deduce the type of x
10    auto y{ };         // The compiler is unable to deduce the type of y
11    auto z{ foo() };   // z can't have type void, so this is invalid
12
13    return 0;
14 }
```

Although using type deduction for fundamental data types only saves a few (if any) keystrokes, in future lessons we will see examples where the types get complex and lengthy (and in some cases, can be hard to figure out). In those cases, using auto can save a lot of typing (and typos).

Related content: The type deduction rules for pointers and references are a bit more complex.

13.8.2 Type deduction drops const / constexpr qualifiers

In most cases, type deduction will drop the const or constexpr qualifier from deduced types. For example:

```
const int x { 5 }; // x has type const int
auto y { x };      // y will be type int (const is dropped)
```

In the above example, x has type const int, but when deducing a type for variable y using x as the initializer, type deduction deduces the type as int, not const int.

If you want a deduced type to be const or constexpr, you must supply the const or constexpr yourself. To do so, simply use the const or constexpr keyword in conjunction with the auto keyword:

```
const int x { 5 }; // x has type const int (compile-time const)
auto y { x };      // y will be type int (const is dropped)

constexpr auto z { x }; // z will be type constexpr int (constexpr is reapplied)
```

13.8.3 Type deduction for string literals

For historical reasons, string literals in C++ have a strange type. Therefore, the following probably won't work as expected:

```
1 | auto s { "Hello, world" }; // s will be type const char*, not std::string
```

If you want the type deduced from a string literal to be std::string or std::string_view, you'll need to use the s or sv literal suffixes.

13.8.4 Type deduction benefits and downsides

Type deduction is not only convenient, but also has a number of other **benefits**:

- First, if two or more variables are defined on sequential lines, the names of the variables will be lined up, helping to **increase readability**:

```
1 // harder to read
2 int a { 5 };
3 double b { 6.7 };
4
5 // easier to read
6 auto c { 5 };
7 auto d { 6.7 };
```

- Second, type deduction only works on variables that have initializers, so if you are in the habit of using type deduction, it can **help avoid unintentionally uninitialized variables**:

```
1 int x; // oops, we forgot to initialize x, but the compiler may not complain
2 auto y; // the compiler will error out because it can't deduce a type for y
```

- Third, you are guaranteed that there will be no unintended performance-impacting conversions:

```
1 std::string_view getString(); // some function that returns a std::string_view
2
3 std::string s1 { getString() }; // bad: expensive conversion from std::string_view to std::string (assuming you didn't want this)
4 auto s2 { getString() }; // good: no conversion required
```

Type deduction also has a few **downsides**:

- First, type deduction **obscures an object's type information in the code**. Although a good IDE should be able to show you the deduced type (e.g. when hovering a variable), it's still a bit easier to make type-based mistakes when using type deduction.

```
auto y { 5 }; // oops, we wanted a double here but we accidentally provided an int literal
```

- Similar cases occur when a variable is unsigned. Since **we don't want to mix signed and unsigned values**, explicitly knowing that a variable has an unsigned type is generally something that shouldn't be obscured.
- Second, **if the type of an initializer changes, the type of a variable using type deduction will also change, perhaps unexpectedly**. Consider:

```
1 auto sum { add(5, 6) + gravity };
```


If the return type of `add` changes from `int` to `double`, or `gravity` changes from `int` to `double`, `sum` will also change type from `int` to `double`.

Overall, the **modern consensus is that type deduction is generally safe to use for objects**, and that doing so **can help make your code more readable** by de-emphasizing type information so the **logic of your code stands out better**.

Best practice:

- Use type deduction for your variables when the type of the object doesn't matter.
- **Favor an explicit type when you require a specific type that differs from the type of the initializer**, or when your object is used in a context where making the type obvious is useful.

13.9 Type deduction for functions

Consider the following program:

```
1 int add(int x, int y)
2 {
3     return x + y;
4 }
```

When this function is compiled, the compiler will determine that `x + y` evaluates to an `int`, then ensure that type of the return value matches the declared return type of the function (or that the return value type can be converted to the declared return type).

Since the compiler already has to deduce the return type from the return statement, **in C++14, the `auto` keyword was extended to do function return type deduction**. This works by **using the `auto` keyword in place of the function's return type**.

```
1 auto add(int x, int y)
2 {
3     return x + y;
4 }
```

Because the return statement is returning an `int` value, the compiler will deduce that the return type of this function is `int`.

When using an auto return type, **all return statements within the function must return values of the same type**, otherwise an error will result. For example:

```
auto someFcn(bool b)
{
    if (b)
        return 5; // return type int
    else
        return 6.7; // return type double
}
```

In the above function, the two return statements return values of different types, so the compiler will give an error.

If such a case is desired for some reason, you can either explicitly specify a return type for your function (in which case the compiler will try to implicitly convert any non-matching return expressions to the explicit return type), or you can explicitly convert all of your return statements to the same type. In the example above, the latter could be done by changing 5 to 5.0, but `static_cast` can also be used for non-literal types.

A major downside of functions that use an auto return type is that such functions must be fully defined before they can be used (a forward declaration is not sufficient). For example:

```
1  #include <iostream>
2
3  auto foo();
4
5  int main()
6  {
7      std::cout << foo() << '\n'; // the compiler has only seen a forward declaration at this point
8
9      return 0;
10 }
11
12 auto foo()
13 {
14     return 5;
15 }
```

This makes sense: a forward declaration does not have enough information for the compiler to deduce the function's return type. This means normal functions that return auto are typically only callable from within the file in which they are defined.

Unlike type deduction for objects, there isn't as much consensus on best practices for function return type deduction. When using type deduction with objects, the initializer is always present as part of the same statement, so it's usually not overly burdensome to determine what type will be deduced. With functions, that is not the case -- when looking at a function's prototype, there is no context to help indicate what type the function returns.

A good programming IDE should make clear what the deduced type of the function is, but in absence of having that available, a user would actually have to dig into the function body itself to determine what type the function returned. The odds of mistakes being made are higher. And the inability for such functions to be forward declared limits their usefulness in multi-file programs.

Best practice: Favor explicit return types over function return type deduction for normal functions.

13.9.1.1 Trailing return type syntax

The `auto` keyword can also be used to declare functions using a **trailing return syntax**, where the return type is specified after the rest of the function prototype.

Consider the following function:

```
int add(int x, int y)
{
    return (x + y);
}
```

Using the trailing return syntax, this could be equivalently written as:

```
auto add(int x, int y) -> int
{
    return (x + y);
}
```

In this case, `auto` does not perform type deduction -- it is just part of the syntax to use a trailing return type.

Why would you want to use this?

One nice thing is that it **makes all of your function names line up**:

```
auto add(int x, int y) -> int;
auto divide(double x, double y) -> double;
auto printSomething() -> void;
auto generateSubstring(const std::string &s, int start, int len) -> std::string;
```

The trailing return syntax is also required for some advanced features of C++, such as lambdas.

For now, **we recommend the continued use of the traditional function return syntax** except in situations that require the trailing return syntax.

13.9.2 *Type deduction can't be used for function parameter types*

Many new programmers who learn about type deduction try something like this:

```
void addAndPrint(auto x, auto y)
{
    std::cout << x + y << '\n';
}
```

Unfortunately, **type deduction doesn't work for function parameters**, and prior to C++20, the above program won't compile (you'll get an error about function parameters not being able to have an auto type).

In C++20, the auto keyword was extended so that the above program will compile and function correctly -- however, auto is not invoking type deduction in this case. Rather, it is triggering a different feature called function templates that was designed to actually handle such cases.

13.10 Summary

The process of converting a value from one data type to another data type is called a **type conversion**.

Implicit type conversion (also called **automatic type conversion** or **coercion**) is performed whenever one data type is expected, but a different data type is supplied. If the compiler can figure out how to do the conversion between the two types, it will. If it doesn't know how, then it will fail with a compile error.

The C++ language defines a number of built-in conversions between its fundamental types (as well as a few conversions for more advanced types) called **standard conversions**. These include numeric promotions, numeric conversions, and arithmetic conversions.

A **numeric promotion** is the conversion of certain smaller numeric types to certain larger numeric types (typically int or double), so that the CPU can operate on data that matches the natural data size for the processor. Numeric promotions include both integral promotions and floating-point promotions. Numeric promotions are **value-preserving**, meaning there is no loss of value or precision. Not all widening conversions are promotions.

A **numeric conversion** is a type conversion between fundamental types that isn't a numeric promotion. A **narrowing conversion** is a numeric conversion that **may result in the loss of value or precision**.

In C++, certain binary operators require that their operands be of the same type. If operands of different types are provided, one or both of the operands will be implicitly converted to matching types using a set of rules called the **usual arithmetic conversions**.

Explicit type conversion is performed when the programmer explicitly requests conversion via a cast. A **cast** represents a request by the programmer to do an explicit type conversion. C++ supports 5 types of casts: C-style casts, static casts, const casts, dynamic casts, and reinterpret casts. Generally, you should avoid C-style casts, const casts, and reinterpret casts. `static_cast` is used to convert a value from one type to a value of another type, and is by far the most used cast in C++.

Typedefs and **type aliases** allow the programmer to create an alias for a data type. These aliases are not new types, and act identically to the aliased type. Typedefs and type aliases do not provide any kind of type safety, and care needs to be taken to not assume the alias is different than the type it is aliasing.

The **auto** keyword has a number of uses. First, auto can be used to do **type deduction** (also called **type inference**), which will deduce a variable's type from its initializer. Type deduction drops const and references, so be sure to add those back if you want them.

Auto can also be used as a function return type to have the compiler infer the function's return type from the function's return statements, though this should be avoided for normal functions. Auto is used as part of the **trailing return syntax**.

14 Function Overloading and Function Templates

14.1 Introduction to function overloading

Consider the following function:

```
1 int add(int x, int y)
2 {
3     return x + y;
4 }
```

This trivial function adds two integers and returns an integer result. However, what if we also want a function that can add two floating point numbers? **This add() function is not suitable, as any floating-point parameters would be converted to integers, causing the floating-point arguments to lose their fractional values.**

One way to work around this issue is to define multiple functions with slightly different names. However, for best effect, this requires that you define a consistent function naming standard for similar functions that have parameters of different types, remember the names of these functions, and actually call the correct one. And then what happens when we want to have a similar function that adds 3 integers instead of 2? **Managing unique names for each function quickly becomes burdensome.**

14.1.1 Introduction to function overloading

Fortunately, C++ has an elegant solution to handle such cases. **Function overloading allows us to create multiple functions with the same name, so long as each identically named function has different parameter types (or the functions can be otherwise differentiated).** Each function sharing a name (in the same scope) is called an overloaded function (sometimes called an overload for short).

To overload our `add()` function, we can simply declare another `add()` function that takes double parameters:

```
int add(int x, int y) // integer version
{
    return x + y;
}

double add(double x, double y) // floating point version
{
    return x + y;
}
```

The above program will compile. Although you might expect these functions to result in a naming conflict, that is not the case here. **Because the parameter types of these functions differ, the compiler is able to differentiate these functions, and will treat them as separate functions that just happen to share a name.**

Key insight: Functions can be overloaded so long as each overloaded function can be differentiated by the compiler. If an overloaded function cannot be differentiated, a compile error will result.

14.1.2 Introduction to overload resolution

Additionally, when a function call is made to a function that has been overloaded, the **compiler will try to match the function call to the appropriate overload based on the arguments used in the function call.** This is called overload resolution.

```
3  int add(int x, int y)
4  {
5      return x + y;
6  }
7
8  double add(double x, double y)
9  {
10     return x + y;
11 }
12
13 int main()
14 {
15     std::cout << add(1, 2); // calls add(int, int)
16     std::cout << '\n';
17     std::cout << add(1.2, 3.4); // calls add(double, double)
18
19     return 0;
20 }
```

When we provide integer arguments in the call to `add(1, 2)`, the compiler will determine that we're trying to call `add(int, int)`. And when we provide floating point arguments in the call to `add(1.2, 3.4)`, the compiler will determine that we're trying to call `add(double, double)`.

14.1.3 Making it compile

In order for a program using overloaded functions to compile, **two things have to be true**:

1. Each overloaded function **has to be differentiated** from the others.
2. Each **call to an overloaded function has to resolve to an overloaded function**.

If an overloaded function is not differentiated, or if a function call to an overloaded function cannot be resolved to an overloaded function, then a compile error will result.

In the next lesson, we'll explore how overloaded functions can be differentiated from each other. Then, in the following lesson, we'll explore how the compiler resolves function calls to overloaded functions.

14.1.4 Conclusion

Function overloading provides a **great way to reduce the complexity of your program** by reducing the number of function names you need to remember. It can and should be used liberally.

Best practice: Use function overloading to make your program simpler.

14.2 Function overload differentiation

In this lesson, we'll take a closer look at how overloaded functions are differentiated. **Overloaded functions that are not properly differentiated will cause the compiler to issue a compile error.**

14.2.1 How overloaded functions are differentiated

Function property	Used for differentiation	Notes
Number of parameters	Yes	
Type of parameters	Yes	Excludes typedefs, type aliases, and const qualifier on value parameters. Includes ellipses.
Return type	No	

Note that a function's return type is not used to differentiate overloaded functions. We'll discuss this more in a bit.

14.2.2 Overloading based on number of parameters

An overloaded function is differentiated so long as each overloaded function has a different number of parameters.

```
int add(int x, int y)
{
    return x + y;
}

int add(int x, int y, int z)
{
    return x + y + z;
}
```

The compiler can easily tell that a function call with two integer parameters should go to `add(int, int)` and a function call with three integer parameters should go to `add(int, int, int)`.

14.2.3 Overloading based on type of parameters

A function can also be differentiated so long as each overloaded function's list of parameter types is distinct. For example, all of the following overloads are differentiated:

```
1 int add(int x, int y); // integer version
2 double add(double x, double y); // floating point version
3 double add(int x, double y); // mixed version
4 double add(double x, int y); // mixed version
```

Because type aliases (or typedefs) are not distinct types, overloaded functions using type aliases are not distinct from overloads using the aliased type.

For parameters passed by value, the `const` qualifier is also not considered. Therefore, the following functions are not considered to be differentiated:

```
1 void print(int);
2 void print(const int); // not differentiated from print(int)
```

14.2.4 The return type of a function is not considered for differentiation

A function's return type is not considered when differentiating overloaded functions.

Consider the case where you want to write a function that returns a random number, but you need a version that will return an int, and another version that will return a double. You might be tempted to do this:

```
1 int getRandomValue();  
2 double getRandomValue();
```

Which of the two overloaded functions would you call? It's not clear. The best way to address this is to give the functions different names.

14.2.5 Type signature

A function's **type signature** (generally called a **signature**) is defined as the parts of the function header that are used for differentiation of the function. In C++, this includes the **function name**, **number of parameters**, **parameter type**, and **function-level qualifiers**. It notably does *not* include the return type.

14.2.6 Name mangling

As an aside...

When the compiler compiles a function, it performs **name mangling**, which means the compiled name of the function is altered ("mangled") **based on various criteria**, such as the number and type of parameters, so that the linker has unique names to work with.

For example, some function with prototype `int fcn()` might compile to name `__fcn_v`, whereas `int fcn(int)` might compile to name `__fcn_i`. So, while in the source code, two overloaded functions share a name, in compiled code, the names are actually unique.

There is no standardization on how names should be mangled, so different compilers will produce different mangled names.

14.3 Function overload resolution and ambiguous matches

In the previous lesson (Function overload differentiation), we discussed which attributes of a function are used to differentiate overloaded functions from each other. If an overloaded function is not properly differentiated from the other overloads of the same name, then the compiler will issue a compile error.

However, having a set of differentiated overloaded functions is only half of the picture. When any function call is made, the compiler must also ensure that a matching function declaration can be found.

With non-overloaded functions (functions with unique names), there is only one function that can potentially match a function call. That function either matches (or can be made to match after type conversions are applied), or it doesn't (and a compile error results). With overloaded functions, there can be many functions that can potentially match a function call. **Since a function call can only resolve to one of them, the compiler has to determine which overloaded function is the best match.** The process of matching function calls to a specific overloaded function is called **overload resolution**.

In simple cases where the type of the function arguments and type of the function parameters match exactly, this is (usually) straightforward. But what happens in cases where the argument types in the function call don't exactly match the parameter types in any of the overloaded functions?

```
3 void print(int x)
4 {
5     std::cout << x << '\n';
6 }
7
8 void print(double d)
9 {
10    std::cout << d << '\n';
11 }
12
13 int main()
14 {
15     print('a'); // char does not match int or double, so what happens?
16     print(5L); // long does not match int or double, so what happens?
17
18     return 0;
19 }
```

Just because there is no exact match here doesn't mean a match can't be found -- after all, a **char** or **long** can be **implicitly type converted** to an **int** or a **double**. But which is the best conversion to make in each case?

In this lesson, we'll explore how the compiler matches a given function call to a specific overloaded function.

14.3.1 Resolving overloaded function calls

When a function call is made to an overloaded function, the **compiler steps through a sequence of rules to determine which (if any) of the overloaded functions is the best match** (we'll cover these steps in the next section below).

At each step, the compiler applies a bunch of different type conversions to the argument(s) in the function call. For each conversion applied, the compiler checks if any of the overloaded functions are now a match. After all the different type conversions have been applied and checked for matches, the step is done. The result will be one of **three possible outcomes**:

- **No matching functions** were found. The **compiler moves to the next step in the sequence.**
- **A single matching function** was found. This **function is considered to be the best match.** The matching **process is now complete**, and subsequent steps are not executed.
- **More than one matching function** was found. The **compiler will issue an ambiguous match compile error.** We'll discuss this case further in a bit.

14.3.2 The argument matching sequence

14.3.2.1 Step 1: find an exact match in two phases

1. First, the compiler will see if there is an overloaded function where the **type of the arguments in the function call exactly matches the type of the parameters** in the overloaded functions.
2. Second, the compiler will apply a number of **trivial conversions** to the arguments in the function call. The **trivial conversions** are a set of specific conversion rules that will **modify types (without modifying the value)** for purposes of finding a match. Matches made via the trivial conversions are considered exact matches.
 - a. lvalue to rvalue conversions
 - b. qualification conversions (e.g. non-const to const)
 - c. non-reference to reference conversions

14.3.2.2 Step 2: numeric promotion

If no exact match is found, the **compiler tries to find a match by applying numeric promotion to the argument(s)**. In lesson (Implicit type conversion), we covered how certain narrow integral and floating-point types can be automatically promoted to wider types, such as int or double. **If, after numeric promotion, a match is found, the function call is resolved.**

14.3.2.3 Step 3: numeric conversion

If no match is found via numeric promotion, the **compiler tries to find a match by applying numeric conversions** to the arguments.

Key insight: Matches made by applying **numeric promotions** take precedence over any matches made by applying **numeric conversions**.

14.3.2.4 Step 4: user-defined conversions

If no match is found via numeric conversion, the **compiler tries to find a match through any user-defined conversions**. Although we haven't covered user-defined conversions yet, **certain types (e.g. classes) can define conversions to other types that can be implicitly invoked.**

14.3.2.5 Step 5: ellipsis

If no match is found via user-defined conversion, the **compiler will look for a matching function that uses ellipsis.**

14.3.2.6 Step 6: compile error

If no matches have been found by this point, the compiler gives up and will issue a compile error about not being able to find a matching function.

14.3.3 Ambiguous matches

With non-overloaded functions, each function call will either resolve to a function, or no match will be found and the compiler will issue a compile error. With overloaded functions, there is a **third possible outcome**: an ambiguous match may be found. An **ambiguous match** occurs when **the compiler finds two or more functions that can be made to match in the same step**. When this occurs, the **compiler will stop matching and issue a compile error stating that it has found an ambiguous function call**.

Since every overloaded function must be differentiated in order to compile, you might be wondering how it is possible that a function call could result in more than one match. Let's take a look at an example that illustrates this:

```
1 void foo(int)
2 {
3 }
4
5 void foo(double)
6 {
7 }
8
9 int main()
10 {
11     foo(5L); // 5L is type long
12
13     return 0;
14 }
```

Since literal 5L is of type long, the compiler will first look to see if it can find an exact match for foo(long), but it will not find one. Next, the compiler will try numeric promotion, but values of type long can't be promoted, so there is no match here either.

Following that, the compiler will try to find a match by applying numeric conversions to the long argument. In the process of checking all the numeric conversion rules, the compiler will find two potential matches. If the long argument is numerically converted into an int, then the function call will match foo(int). If the long argument is instead converted into a double, then it will match foo(double) instead. Since two possible matches via numeric conversion have been found, the function call is considered ambiguous.

Key insight: If the compiler finds multiple matches in a given step, an ambiguous function call will result. **This means no match from a given step is considered to be better than any other match from the same step.**

14.3.4 *Resolving ambiguous matches*

Because ambiguous matches are a compile-time error, an **ambiguous match needs to be disambiguated before your program will compile**. There are a **few ways to resolve** ambiguous matches:

1. Often, the best way is simply to **define a new overloaded function** that takes parameters of exactly the type you are trying to call the function with. Then C++ will be able to find an exact match for the function call.
2. Alternatively, **explicitly cast the ambiguous argument(s)** to match the type of the function you want to call.
3. If your argument is a literal, you can **use the literal suffix to ensure your literal is interpreted as the correct type**.

14.3.5 *Matching for functions with multiple arguments*

If there are multiple arguments, the **compiler applies the matching rules to each argument in turn**. The function chosen is the one for which each argument matches at least as well as all the other functions, with at least one argument matching better than all the other functions. In other words, **the function chosen must provide a better match than all the other candidate functions for at least one parameter, and no worse for all of the other parameters**.

In the case that such a function is found, it is **clearly and unambiguously the best choice**. If no such function can be found, the call will be considered ambiguous (or a non-match).

14.4 Deleting functions

In some cases, it is possible to write functions that don't behave as desired when called with values of certain types.

```
void printInt(int x)
{
    std::cout << x << '\n';
}

int main()
{
    printInt(5);    // okay: prints 5
    printInt('a');  // prints 97 -- does this make sense?
    printInt(true); // print 1 -- does this make sense?
```

While `printInt(5)` is clearly okay, the other two calls to `printInt()` are more questionable. With `printInt('a')`, the compiler will determine that it can promote 'a' to int value 97 in order to match the function call with the function definition. And it will promote `true` to int value 1. And it will do so without complaint.

Let's assume we don't think it makes sense to call `printInt()` with a value of type `char` or `bool`. What can we do?

14.4.1 Deleting a function using the `= delete` specifier

In cases where we have a function that we explicitly do not want to be callable, we can define that function as deleted by using the `= delete` specifier. If the compiler matches a function call to a deleted function, compilation will be halted with a compile error.

```
3 void printInt(int x)
4 {
5     std::cout << x << '\n';
6 }
7
8 void printInt(char) = delete; // calls to this function will halt compilation
9 void printInt(bool) = delete; // calls to this function will halt compilation
10
11 int main()
12 {
13     printInt(97);    // okay
14
15     printInt('a');  // compile error: function deleted
16     printInt(true); // compile error: function deleted
17
18     printInt(5.0);  // compile error: ambiguous match
19
20     return 0;
21 }
```


Let's take a quick look at some of these. First, `printInt('a')` is a direct match for `printInt(char)`, which is deleted. The compiler thus produces a compilation error. `printInt(true)` is a direct match for `printInt(bool)`, which is deleted, and thus also produces a compilation error.

`printInt(5.0)` is an interesting case, with perhaps unexpected results. First, the compiler checks to see if exact match `printInt(double)` exists. It does not. Next, the compiler tries to find a best match. Although `printInt(int)` is the only non-deleted function, the deleted functions are still considered as candidates in function overload resolution. Because none of these functions are unambiguously the best match, the compiler will issue an ambiguous match compilation error.

Key insight: = delete means “I forbid this”, not “this doesn't exist”.

Deleted function participate in all stages of function overload resolution (not just in the exact match stage). **If a deleted function is selected, then a compilation error results.**

14.4.2 Advanced: Deleting all non-matching overloads

Deleting a bunch of individual function overloads works fine, but can be verbose. There may be times when we want a certain function to be called only with arguments whose types exactly match the function parameters. We can do this by using a function template (introduced in upcoming lesson Function templates) as follows:

```
1  #include <iostream>
2
3  // This function will take precedence for arguments of type int
4  void printInt(int x)
5  {
6      std::cout << x << '\n';
7  }
8
9  // This function template will take precedence for arguments of other types
10 // Since this function template is deleted, calls to it will halt compilation
11 template <typename T>
12 void printInt(T x) = delete;
13
14 int main()
15 {
16     printInt(97);    // okay
17     printInt('a');  // compile error
18     printInt(true); // compile error
19
20     return 0;
21 }
```

14.5 Default arguments

A **default argument** is a default value provided for a function parameter. For example:

```
1 void print(int x, int y=10) // 10 is the default argument
2 {
3     std::cout << "x: " << x << '\n';
4     std::cout << "y: " << y << '\n';
5 }
```

When making a function call, **the caller can optionally provide an argument for any function parameter that has a default argument**. If the caller provides an argument, the value of the argument in the function call is used. **If the caller does not provide an argument, the value of the default argument is used.**

Note that you **must use the equals sign to specify a default argument**. Using **parenthesis or brace initialization won't work**.

Key insight: Default arguments are **inserted by the compiler at site of the function call**.

14.5.1 When to use default arguments

Default arguments are an **excellent option when a function needs a value that has a reasonable default value**, but for which you want to let the caller override if they wish.

For example, here are a couple of function prototypes for which default arguments might be commonly used:

```
1 int rollDie(int sides=6);
2 void openLogFile(std::string filename="default.log");
```

Author's note: Because the user can choose whether to supply a specific argument value or use the default value, a parameter with a default value provided is sometimes called an **optional parameter**. However, the term *optional parameter* is also used to refer to several other types of parameters (including parameters passed by address, and parameters using `std::optional`), so we recommend avoiding this term.

14.5.2 Multiple default arguments

A function can have multiple parameters with default arguments.

C++ does not (as of C++23) support a function call syntax such as `print(,,3)` (as a way to provide an explicit value for `z` while using the default arguments for `x` and `y`. This has **three major consequences:**

1. In a function call, any explicitly provided arguments must be the leftmost arguments (arguments with defaults cannot be skipped).
2. If a parameter is given a default argument, **all subsequent parameters (to the right) must also be given default arguments.**
3. If more than one parameter has a default argument, **the leftmost parameter should be the one most likely to be explicitly set by the user.**

14.5.3 Default arguments cannot be redeclared, and must be declared before use

Once declared, a **default argument cannot be redeclared in the same translation unit**. That means for a function with a forward declaration and a function definition, the **default argument can be declared in either the forward declaration or the function definition, but not both**.

The default argument **must also be declared in the translation unit before it can be used**:

```
1  #include <iostream>
2
3  void print(int x, int y); // forward declaration, no default argument
4
5  int main()
6  {
7      print(3); // compile error: default argument for y hasn't been defined yet
8
9      return 0;
10 }
11
12 void print(int x, int y=4)
13 {
14     std::cout << "x: " << x << '\n';
15     std::cout << "y: " << y << '\n';
16 }
```

Best practice: declare the **default argument in the forward declaration** and not in the function definition, as the **forward declaration is more likely to be seen by other files and included before use (particularly if it's in a header file)**.

in foo.h:

```
1 #ifndef F00_H
2 #define F00_H
3 void print(int x, int y=4);
4 #endif
```

in main.cpp:

```
1 #include "foo.h"
2 #include <iostream>
3
4 void print(int x, int y)
5 {
6     std::cout << "x: " << x << '\n';
7     std::cout << "y: " << y << '\n';
8 }
9
10 int main()
11 {
12     print(5);
13
14     return 0;
15 }
```

Note that in the above example, we're able to use the default argument for function `print()` because `main.cpp` includes `foo.h`, which has the forward declaration that defines the default argument.

Best practice: If the function has a forward declaration (especially one in a header file), **put the default argument there**. Otherwise, put the default argument in the function definition.

14.5.4 Default arguments and function overloading

Functions with default arguments may be overloaded. For example, the following is allowed:

```
1  #include <iostream>
2  #include <string_view>
3
4  void print(std::string_view s)
5  {
6      std::cout << s << '\n';
7  }
8
9  void print(char c = ' ')
10 {
11     std::cout << c << '\n';
12 }
13
14 int main()
15 {
16     print("Hello, world"); // resolves to print(std::string_view)
17     print('a');           // resolves to print(char)
18     print();               // resolves to print(char)
19
20     return 0;
21 }
```

The function call to `print()` acts as if the user had explicitly called `print(' ')`, which resolves to `print(char)`.

```
1  void print(int x);           // signature print(int)
2  void print(int x, int y = 10); // signature print(int, int)
3  void print(int x, double y = 20.5); // signature print(int, double)
```

Default values are not part of a function's signature, so the above functions are differentiated overloads. Thus, the above will compile.

```
1  print(1, 2); // will resolve to print(int, int)
2  print(1, 2.5); // will resolve to print(int, double)
3  print(1); // ambiguous function call
```

In the last case, the compiler is unable to tell whether `print(1)` should resolve to `print(int)` or one of the two functions where the second parameter has a default value. The result is an ambiguous function call.

14.5.5 Advanced: Default arguments don't work for functions called through function pointers

We cover this topic in lesson: Function Pointers.

14.5.6 Summary

Default arguments provide a useful mechanism to specify values for parameters that the user may or may not want to override. They are frequently used in C++, and you'll see them a lot in future lessons.

14.6 Function templates

Let's say you wanted to write a function to calculate the maximum of two numbers. You might do so like this:

```
1 int max(int x, int y)
2 {
3     return (x < y) ? y : x;
4     // Note: we use < instead of > because std::max uses <
5 }
```

While the caller can pass different values into the function, **the type of the parameters is fixed**, so the caller can only pass in int values. That means **this function really only works well for integers (and types that can be promoted to int)**.

So, what happens later when you want to find the max of two double values? Because C++ requires us to specify the type of all function parameters, **the solution is to create a new overloaded version of max with parameters of type double**:

```
1 double max(double x, double y)
2 {
3     return (x < y) ? y : x;
4 }
```

Note that the code for the implementation of the double version of max is exactly the same as for the int version of max! In fact, this implementation works for many different types: including int, double, long, long double, and even new types that you've created yourself (which we'll cover how to do in future lessons).

Having to create overloaded functions with the same implementation for each set of parameter types we want to support is a maintenance headache, a recipe for errors, and a clear violation of the DRY (don't repeat yourself) principle. There's a less-obvious challenge here as well: a programmer who wishes to use the max function may wish to call it with an argument type that the author of the max did not anticipate (and thus did not write an overloaded function for).

What we are really missing is some way to write a single version of max that can work with arguments of any type (even types that may not have been anticipated when the code for max was written). Normal functions are simply not up to the task here. Fortunately, C++ supports another feature that was designed specifically to solve this kind of problem.

Welcome to the world of C++ templates.

14.6.1 Introduction to C++ templates

In C++, the template system was **designed to simplify the process of creating functions (or classes) that are able to work with different data types.**

Instead of manually creating a bunch of mostly-identical functions or classes (one for each set of different types), we instead create a single template. Just like a normal definition, **a template describes what a function or class looks like.** Unlike a normal definition (where all types must be specified), **in a template we can use one or more placeholder types.** A **placeholder type** represents some type that is not known at the time the template is written, but that will be provided later.

Once a template is defined, the **compiler can use the template to generate as many overloaded functions (or classes) as needed,** each using different actual types!

The end result is the same -- we end up with a bunch of mostly-identical functions or classes (one for each set of different types). But we only have to create and maintain a single template, and the compiler does all the hard work for us.

Key insight: The compiler can use a single template to generate a family of related functions or classes, each using a different set of types.

As an aside...

Because the concept behind templates can be hard to describe in words, let's try an **analogy**.

If you were to look up the word "template" in the dictionary, you'd find a definition that was similar to the following: "a template is a model that serves as a pattern for creating similar objects". One type of template that is very easy to understand is that of a stencil. A stencil is a thin piece of material (such as a piece of cardboard or plastic) with a shape cut out of it (e.g. a happy face). By placing the stencil on top of another object, then spraying paint through the hole, you can very quickly replicate the cut-out shape. The stencil itself only needs to be created once, and then it can be reused as many times as desired, to create the cut-out shape in as many different colors as you like. Even better, the color of a shape made with the stencil doesn't have to be determined until the stencil is actually used.

A template is essentially a stencil for creating functions or classes. We create the template (our stencil) once, and then we can use it as many times as needed, to stencil out a function or class for a specific set of actual types. Those actual types don't need to be determined until the template is actually used.

Key insight: Templates **can work with types that didn't even exist when the template was written**. This helps make template code both **flexible and future proof!**

In the rest of this lesson, we'll introduce and explore how to create templates for functions, and describe how they work in more detail. We'll save discussion of class templates until after we've covered what classes are.

14.6.2 Function templates

A **function template** is a function-like definition that is used to generate one or more overloaded functions, each with a different set of actual types. This is what will allow us to create functions that can work with many different types.

When we create our function template, we use **placeholder types** (also called **type template parameters**, or informally **template types**) for any parameter types, return types, or types used in the function body that we want to be specified later.

For advanced readers: C++ supports 3 different kinds of template parameters:

- Type template parameters (where the **template parameter represents a type**).
- Non-type template parameters (where the **template parameter represents a constexpr value**).
- Template template parameters (where the **template parameter represents a template**).

Function templates are something that is **best taught by example**, so let's convert our normal `max(int, int)` function from the example above into a function template. It's surprisingly easy, and we'll explain what's happening along the way.

14.6.3 *Creating a templated max function*

Here's the `int` version of `max` again:

```
1 int max(int x, int y)
2 {
3     return (x < y) ? y : x;
4 }
```

Note that we use type `int` three times in this function: once for parameter `x`, once for parameter `y`, and once for the return type of the function.

To create a function template, we're going to do two things. First, we're going to **replace our specific types with type template parameters**. In this case, because we have only one type that needs replacing (`int`), **we only need one type template parameter (which we'll call `T`)**:

Here's our new function that uses a single template type:

```
1 T max(T x, T y) // won't compile because we haven't defined T
2 {
3     return (x < y) ? y : x;
4 }
```

This is a good start -- however, it **won't compile because the compiler doesn't know what `T` is!** And this is still a normal function, not a function template.

Second, we're going to tell the compiler that this is a function template, and that `T` is a type template parameter that is a placeholder for any type. This is done using what is called a **template parameter declaration**. The **scope of a template parameter declaration is limited to the function template (or class template) that follows**. Therefore, each function template (or class) needs its own template parameter declaration.

```
1  template <typename T> // this is the template parameter declaration
2  T max(T x, T y) // this is the function template definition for max<T>
3  {
4      return (x < y) ? y : x;
5  }
```

In our template parameter declaration, we start with the **keyword template**, which tells the compiler that we're creating a template. Next, we specify all of the template parameters that our template will use inside angled brackets (<>). For each type template parameter, we use the keyword **typename** or **class**, followed by the name of the type template parameter (e.g. T).

We discuss how to create function templates with multiple template types in a future lesson.

As an aside...

There is **no difference between the typename and class keywords in this context**. You will often see people use the class keyword since it was introduced into the language earlier. **However, we prefer the newer typename keyword, because it makes it clearer that the type template parameter can be replaced by any type** (such as a fundamental type), **not just class types**.

Believe it or not, we're done! We have created a template version of our max function that can now accept arguments of different types.

Because this function template has one template type named T, we'll refer to it as max<T>. In the next lesson, we'll look at how we use our max<T> function template to generate one or more max() functions with parameters of different types.

14.6.4 Naming template parameters

Much like we often use a single letter for variable names used in trivial situations (e.g. x), **it's conventional to use a single capital letter (starting with T) when the template parameter is used in a trivial or obvious way**. For example, in our max function template we use T.

We don't need to give T a complex name, because it's obviously just a placeholder type for the values being compared, and T can be any type that can be compared (such as int, double, or char, but not nullptr).

If a type template parameter has a **non-obvious usage or specific requirements** that must be met, there are **two common conventions** for such names:

- **Starting with a capital letter** (e.g. Allocator). **The standard library uses this naming convention.**
- **Prefixed with a T**, then starting with a capital letter (e.g. TAllocator). **This makes it easier to see that the type is a type template parameter.**

Which you choose is a matter of personal preference.

Best practice:

- **Use a single capital letter starting with T** (e.g. T, U, V, etc...) to name type template parameters that are used in trivial or obvious ways and represent “any reasonable type”.
- If the type template parameter has a **non-obvious usage or specific requirements that must be met, then a more descriptive name is warranted.** (e.g. Allocator or TAllocator).

14.7 Function template instantiation

14.7.1 Using a function template

Function templates are not actually functions -- their **code isn't compiled or executed directly**. Instead, **function templates have one job**: to **generate functions** (that are compiled and executed).

To use our `max<T>` function template, we can make a function call with the following syntax:

```
max<actual_type>(arg1, arg2); // actual_type is some actual type, like int or double
```

This looks a lot like a normal function call -- the **primary difference is the addition of the type in angled brackets** (called a **template argument**), which specifies the actual type that will be used in place of template type `T`.

```
std::cout << max<int>(1, 2) << '\n'; // instantiates and calls function max<int>(int, int)
```

When the compiler encounters the function call `max<int>(1, 2)`, it will determine that a function definition for `max<int>(int, int)` does not already exist. Consequently, the compiler will implicitly use our `max<T>` function template to create one.

The process of creating functions (with specific types) from function templates (with template types) is called **function template instantiation** (or **instantiation** for short). When a function is instantiated due to a function call, it's called **implicit instantiation**. A function that is instantiated from a template is technically called a **specialization**, but in common language is often called a **function instance**. The template from which a specialization is produced is called a **primary template**. Function instances are normal functions in all regards.

Nomenclature: The term “**specialization**” is more often used to refer to **explicit specialization**, which allows us to explicitly define a specialization (rather than have it implicitly instantiated from the primary template).

The process for instantiating a function is simple: the **compiler essentially clones the primary template and replaces the template type (T) with the actual type** we've specified (int).

Here's the same example as above, showing what the compiler actually compiles after all instantiations are done:

```

1  #include <iostream>
2
3  // a declaration for our function template (we don't need the definition any more)
4  template <typename T>
5  T max(T x, T y);
6
7  template<
8  int max<int>(int x, int y) // the generated function max<int>(int, int)
9  {
10     return (x < y) ? y : x;
11 }
12
13 int main()
14 {
15     std::cout << max<int>(1, 2) << '\n'; // instantiates and calls function max<int>(int, int)
16
17     return 0;
18 }
```

You can compile this yourself and see that it works. A function template is only instantiated the first time a function call is made in each translation unit. **Further calls to the function are routed to the already instantiated function.**

Conversely, if no function call is made to a function template, the function template won't be instantiated in that translation unit.

```
std::cout << max<double>(1, 2) << '\n'; // instantiates and calls function max<double>(double, double)
```

One additional thing to note here: when we instantiate `max<double>`, the instantiated function has parameters of type `double`. Because we've provided `int` arguments, those arguments will be implicitly converted to `double`.

14.7.2 Template argument deduction

In most cases, the actual types we want to use for instantiation will match the type of our function parameters.

In cases where the type of the arguments match the actual type we want, we do not need to specify the actual type -- instead, we can use **template argument deduction** to have the compiler deduce the actual type that should be used from the argument types in the function call.

For example, instead of making a function call like this:

```
1 | std::cout << max<int>(1, 2) << '\n'; // specifying we want to call max<int>
```

We can **do one of these** instead:

```
1 | std::cout << max<>(1, 2) << '\n';  
2 | std::cout << max(1, 2) << '\n';
```

In either case, the **compiler will see that we haven't provided an actual type, so it will attempt to deduce** an actual type from the function arguments that will allow it to generate a `max()` function where all template parameters match the type of the provided arguments. In this example, the compiler will deduce that using function template `max<T>` with actual type `int` allows it to instantiate function `max<int>(int, int)`, so that the **type of both function parameters (int) matches the type of the provided arguments (int)**.

The difference between the two cases has to do with how the compiler resolves the function call from a set of overloaded functions.

In the **top case (with the empty angled brackets)**, the compiler **will only consider `max<int>` template function overloads** when determining which overloaded function to call.

In the **bottom case (with no angled brackets)**, the compiler **will consider both `max<int>` template function overloads and `max` non-template function overloads**. When the bottom case results in both a template function and a non-template function that are equally viable, the non-template function will be preferred.

Key insight: The **normal function call syntax will prefer a non-template function over an equally viable function instantiated from a template**.

For example:

```

1  #include <iostream>
2
3  template <typename T>
4  T max(T x, T y)
5  {
6      std::cout << "called max<int>(int, int)\n";
7      return (x < y) ? y : x;
8  }
9
10 int max(int x, int y)
11 {
12     std::cout << "called max(int, int)\n";
13     return (x < y) ? y : x;
14 }
15
16 int main()
17 {
18     std::cout << max<int>(1, 2) << '\n'; // calls max<int>(int, int)
19     std::cout << max<>>(1, 2) << '\n';   // deduces max<int>(int, int) (non-template functions not considered)
20     std::cout << max(1, 2) << '\n';     // calls max(int, int)
21
22     return 0;
23 }

```

Note how the syntax in the bottom case looks identical to a normal function call! In most cases, this normal function call syntax will be the one we use to call functions instantiated from a function template.

There are a few reasons for this:

- The **syntax is more concise**.
- It's rare that we'll have both a matching non-template function and a function template.
- If we do have a matching non-template function and a matching function template, **we will usually prefer the non-template function to be called**.

That last point may be non-obvious. A function **template has an implementation that works for multiple types** -- but **as a result, it must be generic**. A non-template function only handles a specific combination of types. It can have an implementation that is more optimized or more specialized for those specific types than the function template version.

Best practice: Favor the **normal function call syntax when making calls to a function instantiated from a function template** (unless you need the function template version to be preferred over a matching non-template function).

14.7.3 Function templates with non-template parameters

It's possible to create function templates that have both template parameters and non-template parameters. The type template parameters can be matched to any type, and the non-template parameters work like the parameters of normal functions.

For example:

```
// T is a type template parameter
// double is a non-template parameter
// We don't need to provide names for these parameters since they aren't used
template <typename T>
int someFcn(T, double)
{
    return 5;
}
```

```
someFcn(1, 3.4); // matches someFcn(int, double)
someFcn(1, 3.4f); // matches someFcn(int, double) -- the float is promoted to a double
someFcn(1.2, 3.4); // matches someFcn(double, double)
someFcn(1.2f, 3.4); // matches someFcn(float, double)
someFcn(1.2f, 3.4f); // matches someFcn(float, double) -- the float is promoted to a double
```

This function template has a templated first parameter, but the **second parameter is fixed with type double**. Note that the **return type can also be any type**. In this case, our function will always return an int value.

14.7.4 Instantiated functions may not always compile

Consider the following program:

```
1  #include <iostream>
2  #include <string>
3
4  template <typename T>
5  T addOne(T x)
6  {
7      return x + 1;
8  }
9
10 int main()
11 {
12     std::string hello { "Hello, world!" };
13     std::cout << addOne(hello) << '\n';
14
15     return 0;
16 }
```

When the compiler tries to resolve `addOne(hello)` it **won't find a non-template function match for `addOne(std::string)`**, but it will find our function template for `addOne(T)`, and determine that it can generate an `addOne(std::string)` function from it. Thus, the compiler will generate and compile this:

```
template<>
std::string addOne<std::string>(std::string x)
{
    return x + 1;
}
```

However, this will generate a compile error, because **`x + 1` doesn't make sense when `x` is a `std::string`**. The obvious solution here is simply not to call `addOne()` with an argument of type `std::string`.

14.7.5 *Instantiated functions may not always make sense semantically*

The compiler will successfully compile an instantiated function template as long as it makes sense syntactically. **However, the compiler does not have any way to check that such a function actually makes sense semantically.**

For example:

```
template <typename T>
T addOne(T x)
{
    return x + 1;
}

int main()
{
    std::cout << addOne("Hello, world!") << '\n';

    return 0;
}
```

In this example, we're calling `addOne()` on a C-style string literal. What does that actually mean semantically? Who knows!

Perhaps surprisingly, because C++ syntactically allows addition of an integer value to a string literal (we cover this in future lesson: Pointer arithmetic and subscripting), **the above example compiles, and produces the following result: ello, world!**

Warning: The compiler will instantiate and compile function templates that do not make sense semantically as long as they are syntactically valid. It is your responsibility to make sure you are calling such function templates with arguments that make sense.

14.7.6 *Function templates and default arguments for non-template parameters*

Just like normal functions, **function templates can have default arguments for non-template parameters.** Each function instantiated from the template will use the same default argument.

14.7.7 Beware function templates with modifiable static local variables

We discussed static local variables, which are local variables with static duration (they persist for the lifetime of the program).

When a static local variable is used in a function template, each function instantiated from that template will have a separate version of the static local variable. This is **rarely a problem if the static local variable is const**. But if the static local variable is one that is modified, the results may not be as expected.

For example:

```
1  #include <iostream>
2
3  // Here's a function template with a static local variable that is modified
4  template <typename T>
5  void printIDAndValue(T value)
6  {
7      static int id{ 0 };
8      std::cout << ++id << ") " << value << '\n';
9  }
10
11 int main()
12 {
13     printIDAndValue(12);
14     printIDAndValue(13);
15
16     printIDAndValue(14.5);
17
18     return 0;
19 }
```

This produces the result:

```
1) 12
2) 13
1) 14.5
```

You may have been expecting the last line to print **3)** 14.5. However, this is what the compiler actually compiles and executes:

```
template <typename T>
void printIDAndValue(T value);

template <>
void printIDAndValue<int>(int value)
{
    static int id{ 0 };
    std::cout << ++id << ") " << value << '\n';
}

template <>
void printIDAndValue<double>(double value)
{
    static int id{ 0 };
    std::cout << ++id << ") " << value << '\n';
}
```

Note that `printIDAndValue<int>` and `printIDAndValue<double>` **each have their own independent static local variable named `id`**, not one that is shared between them.

14.7.8 Generic programming

Because template types can be replaced with any actual type, template types are sometimes called **generic types**. And because templates can be written agnostically of specific types, programming with templates is sometimes called **generic programming**. Whereas **C++ typically has a strong focus on types and type checking**, in contrast, **generic programming lets us focus on the logic of algorithms and design of data structures without having to worry so much about type information**.

14.7.9 Conclusion

Once you get used to writing function templates, you'll find they actually don't take much longer to write than functions with actual types. **Function templates can significantly reduce code maintenance and errors by minimizing the amount of code that needs to be written and maintained.**

Function templates do have a few drawbacks, and we would be remiss not to mention them:

- First, the **compiler will create (and compile) a function for each function call with a unique set of argument types**. So, while function templates are compact to write, **they can expand into a crazy amount of code**, which can lead to **code bloat and slow compile times**.
- The bigger downside of function templates is that they tend to **produce crazy-looking, borderline unreadable error messages** that are much harder to decipher than those of regular functions. These error messages can be quite intimidating, but once you understand what they are trying to tell you, the problems they are pinpointing are often quite straightforward to resolve.

These drawbacks are fairly minor compared with the power and safety that templates bring to your programming toolkit, so use templates liberally anywhere you need type flexibility! A good rule of thumb is to create normal functions at first, and then convert them into function templates if you find you need an overload for different parameter types.

Best practice: Use function templates to write generic code that can work with a wide variety of types whenever you have the need.

14.8 Using function templates in multiple files

Consider the following program, which **doesn't work correctly**:

```
main.cpp:
1  #include <iostream>
2
3  template <typename T>
4  T addOne(T x); // function template forward declaration
5
6  int main()
7  {
8      std::cout << addOne(1) << '\n';
9      std::cout << addOne(2.3) << '\n';
10
11     return 0;
12 }

add.cpp:
1  template <typename T>
2  T addOne(T x) // function template definition
3  {
4      return x + 1;
5  }
```

If `addOne` were a non-template function, this program would work fine: In *main.cpp*, the compiler would be satisfied with the forward declaration of `addOne`, and the linker would connect the call to `addOne()` in *main.cpp* to the function definition in *add.cpp*. But **because `addOne` is a template, this program doesn't work, and we get a linker error.**

In *main.cpp*, we call `addOne<int>` and `addOne<double>`. However, since the compiler can't see the definition for function template `addOne`, it can't instantiate those functions inside *main.cpp*. It does see the forward declaration for `addOne` though, and will assume those functions exist elsewhere and will be linked in later.

When the compiler goes to compile *add.cpp*, it will see the definition for function template `addOne`. However, there are no uses of this template in *add.cpp*, so the compiler will not instantiate anything. The end result is that the linker is unable to connect the calls to `addOne<int>` and `addOne<double>` in *main.cpp* to the actual functions, because those functions were never instantiated.

As an aside...

If *add.cpp* had instantiated those functions, the program would have compiled and linked just fine. But such solutions are fragile and should be avoided: if the code in *add.cpp* was later changed so those functions are no longer instantiated, the program would again fail to link. Or if *main.cpp* called a different version of *addOne* (such as *addOne<float>*) that was not instantiated in *add.cpp*, we run into the same problem.

The most conventional way to address this issue is to put all your template code in a header (.h) file instead of a source (.cpp) file:

```
add.h:
1  #ifndef ADD_H
2  #define ADD_H
3
4  template <typename T>
5  T addOne(T x) // function template definition
6  {
7      return x + 1;
8  }
9
10 #endif
```

That way, any files that need access to the template can *#include* the relevant header, and the template definition will be copied by the preprocessor into the source file. The compiler will then be able to instantiate any functions that are needed.

You may be wondering why this doesn't cause a violation of the one-definition rule (ODR). The **ODR says that types, templates, inline functions, and inline variables are allowed to have identical definitions in different files. So, there is no problem if the template definition is copied into multiple files (as long as each definition is identical).**

But **what about the instantiated functions themselves?** If a function is instantiated in multiple files, how does that not cause a violation of the ODR? The answer is that **functions implicitly instantiated from templates are implicitly inline.** And as you know, inline functions can be defined in multiple files, so long as the definition is identical in each.

Key insight:

Template definitions are exempt from the part of the one-definition rule that requires only one definition per program, so it is not a problem to have the same template definition #included into multiple source files. And **functions implicitly instantiated from function templates are implicitly inline**, so they can be defined in multiple files, so long as each definition is identical.

The templates themselves are not inline, as the concept of inline only applies to variables and functions.

Best practice: Templates that are needed in multiple files **should be defined in a header file**, and then #included wherever needed. This allows **the compiler to see the full template definition and instantiate the template when needed.**

14.9 Function templates with multiple template types

In lesson: Function templates, we wrote a function template to calculate the maximum of two values:

```
3  template <typename T>
4  T max(T x, T y)
5  {
6      return (x < y) ? y : x;
7  }
8
9  int main()
10 {
11     std::cout << max(2, 3.5) << '\n'; // compile error
12
13     return 0;
14 }
```

You may be surprised to find that this program won't compile. Instead, the compiler will issue a bunch of (probably crazy looking) error messages. In our function call `max(2, 3.5)`, **we're passing arguments of two different types**: one `int` and one `double`. Because we're making a function call without using angled brackets to specify an actual type, the compiler will first look to see if there is a non-template match for `max(int, double)`. It won't find one.

Next, the compiler will see if it can find a function template match (using template argument deduction, which we covered in lesson: Function template instantiation). However, this will also fail, for a simple reason: `T` can only represent a single type. There is no type for `T` that would allow the compiler to instantiate function template `max<T>(T, T)` into a function with two different parameter types. Put another way, because both parameters in the function template are of type `T`, they must resolve to the same actual type.

Since both a non-template match and a template match couldn't be found, the function call fails to resolve, and we get a compile error.

You might wonder why the compiler didn't generate function `max<double>(double, double)` and then use numeric conversion to type convert the `int` argument to a `double`. The answer is simple: **type conversion is done only when resolving function overloads, not when performing template argument deduction.**

This lack of type conversion is intentional for at least two reasons.

- First, it **helps keep things simple**: we **either find an exact match** between the function call arguments and template type parameters, **or we don't**.
- Second, it **allows us to create function templates for cases where we want to ensure that two or more parameters have the same type** (as in the example above).

We'll have to find another solution. Fortunately, we can solve this problem in (at least) **three ways**:

- Use **`static_cast`** to convert the arguments to matching types. Use `static_cast` to convert the arguments to matching types. However, this solution is **awkward and hard to read**.
- Provide an **explicit type template argument**. If we had written a non-template **`max(double, double)`** function, then we would be able to call `max(int, double)` and **let the implicit type conversion rules convert our `int` argument into a `double`** so the function call could be resolved. While this is more readable than using `static_cast`, **it would be even nicer if we didn't even have to think about the types** when making a function call to `max` at all.
- Function **templates with multiple template type parameters**

14.9.1 Function templates with multiple template type parameters

The **root of our problem** is that we've only defined the single template type (T) for our function template, and then specified that both parameters must be of this same type.

The **best way to solve this problem** is to rewrite our function template in such a way that our **parameters can resolve to different types**. Rather than using one template type parameter T, we'll now use two (T and U):

```
template <typename T, typename U> // We're using two template type parameters named T and U
T max(T x, U y) // x can resolve to type T, and y can resolve to type U
{
    return (x < y) ? y : x; // uh oh, we have a narrowing conversion problem here
}
```

Key insight: Because T and U are independent template parameters, **they resolve their types independent of each other**. This means T and U can resolve to **different types**, or they can **resolve to the same type**.

However, the **above code still has a problem**: using the usual arithmetic rules (Arithmetic conversions), **double takes precedence over int**, so our **conditional operator will return a double**. But our **function is defined as returning a T** -- in cases where **T resolves to an int**, our **double return value will undergo a narrowing conversion to an int**, which will produce a **warning (and possible loss of data)**.

Making the return type a U instead doesn't solve the problem, as we can always flip the order of the operands in the function call to flip the types of T and U.

How do we solve this? This is a **good use for an auto return type** -- we'll let the compiler deduce what the return type should be from the return statement:

```
template <typename T, typename U>
auto max(T x, U y)
{
    return (x < y) ? y : x;
}
```

14.9.2 Abbreviated function templates: C++20

C++20 introduces a new use of the **auto** keyword: When the **auto** keyword is used as a parameter type in a normal function, the compiler will automatically convert the function into a function template with each **auto** parameter becoming an independent template type parameter. This method for creating a function template is called an **abbreviated function template**.

The above example is the C++20 abbreviated template and the **template** underneath does exactly the same as the one above:

```
✓ auto Max(auto x, auto y)
{
    return (x < y) ? y : x;
}

template <typename T, typename U>
✓ auto Max(T x, U y)
{
    return (x < y) ? y : x;
}
```

In cases where you want each template type parameter to be an independent type, this form is **preferred** as the **removal of the template parameter declaration line makes your code more concise and readable**.

There isn't a concise way to use abbreviated function templates when you want more than one **auto** parameter to be the same type. That is, **there isn't an easy abbreviated function template for something like this**:

```
1  template <typename T>
2  T max(T x, T y) // two parameters of the same type
3  {
4      return (x < y) ? y : x;
5  }
```

Best practice: Feel free to use abbreviated function templates with a single **auto** parameter, or where each **auto** parameter should be an independent type (and your language standard is set to C++20 or newer).

Function templates may be overloaded:

Just like functions may be overloaded, **function templates may also be overloaded**. Such overloads can have a different number of template types and/or a different number or type of function parameters:

```
#include <iostream>

// Add two values with matching types
template <typename T>
T add(T x, T y)
{
    return x + y;
}

// Add two values with non-matching types
// As of C++20 we could also use auto add(auto x, auto y)
template <typename T, typename U>
T add(T x, U y)
{
    return x + y;
}

// Add three values with any type
// As of C++20 we could also use auto add(auto x, auto y, auto z)
template <typename T, typename U, typename V>
T add(T x, U y, V z)
{
    return x + y + z;
}
```

One interesting note here is that for the call to `add(1.2, 3.4)`, the compiler will prefer `add<T>` over `add<T, U>` even though both could possibly match.

The rules for determining which of multiple matching function templates should be preferred are called “partial ordering of function templates”. In short, whichever function template is more restrictive/specialized will be preferred. `add<T>` is the more restrictive function template in this case (since it only has one template parameter), so it is preferred.

If multiple function templates can match a call and the compiler can’t determine which is more restrictive, the compiler will error with an ambiguous match.

14.10 Non-type template parameters

In the previous lessons, we discussed how to create function templates that use type template parameters. A type template parameter serves as a placeholder for an actual type that is passed in as a template argument.

While type template parameters are by far the most common type of template parameter used, there is another kind of template parameter worth knowing about: **the non-type template parameter**.

14.10.1 Non-type template parameters

A **non-type template parameter** is a template parameter with a fixed type that serves as a placeholder for a **constexpr** value passed in as a template argument.

A non-type template parameter can be any of the following types:

- An integral type
- An enumeration type
- `std::nullptr_t`
- A floating-point type (since C++20)
- A pointer or reference to an object
- A pointer or reference to a function
- A pointer or reference to a member function
- A literal class type (since C++20)

We saw our first example of a non-type template parameter when we discussed `std::bitset`.

In the case of `std::bitset`, the non-type template parameter is used to tell the `std::bitset` how many bits we want it to store.

```
std::bitset<8> bits{ 0b0000'0101 }; // The <8> is a non-type template parameter
```

14.10.2 Defining our own non-type template parameters

Here's a simple example of a function that uses an int non-type template parameter:

```
template <int N> // declare a non-type template parameter of type int named N
void print()
{
    std::cout << N << '\n'; // use value of N here
}

int main()
{
    print<5>(); // 5 is our non-type template argument

    return 0;
}
```

Much like T is typically used as the name for the first type template parameter, **N is conventionally used as the name of an int non-type template parameter.**

Best practice: Use N as the name of an int non-type template parameter.

14.10.3 What are non-type template parameters useful for?

As of C++20, function parameters cannot be constexpr. This is true for normal functions, constexpr functions (which makes sense, as they must be able to be run at runtime), and perhaps **surprisingly, even constexpr functions.**

So, let's say we have some function like this one:

```
double getSqrt(double d)
{
    assert(d >= 0.0 && "getSqrt(): d must be non-negative");

    if (d >= 0) { return std::sqrt(d); }
    return 0.0;
}
```

When run, the call to `getSqrt(-5.0)` will runtime assert out. While this is better than nothing, because `-5.0` is a literal (and implicitly constexpr), it **would be better if we could static_assert** so that errors such as this one would be caught at compile-time. However, **static_assert requires a constant expression**, and function parameters can't be constexpr...

However, if we change the function parameter to a non-type template parameter instead, then we can do exactly as we want:

```
template <double D> // requires C++20 for floating point non-type parameters
double getSqrt()
{
    static_assert(D >= 0.0, "getSqrt(): D must be non-negative");

    if constexpr (D >= 0) // ignore the constexpr here for this example
        return std::sqrt(D); // strangely, std::sqrt isn't a constexpr function (until C++26)

    return 0.0;
}
```

This version fails to compile. When the compiler encounters `getSqrt<-5.0>()`, it will instantiate and call a function that **looks something like this**:

```
template <>
double getSqrt<-5.0>()
{
    static_assert(-5.0 >= 0.0, "getSqrt(): D must be non-negative");

    if constexpr (-5.0 >= 0) // ignore the constexpr here for this example
        return std::sqrt(-5.0);

    return 0.0;
}
```

The `static_assert` condition is false, so the **compiler asserts out**.

Key insight: Non-type template parameters are used primarily **when we need to pass constexpr values to functions** (or class types) so they can be used in contexts that require a constant expression.

Author's note: Having to use non-type template parameters to **circumvent the restriction that function parameters can't be constexpr** isn't great. There are quite a few different proposals being evaluated to help address situations like this. I expect that we might see a better solution to this in a future C++ language standard.

14.10.4 Implicit conversions for non-type template arguments: Optional

Certain non-type template **arguments can be implicitly converted in order to match a non-type template parameter** of a different type. For example:

```
template <int N> // int non-type template parameter
void print()
{
    std::cout << N << '\n';
}

int main()
{
    print<5>(); // no conversion necessary
    print<'c'>(); // 'c' converted to type int, prints 99

    return 0;
}
```

In this context, only certain types of constexpr conversions are allowed. The most common types of allowed conversions include:

- Integral promotions (e.g. char to int)
- Integral conversions (e.g. char to long or int to char)
- User-defined conversions (e.g. some program-defined class to int)
- Lvalue to rvalue conversions (e.g. some variable x to the value of x)

Note that this list is less permissive than the type of implicit conversions allowed for list initialization.

Unlike with normal functions, the **algorithm for matching function template calls to function template definitions is not sophisticated**, and **certain matches are not prioritized over others based on the type of conversion required** (or lack thereof). This means that if a function template is overloaded for different kinds of non-type template parameters, **it can very easily result in an ambiguous match**.

Perhaps surprisingly, **both of these calls to print() result in ambiguous matches**.

14.10.5 Type deduction for non-type template parameters using auto: C++17

As of C++17, non-type template parameters may use `auto` to have the compiler deduce the non-type template parameter from the template argument:

```
template <auto N> // deduce non-type template parameter from template argument
void print()
{
    std::cout << N << '\n';
}
```

14.11 Summary

Function overloading allows us to create multiple functions with the same name, so long as each identically named function has different set of parameter types (or the functions can be otherwise differentiated). Such a function is called an **overloaded function** (or **overload** for short). Return types are not considered for differentiation.

When resolving overloaded functions, if an exact match isn't found, the compiler will favor overloaded functions that can be matched via numeric promotions over those that require numeric conversions. When a function call is made to function that has been overloaded, the compiler will try to match the function call to the appropriate overload based on the arguments used in the function call. This is called **overload resolution**.

An **ambiguous match** occurs when the compiler finds two or more functions that can match a function call to an overloaded function and can't determine which one is best.

A **default argument** is a default value provided for a function parameter. Parameters with default arguments must always be the rightmost parameters, and they are not used to differentiate functions when resolving overloaded functions.

Function templates allow us to create a function-like definition that serves as a pattern for creating related functions. In a function template, we use **type template parameters** as placeholders for any types we want to be specified later. The syntax that tells the compiler we're defining a template and declares the template types is called a **template parameter declaration**.

The process of creating functions (with specific types) from function templates (with template types) is called **function template instantiation** (or **instantiation**) for short. When this process happens due to a function call, it's called **implicit instantiation**. An instantiated function is called a **function instance** (or **instance** for short, or sometimes a **template function**).

Template argument deduction allows the compiler to deduce the actual type that should be used to instantiate a function from the arguments of the function call. Template argument deduction does not do type conversion.

Template types are sometimes called **generic types**, and programming using templates is sometimes called **generic programming**.

In C++20, when the `auto` keyword is used as a parameter type in a normal function, the compiler will automatically convert the function into a function template with each `auto` parameter becoming an independent template type parameter. This method for creating a function template is called an **abbreviated function template**.

A **non-type template parameter** is a template parameter with a fixed type that serves as a placeholder for a `constexpr` value passed in as a template argument.

15 Compound Types: References and Pointers

15.1 Introduction to compound data types

In lesson: Introduction to fundamental data types, we introduced the fundamental data types, which are the basic data types that C++ provides as part of the core language.

We've made much use of these fundamental types in our programs so far, especially the `int` data type. And while these fundamental types are extremely useful for straightforward uses, they don't cover our full range of needs as we begin to do more complicated things.

For example, imagine you were writing a math program to multiply two fractions. How would you represent a fraction in your program? You might use a pair of integers (one for the numerator, one for the denominator), like this:

```
int main()
{
    // Our first fraction
    int num1 {};
    int den1 {};

    // Our second fraction
    int num2 {};
    int den2 {};

    // Used to eat (remove) the slash between the numerator and denominator
    char ignore {};

    std::cout << "Enter a fraction: ";
    std::cin >> num1 >> ignore >> den1;

    std::cout << "Enter a fraction: ";
    std::cin >> num2 >> ignore >> den2;

    std::cout << "The two fractions multiplied: "
                << num1 * num2 << '/' << den1 * den2 << '\n';

    return 0;
}
```

While this program works, it introduces a couple of challenges for us to improve upon. First, each pair of integers is only **loosely linked** -- outside of comments and the context of how they are used in the code, there's little to suggest that each numerator and denominator pair are related. Second, following the DRY (don't repeat yourself) principle, **we should create a function** to handle the user inputting a fraction (along with some error handling). However, **functions can only return a single value**, so how would we return the numerator and denominator back to the caller?

Now imagine another case where you're writing a program that needs to keep a list of employee IDs. How might you do so? You might try something like this:

```
int id1 { 42 };  
int id2 { 57 };  
int id3 { 162 };  
// and so on
```

But what if you had 100 employees? First, you'd need to type in 100 variable names. And what if you needed to print them all? Or pass them to a function? We'd be in for a lot of typing. This simply doesn't scale.

Clearly fundamental types will only carry us so far.

15.1.1 Compound data types

Fortunately, C++ supports a second set of data types called **compound data types**. Compound data types (also sometimes called **composite data types**) are **data types that can be constructed from fundamental data types (or other compound data types)**. Each compound data type has its own unique properties as well.

As we'll show in this chapter and future chapters, we can use compound data types to elegantly solve all of the challenges we presented above.

C++ supports the following compound types:

- Functions
- Arrays
- Pointer types:
 - Pointer to object
 - Pointer to function
- Pointer to member types:
 - Pointer to data member
 - Pointer to member function
- Reference types:
 - L-value references
 - R-value references
- Enumerated types:
 - Unscoped enumerations
 - Scoped enumerations
- Class types:
 - Structs
 - Classes
 - Unions

You've already been using one compound type regularly: **functions**. For example, consider this function:

```
1 void doSomething(int x, double y)
2 {
3 }
```

The type of this function is `void(int, double)`. Note that this type is **composed of fundamental types**, making it a compound type. Of course, functions also have their own special behaviors as well (e.g. being callable).

Because there's a lot of material to cover here, we'll do it over multiple chapters. **In this chapter, we'll cover some of the more straightforward compound types**, including **l-value references, and pointers**. Next chapter, we'll cover unscoped enumerations, scoped enumerations, and basic structs. Then, in the chapters beyond that, we'll introduce class types and dig into some of the more useful array types. This includes `std::string` which is actually a class type!

15.2 Value categories (lvalues and rvalues)

Before we talk about our first compound type (**lvalue references**), we're going to take a little detour and talk about what an **lvalue** is.

In lesson: Introduction to expressions, we defined an expression as “a **combination of literals, variables, operators, and function calls that can be executed to produce a singular value**”.

For example:

```
std::cout << 2 + 3 << '\n'; // The expression 2 + 3 produces the value 5
```

In lesson: Increment/decrement operators, and side effects, we also noted that **expressions can produce side effects that outlive the expression**:

```
int x { 5 };
++x; // This expression statement has the side-effect of incrementing x
std::cout << x << '\n'; // prints 6
```

In the above program, the expression `++x` increments the value of `x`, and that value remains changed even after the expression has finished evaluating.

Besides producing values and side effects, **expressions can do one more thing**: they can **evaluate to objects or functions**. We'll explore this point further in just a moment.

15.2.1 *The properties of an expression*

To help determine how expressions should evaluate and where they can be used, all **expressions in C++ have two properties**: a **type** and a **value category**.

15.2.2 *The type of an expression*

The **type of an expression is equivalent to the type of the value, object, or function** that results from the evaluated expression. For example:

```
auto v1 { 12 / 4 }; // int / int => int
auto v2 { 12.0 / 4 }; // double / int => double
```

The compiler can use the type of an expression to determine whether an expression is valid in a given context. For example:

```
print("foo"); // error: print() was expecting an int argument, we tried to pass in a string literal
```

In the above program, the `print(int)` function is expecting an `int` parameter. However, the type of the expression we're passing in (the string literal `"foo"`) does not match, and no conversion can be found. So, a compile error results.

Note that the **type of an expression must be determinable at compile time (otherwise type checking and type deduction wouldn't work)** -- however, the **value of an expression may be determined at either compile time** (if the expression is `constexpr`) **or runtime** (if the expression is not `constexpr`).

15.2.3 *The value category of an expression*

Now consider the following program:

```
x = 5; // valid: we can assign 5 to x
5 = x; // error: can not assign value of x to literal value 5
```

One of these assignment statements is valid (assigning value 5 to variable `x`) and one is not (what would it mean to assign the value of `x` to the literal value 5?). So **how does the compiler know which expressions can legally appear on either side of an assignment statement?**

The answer lies in the second property of expressions: the value category. The **value category** of an expression (or subexpression) **indicates whether an expression resolves to a value, a function, or an object of some kind.**

Prior to C++11, there were only two possible value categories: **lvalue** and **rvalue**.

In C++11, three additional value categories (**glvalue**, **prvalue**, and **xvalue**) were added to support a new feature called move semantics.

Author's note: In this lesson, we'll stick to the pre-C++11 view of value categories, as this makes for a gentler introduction to value categories (and is **all that we need for the moment**). We'll cover move semantics (and the additional three value categories) in a future chapter.

15.2.4 *Lvalue and rvalue expressions*

An **lvalue** (pronounced “ell-value”, short for “**left value**” or “**locator value**”, and sometimes written as “l-value”) is an **expression that evaluates to an identifiable object or function** (or bit-field).

The term “identity” is used by the C++ standard, but is not well-defined. **An entity (such as an object or function) that has an identity can be differentiated from other similar entities (typically by comparing the addresses of the entity).**

Entities with identities can be accessed via an identifier, reference, or pointer, and **typically have a lifetime longer than a single expression or statement.**

```
int x { 5 };  
int y { x }; // x is an lvalue expression
```

In the above program, the expression `x` is an lvalue expression as it evaluates to variable `x` (which has an identifier).

Since the introduction of constants into the language, lvalues come in **two subtypes**:

- A **modifiable lvalue** is an lvalue whose value can be modified.
- A **non-modifiable lvalue** is an lvalue whose value can't be modified (because the lvalue is `const` or `constexpr`).

```
int x{};
const double d{};

int y { x }; // x is a modifiable lvalue expression
const double e { d }; // d is a non-modifiable lvalue expression
```

An **rvalue** (pronounced “arr-value”, short for “**right value**”, and sometimes written as r-value) is an expression that is not an lvalue. **Rvalue expressions evaluate to a value.** Commonly seen rvalues include literals (except **C-style string literals, which are lvalues**) and the return value of functions and operators that return by value. **Rvalues aren't identifiable** (meaning they have to be used immediately), and **only exist within the scope of the expression in which they are used.**

Key insight: Lvalue expressions evaluate to an **identifiable object**. Rvalue expressions evaluate to a **value**.

Now we can answer the question about why `x = 5` is valid but `5 = x` is not: **an assignment operation requires the left operand of the assignment to be a modifiable lvalue expression**, and the right operand to be an rvalue expression. The latter assignment (`5 = x`) fails because the left operand expression `5` isn't an lvalue.

```
// Assignment requires the left operand to be a modifiable lvalue expression and the right operand to be an rvalue expression
x = 5; // valid: x is a modifiable lvalue expression and 5 is an rvalue expression
5 = x; // error: 5 is an rvalue expression and x is a modifiable lvalue expression
```

Tip: If you're not sure whether an expression is an lvalue or rvalue, try taking its address using operator `&`, which requires its operand to be an lvalue.

If `&(expression)`; compiles, expression must be an lvalue.

Key insight: A **C-style string literal is an lvalue** because C-style strings (**which are C-style arrays**) **decay to a pointer**. The **decay process only works if the array is an lvalue** (and thus has an address that can be stored in the pointer). **C++ inherited this for backwards compatibility.**

15.2.5 Lvalue to rvalue conversion

Let's take a look at this example again:

```
int x { 5 };  
int y { x }; // x is an lvalue expression
```

If x is an lvalue expression that evaluates to variable x, then how does y end up with value 5?

The answer is that **lvalue expressions will implicitly convert to rvalue expressions in contexts where an rvalue is expected but an lvalue is provided**. The initializer for an int variable is expected to be an rvalue expression. Thus, lvalue expression x undergoes an lvalue-to-rvalue conversion, which evaluates to value 5, which is then used to initialize y.

We said above that the assignment operator expects the right operand to be an rvalue expression, so why does code like this work?

```
int x{ 1 };  
int y{ 2 };  
  
x = y; // y is a modifiable lvalue, not an rvalue, but this is legal
```

In this case, y is an lvalue expression, but it undergoes an **lvalue-to-rvalue conversion**, which **evaluates to the value of y (2)**, which is then assigned to x.

Now that we've covered lvalues, we can get to our first compound type: the **lvalue reference**.

Key insight: A rule of thumb to identify lvalue and rvalue expressions:

- **Lvalue** expressions are those that **evaluate to functions or identifiable objects** (including variables) that **persist beyond the end of the expression**. Basically, anything that has an address in memory (C-style strings too).
- **Rvalue** expressions are those that **evaluate to values**, including literals and temporary objects that **do not persist beyond the end of the expression**.

15.3 Lvalue references

In C++, a **reference** is an **alias for an existing object**. Once a reference has been defined, any operation on the reference is applied to the object being referenced.

Key insight: A reference is essentially identical to the object being referenced.

This means **we can use a reference to read or modify the object being referenced**. Although references might seem silly, useless, or redundant at first, references are used everywhere in C++ (we'll see examples of this in a few lessons). You can also create references to functions, though this is done less often.

Modern C++ contains **two types of references**:

- **lvalue** references
- **rvalue** references (covered in the chapter on move semantics)

15.3.1 Lvalue reference types

An **lvalue reference** (commonly just called a reference since **prior to C++11 there was only one type of reference**) acts as an **alias for an existing lvalue** (such as a variable).

To declare an lvalue reference type, we use an **ampersand (&)** in the type declaration:

```
1 | int      // a normal int type
2 | int&     // an lvalue reference to an int object
3 | double& // an lvalue reference to a double object
```

15.3.2 Lvalue reference variables

One of the things we can do with an lvalue reference type is create an lvalue reference variable. An lvalue reference variable is a variable that acts as a reference to an lvalue (usually another variable).

To create an lvalue reference variable, we simply define a variable with an lvalue reference type:

```
int x { 5 };    // x is a normal integer variable
int& ref { x }; // ref is an lvalue reference variable that can now be used as an alias for variable x

std::cout << x << '\n'; // print the value of x (5)
std::cout << ref << '\n'; // print the value of x via ref (5)
```

In the above example, the type `int&` defines `ref` as an lvalue reference to an `int`, which we then initialize with lvalue expression `x`. Thereafter, `ref` and `x` can be used synonymously. This program thus prints:

5

5

From the compiler's perspective, it doesn't matter whether the ampersand is "attached" to the type name (`int& ref`) or the variable's name (`int &ref`), and **which you choose is a matter of style**. Modern C++ programmers tend to prefer attaching the ampersand to the type, as it makes it clearer that the reference is part of the type information, not the identifier.

Best practice: When defining a reference, place the ampersand next to the type (not the reference variable's name).

For advanced readers: For those of you already familiar with pointers, the ampersand in this context does not mean "address of", it means "lvalue reference to".

15.3.3 Modifying values through an lvalue reference

In the above example, we showed that we can use a reference to read the value of the object being referenced. **We can also use a reference to modify the value of the object being referenced:**

```
3 int main()
4 {
5     int x { 5 }; // normal integer variable
6     int& ref { x }; // ref is now an alias for variable x
7
8     std::cout << x << ref << '\n'; // print 55
9
10    x = 6; // x now has value 6
11
12    std::cout << x << ref << '\n'; // prints 66
13
14    ref = 7; // the object being referenced (x) now has value 7
15
16    std::cout << x << ref << '\n'; // prints 77
17
18    return 0;
19 }
```

This code prints:

55

66

77

In the above example, **ref** is an alias for **x**, so we are able to **change the value of x** through either **x** or **ref**.

15.3.4 Initialization of lvalue references

Much like constants, **all references must be initialized**.

```
int& invalidRef; // error: references must be initialized

int x { 5 };
int& ref { x }; // okay: reference to int is bound to int variable
```

When a reference is initialized with an object (or function), we say it is **bound** to that object (or function). The process by which such a reference is bound is called **reference binding**. The object (or function) being referenced is sometimes called the **referent**.

Lvalue references must be bound to a modifiable lvalue.

```
int x { 5 };
int& ref { x }; // valid: lvalue reference bound to a modifiable lvalue

const int y { 5 };
int& invalidRef { y }; // invalid: can't bind to a non-modifiable lvalue
int& invalidRef2 { 0 }; // invalid: can't bind to an rvalue
```

Lvalue references can't be bound to non-modifiable lvalues or rvalues (otherwise you'd be able to change those values through the reference, which would be a violation of their const-ness). For this reason, lvalue references are occasionally called **lvalue references to non-const** (sometimes shortened to **non-const reference**).

In most cases, the type of the reference must match the type of the referent (there are some exceptions to this rule that we'll discuss when we get into inheritance):

```
int x { 5 };
int& ref { x }; // okay: reference to int is bound to int variable

double y { 6.0 };
int& invalidRef { y }; // invalid; reference to int cannot bind to double variable
double& invalidRef2 { x }; // invalid: reference to double cannot bind to int variable
```

Lvalue references to void are disallowed (what would be the point?).

15.3.5 References can't be reseated (changed to refer to another object)

Once initialized, a **reference in C++ cannot be reseated**, meaning it **cannot be changed to reference another object**.

New C++ programmers often try to reseat a reference by using assignment to provide the reference with another variable to reference. **This will compile and run -- but not function as expected.** Consider the following program:

```
3 int main()
4 {
5     int x { 5 };
6     int y { 6 };
7
8     int& ref { x }; // ref is now an alias for x
9
10    ref = y; // assigns 6 (the value of y) to x (the object being referenced by ref)
11    // The above line does NOT change ref into a reference to variable y!
12
13    std::cout << x << '\n'; // user is expecting this to print 5
14
15    return 0;
16 }
```

Perhaps surprisingly, this prints:

6

When a reference is evaluated in an expression, it resolves to the object it's referencing. So, **ref = y doesn't change ref to now reference y**. Rather, because ref is an alias for x, **the expression evaluates as if it was written x = y** -- and since y evaluates to value 6, x is assigned the value 6.

15.3.6 Lvalue reference scope and duration

Reference variables **follow the same scoping and duration rules that normal variables** do:

```
1  #include <iostream>
2
3  int main()
4  {
5      int x { 5 }; // normal integer
6      int& ref { x }; // reference to variable value
7
8      return 0;
9  } // x and ref die here
```

15.3.7 References and referents have independent lifetimes

With one exception (that we'll cover next lesson), the **lifetime of a reference and the lifetime of its referent are independent**. In other words, both of the following are true:

- A reference can be destroyed before the object it is referencing.
- The object being referenced can be destroyed before the reference.

When a reference is destroyed before the referent, the referent is not impacted. The following program demonstrates this:

```
3  int main()
4  {
5      int x { 5 };
6
7      {
8          int& ref { x }; // ref is a reference to x
9          std::cout << ref << '\n'; // prints value of ref (5)
10     } // ref is destroyed here -- x is unaware of this
11
12     std::cout << x << '\n'; // prints value of x (5)
13
14     return 0;
15 } // x destroyed here
```

15.3.8 Dangling references

When an **object being referenced is destroyed before a reference to it, the reference is left referencing an object that no longer exists**. Such a reference is called a **dangling reference**. **Accessing a dangling reference leads to undefined behavior**.

Dangling references are fairly easy to avoid, but we'll show a case where this can happen in practice in lesson: Return by reference and return by address.

15.3.9 References aren't objects

Perhaps surprisingly, **references are not objects in C++**. A reference is not required to exist or occupy storage. If possible, the **compiler will optimize references away by replacing all occurrences of a reference with the referent**. However, **this isn't always possible, and in such cases, references may require storage**.

This also means that the term "reference variable" is a bit of a misnomer, as variables are objects with a name, and **references aren't objects**.

Because references aren't objects, they can't be used anywhere an object is required (e.g. you can't have a reference to a reference, since an lvalue reference must reference an identifiable object). In cases where you need a reference that is an object or a reference that can be reseated, `std::reference_wrapper` provides a solution.

As an aside...

Consider the following variables:

```
1 int var{};
2 int& ref1{ var }; // an lvalue reference bound to var
3 int& ref2{ ref1 }; // an lvalue reference bound to var
```

Because `ref2` (a reference) is initialized with `ref1` (a reference), **you might be tempted to conclude that `ref2` is a reference to a reference**. It is not. Because `ref1` is a reference to `var`, when used in an expression (such as an initializer), **`ref1` evaluates to `var`**. So `ref2` is just a normal **lvalue reference** (as indicated by its type `int&`), bound to `var`.

A reference to a reference (to an `int`) would have syntax `int&&` -- **but since C++ doesn't support references to references, this syntax was repurposed in C++11 to indicate an rvalue reference**.

15.4 Lvalue references to const

In the previous lesson (Lvalue references), we discussed how an lvalue reference can only bind to a modifiable lvalue. This means the following is illegal:

```
const int x { 5 }; // x is a non-modifiable (const) lvalue
int& ref { x }; // error: ref can not bind to non-modifiable lvalue
```

This is disallowed because it would allow us to modify a const variable (x) through the non-const reference (ref). But what if we want to have a const variable we want to create a reference to? A normal lvalue reference (to a non-const value) won't do.

15.4.1 Lvalue reference to const

By using the const keyword when declaring an lvalue reference, we tell an lvalue reference to treat the object it is referencing as const. Such a reference is called an **lvalue reference to a const value** (sometimes called a **reference to const** or a **const reference**).

Lvalue references to const can bind to non-modifiable lvalues:

```
const int x { 5 }; // x is a non-modifiable lvalue
const int& ref { x }; // okay: ref is a an lvalue reference to a const value
```

Because lvalue references to const treat the object they are referencing as const, they can be used to access but not modify the value being referenced:

```
const int x { 5 }; // x is a non-modifiable lvalue
const int& ref { x }; // okay: ref is a an lvalue reference to a const value

std::cout << ref << '\n'; // okay: we can access the const object
ref = 6; // error: we can not modify an object through a const reference
```


15.4.2 *Initializing an lvalue reference to const with a modifiable lvalue*

Lvalue references to const can also bind to modifiable lvalues. In such a case, the object being referenced is treated as const when accessed through the reference (even though the underlying object is non-const):

```
int x { 5 };           // x is a modifiable lvalue
const int& ref { x };  // okay: we can bind a const reference to a modifiable lvalue

std::cout << ref << '\n'; // okay: we can access the object through our const reference
ref = 7;                // error: we can not modify an object through a const reference

x = 6;                 // okay: x is a modifiable lvalue, we can still modify it through the original identifier
```

In the above program, we bind const reference ref to modifiable lvalue x. We can then use ref to access x, but because ref is const, we cannot modify the value of x through ref. However, we still can modify the value of x directly (using the identifier x).

Best practice: Favor lvalue references to const over lvalue references to non-const unless you need to modify the object being referenced.

15.4.3 *Initializing an lvalue reference to const with an rvalue*

Perhaps surprisingly, lvalues references to const can also bind to rvalues:

```
const int& ref { 5 }; // okay: 5 is an rvalue

std::cout << ref << '\n'; // prints 5
```

When this happens, a temporary object is created and initialized with the rvalue, and the reference to const is bound to that temporary object.

15.4.4 *Initializing an lvalue reference to const with a value of a different type*

Lvalue references to const can even bind to values of a different type, so long as those values can be implicitly converted to the reference type:

```
// case 1
const double& r1 { 5 }; // temporary double initialized with value 5, r1 binds to temporary

std::cout << r1 << '\n'; // prints 5

// case 2
char c { 'a' };
const int& r2 { c }; // temporary int initialized with value 'a', r2 binds to temporary

std::cout << r2 << '\n'; // prints 97 (since r2 is a reference to int)
```

In both cases, the **type of the reference and the type of the temporary match**.

Key insight: If you try to bind a const lvalue reference to a value of a different type, the **compiler will create a temporary object of the same type as the reference, initialize it using the value, and then bind the reference to the temporary**.

Also note that when we print r2 it prints as an int rather than a char. This is because r2 is a reference to an int object (the temporary int that was created), not to char c.

Although it may seem strange to allow this, we'll see examples where this is useful in lesson 12.6: Pass by const lvalue reference.

15.4.5 *Const references bound to temporary objects extend the lifetime of the temporary object*

Temporary objects are normally destroyed at the end of the expression in which they are created.

However, consider what would happen in the above example if the temporary object created to hold value 5 was destroyed at the end of the expression that initializes ref. **Reference ref would be left dangling** (referencing an object that had been destroyed), and **we'd get undefined behavior** when we tried to access ref.

To avoid dangling references in such cases, C++ has a special rule: When a const lvalue reference is directly bound to a temporary object, the lifetime of the temporary object is extended to match the lifetime of the reference.

```
const int& ref { 5 }; // The temporary object holding value 5 has its lifetime extended to match ref
std::cout << ref << '\n'; // Therefore, we can safely use it here
```

In the above example, when `ref` is initialized with rvalue 5, a temporary object is created and `ref` is bound to that temporary object. The lifetime of the temporary object matches the lifetime of `ref`. Thus, we can safely print the value of `ref` in the next statement. Then both `ref` and the temporary object go out of scope and are destroyed at the end of the block.

Key insight: Lvalue references can only bind to modifiable lvalues. Lvalue references to `const` can bind to modifiable lvalues, non-modifiable lvalues, and rvalues. This makes them a much more flexible type of reference.

For advanced readers: Lifetime extension only works when a `const` reference is directly bound to a temporary. Temporaries returned from a function (even ones returned by `const` reference) are not eligible for lifetime extension.

15.4.6 *constexpr lvalue references: optional*

When applied to a reference, **constexpr** allows the reference to be used in a constant expression. **constexpr** references have a particular limitation: they can only be bound to objects with static duration (either globals or static locals). This is because the compiler knows where static objects will be instantiated in memory, so it can treat that address as a compile-time constant.

A **constexpr** reference cannot bind to a (non-static) local variable. This is because the address of local variables is not known until the function they are defined within is actually called.

```
1  int g_x { 5 };
2
3  int main()
4  {
5      [[maybe_unused]] constexpr int& ref1 { g_x }; // ok, can bind to global
6
7      static int s_x { 6 };
8      [[maybe_unused]] constexpr int& ref2 { s_x }; // ok, can bind to static local
9
10     int x { 6 };
11     [[maybe_unused]] constexpr int& ref3 { x }; // compile error: can't bind to non-static object
12
13     return 0;
14 }
```

When defining a constexpr reference to a const variable, we need to apply both constexpr (which applies to the reference) and const (which applies to the type being referenced).

```
static const int s_x { 6 }; // a const int
[[maybe_unused]] constexpr const int& ref2 { s_x }; // needs both constexpr and const
```

Given these limitations, constexpr references typically don't see much use.

15.5 Pass by lvalue reference

In the previous lessons, we introduced lvalue references and lvalue references to const. In isolation, these may not have seemed very useful -- **why create an alias to a variable when you can just use the variable itself?**

In this lesson, we'll finally provide some insight into what makes references useful. And then starting later in this chapter, you'll see references used regularly.

First, some context. Back in lesson: Introduction to function parameters and arguments we discussed pass by value, where an argument passed to a function is copied into the function's parameter:

```
3 void printValue(int y)
4 {
5     std::cout << y << '\n';
6 } // y is destroyed here
7
8 int main()
9 {
10     int x { 2 };
11
12     printValue(x); // x is passed by value (copied) into parameter y (inexpensive)
13
14     return 0;
15 }
```

In the above program, when printValue(x) is called, the value of x (2) is *copied* into parameter y. Then, at the end of the function, object y is destroyed.

This means that when we called the function, we made a copy of our argument's value, only to use it briefly and then destroy it! Fortunately, because fundamental types are cheap to copy, this isn't a problem.

15.5.1 Some objects are expensive to copy

Most of the types provided by the standard library (such as `std::string`) are class types. Class types are **usually expensive to copy**. Whenever possible, we want to avoid making unnecessary copies of objects that are expensive to copy, **especially when we will destroy those copies almost immediately**.

Consider the following program illustrating this point:

```
4 void printValue(std::string y)
5 {
6     std::cout << y << '\n';
7 } // y is destroyed here
8
9 int main()
10 {
11     std::string x { "Hello, world!" }; // x is a std::string
12
13     printValue(x); // x is passed by value (copied) into parameter y (expensive)
14
15     return 0;
16 }
```

While this program behaves like we expect, it's also **inefficient**. Identically to the prior example, when `printValue()` is called, argument `x` is copied into `printValue()` parameter `y`. However, in this example, the argument is a `std::string` instead of an `int`, and `std::string` is a class type that is expensive to copy. And **this expensive copy is made every time `printValue()` is called!**

We can do better.

15.5.2 Pass by reference

One way to avoid making an expensive copy of an argument when calling a function is to **use pass by reference instead of pass by value**. When using pass by reference, we **declare a function parameter as a reference type (or `const` reference type)** rather than as a normal type. When the function is called, each reference parameter is bound to the appropriate argument. **Because the reference acts as an alias for the argument, no copy of the argument is made.**

```
4 void printValue(std::string& y) // type changed to std::string&
5 {
6     std::cout << y << '\n';
7 } // y is destroyed here
8
9 int main()
10 {
11     std::string x { "Hello, world!" };
12
13     printValue(x); // x is now passed by reference into reference parameter y (inexpensive)
14
15     return 0;
16 }
```

This program is identical to the prior one, except the type of parameter `y` has been changed from `std::string` to `std::string&` (an lvalue reference). Now, when `printValue(x)` is called, lvalue reference parameter `y` is bound to argument `x`. Binding a reference is always inexpensive, and no copy of `x` needs to be made. Because a reference acts as an alias for the object being referenced, when `printValue()` uses reference `y`, it's accessing the actual argument `x` (rather than a copy of `x`).

Key insight: Pass by reference **allows us to pass arguments to a function without making copies** of those arguments **each time the function is called**.

15.5.3 Pass by reference allows us to change the value of an argument

When an object is passed by value, the function parameter receives a copy of the argument. **This means that any changes to the value of the parameter are made to the copy of the argument, not the argument itself:**

```
3 void addOne(int y) // y is a copy of x
4 {
5     ++y; // this modifies the copy of x, not the actual object x
6 }
7
8 int main()
9 {
10     int x { 5 };
11
12     std::cout << "value = " << x << '\n';
13
14     addOne(x);
15
16     std::cout << "value = " << x << '\n'; // x has not been modified
17
18     return 0;
19 }
```

In the above program, because value parameter `y` is a copy of `x`, when we increment `y`, this only affects `y`. This program outputs:

value = 5

value = 5

However, since a reference acts identically to the object being referenced, **when using pass by reference, any changes made to the reference parameter *will* affect the argument:**

```
3 void addOne(int& y) // y is bound to the actual object x
4 {
5     ++y; // this modifies the actual object x
6 }
7
8 int main()
9 {
10     int x { 5 };
11
12     std::cout << "value = " << x << '\n';
13
14     addOne(x);
15
16     std::cout << "value = " << x << '\n'; // x has been modified
17
18     return 0;
19 }
```

This program outputs:

value = 5

value = 6

Key insight: Passing values by reference to non-const **allows us to write functions that modify the value of arguments** passed in.

The ability for functions to modify the value of arguments passed in can be useful. Imagine you've written a function that determines whether a monster has successfully attacked the player. If so, the monster should do some amount of damage to the player's health. If you pass your player object by reference, the function can directly modify the health of the actual player object that was passed in. If you pass the player object by value, you could only modify the health of a copy of the player object, which isn't as useful.

15.5.4 *Pass by reference can only accept modifiable lvalue arguments*

Because a reference to a non-const value can only bind to a modifiable lvalue (essentially a non-const variable), this means that **pass by reference only works with arguments that are modifiable lvalues**. In practical terms, **this significantly limits the usefulness of pass by reference to non-const**, as it means **we cannot pass const variables or literals**.

Fortunately, there's an easy way around this, which we will discuss next lesson. We'll also take a look at when to pass by value vs. pass by reference.

15.6 Pass by const lvalue reference

Unlike a reference to non-const (which can only bind to modifiable lvalues), **a reference to const can bind to modifiable lvalues, non-modifiable lvalues, and rvalues**. Therefore, if we make a reference parameter const, then it will be able to bind to any type of argument:

```
2
3 void printRef(const int& y) // y is a const reference
4 {
5     std::cout << y << '\n';
6 }
7
8 int main()
9 {
10     int x { 5 };
11     printRef(x); // ok: x is a modifiable lvalue, y binds to x
12
13     const int z { 5 };
14     printRef(z); // ok: z is a non-modifiable lvalue, y binds to z
15
16     printRef(5); // ok: 5 is rvalue literal, y binds to temporary int object
17
18     return 0;
19 }
```

Passing by const reference offers the same primary benefit as pass by reference (avoiding making a copy of the argument), while also guaranteeing that the function cannot change the value being referenced.

For example, the following is **disallowed**, because ref is const:

```
1 void addOne(const int& ref)
2 {
3     ++ref; // not allowed: ref is const
4 }
```

In most cases, we don't want our functions modifying the value of arguments.

Best practice: Favor passing by const reference over passing by non-const reference unless you have a specific reason to do otherwise (e.g. the function needs to change the value of an argument).

Now we can understand the motivation for allowing const lvalue references to bind to rvalues: **without that capability, there would be no way to pass literals (or other rvalues) to functions that used pass by reference!**

15.6.1 Passing values of a different type to a const lvalue reference parameter

In lesson: Lvalue references to const, we noted that a const lvalue reference can bind to a value of a different type, as long as that value is convertible to the type of the reference. The primary motivation for allowing this is so that we can pass a value as an argument to either a value parameter or a const reference parameter in exactly the same way:

```
2
3 void printVal(double d)
4 {
5     std::cout << d << '\n';
6 }
7
8 void printRef(const double& d)
9 {
10    std::cout << d << '\n';
11 }
12
13 int main()
14 {
15     printVal(5); // 5 converted to temporary double, copied to parameter d
16     printRef(5); // 5 converted to temporary double, bound to parameter d
17
18     return 0;
19 }
```

15.6.2 Mixing pass by value and pass by reference

A function with multiple parameters can determine whether each parameter is passed by value or passed by reference individually.

15.6.3 When to pass by (const) reference

Because class types can be expensive to copy (sometimes significantly so), class types are usually passed by const reference instead of by value to **avoid making an expensive copy** of the argument. Fundamental types are cheap to copy, so they are typically passed by value.

Best practice

As a rule of thumb, **pass fundamental types by value**, and **class (or struct) types by const reference**.

Other common types to pass by value: enumerated types and `std::string_view`.

Other common types to pass by (const) reference: `std::string`, `std::array`, and `std::vector`.

15.6.4 *The cost of pass by value vs pass by reference: Advanced*

Not all class types need to be passed by reference. And you may be wondering why we don't just pass everything by reference. In this section (which is optional reading), we discuss **the cost of pass by value vs pass by reference**, and **refine our best practice as to when we should use each**.

There are **two key points** that will help us understand when we should pass by value vs pass by reference:

- The **size of the object**. Objects that use more memory take more time to copy.
- Any **additional setup costs**. Some class types do additional setup when they are instantiated (e.g. such as **opening a file or database, or allocating a certain amount of dynamic memory to hold an object of a variable size**). These **setup costs must be paid each time an object is copied**.

On the other hand, **binding a reference to an object is always fast** (about the **same speed as copying a fundamental type**).

Second, **accessing an object through a reference is slightly more expensive than accessing an object through a normal variable identifier**. With a variable identifier, the running program can just go to the memory address assigned to that variable and access the value directly. **With a reference, there usually is an extra step: the program must first access the reference to determine which object is being referenced, and only then can it go to that memory address for that object and access the value**. The compiler can also sometimes optimize code using objects passed by value more highly than code using objects passed by reference. This means **code generated to access objects passed by reference is typically slower than the code generated for objects passed by value**.

We can now answer the question of why we don't pass everything by reference:

- For objects that are cheap to copy, the **cost of copying is similar to the cost of binding**, so we favor pass by value so the code generated will be faster.
- For objects that are expensive to copy, the **cost of the copy dominates**, so we favor pass by (const) reference to avoid making a copy.

Best practice: Prefer pass by value for objects that are cheap to copy, and pass by const reference for objects that are expensive to copy. If you're not sure whether an object is cheap or expensive to copy, favor pass by const reference.

The last question then is, how do we define “cheap to copy”? There is no absolute answer here, as this varies by compiler, use case, and architecture. However, we can formulate a **good rule of thumb: An object is cheap to copy if it uses 2 or fewer “words” of memory (where a “word” is approximated by the size of a memory address on that architecture) and it has no setup costs. In most cases this is 8 bytes.**

The following program defines a function-like macro that can be used to determine if a type (or object) is cheap to copy accordingly:

```

2 // Function-like macro that evaluates to true if the type (or object) is equal to or smaller than
3 // the size of two memory addresses
4 #define isSmall(T) (sizeof(T) <= 2 * sizeof(void*))
5
6
7 struct S
8 {
9     double a;
10    double b;
11    double c;
12 };
13
14 int main()
15 {
16     std::cout << std::boolalpha; // print true or false rather than 1 or 0
17     std::cout << isSmall(int) << '\n'; // true
18
19     double d {};
20     std::cout << isSmall(d) << '\n'; // true
21     std::cout << isSmall(S) << '\n'; // false
22
23     return 0;
24 }

```

As an aside... We use a preprocessor function-like macro here so that we can provide either an object OR a type name as a parameter (as C++ functions disallow passing types as a parameter).

However, it **can be hard to know whether a class type object has setup costs or not**. It's **best to assume that most standard library classes have setup costs**, unless you know otherwise that they don't.

Tip: An object of type T is cheap to copy if `sizeof(T) <= 2 * sizeof(void*)` and has no additional setup costs.

15.6.5 For function parameters, prefer `std::string_view` over `const std::string&` in most cases

One question that comes up often in modern C++: when writing a function that has a string parameter, should the type of the parameter be `const std::string&` or `std::string_view`?

In most cases, `std::string_view` is the better choice, as it can handle a wider range of argument types efficiently.

There are a few cases where using a `const std::string&` parameter may be more appropriate:

- If **you're using C++14 or older**, `std::string_view` isn't available.
- If **your function needs to call some other function that takes a C-style string or `std::string` parameter, then `const std::string&` may be a better choice**, as `std::string_view` is not guaranteed to be null-terminated (something that C-style string functions expect) and does not efficiently convert back to a `std::string`.

Best practice: Prefer passing strings using `std::string_view` (by value) instead of `const std::string&`, unless your function calls other functions that require C-style strings or `std::string` parameters.

15.7 Introduction to pointers

Pointers are one of **C++'s historical boogeymen**, and a place where many aspiring C++ learners have gotten stuck. However, as you'll see shortly, **pointers are nothing to be scared of**.

In fact, **pointers behave a lot like lvalue references**. But before we explain that further, let's do some setup.

If you're rusty or not familiar with lvalue references, now would be a good time to review them.

Consider a normal variable, like this one:

```
1 | char x {}; // chars use 1 byte of memory
```

Simplifying a bit, when the code generated for this definition is executed, **a piece of memory from RAM will be assigned to this object**. For the sake of **example, let's say that the variable x is assigned memory address 140**. Whenever we use variable x in an expression or statement, **the program will go to memory address 140 to access the value stored there**.

The nice thing about variables is that we don't need to worry about what specific memory addresses are assigned, or how many bytes are required to store the object's value. We just refer to the variable by its given identifier, and **the compiler translates this name into the appropriately assigned memory address. The compiler takes care of all the addressing**.

This is also true with references:

```
char x {}; // assume this is assigned memory address 140  
char& ref { x }; // ref is an lvalue reference to x (when used with a type, & means lvalue reference)
```

Because ref acts as an alias for x, **whenever we use ref, the program will go to memory address 140 to access the value**. Again, the **compiler takes care of the addressing**, so that we don't have to think about it.

15.7.1 The address-of operator (&)

Although the memory addresses used by variables aren't exposed to us by default, we do have access to this information. The **address-of operator (&)** returns the **memory address** of its operand. This is pretty straightforward:

```
int x{ 5 };  
std::cout << x << '\n'; // print the value of variable x  
std::cout << &x << '\n'; // print the memory address of variable x
```

On the author's machine, the above program printed:

5

0027FEA0

In the above example, we use the address-of operator (&) to retrieve the address assigned to variable x and print that address to the console. **Memory addresses are typically printed as hexadecimal values** (we covered hex in lesson: Numeral systems (decimal, binary, hexadecimal, and octal)), **often without the 0x prefix**.

For objects that use more than one byte of memory, address-of will return the memory address of the first byte used by the object.

Tip: The **& symbol** tends to cause confusion because it **has different meanings depending on context**:

- When following a **type name**, & denotes an **lvalue reference**: `int& ref`.
- When used in a **unary context** in an expression, & is the **address-of operator**: `std::cout << &x`.
- When used in a **binary context** in an expression, & is the **Bitwise AND operator**: `std::cout << x & y`.

15.7.2 The dereference operator (*)

The **most useful thing we can do with an address is access the value stored at that address**. The **dereference operator (*)** (also occasionally called the **indirection operator**) returns the value at a given memory address as an lvalue:

```
int x{ 5 };
std::cout << x << '\n'; // print the value of variable x
std::cout << &x << '\n'; // print the memory address of variable x

std::cout << *(&x) << '\n'; // print the value at the memory address of variable x
// (parentheses not required, but make it easier to read)
```

On the author's machine, the above program printed:

5

0027FEA0

5

This program is pretty simple. **First we declare a variable x and print its value. Then we print the address of variable x. Finally, we use the dereference operator to get the value at the memory address of variable x (which is just the value of x), which we print to the console.**

Key insight:

Given a memory address, we can use the dereference operator (*) to get the value at that address (as an lvalue).

The address-of operator (&) and dereference operator (*) work as opposites:

- **address-of (&)** gets the address of an object
- **dereference (*)** gets the object at an address.

Tip: Although the dereference operator looks just like the multiplication operator, you can distinguish them because the **dereference operator is unary**, whereas the **multiplication operator is binary**.

Getting the memory address of a variable and then immediately dereferencing that address to get a value isn't that useful either (after all, we can just use the variable to access the value).

But **now that we have the address-of operator (&) and dereference operator (*) added to our toolkits, we're ready to talk about pointers**.

15.7.3 Pointers

A **pointer** is an **object that holds a *memory address*** (typically of another variable) as its value. This **allows us to store the address of some other object to use later**.

As an aside...

In modern C++, the pointers we are talking about here are sometimes called “**raw pointers**” or “**dumb pointers**”, to help **differentiate them from “smart pointers”** that were introduced into the language more recently.

Much like reference types are declared using an ampersand (&) character, pointer types are declared using an asterisk (*):

```
int; // a normal int
int& // an lvalue reference to an int value

int*; // a pointer to an int value (holds the address of an integer value)
```

Note that this **asterisk is part of the declaration syntax for pointers, not a use of the dereference operator**.

Best practice: When declaring a pointer type, place the asterisk next to the type name.

Warning: Although **you generally should not declare multiple variables on a single line**, if you do, the asterisk has to be included with each variable.

```
1 int* ptr1, ptr2; // incorrect: ptr1 is a pointer to an int, but ptr2 is just a plain int!
2 int* ptr3, * ptr4; // correct: ptr3 and ptr4 are both pointers to an int
```

Although this is sometimes used as an argument to not place the asterisk with the type name (instead placing it next to the variable name), it's a better argument for avoiding defining multiple variables in the same statement.

15.7.4 Pointer initialization

Like normal variables, **pointers are *not* initialized by default**. A pointer that has not been initialized is sometimes called a **wild pointer**. Wild pointers contain a **garbage address**, and dereferencing a wild pointer will result in undefined behavior. Because of this, you should always initialize your pointers to a known value.

Best practice: Always initialize your pointers.

```
int* ptr;           // an uninitialized pointer (holds a garbage address)
int* ptr2{};        // a null pointer (we'll discuss these in the next lesson)
int* ptr3{ &x };    // a pointer initialized with the address of variable x
```

Since pointers hold addresses, when we initialize or assign a value to a pointer, **that value has to be an address**. Typically, pointers are used to hold the address of another variable (which we can get **using the address-of operator (&)**).

Once we have a pointer holding the address of another object, we can then **use the dereference operator (*) to access the value at that address**. For example:

```
int x{ 5 };
std::cout << x << '\n'; // print the value of variable x

int* ptr{ &x }; // ptr holds the address of x
std::cout << *ptr << '\n'; // use dereference operator to print the value at the address that ptr is holding (which is x's address)
```

This prints:

5

5

Conceptually, you can think of the above snippet like this:



This is where pointers get their name from -- ptr is holding the address of x, so we say that ptr is “pointing to” x.

Author’s note: A note on pointer nomenclature: “X pointer” (where X is some type) is a commonly used shorthand for “pointer to an X”. **So, when we say, “an integer pointer”, we really mean “a pointer to an integer”.** This understanding will be valuable when we talk about const pointers.

Much like the type of a reference has to match the type of object being referred to, the **type of the pointer has to match the type of the object being pointed to.**

With one exception that we’ll discuss next lesson, **initializing a pointer with a literal value is disallowed:**

```
1 int* ptr{ 5 }; // not okay
2 int* ptr{ 0x0012FF7C }; // not okay, 0x0012FF7C is treated as an integer literal
```

15.7.5 Pointers and assignment

We can use assignment with pointers in two different ways:

- To change what the pointer is pointing at (by **assigning the pointer a new address**)
- To change the value being pointed at (by **assigning the dereferenced pointer a new value**)

First, let's look at a case where a **pointer is changed to point at a different object**:

```
int x{ 5 };
int* ptr{ &x }; // ptr initialized to point at x

std::cout << *ptr << '\n'; // print the value at the address being pointed to (x's address)

int y{ 6 };
ptr = &y; // // change ptr to point at y

std::cout << *ptr << '\n'; // print the value at the address being pointed to (y's address)
```

The above prints:

5

6

In the above example, we define pointer `ptr`, initialize it with the address of `x`, and dereference the pointer to print the value being pointed to (5). We then use the assignment operator to change the address that `ptr` is holding to the address of `y`. We then dereference the pointer again to print the value being pointed to (which is now 6).

Now let's look at how we can also use a **pointer to change the value being pointed at**:

```
int x{ 5 };
int* ptr{ &x }; // initialize ptr with address of variable x

std::cout << x << '\n'; // print x's value
std::cout << *ptr << '\n'; // print the value at the address that ptr is holding (x's address)

*ptr = 6; // The object at the address held by ptr (x) assigned value 6 (note that ptr is dereferenced here)

std::cout << x << '\n';
std::cout << *ptr << '\n'; // print the value at the address that ptr is holding (x's address)
```

This program prints:

5

5

6

6

In this example, we define pointer `ptr`, initialize it with the address of `x`, and then print the value of both `x` and `*ptr` (5). Because `*ptr` returns an lvalue, we can use this on the left-hand side of an assignment statement, which we do to change the value being pointed at by `ptr` to 6. We then print the value of both `x` and `*ptr` again to show that the value has been updated as expected.

Key insight

When we use a pointer **without a dereference** (`ptr`), **we are accessing the address held** by the pointer. Modifying this (`ptr = &y`) **changes what the pointer is pointing at**.

When we dereference a pointer (`*ptr`), **we are accessing the object being pointed at**. Modifying this (`*ptr = 6;`) **changes the value of the object being pointed at**.

15.7.6 Pointers behave much like lvalue references

Pointers and lvalue references behave similarly. Consider the following program:

```
int x{ 5 };
int& ref { x }; // get a reference to x
int* ptr { &x }; // get a pointer to x

std::cout << x;
std::cout << ref; // use the reference to print x's value (5)
std::cout << *ptr << '\n'; // use the pointer to print x's value (5)

ref = 6; // use the reference to change the value of x
std::cout << x;
std::cout << ref; // use the reference to print x's value (6)
std::cout << *ptr << '\n'; // use the pointer to print x's value (6)

*ptr = 7; // use the pointer to change the value of x
std::cout << x;
std::cout << ref; // use the reference to print x's value (7)
std::cout << *ptr << '\n'; // use the pointer to print x's value (7)
```

This program prints:

555

666

777

In the above program, we create a normal variable `x` with value 5, and then create an lvalue reference and a pointer to `x`. Next, we use the lvalue reference to change the value from 5 to 6, and show that we can access that updated value via all three methods. Finally, we use the dereferenced pointer to change the value from 6 to 7, and again show that we can access the updated value via all three methods.

Thus, **pointers and references both provide a way to indirectly access another object**. The **primary difference** is that with pointers, we need to explicitly get the address to point at, and we have to explicitly dereference the pointer to get the value. With references, the address-of and dereference happens implicitly.

There are some other differences between pointers and references worth mentioning:

- **References must be initialized**, pointers are not required to be initialized (but should be).
- **References are not objects**, pointers are.
- **References cannot be reseated** (changed to reference something else), pointers can change what they are pointing at.
- **References must always be bound to an object**, pointers can point to nothing (we'll see an example of this in the next lesson).
- **References are "safe" (outside of dangling references), pointers are inherently dangerous** (we'll also discuss this in the next lesson).

15.7.7 The address-of operator returns a pointer

It's worth noting that the **address-of operator (&)** doesn't return the address of its operand as a literal. Instead, it returns a pointer containing the address of the operand, whose type is derived from the argument (e.g. taking the address of an int will return the address in an int pointer).

We can see this in the following example:

```
int x{ 4 };  
std::cout << typeid(&x).name() << '\n'; // print the type of &x
```

On Visual Studio, this printed:

int *

With gcc, this prints "pi" (pointer to int) instead. Because the result of `typeid().name()` is compiler-dependent, your compiler may print something different, but it will have the same meaning.

15.7.8 The size of pointers

The size of a pointer is **dependent upon the architecture** the executable is compiled for a **32-bit executable uses 32-bit memory addresses** consequently, a **pointer on a 32-bit machine is 32 bits** (4 bytes). With a **64-bit executable, a pointer would be 64 bits** (8 bytes). Note that this is true **regardless of the size of the object being pointed to**:

```
char* chPtr{};           // chars are 1 byte
int* iPtr{};             // ints are usually 4 bytes
long double* ldPtr{};    // long doubles are usually 8 or 12 bytes

std::cout << sizeof(chPtr) << '\n'; // prints 4
std::cout << sizeof(iPtr) << '\n';  // prints 4
std::cout << sizeof(ldPtr) << '\n'; // prints 4
```

The **size of the pointer** is always the same. This is because a **pointer is just a memory address**, and the number of bits needed to access a memory address is constant.

Dangling pointers

Much like a dangling reference, a **dangling pointer** is a pointer that is **holding the address of an object that is no longer valid** (e.g. because it has been destroyed).

Dereferencing a dangling pointer (e.g. in order to print the value being pointed at) **will lead to undefined behavior**, as you are trying to access an object that is no longer valid.

Perhaps surprisingly, the standard says “**Any other use of an invalid pointer value has implementation-defined behavior**”. This means that you can assign an invalid pointer a new value, such as `nullptr` (because this doesn’t use the invalid pointer’s value). However, any **other operations** that use the invalid pointer’s value (such as copying or incrementing an invalid pointer) will **yield undefined behavior**.

Key insight: Dereferencing an invalid pointer will lead to undefined behavior. Any other use of an invalid pointer value is implementation-defined.

```
{
    int y{ 6 };
    ptr = &y;

    std::cout << *ptr << '\n'; // valid
} // y goes out of scope, and ptr is now dangling

std::cout << *ptr << '\n'; // undefined behavior from dereferencing a dangling pointer
```

15.7.9 Conclusion

Pointers are variables that hold a memory address. They can be dereferenced using the dereference operator (*) to retrieve the value at the address they are holding. **Dereferencing a wild or dangling (or null) pointer will result in undefined behavior and will probably crash your application.**

Pointers are both more flexible than references and more dangerous. We'll continue to explore this in the upcoming lessons.

15.8 Null pointers

In the previous lesson (Introduction to pointers), we covered the basics of pointers, which are objects that hold the address of another object. This address can be dereferenced using the dereference operator (*) to get the object at that address:

```
int* ptr{ &x }; // ptr holds the address of x
std::cout << *ptr << '\n'; // use dereference operator to print the value of the object
```

In the prior lesson, we also noted that **pointers do not need to point to anything**. In this lesson, we'll explore such pointers (and the various implications of pointing to nothing) further.

15.8.1 Null pointers

Besides a memory address, there is **one additional value that a pointer can hold**: a **null** value. A **null value** (often shortened to **null**) is a special value that **means something has no value**. When a pointer is holding a null value, it means the pointer is not pointing at anything. Such a pointer is called a **null pointer**.

The easiest way to create a null pointer is to use **value initialization**:

```
int* ptr {}; // ptr is now a null pointer, and is not holding an address
```

Best practice: Value initialize your pointers (to be null pointers) if you are not initializing them with the address of a valid object.

Because we can use assignment to change what a pointer is pointing at, a **pointer that is initially set to null can later be changed to point at a valid object**.

15.8.2 The `nullptr` keyword

Much like the keywords `true` and `false` represent Boolean literal values, the **`nullptr` keyword represents a null pointer literal**. We can use `nullptr` to **explicitly initialize or assign a pointer a null value**.

```
int* ptr { nullptr }; // can use nullptr to initialize a pointer to be a null pointer
```

Best practice: Use `nullptr` when you need a null pointer literal for initialization, assignment, or passing a null pointer to a function.

15.8.3 Dereferencing a null pointer results in undefined behavior

Much like dereferencing a dangling (or wild) pointer leads to undefined behavior, **dereferencing a null pointer also leads to undefined behavior**. In most cases, it will crash your application.

The following program illustrates this, and **will probably crash or terminate your application abnormally** when you run it (go ahead, try it, you won't harm your machine):

```
int main()
{
    int* ptr {}; // Create a null pointer
    std::cout << *ptr << '\n'; // Dereference the null pointer

    return 0;
}
```

Conceptually, this makes sense. Dereferencing a pointer means “go to the address the pointer is pointing at and access the value there”. A null pointer holds a null value, which semantically means the pointer is not pointing at anything. So, what value would it access?

Accidentally dereferencing null and dangling pointers is one of the most common mistakes C++ programmers make, and is probably the most common reason that C++ programs crash in practice.

Warning: Whenever you are using pointers, you'll need to be extra careful that your code isn't dereferencing null or dangling pointers, as this will cause undefined behavior (probably an application crash)

15.8.4 Checking for null pointers

Much like we can use a conditional to test Boolean values for true or false, we can use a **conditional to test whether a pointer has value nullptr or not**:

```
if (ptr == nullptr) // explicit test for equivalence
    std::cout << "ptr is null\n";
else
    std::cout << "ptr is non-null\n";
```

In lesson: Boolean values, we noted that integral values will implicitly convert into Boolean values: an integral value of 0 converts to Boolean value false, and any other integral value converts to Boolean value true.

Similarly, **pointers will also implicitly convert to Boolean values**: a null pointer converts to Boolean value false, and a non-null pointer converts to Boolean value true. This allows us to skip explicitly testing for nullptr and just use the implicit conversion to Boolean to test whether a pointer is a null pointer. The **following program is equivalent to the prior one**:

```
// pointers convert to Boolean false if they are null, and Boolean true if they are non-null
if (ptr) // implicit conversion to Boolean
    std::cout << "ptr is non-null\n";
else
    std::cout << "ptr is null\n";
```

Warning: Conditionals can only be used to differentiate null pointers from non-null pointers. **There is no convenient way to determine whether a non-null pointer is pointing to a valid object or dangling (pointing to an invalid object).**

15.8.5 Use `nullptr` to avoid dangling pointers

Above, we mentioned that dereferencing a pointer that is either null or dangling will result in undefined behavior. Therefore, we need to ensure our code does not do either of these things.

We can **easily avoid dereferencing a null pointer** by using a conditional to ensure a pointer is non-null before trying to dereference it:

```
// Assume ptr is some pointer that may or may not be a null pointer
if (ptr) // if ptr is not a null pointer
    std::cout << *ptr << '\n'; // okay to dereference
```

But what about dangling pointers? Because there is no way to detect whether a pointer is dangling, **we need to avoid having any dangling pointers** in our program in the first place. We do that by ensuring that any pointer that is not pointing at a valid object is set to `nullptr`.

That way, before dereferencing a pointer, we only need to test whether it is null -- **if it is non-null, we assume the pointer is not dangling**.

Best practice: A pointer **should either hold the address of a valid object, or be set to `nullptr`**. That way we only need to test pointers for null, and can assume any non-null pointer is valid.

Warning: Unfortunately, avoiding dangling pointers isn't always easy: when an object is destroyed, any pointers to that object will be left dangling. Such pointers are *not* nulled automatically! **It is the programmer's responsibility to ensure that all pointers to an object that has just been destroyed are properly set to `nullptr`**.

15.8.6 Legacy null pointer literals: `0` and `NULL`

In older code, you may see two other literal values used instead of `nullptr`.

The first is the literal `0`. In the context of a pointer, the literal `0` is **specially defined to mean a null value**, and is the only time you can assign an integral literal to a pointer.

Additionally, there is a **preprocessor macro named `NULL`** (defined in the `<cstdlib>` header). This macro is inherited from C, where it is commonly used to indicate a null pointer.

```
float* ptr { 0 }; // ptr is now a null pointer (for example only, don't do this)
```

```
double* ptr { NULL }; // ptr is a null pointer
```

Both `0` and `NULL` **should be avoided in modern C++** (use `nullptr` instead).

15.8.7 Favor references over pointers whenever possible

Pointers and references both give us the ability to access some other object indirectly.

Pointers have the additional abilities of being able to change what they are pointing at, and to be pointed at null. However, these **pointer abilities are also inherently dangerous**: A null pointer runs the **risk of being dereferenced**, and the ability to change what a pointer is pointing at can make creating dangling pointers easier.

Since references can't be bound to null, we don't have to worry about null references. And **because references must be bound to a valid object upon creation and then cannot be reseated, dangling references are harder to create.**

Because they are safer, **references should be favored over pointers**, unless the additional capabilities provided by pointers are required.

Best practice: Favor references over pointers unless the additional capabilities provided by pointers are needed.

15.9 Pointers and const

```
int* ptr { &x }; // ptr is a normal (non-const) pointer

int y { 6 };
ptr = &y; // we can point at another value

*ptr = 7; // we can change the value at the address being held
```

With normal (non-const) pointers, we can change both what the pointer is pointing at (by assigning the pointer a new address to hold) or change the value at the address being held (by assigning a new value to the dereferenced pointer).

However, what happens **if the value we want to point at is const**?

```
const int x { 5 }; // x is now const
int* ptr { &x }; // compile error: cannot convert from const int* to int*
```

The above snippet won't compile -- we **can't set a normal pointer to point at a const variable**. This makes sense: a const variable is one whose value cannot be changed. Allowing the programmer to set a non-const pointer to a const value would allow the programmer to dereference the pointer and change the value. That would violate the const-ness of the variable.

15.9.1 Pointer to const value

A **pointer to a const value** (sometimes called a **pointer to const for short**) is a (non-const) pointer that points to a constant value.

To declare a pointer to a const value, use the `const` keyword before the pointer's data type:

To declare a pointer to a const value, use the **const keyword before the pointer's data type**:

```
const int x{ 5 };  
const int* ptr { &x }; // okay: ptr is pointing to a "const int"
```

In the above example, `ptr` points to a const int. Because the data type being pointed to is const, the value being pointed to can't be changed.

However, because a **pointer to const is not const itself** (it just points to a const value), we can change what the pointer is pointing at by assigning the pointer a new address:

```
const int x{ 5 };  
const int* ptr { &x }; // ptr points to const int x  
  
const int y{ 6 };  
ptr = &y; // okay: ptr now points at const int y
```

Just like a reference to const, a pointer to const can point to non-const variables too. A pointer to const **treats the value being pointed to as constant**, regardless of whether the object at that address was initially defined as const or not:

```
int x{ 5 }; // non-const  
const int* ptr { &x }; // ptr points to a "const int"  
  
*ptr = 6; // not allowed: ptr points to a "const int" so we can't change the value through ptr  
x = 6; // allowed: the value is still non-const when accessed through non-const identifier x
```

15.9.2 Const pointers

We can also make a pointer itself constant. A **const pointer** is a pointer whose address cannot be changed after initialization.

To declare a const pointer, use the const keyword after the asterisk in the pointer declaration. Just like a normal const variable, a const pointer must be initialized upon definition, and this **value can't be changed via assignment**:

```
int x{ 5 };  
int y{ 6 };  
  
int* const ptr { &x }; // okay: the const pointer is initialized to the address of x  
ptr = &y; // error: once initialized, a const pointer can not be changed.
```

However, because the *value* being pointed to is non-const, it is possible to change the value being pointed to via dereferencing the const pointer:

```
int x{ 5 };  
int* const ptr { &x }; // ptr will always point to x  
  
*ptr = 6; // okay: the value being pointed to is non-const
```

15.9.3 Const pointer to a const value

Finally, it is possible to declare a **const pointer to a const value** by using the const keyword both before the type and after the asterisk:

```
int value { 5 };  
const int* const ptr { &value }; // a const pointer to a const value
```

A const pointer to a const value **cannot have its address changed, nor can the value it is pointing to be changed through the pointer**. It can only be dereferenced to get the value it is pointing at.

15.9.4 Pointer and const recap

To summarize, you only need to remember **4 rules**, and they are pretty logical:

- A **non-const** pointer can be **assigned another address to change what it is pointing at**.
- A **pointer to a non-const** value **can change the value it is pointing to**. These cannot point to a const value.
- A **const pointer** always **points to the same address**, and this address cannot be changed.
- A **pointer to a const value** **treats the value as const when accessed through the pointer**, and thus cannot change the value it is pointing to. These can be pointed to const or non-const l-values (but not r-values, which don't have an address).

Keeping the declaration syntax straight can be a bit challenging:

- A **const before the asterisk** is associated with the **type being pointed to**. Therefore, this is a **pointer to a const value**, and the value cannot be modified through the pointer.
- A **const after the asterisk** is associated with the **pointer itself**. Therefore, this pointer cannot be assigned a new address.

```
int v{ 5 };  
  
int* ptr0 { &v };           // points to an "int" but is not const itself, so this is a normal pointer.  
const int* ptr1 { &v };      // points to a "const int" but is not const itself, so this is a pointer to a const value.  
int* const ptr2 { &v };      // points to an "int" and is const itself, so this is a const pointer (to a non-const value).  
const int* const ptr3 { &v }; // points to a "const int" and is const itself, so this is a const pointer to a const value.  
  
// if the const is on the left side of the *, the const belongs to the value  
// if the const is on the right side of the *, the const belongs to the pointer
```

15.10 Pass by address 1

In prior lessons, we've covered two different ways to pass an argument to a function: pass by value (Introduction to function parameters and arguments) and pass by reference (Pass by lvalue reference).

15.10.1 Pass by address

C++ provides a third way to pass values to a function, called pass by address. With **pass by address**, instead of providing an object as an argument, the **caller provides an object's address (via a pointer)**. This pointer (holding the address of the object) is copied into a pointer parameter of the called function (which now also holds the address of the object). **The function can then dereference that pointer to access the object whose address was passed.**

Here's a version of the above program that adds a pass by address variant:

```
void printByValue(std::string val) // The function parameter is a copy of str
{
    std::cout << val << '\n'; // print the value via the copy
}

void printByReference(const std::string& ref) // The function parameter is a reference that binds to str
{
    std::cout << ref << '\n'; // print the value via the reference
}

void printByAddress(const std::string* ptr) // The function parameter is a pointer that holds the address of str
{
    std::cout << *ptr << '\n'; // print the value via the dereferenced pointer
}
```

Note how similar all three of these versions are. **Let's explore the pass by address version in more detail.**

First, because we want our printByAddress() function to use pass by address, we've made our function parameter a pointer named ptr. Since printByAddress() will use ptr in a **read-only manner**, ptr is a **pointer to a const value**.

```
void printByAddress(const std::string* ptr)
{
    std::cout << *ptr << '\n'; // print the value via the dereferenced pointer
}
```

Inside the printByAddress() function, we dereference ptr parameter to access the value of the object being pointed to.

Second, when the function is called, we can't just pass in the str object -- we need to pass in the address of str. The easiest way to do that is to use the address-of operator (&) to get a pointer holding the address of str:

```
1 | printByAddress(&str); // use address-of operator (&) to get pointer holding address of str
```


When this call is executed, **&str will create a pointer holding the address of str**. This address is then copied into function parameter ptr as part of the function call. Because ptr now holds the address of str, when the function dereferences ptr, it will get the value of str, which the function prints to the console.

That's it.

Although we use the address-of operator in the above example to get the address of str, if we already had a pointer variable holding the address of str, we could use that instead.

15.10.2 Pass by address does not make a copy of the object being pointed to

As we noted in: Pass by lvalue reference, copying a std::string is expensive, so that's something we want to avoid. **When we pass a std::string by address, we're not copying the actual std::string object** -- we're just copying the pointer (holding the address of the object) from the caller to the called function. **Since an address is typically only 4 or 8 bytes, a pointer is only 4 or 8 bytes, so copying a pointer is always fast.**

Thus, **just like pass by reference, pass by address is fast**, and avoids making a copy of the argument object.

15.10.3 Pass by address allows the function to modify the argument's value

When we pass an object by address, the function receives the address of the passed object, which it can **access via dereferencing**. Because this is the address of the actual argument object being passed (not a copy of the object), **if the function parameter is a pointer to non-const**, the function can modify the argument via the pointer parameter:

```
void changeValue(int* ptr) // note: ptr is a pointer to non-const in this example
{
    *ptr = 6; // change the value to 6
}
```

```
int x{ 5 };

std::cout << "x = " << x << '\n';

changeValue(&x); // we're passing the address of x to the function
```

If a function is not supposed to modify the object being passed in, the function parameter can be made a pointer to const:

```
void changeValue(const int* ptr) // note: ptr is now a pointer to a const
{
    *ptr = 6; // error: can not change const value
}
```

15.10.4 Null checking

When passing a parameter by address, **care should be taken to ensure the pointer is not a null pointer before you dereference the value**. One way to do that is to use a conditional statement:

```
void print(int* ptr)
{
    if (ptr) // if ptr is not a null pointer
    {
        std::cout << *ptr << '\n';
    }
}
```

In the above program, we're testing ptr to ensure it is not null before we dereference it. While this is fine for such a simple function, **in more complicated functions this can result in redundant logic (testing if ptr is not null multiple times)** or nesting of the primary logic of the function (if contained in a block).

In most cases, it is more effective to do the opposite: test whether the function parameter is null as a precondition (Assert and static_assert) and handle the negative case immediately:

```
void print(int* ptr)
{
    if (!ptr) // if ptr is a null pointer, early return back to the caller
        return;

    // if we reached this point, we can assume ptr is valid
    // so no more testing or nesting required

    std::cout << *ptr << '\n';
}
```

If a null pointer should never be passed to the function, an **assert** (which we covered in lesson: Assert and static_assert) **can be used instead** (or also) (as asserts are intended to document things that should never happen):

```
void print(const int* ptr) // now a pointer to a const int
{
    assert(ptr); // fail the program in debug mode if a null pointer is passed (since this should never happen)

    // (optionally) handle this as an error case in production mode so we don't crash if it does happen
    if (!ptr)
        return;

    std::cout << *ptr << '\n';
}
```

15.10.5 Prefer pass by (const) reference

Note that function print() in the example above doesn't handle null values very well -- it effectively just aborts the function. Given this, **why allow a user to pass in a null value at all?** **Pass by reference has the same benefits as pass by address without the risk of inadvertently dereferencing a null pointer.**

Pass by const reference has a few other advantages over pass by address.

First, because an object being passed by address must have an address, **only lvalues can be passed by address (as rvalues don't have addresses)**. Pass by **const reference is more flexible, as it can accept lvalues and rvalues**:

```
void printByValue(int val) // The function parameter is a copy of the argument
{
    std::cout << val << '\n'; // print the value via the copy
}

void printByReference(const int& ref) // The function parameter is a reference that binds to the argument
{
    std::cout << ref << '\n'; // print the value via the reference
}

void printByAddress(const int* ptr) // The function parameter is a pointer that holds the address of the argument
{
    std::cout << *ptr << '\n'; // print the value via the dereferenced pointer
}
```

Second, the syntax for pass by reference is natural, as we can just pass in literals or objects.

With pass by address, our code ends up littered with ampersands (&) and asterisks (*).

In modern C++, most things that can be done with pass by address are better accomplished through other methods. Follow this common maxim: **“Pass by reference when you can, pass by address when you must”**.

Best practice: Prefer **pass by reference** to pass by address unless you have a specific reason to use pass by address.

15.11 Pass by address 2

This lesson is a continuation of Pass by address 1.

15.11.1 Pass by address for “optional” arguments

One of the more common uses for pass by address is to allow a function to accept an **“optional”** argument. This is easier to illustrate by example than to describe:

```
void printIDNumber(const int *id=nullptr)
{
    if (id)
        std::cout << "Your ID number is " << *id << ".\n";
    else
        std::cout << "Your ID number is not known.\n";
}
```

However, in many cases, **function overloading** is a better alternative to achieve the same result.

This has a number of advantages: **we no longer have to worry about null dereferences**, and we can pass in literals or other rvalues as an argument.

15.11.2 Changing what a pointer parameter points at

When we pass an address to a function, that **address is copied from the argument into the pointer parameter** (which is fine, because copying an address is fast). Now consider the following program:

```
// [[maybe_unused]] gets rid of compiler warnings about ptr2 being set but not used
void nullify([[maybe_unused]] int* ptr2)
{
    ptr2 = nullptr; // Make the function parameter a null pointer
}

int main()
{
    int x{ 5 };
    int* ptr{ &x }; // ptr points to x

    std::cout << "ptr is " << (ptr ? "non-null\n" : "null\n");

    nullify(ptr);

    std::cout << "ptr is " << (ptr ? "non-null\n" : "null\n");
    return 0;
}
```

This program prints:

ptr is non-null

ptr is non-null

As you can see, **changing the address held by the pointer parameter had no impact on the address held by the argument** (ptr still points at x). When function nullify() is called, **ptr2 receives a copy** of the address passed in (in this case, the address held by ptr, which is the address of x). When the function changes what ptr2 points at, this only affects the copy held by ptr2.

So, what if we want to allow a function to change what a pointer argument points to?

15.11.3 Pass by address... by reference?

Yup, it's a thing. Just like we can pass a normal variable by reference, **we can also pass pointers by reference**. Here's the same program as above with ptr2 changed to be a reference to an address:

```
void nullify(int*& refptr) // refptr is now a reference to a pointer
{
    refptr = nullptr; // Make the function parameter a null pointer
}

int main()
{
    int x{ 5 };
    int* ptr{ &x }; // ptr points to x

    std::cout << "ptr is " << (ptr ? "non-null\n" : "null\n");

    nullify(ptr);

    std::cout << "ptr is " << (ptr ? "non-null\n" : "null\n");
    return 0;
}
```

This program prints:

ptr is non-null

ptr is null

Because refptr is now a reference to a pointer, **when ptr is passed as an argument, refptr is bound to ptr**. This means any changes to refptr are made to ptr.

15.11.4 Why using 0 or NULL is no longer preferred (optional)

In this subsection, we'll explain why using 0 or NULL is no longer preferred.

The literal 0 **can be interpreted as either an integer literal, or as a null pointer literal**. In certain cases, **it can be ambiguous** which one we intend -- and in some of those cases, the **compiler may assume we mean one when we mean the other** -- with unintended consequences to the behavior of our program.

The definition of preprocessor macro-NULL is **not defined by the language standard**. It can be defined as 0, 0L, ((void*)0), or something else entirely.

Using nullptr removes this ambiguity (it will always call print(int*)), since nullptr will only match a pointer type.

std::nullptr_t (optional)

Since nullptr can be differentiated from integer values in function overloads, it must have a different type. **So what type is nullptr?** The answer is that **nullptr has type std::nullptr_t** (defined in header <cstdlib>). std::nullptr_t can only hold one value: nullptr! While this may seem kind of silly, it's useful in one situation. **If we want to write a function that accepts only a nullptr literal argument, we can make the parameter a std::nullptr_t.**

You probably won't ever need to use this, but it's good to know, just in case.

15.11.5 There is only pass by value

Now that you understand the basic differences between passing by reference, address, and value, **let's get reductionist for a moment.** :)

While the compiler can often optimize references away entirely, there are cases where this is not possible and a reference is actually needed. **References are normally implemented by the compiler using pointers.** This means that **behind the scenes, pass by reference is essentially just a pass by address.**

And in the previous lesson, we mentioned that **pass by address just copies an address from the caller to the called function -- which is just passing an address by value.**

Therefore, **we can conclude that C++ really passes everything by value!** The properties of pass by address (and reference) come solely from the fact that we can dereference the passed address to change the argument, which we cannot do with a normal value parameter!

15.12 Return by reference and return by address

In previous lessons, we discussed that when passing an argument by value, a copy of the argument is made into the function parameter. For fundamental types (which are cheap to copy), this is fine. But copying is typically expensive for class types (such as `std::string`). We can avoid making an expensive copy by utilizing passing by (const) reference (or pass by address) instead.

We encounter a similar situation when returning by value: **a copy of the return value is passed back to the caller.** If the return type of the function is a class type, **this can be expensive.**

```
1 | std::string returnByValue(); // returns a copy of a std::string (expensive)
```


15.12.1 Return by reference

In cases where we're passing a class type back to the caller, we may (or may not) want to return by reference instead. **Return by reference** returns a reference that is **bound to the object being returned, which avoids making a copy of the return value**. To return by reference, we **simply define the return value of the function to be a reference type**:

```
std::string&    returnByReference(); // returns a reference to an existing std::string (cheap)
const std::string& returnByReferenceToConst(); // returns a const reference to an existing std::string (cheap)

const std::string& getProgramName() // returns a const reference
{
    static const std::string s_programName { "Calculator" }; // has static duration, destroyed at end of program

    return s_programName;
}
```

Because `getProgramName()` returns a const reference, **when the line `return s_programName` is executed, `getProgramName()` will return a const reference to `s_programName` (thus avoiding making a copy)**. That const reference can then be used by the caller to access the value of `s_programName`, which is printed.

15.12.2 The object being returned by reference must exist after the function returns

Using return by reference has **one major caveat**: the programmer **must be sure that the object being referenced outlives the function returning the reference**. Otherwise, the reference being returned will be left dangling (referencing an object that has been destroyed), and use of that reference will result in undefined behavior.

In the program above, **because `s_programName` has static duration, `s_programName` will exist until the end of the program**. When `main()` accesses the returned reference, it is actually accessing `s_programName`, which is fine, because `s_programName` won't be destroyed until later.

Modern compilers will produce a warning or error if you try to return a local variable by reference (so the above program may not even compile), but compilers sometimes have trouble detecting more complicated cases.

Warning: Objects returned by reference must live beyond the scope of the function returning the reference, or a dangling reference will result. **Never return a (non-static) local variable or temporary by reference.**

15.12.3 Lifetime extension doesn't work across function boundaries

Let's take a look at an example where we return a temporary by reference:

```
const int& returnByConstReference()
{
    return 5; // returns const reference to temporary object
}

int main()
{
    const int& ref { returnByConstReference() };

    std::cout << ref; // undefined behavior
}
```

In the above program, `returnByConstReference()` is returning an integer literal, but the return type of the function is `const int&`. **This results in the creation of a temporary reference bound to a temporary object holding value 5.** This temporary reference to a temporary object is then returned. The temporary object then goes out of scope, **leaving the reference dangling.**

By the time the return value is bound to another const reference (in `main()`), it is too late to extend the lifetime of the temporary object -- as it has already been destroyed. Thus, `ref` is bound to a dangling reference, and use of the value of `ref` will result in undefined behavior.

Here's a less obvious example that similarly doesn't work:

```
// case 1: direct binding
const int& ref1 { 5 }; // extends lifetime
std::cout << ref1 << '\n'; // okay

// case 2: indirect binding
const int& ref2 { returnByConstReference(5) }; // binds to dangling reference
std::cout << ref2 << '\n'; // undefined behavior
```

In case 2, a temporary object is created to hold value 5, which function parameter `ref` binds to. The function just returns this reference back to the caller, which then uses the reference to initialize `ref2`. Because this is not a direct binding to the temporary object (as the reference was bounced through a function), lifetime extension doesn't apply. This leaves `ref2` dangling, and its subsequent use is undefined behavior.

Warning: Reference lifetime extension does not work across function boundaries.

15.12.4 Don't return non-const static local variables by reference

In the original example above, we returned a const static local variable by reference to illustrate the mechanics of return by reference in a simple way. However, returning **non-const static local variables by reference is fairly non-idiomatic, and should generally be avoided**. Here's a simplified example that illustrates one such problem that can occur:

```
const int& getNextId()
{
    static int s_x{ 0 }; // note: variable is non-const
    ++s_x; // generate the next id
    return s_x; // and return a reference to it
}

int main()
{
    const int& id1 { getNextId() }; // id1 is a reference
    const int& id2 { getNextId() }; // id2 is a reference

    std::cout << id1 << id2 << '\n';
}
```

This program prints:

22

This happens because `id1` and `id2` are referencing the same object (the static variable `s_x`), so **when anything (e.g. `getNextId()`) modifies that value, all references are now accessing the modified value**.

This above example **can be fixed by making `id1` and `id2` normal variables** (rather than references) so they **save a copy of the return value rather than a reference to `s_x`**.

Another issue that commonly occurs with programs that return a non-const static local by reference is that **there is no standardized way to reset `s_x` back to the default state**. Such programs must either use a non-conventional solution (e.g. a reset function parameter), or can only be reset by quitting and restarting the program.

Best practice: Avoid returning references to non-const local static variables.

Returning a const reference to a *const* local static variable is sometimes done if the local variable being returned by reference is expensive to create and/or initialize (so we don't have to recreate the variable every function call). But this is rare.

Returning a const reference to a *const* global variable is also sometimes done as a way to encapsulate access to a global variable.

15.12.5 Assigning/initializing a normal variable with a returned reference makes a copy

If a function returns a reference, and that **reference is used to initialize** or assign to a non-reference variable, **the return value will be copied (as if it had been returned by value)**.

```
const int& getNextId()
{
    static int s_x{ 0 };
    ++s_x;
    return s_x;
}

int main()
{
    const int id1 { getNextId() }; // id1 is a normal variable now and receives a copy of the value returned by reference from getNextId()
    const int id2 { getNextId() }; // id2 is a normal variable now and receives a copy of the value returned by reference from getNextId()

    std::cout << id1 << id2 << '\n';
}
```

In the above example, `getNextId()` is returning a reference, but `id1` and `id2` are non-reference variables. In such a case, the value of the returned reference is copied into the normal variable. Thus, this program prints:

12

Also note that if a program returns a dangling reference, the reference is left dangling before the copy is made, which will lead to undefined behavior.

15.12.6 It's okay to return reference parameters by reference

There are quite a few cases where returning objects by reference makes sense, and we'll encounter many of those in future lessons. However, there is one useful example that we can show now.

If a parameter is passed into a function by reference, it's safe to return that parameter by reference. This makes sense: in order to pass an argument to a function, the argument must exist in the scope of the caller. **When the called function returns, that object must still exist in the scope of the caller.**

15.12.7 The caller can modify values through the reference

When an argument is passed to a function by non-const reference, the function can use the reference to modify the value of the argument.

Similarly, when a non-const reference is returned from a function, the caller can use the reference to modify the value being returned.

Here's an illustrative example:

```
// takes two integers by non-const reference, and returns the greater by reference
int& max(int& x, int& y)
{
    return (x > y) ? x : y;
}

int main()
{
    int a{ 5 };
    int b{ 6 };

    max(a, b) = 7; // sets the greater of a or b to 7

    std::cout << a << b << '\n';
}
```

The expression `max(a, b) = 7` effectively resolves to `b = 7`.

This prints:

57

15.12.8 Return by address

Return by address works almost identically to return by reference, except a pointer to an object is returned instead of a reference to an object. Return by address has the same primary caveat as return by reference -- the object being returned by address must outlive the scope of the function returning the address, otherwise the caller will receive a dangling pointer.

The major advantage of return by address over return by reference is that we can have the function return `nullptr` if there is no valid object to return. For example, let's say we have a list of students that we want to search. If we find the student we are looking for in the list, we can return a pointer to the object representing the matching student. If we don't find any students matching, we can return `nullptr` to indicate a matching student object was not found.

The major disadvantage of return by address is that the caller has to remember to do a `nullptr` check before dereferencing the return value, otherwise a null pointer dereference may occur and undefined behavior will result. Because of this danger, return by reference should be preferred over return by address unless the ability to return "no object" is needed.

Best practice: Prefer return by reference over return by address unless the ability to return "no object" (using `nullptr`) is important.

15.13 In and out parameters

A function and its caller communicate with each other via **two mechanisms**: parameters and return values. When a function is called, the caller provides arguments, which the function receives via its parameters. These arguments can be passed by value, reference, or address.

Typically, we'll pass arguments by value or by const reference. But there are times when we may need to do otherwise.

15.13.1 In parameters

In most cases, a function parameter is used **only to receive an input from the caller**. Parameters that are used only for receiving input from the caller are sometimes called **in parameters**.

In-parameters are typically passed by value or by const reference.

```
void print(int x) // x is an in parameter
{
    std::cout << x << '\n';
}

void print(const std::string& s) // s is an in parameter
{
    std::cout << s << '\n';
}
```

15.13.2 Out parameters

A function argument passed by **(non-const) reference (or by address)** allows the function to **modify the value of an object passed as an argument**. This provides a way for a function to **return data back to the caller** in cases where using a return value is not sufficient for some reason.

A function parameter that is used **only for the purpose of returning information back to the caller** is called an out parameter.

For example:

```
// sinOut and cosOut are out parameters
void getSinCos(double degrees, double& sinOut, double& cosOut)
{
    // sin() and cos() take radians, not degrees, so we need to convert
    constexpr double pi { 3.14159265358979323846 }; // the value of pi
    double radians = degrees * pi / 180.0;
    sinOut = std::sin(radians);
    cosOut = std::cos(radians);
}
```

15.13.3 In essence:

- In parameters: take the argument and modify it inside of the function, whether it returns the value or not.
- Out parameters: take the argument and modify it inside of the function BUT the **modified object is also modified outside of the function's scope**, whether it returns the value or not. This **allows us to "return" more** back to the caller.

15.13.4 Out parameters have an unnatural syntax

Out-parameters, while functional, have a **few downsides**.

First, the **caller must instantiate (and initialize) objects and pass them as arguments**, even if it doesn't intend to use them. These **objects must be able to be assigned to, which means they can't be made const**.

Second, because the caller must pass in objects, these **values can't be used as temporaries, or easily used in a single expression**.

```
void getByReference(int& x)
{
    x = 5;
}

int main()
{
    // return by value
    [[maybe_unused]] int x{ getByValue() }; // can use to initialize object
    std::cout << getByValue() << '\n';      // can use temporary return value in expression

    // return by out parameter
    int y{}; // must first allocate an assignable object
    getByReference(y); // then pass to function to assign the desired value
    std::cout << y << '\n'; // and only then can we use that value

    return 0;
}
```

As you can see, the syntax of using out-parameters is a bit unnatural.

15.13.5 Out-parameters by reference don't make it obvious the arguments will be modified

When we assign a function's return value to an object, it is clear that the value of the object is being modified.

```
1 | x = getByValue(); // obvious that x is being modified
```

Using pass by address instead of pass by reference can in some case help make out-parameters more obvious by requiring the caller to pass in the address of objects as arguments.

```
void foo1(int x); // pass by value
void foo2(int& x); // pass by reference
void foo3(int* x); // pass by address

int main()
{
    int i{};

    foo1(i); // can't modify i
    foo2(i); // can modify i (not obvious)
    foo3(&i); // can modify i

    int *ptr { &i };
    foo3(ptr); // can modify i (not obvious)
```

The caller may also think they can pass in nullptr or a null pointer as a valid argument when this is disallowed. And the function is now required to do null pointer checking and handling, which adds more complexity. This need for added null pointer handling often causes more issues than just sticking with pass by reference.

Best practice:

Avoid out-parameters (except in the rare case where no better options exist).

Prefer pass by reference for non-optional out-parameters.

15.13.6 In/out parameters

In rare cases, a function will actually use the value of an out-parameter before overwriting its value. Such a parameter is called an **in-out parameter**. In-out-parameters work identically to out-parameters and have all the same challenges.

15.13.7 When to pass by non-const reference

If you're going to pass by reference in order to avoid making a copy of the argument, you should almost always **pass by const reference**.

However, there are **two primary cases where pass by non-const reference may be the better choice**:

First, use pass by non-const reference **when a parameter is an in-out-parameter**. Since we're already passing in the object we need back out, it's often more straightforward and performant to just modify that object.

Second, use pass by non-const reference **when a function would otherwise return an object by value to the caller, but making a copy of that object is *extremely* expensive**. Especially if the function is called many times in a performance-critical section of code.

That said, objects are rarely so expensive to copy that resorting to non-conventional methods of returning those objects is worthwhile.

15.14 Type deduction with pointers, references, and const

In lesson: Type deduction for objects using the auto keyword, we discussed how the auto keyword can be used to have the compiler deduce the type of a variable from the initializer:

```
int getVal(); // some function that returns an int by value

int main()
{
    auto val { getVal() }; // val deduced as type int
}
```

We also noted that **by default, type deduction will drop const (and constexpr) qualifiers**.

Const (or constexpr) **can be (re)applied by adding the const (or constexpr) qualifier** in the definition:

```
const auto x{ foo() }; // const double (const dropped, const reapplied)
constexpr auto y{ cd }; // constexpr double (constexpr dropped, constexpr reapplied)
const auto z { cd }; // const double (constexpr dropped, const applied)
```

15.14.1 Type deduction drops references

In addition to dropping const/constexpr qualifiers, **type deduction will also drop references**:

```
std::string& getRef(); // some function that returns a reference

int main()
{
    auto ref { getRef() }; // type deduced as std::string (not std::string&)
```

Just like with the dropped const qualifier, if you want the deduced type to be a reference, **you can reapply the reference at the point of definition**:

```
std::string& getRef(); // some function that returns a reference

int main()
{
    auto ref1 { getRef() }; // std::string (reference dropped)
    auto& ref2 { getRef() }; // std::string& (reference reapplied)
```

15.14.2 Top-level const and low-level const

A **top-level const** is a const qualifier that **applies to an object itself**. For example:

```
1 | const int x;    // this const applies to x, so it is top-level
2 | int* const ptr; // this const applies to ptr, so it is top-level
```

In contrast, a **low-level const** is a const qualifier that **applies to the object being referenced or pointed to**:

```
1 | const int& ref; // this const applies to the object being referenced, so it is low-level
2 | const int* ptr; // this const applies to the object being pointed to, so it is low-level
```

A reference to a const value is always a low-level const. A pointer can have a top-level, low-level, or both kinds of const.

When we say that type deduction drops const qualifiers, it only drops top-level consts. Low-level consts are not dropped.

15.14.3 Type deduction and const references

If the initializer is a reference to const (or constexpr), the reference is dropped first (and then reapplied if applicable), and then any top-level const or constexpr is dropped from the result.

```
const std::string& getConstRef(); // some function that returns a reference to const

int main()
{
    auto ref1{ getConstRef() }; // std::string (reference dropped, then top-level const dropped from result)
```

In the above example, since `getConstRef()` returns a `const std::string&`, the **reference is dropped first**, leaving us with a `const std::string`. This const is now a **top-level const**, so it is also dropped, leaving the deduced type as `std::string`.

Best practice: If you want a const reference, **reapply the const qualifier even when it's not strictly necessary**, as it makes your intent clear and helps prevent mistakes.

15.14.4 What about constexpr references?

Constexpr is not part of an expression's type, so it is not deduced by auto.

A reminder:

When defining a const reference (e.g. `const int&`), the `const` applies to the object being referenced, not the reference itself.

When defining a constexpr reference to a const variable (e.g. `constexpr const int&`), we need to apply both `constexpr` (which applies to the reference) and `const` (which applies to the type being referenced).

15.14.5 Type deduction and pointers

Unlike references, **type deduction does not drop pointers**.

Key insight:

The reason that references are dropped during type deduction but pointers are not dropped is because references and pointers have different semantics.

When we evaluate a reference, we're really evaluating the object being referenced. Therefore, when deducing a type, it makes sense that we should deduce the type of the thing being referenced, not the reference itself. Also, since we deduce a non-reference, it's really easy to make it a reference by using `auto&`. If type deduction were to deduce a reference instead, the syntax for removing a reference if we didn't want it is much more complicated.

On the other hand, pointers hold the address of an object. When we evaluate a pointer, we are evaluating the pointer, not the object being pointed to (if we want that, we can dereference the pointer). Therefore, it makes sense that we should deduce the type of the pointer, not the thing being pointed to.

15.14.5.1 Type deduction and const pointers (Optional)

Since **pointers aren't dropped**, we don't have to worry about that. But with pointers, we have **both the const pointer and the pointer to const cases to think about**, and we also have `auto` vs `auto*`. Just like with references, only **top-level const is dropped during pointer type deduction**.

Best practice: If you want a const pointer, pointer to const, or const pointer to const, **reapply the const qualifier(s) even when it's not strictly necessary**, as it makes your intent clear and helps prevent mistakes.

```
std::string s{};
const std::string* const ptr { &s };

auto ptr1{ ptr }; // const std::string*
auto* ptr2{ ptr }; // const std::string*

auto const ptr3{ ptr }; // const std::string* const
const auto ptr4{ ptr }; // const std::string* const

auto* const ptr5{ ptr }; // const std::string* const
const auto* ptr6{ ptr }; // const std::string*

const auto const ptr7{ ptr }; // error: const qualifer can not be applied twice
const auto* const ptr8{ ptr }; // const std::string* const
```

15.15 std::optional

In lesson: Detecting and handling errors, we discussed cases where a function encounters an error that it cannot reasonably handle itself. For example, consider a function that divides and returns a value but the caller passes in a 0. So, what are we to do?

First, we could pick some (hopefully) **uncommon return value as our sentinel** and use it to indicate an error:

```
// returns std::numeric_limits<int>::lowest() on failure
int doIntDivision(int x, int y)
{
    if (y == 0)
        return std::numeric_limits<int>::lowest();
    return x / y;
}
```

Second, we could **abandon using return values to return errors and use some other mechanism** (e.g. exceptions). However, exceptions have their own complications and performance costs, and may not be appropriate or desired. That's **probably overkill for something like this**.

Third, we could abandon returning a single value and return two values instead: one (of type bool) that indicates whether the function succeeded, and the other (of the desired return type) that holds the actual return value (if the function succeeded) or an indeterminate value (if the function failed). This is **probably the best option of the bunch**.

Prior to C++17, choosing this latter option required you to implement it yourself. And while C++ provides multiple ways to do so, **any roll-your-own approach will inevitably lead to inconsistencies and errors**.

15.15.1 Returning a `std::optional`

C++17 introduces `std::optional`, which is a class template type that implements an optional value. That is, a `std::optional<T>` can either have a value of type `T`, or not. We can use this to implement the third option above:

```
std::optional<int> doIntDivision(int x, int y)
{
    if (y == 0)
        return {}; // or return std::nullopt
    return x / y;
}
```

Using `std::optional` is quite easy. We can construct a `std::optional<T>` either with or without a value:

```
1 std::optional<int> o1 { 5 };           // initialize with a value
2 std::optional<int> o2 {};             // initialize with no value
3 std::optional<int> o3 { std::nullopt }; // initialize with no value
```

To see if a `std::optional` has a value, we can choose one of the following:

```
1 if (o1.has_value()) // call has_value() to check if o1 has a value
2 if (o2)             // use implicit conversion to bool to check if o2 has a value
```

To get the value from a `std::optional`, we can choose one of the following:

```
1 std::cout << *o1;
2 std::cout << o2.value();
  value)
3 std::cout << o3.value_or(42);
```

Note that `std::optional` has a usage syntax that is essentially identical to a pointer:

Behavior	Pointer	<code>std::optional</code>
Hold no value	initialize/assign <code>{}</code> or <code>std::nullptr</code>	initialize/assign <code>{}</code> or <code>std::nullopt</code>
Hold a value	initialize/assign an address	initialize/assign a value
Check if has value	implicit conversion to bool	implicit conversion to bool or <code>has_value()</code>
Get value	dereference	dereference or <code>value()</code>

15.15.2 Pros and cons of returning a `std::optional`

Returning a `std::optional` is nice for a number of reasons:

- Using `std::optional` **effectively documents that a function may return a value or not.**
- **We don't have to remember which value is being returned as a sentinel.**
- The **syntax for using `std::optional` is convenient** and intuitive.

Returning a `std::optional` does come with a few downsides:

- We have to **make sure the `std::optional` contains a value before getting the value.** If we dereference a `std::optional` that does not contain a value, we get undefined behavior.
- `std::optional` **does not provide a way to pass back information about why the function failed.**

Unless your function needs to return additional information about why it failed (either to better understand the failure, or to differentiate different kinds of failure), `std::optional` is an excellent choice for functions that may return a value or fail.

Best practice: Return a `std::optional` (instead of a sentinel value) for functions that may fail, unless your function needs to return additional information about why it failed.

15.15.3 Using `std::optional` as an optional function parameter

In lesson: Pass by address (part 2), we discussed how pass by address can be used to allow a function to accept an “optional” argument (that is, the caller can either pass in `nullptr` to represent “no argument” or an object). However, one downside of this approach is that a non-`nullptr` argument must be an lvalue (so that its address can be passed to the function).

Perhaps unsurprisingly (given the name), **`std::optional` is an alternative way for a function to accept an optional argument (that is used as an in-parameter only).**

Best practice:

Prefer `std::optional` for optional return types.

Prefer function overloading for optional function parameters (when possible). Otherwise, use `std::optional<T>` for optional arguments when T would normally be passed by value. Favor `const T*` when T is expensive to copy.

15.16 Summary

Compound data types (also called **composite data type**) are data types that can be constructed from fundamental data types (or other compound data types).

The **value category** of an expression indicates whether an expression resolves to a value, a function, or an object of some kind.

An **lvalue** is an expression that evaluates to a function or an object that has an **identity**. An object or function with an identity has an identifier or an identifiable memory address. Lvalues come in two subtypes: **modifiable lvalues** are lvalues that can be modified, and **non-modifiable lvalues** are lvalues whose values can't be modified (typically because they are `const` or `constexpr`).

An **rvalue** is an expression that is not an l-value. This includes literals (except string literals) and the return values of functions or operators (when returned by value).

A **reference** is an alias for an existing object. Once a reference has been defined, any operation on the reference is applied to the object being referenced. C++ contains two types of references: lvalue references and rvalue references. An **lvalue reference** (commonly just called a **reference**) acts as an alias for an existing lvalue (such as a variable). An **lvalue reference variable** is a variable that acts as a reference to an lvalue (usually another variable).

When a reference is initialized with an object (or function), we say it is **bound** to that object (or function). The object (or function) being referenced is sometimes called the referent.

Lvalue references can't be bound to non-modifiable lvalues or rvalues (otherwise you'd be able to change those values through the reference, which would be a violation of their `const`-ness). For this reason, lvalue references are occasionally called **lvalue references to non-const** (sometimes shortened to **non-const reference**).

Once initialized, a reference in C++ cannot be **reseated**, meaning it cannot be changed to reference another object.

When an object being referenced is destroyed before a reference to it, the reference is left referencing an object that no longer exists. Such a reference is called a **dangling reference**. Accessing a dangling reference leads to undefined behavior.

By using the `const` keyword when declaring an lvalue reference, we tell an lvalue reference to treat the object it is referencing as `const`. Such a reference is called an **lvalue reference to a const value** (sometimes called a **reference to const** or a **const reference**). `Const` references can bind to modifiable lvalues, non-modifiable lvalues, and rvalues.

A **temporary object** (also sometimes called an **unnamed object** or **anonymous object**) is an object that is created for temporary use (and then destroyed) within a single expression.

When using **pass by reference**, we declare a function parameter as a reference (or const reference) rather than as a normal variable. When the function is called, each reference parameter is bound to the appropriate argument. Because the reference acts as an alias for the argument, no copy of the argument is made.

The **address-of operator** (&) returns the memory address of its operand. The **dereference operator** (*) returns the value at a given memory address as an lvalue.

A pointer is an object that holds a memory address (typically of another variable) as its value. This allows us to store the address of some other object to use later. Like normal variables, pointers are not initialized by default. A pointer that has not been initialized is sometimes called a **wild pointer**. A **dangling pointer** is a pointer that is holding the address of an object that is no longer valid (e.g. because it has been destroyed).

Besides a memory address, there is one additional value that a pointer can hold: a null value. A **null value** (often shortened to **null**) is a special value that means something has no value. When a pointer is holding a null value, it means the pointer is not pointing at anything. Such a pointer is called a **null pointer**. The **nullptr** keyword represents a null pointer literal. We can use **nullptr** to explicitly initialize or assign a pointer a null value.

A pointer should either hold the address of a valid object, or be set to **nullptr**. That way we only need to test pointers for null, and can assume any non-null pointer is valid.

A **pointer to a const value** (sometimes called a **pointer to const** for short) is a (non-const) pointer that points to a constant value.

A **const pointer** is a pointer whose address cannot be changed after initialization.

A **const pointer to a const value** cannot have its address changed, nor can the value it is pointing to be changed through the pointer.

With **pass by address**, instead of providing an object as an argument, the caller provides an object's address (via a pointer). This pointer (holding the address of the object) is copied into a pointer parameter of the called function (which now also holds the address of the object). The function can then dereference that pointer to access the object whose address was passed.

Return by reference returns a reference that is bound to the object being returned, which avoids making a copy of the return value. Using return by reference has one major caveat: the programmer must be sure that the object being referenced outlives the function returning the reference. Otherwise, the reference being returned will be left dangling (referencing an object

that has been destroyed), and use of that reference will result in undefined behavior. If a parameter is passed into a function by reference, it's safe to return that parameter by reference.

If a function returns a reference, and that reference is used to initialize or assign to a non-reference variable, the return value will be copied (as if it had been returned by value).

Type deduction for variables (via the `auto` keyword) will drop any reference or top-level `const` qualifiers from the deduced type. These can be reapplied as part of the variable declaration if desired.

Return by address works almost identically to return by reference, except a pointer to an object is returned instead of a reference to an object.

16 Compound Types: Enums and Structs

16.1 Introduction to program-defined (user-defined) types

Because fundamental types are defined as part of the core C++ language, they are available for immediate use. For example, if we want to define a variable with a type of `int` or `double`, we can just do so. This is also true for the compound types that are simple extensions of fundamental types (including functions, pointers, references, and arrays).

This works because the **C++ language already knows what the type names (and symbols) for these types mean** -- we do not need to provide or import any definitions.

However, consider the case of a type alias (introduced in lesson: Typedefs and type aliases), which allows us to define a new name for an existing type. Because a type alias introduces a new identifier into the program, a type alias must be defined before it can be used to tell the compiler what the alias is so that it can be used later.

16.1.1 What are user-defined / program-defined types?

Back in the introduction to the previous chapter (Introduction to compound data types), we introduced the challenge of wanting to store a fraction, which has a numerator and denominator that are conceptually linked together. In that lesson, we discussed some of the challenges with using two separate integers to store a fraction's numerator and denominator independently.

If C++ had a built-in fraction type, that would have been perfect -- but it doesn't. And there are **hundreds of other potentially useful types that C++ doesn't include because it's just not possible to anticipate everything that someone might need (let alone implement and test those things)**.

Instead, **C++ solves such problems in a different way: by allowing us to create entirely new, custom types** for use in our programs! Such types are often called **user-defined types** (though we think **the term program-defined types is better** -- we'll discuss the difference later in this lesson).

C++ has two different categories of compound types that can be used to create program-defined types:

- **Enumerated types** (including unscoped and scoped enumerations)
- **Class types** (including structs, classes, and unions).

16.1.2 Defining program-defined types

Just like type aliases, program-defined types must also be defined before they can be used. The definition for a program-defined type is called a type definition.

Although we haven't covered what a struct is yet, here's an example showing the definition of custom Fraction type and an instantiation of an object using that type:

```
struct Fraction
{
    int numerator {};
    int denominator {};
};
```

Program-defined type definitions must end in a semicolon. Failure to include the semicolon at the end of a type definition is a **common programmer error, and one that can be hard to debug** because the compiler may error on the line *after* the type definition.

Warning: Don't forget to end your type definitions with a semicolon.

16.1.3 Naming program-defined types

By convention, program-defined types are named starting with a capital letter and don't use a suffix (e.g. **Fraction**, not fraction, fraction_t, or Fraction_t).

Best practice: Name your program-defined types starting with a capital letter and do not use a suffix.

New programmers sometimes find variable definitions such as the following confusing because of the similarity between the type name and variable name:

```
1 | Fraction fraction {}; // Instantiates a variable named fraction of type Fraction
```

This is no different than any other variable definition: the type (Fraction) comes first (and because Fraction is capitalized, we know it's a program-defined type), then the variable name (fraction), and then an optional initializer. Because C++ is case-sensitive, there is no naming conflict here!

16.1.4 Using program-defined types throughout a multi-file program

Every code file that uses a program-defined type needs to see the full type definition before it is used. A forward declaration is not sufficient. This is required so that the compiler knows how much memory to allocate for objects of that type.

To propagate type definitions into the code files that need them, **program-defined types are typically defined in header files, and then #included into any code file that requires that type definition.** These **header files are typically given the same name as the program-defined type** (e.g. a program-defined type named **Fraction** would be defined in **Fraction.h**)

Best practice: A program-defined type used in only one code file **should be defined in that code file as close to the first point of use as possible.**

A program-defined type used in multiple code files should be defined in a header file with the same name as the program-defined type and then #included into each code file as needed.

Here's an example of what our Fraction type would look like if we moved it to a header file (named Fraction.h) so that it could be included into multiple code files:

Fraction.h:

```
1  #ifndef FRACTION_H
2  #define FRACTION_H
3
4  // Define a new type named Fraction
5  // This only defines what a Fraction looks like, it doesn't create one
6  // Note that this is a full definition, not a forward declaration
7  struct Fraction
8  {
9      int numerator {} ;
10     int denominator {} ;
11 };
12
13 #endif
```

Fraction.cpp:

```
1  #include "Fraction.h" // include our Fraction definition in this code file
2
3  // Now we can make use of our Fraction type
4  int main()
5  {
6      Fraction f{ 3, 4 }; // this actually creates a Fraction object named f
7
8      return 0;
9  }
```

16.1.5 Type definitions are partially exempt from the one-definition rule (ODR)

Type definitions are partially **exempt from the one-definition rule (ODR)** allowing them to be defined in multiple code files. While functions and global variables require only one definition per program, types can be defined in multiple files. This is necessary because the compiler often needs the full definition of a type, not just a forward declaration.

Two key caveats:

1. A type can only be **defined once per code file** (header guards help with this).
2. **All definitions of a given type across different files must be identical**; otherwise, undefined behavior may occur.

16.1.6 Nomenclature: user-defined types vs program-defined types

The term “user-defined type” sometimes comes up in casual conversation, as well as being mentioned (but **not defined**) in the **C++ language standard**. In casual conversation, the term tends to mean “**a type defined within your own programs**” (such as the Fraction type example above).

The C++ language standard uses the term “user-defined type” in a non-conventional manner. In the language standard, a “**user-defined type**” is any **class type or enumerated type that is defined by you, the standard library, or the implementation** (e.g. types defined by the compiler to support language extensions). Perhaps counter-intuitively, **this means `std::string`** (a class type defined in the standard library) **is considered to be a user-defined type!**

To provide additional differentiation, the **C++20 language standard helpfully defines the term “program-defined type”** to mean **class types and enumerated types that are not defined as part of the standard library, implementation, or core language**. In other words, “program-defined types” **only include class types and enum types that are defined by us (or a third-party library)**.

Type	Meaning	Examples
Fundamental	A type built into the core C++ language	<code>int</code> , <code>std::nullptr_t</code>
Compound	A type built from fundamental types	<code>int&</code> , <code>double*</code> , <code>std::string</code> , <code>Fraction</code>
User-defined	A class type or enumerated type (Includes those defined in the standard library or implementation) (In casual use, typically used to mean program-defined types)	<code>std::string</code> , <code>Fraction</code>
Program-defined	A class type or enumerated type (Excludes those defined in standard library or implementation)	<code>Fraction</code>

16.2 Unscoped enumerations

C++ contains many useful fundamental and compound data types (which we introduced in lessons: Introduction to fundamental data types and Introduction to compound data types). But these types aren't always sufficient for the kinds of things we want to do.

For example, let's say you're writing a program that needs to keep track of whether an apple is red, yellow, or green, or what color a shirt is (from a preset list of colors). If only fundamental types were available, how might you do this?

You might store the color as an integer value, using some kind of implicit mapping (0 = red , 1 = green, 2 = blue). But this isn't at all intuitive, and we've already discussed why magic numbers are bad. We can get rid of the magic numbers by using symbolic constants:

```
constexpr int red{ 0 };
constexpr int green{ 1 };
constexpr int blue{ 2 };

int main()
{
    int appleColor{ red };
    int shirtColor{ green };
}
```

While this is a bit better for reading, the programmer is still left to deduce that `appleColor` and `shirtColor` (which are of type `int`) are meant to hold one of the values defined in the set of color symbolic constants (which are likely defined elsewhere, probably in a separate file).

We can make this program a little clearer by using a type alias:

```
using Color = int; // define a type alias named Color

// The following color values should be used for a Color
constexpr Color red{ 0 };
constexpr Color green{ 1 };
constexpr Color blue{ 2 };
```

We're getting closer. Someone reading this code still has to understand that these color symbolic constants are meant to be used with variables of type `Color`, but at least the type has a unique name now so someone searching for `Color` would be able to find the set of associated symbolic constants.

However, because `Color` is just an alias for an `int`, **we still have the problem that nothing enforces proper usage of these color symbolic constants**. We can still do something like this:

```
Color eyeColor{ 8 }; // syntactically valid, semantically meaningless
```


Also, if we debug any of these variables in our debugger, we'll only see the integer value of the color (e.g. 0), not the symbolic meaning (red), which can make it harder to tell if our program is correct.

Fortunately, we can do better.

As inspiration, consider the `bool` type. What makes `bool` particularly interesting is that it only has two defined values: `true` and `false`. We can use `true` or `false` directly (as literals), or we can instantiate a `bool` object and have it hold either one of those values. Additionally, the compiler is able to differentiate `bool` from other types. This means we can overload functions, and customize how those functions behave when passed a boolean value.

If we had the ability to define our own custom types, where we could define the set of named values associated with that type, then we would have the perfect tool to elegantly solve the challenge above...

16.2.1 Enumerations

An **enumeration** (also called an enumerated type or an enum) is a **compound data type whose values are restricted to a set of named symbolic constants (called enumerators)**.

C++ supports two kinds of enumerations: unscoped enumerations (which we'll cover now) and scoped enumerations (which we'll cover later in this chapter).

Because enumerations are program-defined types, **each enumeration needs to be fully defined before we can use it (a forward declaration is not sufficient)**.

16.2.2 Unscoped enumerations

Unscoped enumerations are defined via the `enum` keyword.

Enumerated types are best taught by example, so let's define an unscoped enumeration that can hold some color values. We'll explain how it all works below.

```
// Define a new unscoped enumeration named Color
enum Color
{
    // Here are the enumerators
    // These symbolic constants define all the possible values this type can hold
    // Each enumerator is separated by a comma, not a semicolon
    red,
    green,
    blue, // trailing comma optional but recommended
}; // the enum definition must end with a semicolon

int main()
{
    // Define a few variables of enumerated type Color
    Color apple { red }; // my apple is red
    Color shirt { green }; // my shirt is green
    Color cup { blue }; // my cup is blue

    Color socks { white }; // error: white is not an enumerator of Color
    Color hat { 2 }; // error: 2 is not an enumerator of Color
}
```

We start our example by using the **`enum` keyword** to tell the compiler that we are defining an **unscoped enumeration**, which we've named **`Color`**.

Inside a pair of curly braces, we define the enumerators for the `Color` type: `red`, `green`, and `blue`. These enumerators define the specific values that type `Color` is restricted to. **Each enumerator must be separated by a comma (not a semicolon)** -- a **trailing comma after the last enumerator is optional but recommended for consistency**.

It is **most common to define one enumerator per line**, but in simple cases (where there are a small number of enumerators and no comments are needed), they may all be defined on a single line.

The **type definition for `Color` ends with a semicolon**. We've now fully defined what enumerated type `Color` is!

Inside `main()`, we instantiate three variables of type `Color`: `apple` is initialized with the color red, `shirt` is initialized with the color green, and `cup` is initialized with the color blue. Memory is allocated for each of these objects. Note that the initializer for an enumerated type must be one of the defined enumerators for that type. The variables `socks` and `hat` cause compile errors because the initializers `white` and `2` are not enumerators of `Color`.

Enumerators are implicitly constexpr.

Reminder: To quickly recap on nomenclature:

- An **enumeration** or **enumerated type** is the program-defined type itself (e.g. `Color`).
- An **enumerator** is a specific named value belonging to the enumeration (e.g. `red`).

If you're wondering how we can use an enumerator where an integral value is expected, **unscoped enumerators will implicitly convert to integral values.**

16.2.3 The scope of unscoped enumerations

Unscoped enumerations are named such because **they put their enumerator names into the same scope as the enumeration definition itself** (as opposed to creating a new scope region like a namespace does).

For example, given this program:

```
enum Color
{
    red,
    green,
    blue, // blue is put into the global namespace
};

enum Feeling
{
    happy,
    tired,
    blue, // error: naming collision with the above blue
};

int main()
{
    Color apple { red }; // my apple is red
    Feeling me { happy }; // I'm happy right now (even though my program doesn't compile)
```

Most often, **unscoped enumerators are accessed without using the scope resolution operator.**

Best practice: Prefer putting your enumerations inside a **named scope region** (such as a namespace or class) so the enumerators don't pollute the global namespace.

16.2.4 Comparing against enumerators

We can use the equality operators (operator== and operator!=) to test whether an enumeration has the value of a particular enumerator or not:

```
Color shirt{ blue };

if (shirt == blue) // if the shirt is blue
    std::cout << "Your shirt is blue!";
else
    std::cout << "Your shirt is not blue!";
```

16.3 Unscoped enumerator integral conversions

In the prior lesson (Unscoped enumerations), we mentioned that **enumerators are symbolic constants**. What we didn't tell you then is that **these enumerators have values that are of an integral type**.

This is similar to the case with chars. A char is really just a 1-byte integral value, and the character 'A' gets converted to an integral value (in this case, 65) and stored.

When we define an enumeration, **each enumerator is automatically associated with an integer value based on its position in the enumerator list**. By default, the **first enumerator is given the integral value 0**, and each subsequent enumerator has a value one greater than the previous enumerator:

```
enum Color
{
    black, // 0
    red,   // 1
    blue,  // 2
    green, // 3
    white, // 4
    cyan,  // 5
    yellow, // 6
    magenta, // 7
};
```

It is possible to explicitly define the value of enumerators. These integral values can be positive or negative, and can share the same value as other enumerators. **Any non-defined enumerators are given a value one greater than the previous enumerator**.

```
enum Animal
{
    cat = -3,    // values can be negative
    dog,        // -2
    pig,        // -1
    horse = 5,
    giraffe = 5, // shares same value as horse
    chicken,    // 6
};
```

Note in this case, horse and giraffe have been given the same value. When this happens, the enumerators become non-distinct -- essentially, **horse and giraffe are interchangeable**. Although **C++ allows it**, assigning the same value to two enumerators in the same enumeration **should generally be avoided**.

Most of the time, the default values for enumerators will be exactly what you want, so **do not provide your own values unless you have a specific reason to do so**.

Best practice: Avoid assigning explicit values to your enumerators unless you have a compelling reason to do so.

16.3.1 Value-initializing an enumeration

If an enumeration is **zero-initialized** (which happens when we use value-initialization), the **enumeration will be given value 0, even if there is no corresponding enumerator with that value**:

```
enum Animal
{
    cat = -3,    // -3
    dog,        // -2
    pig,        // -1
    // note: no enumerator with value 0 in this list
    horse = 5,   // 5
    giraffe = 5, // 5
    chicken,    // 6
};

int main()
{
    Animal a {}; // value-initialization zero-initializes a to value 0
    std::cout << a; // prints 0
}
```

Best practice: Make the enumerator representing 0 the one that is the best default meaning for your enumeration. If no good default meaning exists, consider adding an “invalid” or “unknown” enumerator that has value 0, so that state is explicitly documented and can be explicitly handled where appropriate.

16.3.2 *Unscoped enumerations will implicitly convert to integral values*

Even though enumerations store integral values, they are not considered to be an integral type (they are a compound type). However, **an unscoped enumeration will implicitly convert to an integral value. Because enumerators are compile-time constants, this is a constexpr conversion** (we cover these in lesson: Narrowing conversions, list initialization, and constexpr initializers).

16.3.3 *Enumeration size and underlying type (base)*

Enumerators have values that are of an integral type. But what integral type? The specific integral type used to represent the value of enumerators is called the enumeration's **underlying type** (or **base**).

For unscoped enumerations, the C++ standard does not specify which specific integral type should be used as the underlying type, so the choice is **implementation-defined**. Most compilers will use **int** as the underlying type (meaning an unscoped enum will be the same size as an int), **unless a larger type is required to store the enumerator values**. But you shouldn't assume this will hold true for every compiler or platform.

It is **possible to explicitly specify an underlying type for an enumeration**. The underlying type must be an integral type. For example, if you are working in some bandwidth-sensitive context (e.g. sending data over a network) you may want to specify a smaller type for your enumeration:

```
// Use an 8-bit integer as the enum underlying type
enum Color : std::int8_t
{
    black,
    red,
    blue,
};
```

Best practice: Specify the base type of an enumeration only when necessary.

16.3.4 Integer to unscoped enumerator conversion

While the compiler will implicitly convert an unscoped enumeration to an integer, it **will not implicitly convert an integer to an unscoped enumeration**. The following will produce a compiler error:

```
enum Pet // no specified base
{
    cat, // assigned 0
    dog, // assigned 1
    pig, // assigned 2
    whale, // assigned 3
};

int main()
{
    Pet pet { 2 }; // compile error: integer value 2 won't implicitly convert to a Pet
    pet = 3;       // compile error: integer value 3 won't implicitly convert to a Pet
}
```

There are **two ways to work around this**:

- **explicitly convert** an integer to an unscoped enumerator using `static_cast`
- As of C++17, if an unscoped enumeration has an **explicitly specified base**, then the compiler will allow you to list initialize an unscoped enumeration using an integral value.

```
enum Pet: int // we've specified a base
{
    cat, // assigned 0
    dog, // assigned 1
    pig, // assigned 2
    whale, // assigned 3
};

int main()
{
    Pet pet1 { 2 }; // ok: can brace initialize unscoped enumeration with specified base with integer (C++17)
    Pet pet2 (2);   // compile error: cannot direct initialize with integer
    Pet pet3 = 2;   // compile error: cannot copy initialize with integer

    pet1 = 3;       // compile error: cannot assign with integer
}
```

16.4 Converting an enumeration to and from a string

Instead of working with integral values, we typically want to print the name of whatever the enumerator represents (e.g. blue). **C++ doesn't come with an out-of-the-box way to do this**, so we'll have to find a solution ourselves. Fortunately, that's not very difficult.

16.4.1 *Getting the name of an enumerator*

The typical way to get the name of an enumerator is to write a function that allows us to pass in an enumerator and returns the enumerator's name as a string. But that requires some way to determine which string should be returned for a given enumerator.

There are **two common ways to do this**:

```
constexpr std::string_view getColorName(Color color)
{
    switch (color)
    {
        case black: return "black";
        case red:   return "red";
        case blue:  return "blue";
        default:    return "???";
    }
}
```

The second way to solve the problem of mapping enumerators to strings is to use an array. We cover this in lesson: `std::array` and enumerations.

16.4.2 *Unscoped enumerator input*

Now let's take a look at an input case. In the following example, we define a `Pet` enumeration. Because `Pet` is a program-defined type, the language doesn't know how to input a `Pet` using `std::cin`:

```
Pet pet { pig };
std::cin >> pet; // compile error: std::cin doesn't know how to input a Pet
```

use `static_cast` to convert the integer to an enumerator of the appropriate enumerated type:

```
int input{};
std::cin >> input; // input an integer
```

```
Pet pet{ static_cast<Pet>(input) };
```


16.4.3 Getting an enumeration from a string

```
constexpr std::optional<Pet> getPetFromString(std::string_view sv)
{
    if (sv == "cat")    return cat;
    if (sv == "dog")    return dog;
    if (sv == "pig")    return pig;
    if (sv == "whale")  return whale;

    return {};
}
```

16.5 Introduction to overloading the I/O operators

In the prior lesson, we showed a code snippet, where we used a function to convert an enumeration into an equivalent string:

```
enum Color
{
    black,
    red,
    blue,
};

constexpr std::string_view getColorName(Color color)
{
    switch (color)
    {
        case black: return "black";
        case red:   return "red";
        case blue:  return "blue";
        default:    return "???";
    }
}

int main()
{
    constexpr Color shirt{ blue };

    std::cout << "Your shirt is " << getColorName(shirt) << '\n';
}
```

Although the above example works just fine, there are two downsides:

- We have to remember the name of the function we created to get the enumerator name.
- Having to call such a function adds clutter to our output statement.

Ideally, it would be nice if we could somehow teach `operator<<` to output an enumeration, so we could do something like this: `std::cout << shirt` and have it do what we expect.

16.5.1 Introduction to operator overloading

In lesson: Introduction to function overloading, we introduced function overloading, which allows us to create multiple functions with the same name so long as each function has a unique function prototype. Using function overloading, we can create variations of a function that work with different data types, without having to think up a unique name for each variant.

Similarly, C++ also supports operator overloading, which lets us define overloads of existing operators, so that we can make those operators work with our program-defined data types.

Basic operator overloading is fairly straightforward:

- Define a **function using the name of the operator** as the function's name.
- Add a **parameter of the appropriate type for each operand (in left-to-right order)**. One of these parameters must be a user-defined type (a class type or an enumerated type), otherwise the compiler will error.
- **Set the return type to whatever type makes sense.**
- Use a **return statement to return the result** of the operation.

When the compiler encounters the **use of an operator in an expression and one or more of the operands is a user-defined type, the compiler will check to see if there is an overloaded operator function that it can use to resolve that call**. For example, given some expression $x + y$, the compiler will use function overload resolution to see if there is an `operator+(x, y)` function call that it can use to evaluate the operation. If a non-ambiguous `operator+` function can be found, it will be called, and the result of the operation returned as the return value.

16.5.2 Overloading operator<< to print an enumerator

Before we proceed, let's quickly recap how operator<< works when used for output.

Consider a simple expression like `std::cout << 5`. `std::cout` has type `std::ostream` (which is a user-defined type in the standard library), and 5 is a literal of type `int`.

When this expression is evaluated, the compiler will look for an overloaded operator<< function that can handle arguments of type `std::ostream` and `int`. It will find such a function (also defined as part of the standard I/O library) and call it. Inside that function, `std::cout` is used to output `x` to the console (exactly how is implementation-defined). Finally, the operator<< function returns its left-operand (which in this case is `std::cout`), so that subsequent calls to operator<< can be chained.

With the above in mind, let's implement an overload of operator<< to print a `Color`:

```
enum Color
{
    black,
    red,
    blue,
};

constexpr std::string_view getColorName(Color color)
{
    switch (color)
    {
        case black: return "black";
        case red:   return "red";
        case blue:  return "blue";
        default:    return "???";
    }
}
```

```
// Teach operator<< how to print a Color
// std::ostream is the type of std::cout, std::cerr, etc...
// The return type and parameter type are references (to prevent copies from being made)
std::ostream& operator<<(std::ostream& out, Color color)
{
    out << getColorName(color); // print our color's name to whatever output stream out
    return out;                // operator<< conventionally returns its left operand

    // The above can be condensed to the following single line:
    // return out << getColorName(color)
}
```

16.5.3 Overloading operator>> to input an enumerator

Similar to how we were able to teach operator<< to output an enumeration above, we can also teach operator>> how to input an enumeration:

```
// pet is an in/out parameter
std::istream& operator>>(std::istream& in, Pet& pet)
{
    std::string s{};
    in >> s; // get input string from user

    std::optional<Pet> match { getPetFromString(s) };
    if (match) // if we found a match
    {
        pet = *match; // dereference std::optional to get matching enumerator
        return in;
    }

    // We didn't find a match, so input must have been invalid
    // so we will set input stream to fail state
    in.setstate(std::ios_base::failbit);

    // On an extraction failure, operator>> zero-initializes fundamental types
    // Uncomment the following line to make this operator do the same thing
    // pet = {};

    return in;
}
```

16.6 Scoped enumerations (enum classes)

Although unscoped enumerations are distinct types in C++, they are not type safe, and in some cases will allow you to do things that don't make sense. Consider the following case:

```
enum Color
{
    red,
    blue,
};

enum Fruit
{
    banana,
    apple,
};

Color color { red };
Fruit fruit { banana };

if (color == fruit) // The compiler will compare color and fruit as integers
    std::cout << "color and fruit are equal\n"; // and find they are equal!
else
    std::cout << "color and fruit are not equal\n";
```

This **doesn't make sense semantically** since color and fruit are from different enumerations and are not intended to be comparable. With standard enumerators, there's no easy way to prevent this.

Because of such challenges, as well as the **namespace pollution problem** (unscoped enumerations defined in the global scope put their enumerators in the global namespace), the **C++ designers determined that a cleaner solution for enumerations would be of use.**

16.6.1 *Scoped enumerations*

That solution is the **scoped enumeration** (often called an **enum class** in C++ for reasons that will become obvious shortly).

Scoped enumerations work similarly to unscoped enumerations, but have two primary differences:

- They **won't implicitly convert** to integers
- The **enumerators are only placed into the scope region of the enumeration** (not into the scope region where the enumeration is defined).

Unlike unscoped enumerations, which place their enumerators in the same scope as the enumeration itself, scoped enumerations place their enumerators *only* in the scope region of the enumeration. In other words, scoped enumerations act like a namespace for their enumerators. This built-in namespacing helps reduce global namespace pollution and the potential for name conflicts when scoped enumerations are used in the global scope.

To make a scoped enumeration, we use the keywords `enum class`. The rest of the scoped enumeration definition is the same as an unscoped enumeration definition. Here's an example:

```
enum class Color // "enum class" defines this as a scoped enumeration rather than an unscoped enumeration
{
    red, // red is considered part of Color's scope region
    blue,
};

enum class Fruit
{
    banana, // banana is considered part of Fruit's scope region
    apple,
};

Color color { Color::red }; // note: red is not directly accessible, we have to use Color::red
Fruit fruit { Fruit::banana }; // note: banana is not directly accessible, we have to use Fruit::banana

if (color == fruit) // compile error: the compiler doesn't know how to compare different types Color and Fruit
    std::cout << "color and fruit are equal\n";
else
    std::cout << "color and fruit are not equal\n";
```

Best practice: Favor scoped enumerations over unscoped enumerations unless there's a compelling reason to do otherwise.

Despite the benefits that scoped enumerations offer, **unscoped enumerations are still commonly used in C++ because there are situations where we desire the implicit conversion to int** (doing lots of static_casting gets annoying) and we don't need the extra namespacing.

16.6.2 *using enum statements C++20*

Introduced in C++20, a **using enum statement imports all of the enumerators from an enum into the current scope**. When used with an enum class type, this allows us to access the enum class enumerators without having to prefix each with the name of the enum class.

This can be useful in cases where we would otherwise have many identical, repeated prefixes, such as within a switch statement:

```
constexpr std::string_view getColor(Color color)
{
    using enum Color; // bring all Color enumerators into current scope (C++20)
    // We can now access the enumerators of Color without using a Color:: prefix

    switch (color)
    {
        case black: return "black"; // note: black instead of Color::black
        case red:   return "red";
        case blue:  return "blue";
        default:    return "???";
    }
}
```

16.7 Introduction to structs, members, and member selection

There are many instances in programming where we need more than one variable in order to represent something of interest. As we discussed in the introduction to the previous chapter (Introduction to compound data types), a fraction has a numerator and denominator that are linked together into a single mathematical object.

Alternatively, let's say we want to write a program where we need to store information about the employees in a company. We might be interested in keeping track of attributes such as the **employee's name, title, age, employee id, manager id, wage, birthday, hire date, etc...**

If we were to use independent variables to track all of this information, that might look something like this:

```
1  std::string name;  
2  std::string title;  
3  int age;  
4  int id;  
5  int managerId;  
6  double wage;  
7  int birthdayYear;  
8  int birthdayMonth;  
9  int birthdayDay;  
10 int hireYear;  
11 int hireMonth;  
12 int hireDay;
```

However, there are a **number of problems with this approach**. First, it's **not immediately clear whether these variables are actually related** or not (you'd have to read comments, or see how they are used in context). Second, there are now **12 variables to manage**. If we wanted to pass this employee to a function, **we'd have to pass 12 arguments** (and in the correct order), which would make a mess of our function prototypes and function calls. And since a function can only return a single value, how would a function even return an employee?

And if we wanted more than one employee, we'd need to define **12 more variables for each additional employee** (each of which would require a unique name)! **This clearly doesn't scale** at all. What we really need is some way to organize all of these related pieces of data together, to make them easier to manage.

Fortunately, **C++ comes with two compound types designed to solve such challenges: structs** (which we'll introduce now) **and classes** (which we'll explore soon). A **struct** (short for **structure**) is a program-defined data type (Introduction to program-defined (user-defined) types) that **allows us to bundle multiple variables together into a single type**. As you'll see shortly, this makes management of related sets of variables much simpler!

16.7.1 Defining structs

Essentially the same rules as enums apply.

Tip: In C++, a **member** is a variable, function, or type that belongs to a struct (or class). All members must be declared within the struct (or class) definition.

```
struct Employee
{
    int id {};
    int age {};
    double wage {};
};
```

16.7.2 Defining struct objects

```
Employee joe {};
```

This defines a variable of type Employee named joe. When the code is executed, an Employee **object is instantiated that contains the 3 data members**. The empty braces ensures our object is value-initialized.

Just like any other type, it is **possible to define multiple variables of the same struct type**.

In the above example, the name joe refers to the entire struct object (which contains the member variables). To access a specific member variable, we use the **member selection operator** (operator.) in between the struct variable name and the member name. For example, to access Joe's age member, we'd use joe.age:

```
Employee joe {};  
  
joe.age = 32; // use member selection operator (.) to select the age member of variable joe  
  
std::cout << joe.age << '\n'; // print joe's age
```

16.8 Struct aggregate initialization

We talked about how to define structs, instantiate struct objects, and access their members. In this lesson, we'll discuss how structs are initialized.

16.8.1 Data members are not initialized by default

Much like normal variables, data members are not initialized by default. Consider the following struct:

```
struct Employee
{
    int id; // note: no initializer here
    int age;
    double wage;
};

int main()
{
    Employee joe; // note: no initializer here either
    std::cout << joe.id << '\n';
}
```

Because we have not provided any initializers, when joe is instantiated, joe.id, joe.age, and joe.wage will all be uninitialized. We will then get **undefined behavior when we try to print the value of joe.id**.

However, before we show you how to initialize a struct, let's take a short **detour**.

16.8.2 What is an aggregate?

In general programming, an **aggregate data type** (also called an **aggregate**) is any type that can contain multiple data members. Some types of aggregates allow members to have different types (e.g. structs), while others require that all members must be of a single type (e.g. arrays).

In C++, the definition of an aggregate is narrower and quite a bit more complicated.

Advanced:

To simplify a bit, an aggregate in C++ is either a **C-style array**, or a **class type** (**struct**, **class**, or **union**) that has:

- No user-declared constructors
- No private or protected non-static data members
- No virtual functions

The **key thing to understand at this point is that structs with only data members are aggregates**.

16.8.3 Aggregate initialization of a struct

When we define an object with a struct type, **we need some way to initialize multiple members at initialization time**.

Aggregates use a form of initialization called **aggregate initialization**, which allows us to directly initialize the members of aggregates. To do this, we provide an **initializer list** as an initializer, which is just a braced list of comma-separated values.

There are **2 primary forms** of aggregate initialization:

```
struct Employee
{
    int id {};
    int age {};
    double wage {};
};

int main()
{
    Employee frank = { 1, 32, 60000.0 }; // copy-list initialization using braced list
    Employee joe { 2, 28, 45000.0 };     // list initialization using braced list (preferred)
```

Best practice: Prefer the (non-copy) braced list form when initializing aggregates.

In **C++20**, we can also initialize (some) aggregates using a parenthesized list of values. **We recommend avoiding this** last form as much as possible, as it doesn't currently work with aggregates that utilize brace elision (notably `std::array`).

16.8.4 Missing initializers in an initializer list

If an aggregate is initialized but the number of initialization values is fewer than the number of members, then **all remaining members are initialized with an empty initializer list**. In most cases, this will perform value-initialization on those members.

```
Employee joe { 2, 28 }; // joe.wage will be value-initialized to 0.0
```

This means we can generally use an empty initialization list to value-initialize all members of the struct:

```
Employee joe {}; // value-initialize all members
```

16.8.5 Overloading operator<< to print a struct

```
std::ostream& operator<<(std::ostream& out, const Employee& e)
{
    out << e.id << ' ' << e.age << ' ' << e.wage;
    return out;
}
```

16.8.6 Const structs

Variables of a struct type can be const (or constexpr), and just like all const variables, they must be initialized.

```
struct Rectangle
{
    double length {};
    double width {};
};

int main()
{
    const Rectangle unit { 1.0, 1.0 };
    const Rectangle zero { }; // value-initialize all members
}
```

16.8.7 Designated initializers C++20

When initializing a struct from a list of values, the initializers are applied to the members in order of declaration.

Now consider what would happen if you were to update this struct definition to add a new member that is **not the last member**:

```
struct Foo
{
    int a {};  
    int b {}; // just added  
    int c {};  
};
```

Now all your initialization values have shifted, and worse, the compiler may not detect this as an error (after all, the syntax is still valid).

To help avoid this, C++20 adds a new way to initialize struct members called **designated initializers**. Designated initializers allow you to explicitly define which initialization values map to which members. The members can be initialized using list or copy initialization, and must be initialized in the same order in which they are declared in the struct, otherwise a warning or error will result. Members not designated an initializer will be value initialized.

```
Foo f1{ .a{ 1 }, .c{ 3 } }; // ok: f1.a = 1, f1.b = 0 (value initialized), f1.c = 3  
Foo f2{ .a = 1, .c = 3 };   // ok: f2.a = 1, f2.b = 0 (value initialized), f2.c = 3  
Foo f3{ .b{ 2 }, .a{ 1 } }; // error: initialization order does not match order of declaration in struct
```

Also, because there's no enforcement that designated initializers are being used consistently everywhere an aggregate is initialized, it's a good idea to avoid adding new members to the middle of an existing aggregate definition, to avoid the risk of initializer shifting.

Best practice: When adding a new member to an aggregate, it's safest to add it to the bottom of the definition list so the initializers for other members don't shift.

16.8.8 Assignment with an initializer list

```
Employee joe { 1, 32, 60000.0 };  
joe = { joe.id, 33, 66000.0 }; // Joe had a birthday and got a raise
```

16.8.9 Assignment with designated initializers C++20

```
Employee joe { 1, 32, 60000.0 };  
joe = { .id = joe.id, .age = 33, .wage = 66000.0 }; // Joe had a birthday and got a raise
```

16.8.10 Initializing a struct with another struct of the same type

A struct may also be initialized using another struct of the same type:

```
Foo foo { 1, 2, 3 };  
  
Foo x = foo; // copy initialization  
Foo y(foo); // direct initialization  
Foo z {foo}; // list initialization
```

Note that this uses the standard forms of initialization that we're familiar with (copy, direct, or list initialization) rather than aggregate initialization.

This is most commonly seen when initializing a struct with the return value of a function that returns a struct of the same type.

16.9 Default member initialization

When we define a struct (or class) type, **we can provide a default initialization value for each member** as part of the type definition. For members not marked as static, this process is sometimes called **non-static member initialization**. The initialization value is called a **default member initializer**.

```
struct Something  
{  
    int x;        // no initialization value (bad)  
    int y {};     // value-initialized by default  
    int z { 2 };  // explicit default value  
};
```

Key insight: Using default member initializers (or other mechanisms that we'll cover later), structs and classes can self-initialize even when no explicit initializers are provided!

16.9.1 Explicit initialization values take precedence over default values

Explicit values in a list initializer always take precedence over default member initialization values:

```
Something s2 { 5, 6, 7 }; // use explicit initializers for s2.x, s2.y, and s2.z (no default values are used)
```

16.9.2 Missing initializers in an initializer list when default values exist

In the previous lesson (Struct aggregate initialization) we noted that if an aggregate is initialized but the number of initialization values is fewer than the number of members, then all remaining members will be value-initialized. **However, if a default member initializer is provided for a given member, that default member initializer will be used instead.**

```
struct Something
{
    int x;      // no default initialization value (bad)
    int y {};   // value-initialized by default
    int z { 2 }; // explicit default value
};

int main()
{
    Something s3 {}; // value initialize s3.x, use default values for s3.y and s3.z
```

In the above case, `s3` is list initialized with an empty list, so all initializers are missing. This means that a default member initializer will be used if it exists, and value initialization will occur otherwise. Thus, `s3.x` (which has no default member initializer) is **value initialized to 0**, `s3.y` is **value initialized by default to 0**, and `s3.z` is **defaulted to value 2**.

16.9.3 Recapping the initialization possibilities

If an aggregate is **defined with an initialization list**:

- If an explicit initialization value exists, that explicit value is used.
- If an initializer is missing and a default member initializer exists, the default is used.
- If an initializer is missing and no default member initializer exists, value initialization occurs.

If an aggregate is **defined with no initialization list**:

- If a default member initializer exists, the default is used.
- If no default member initializer exists, the member remains uninitialized.

Members are always initialized in the order of declaration.

The following example recaps all possibilities:

```
struct Something
{
    int x;        // no default initialization value (bad)
    int y {};     // value-initialized by default
    int z { 2 };  // explicit default value
};

int main()
{
    Something s1;           // No initializer list: s1.x is uninitialized, s1.y and s1.z use defaults
    Something s2 { 5, 6, 7 }; // Explicit initializers: s2.x, s2.y, and s2.z use explicit values (no default values are used)
    Something s3 {};        // Missing initializers: s3.x is value initialized, s3.y and s3.z use defaults

    return 0;
}
```

The **case we want to watch out for is s1.x**. Because s1 has no initializer list and x has no default member initializer, s1.x remains uninitialized (which is bad, since we should always initialize our variables).

Always provide default values for your members

To avoid the possibility of uninitialized members, simply ensure that each member has a default value (either an **explicit default value**, or an **empty pair of braces**). That way, our members will be initialized with some value regardless of whether we provide an initializer list or not.

Best practice: Provide a default value for all members. This ensures that your members will be initialized even if the variable definition doesn't include an initializer list.

16.9.4 Default initialization vs value initialization for aggregates

```
struct Fraction
{
    int numerator { }; // we should use { 0 } here, but for the sake of example we'll use value initialization instead
    int denominator { 1 };
};

int main()
{
    Fraction f1;          // f1.numerator value initialized to 0, f1.denominator defaulted to 1
    Fraction f2 {};       // f2.numerator value initialized to 0, f2.denominator defaulted to 1
}
```

You'll note that f1 is default initialized and f2 is value initialized, yet the results are the same (numerator is initialized to 0 and denominator is initialized to 1). So, **which should we prefer?**

The **value initialization case (f2) is safer**, because it will ensure any members with no default values are value initialized (and although we should always provide default values for members, this protects against the case where one is missed).

Preferring value initialization has **one more benefit -- it's consistent with how we initialize objects of other types**. Consistency helps prevent errors.

Best practice: For aggregates, prefer value initialization (with an empty braces initializer) to default initialization (with no braces).

16.10 Passing and returning structs

16.10.1 Passing structs (by reference)

A big advantage of using structs over individual variables is that we can pass the entire struct to a function that needs to work with the members. **Structs are generally passed by reference (typically by const reference) to avoid making copies.**

```
void printEmployee(const Employee& employee) // note pass by reference here
{
    std::cout << "ID:   " << employee.id << '\n';
    std::cout << "Age:  " << employee.age << '\n';
    std::cout << "Wage: " << employee.wage << '\n';
}
```

16.10.2 Passing temporary structs

In the prior example, we created Employee variable `joe` prior to passing it to the `printEmployee()` function. This allows us to give the Employee variable a name, which can be useful for documentation purposes. But it also **requires two statements (one to create `joe`, one to use `joe`)**.

In cases where we only use a variable once, having to give the variable a name and separate the creation and use of that variable can increase complexity. **In such cases, it may be preferable to use a temporary object** instead. A temporary object is not a variable, so it does not have an identifier.

Here's the same example as above, but we've replaced variables `joe` and `frank` with temporary objects:

```
// Print Joe's information
printEmployee(Employee { 14, 32, 24.15 }); // construct a temporary Employee to pass to function (type explicitly specified) (preferred)

std::cout << '\n';

// Print Frank's information
printEmployee({ 15, 28, 18.27 }); // construct a temporary Employee to pass to function (type deduced from parameter)
```

A few more things about temporary objects: **they are created and initialized at the point of definition**, and are **destroyed at the end of the full expression in which they are created**. And evaluation of a temporary object is an rvalue expression, it can only be used in places where rvalues are accepted. When a temporary object is used as a function argument, it will only bind to parameters that accept rvalues. This includes pass by value and pass by const reference, and excludes pass by non-const reference and pass by address.

16.10.3 Returning structs

```
struct Point3d
{
    double x { 0.0 };
    double y { 0.0 };
    double z { 0.0 };
};

Point3d getZeroPoint()
{
    // We can create a variable and return the variable (we'll improve this below)
    Point3d temp { 0.0, 0.0, 0.0 };
    return temp;
}
```

Structs are usually returned by value, so as not to return a dangling reference.

In this case, a temporary Point3d is constructed, copied back to the caller, and then destroyed at the end of the expression. Note how much cleaner this is (one line vs two, and no need to understand whether temp is used more than once).

16.10.4 Deducing the return type

In the case where the function has an explicit return type (e.g. Point3d), we can even omit the type in the return statement:

```
Point3d getZeroPoint()
{
    // We can use empty curly braces to value-initialize all members
    return {};
}
```

16.10.5 Structs are an important building block

While structs are useful in and of themselves, **classes** (which are the heart of C++ and object-oriented programming) **build directly on top of the concepts we've introduced here**. Having a **good understanding of structs** (especially data members, member selection, and default member initialization) **will make your transition to classes that much easier**.

16.11 Struct miscellany

In C++, structs (and classes) can have members that are other program-defined types. There are **two ways to do this**:

- We can **define one program-defined type** (in the global scope) and then **use it as a member of another program-defined type**.

```
struct Employee
{
    int id {};
    int age {};
    double wage {};
};

struct Company
{
    int numberOfEmployees {};
    Employee CEO {}; // Employee is a struct within the Company struct
};
```

- Second, **types can also be nested inside other types**, so if an Employee only existed as part of a Company, the Employee type could be nested inside the Company struct.

```
struct Company
{
    struct Employee // accessed via Company::Employee
    {
        int id{};
        int age{};
        double wage{};
    };

    int numberOfEmployees{};
    Employee CEO{}; // Employee is a struct within the Company struct
};
```

16.11.1 Structs that are owners should have data members that are owners

In lesson: `std::string_view` (part 2), we introduced the **dual concepts of owners and viewers**. Owners manage their own data, and control when it is destroyed. Viewers view someone else's data, and do not control when it is altered or destroyed.

In most cases, **we want our structs (and classes) to be owners of the data they contain**. This provides a few useful benefits:

- The data members will be valid for as long as the struct (or class) is.
- The value of those data members won't change unexpectedly.

The easiest way to make a struct (or class) an owner is to **give each data member a type that is an owner** (e.g. not a viewer, pointer, or reference). If a struct or class has data members that are all owners, then the struct or class itself is automatically an owner.

If a struct (or class) has a data member that is a viewer, it is **possible that the object being viewed by that member will be destroyed before the data member that is viewing it**. If this happens, the **struct will be left with a dangling member**, and accessing that member will lead to **undefined behavior**.

Best practice: In most cases, we want our structs (and classes) to be owners. The easiest way to enable this is to ensure each data member has an owning type (e.g. not a viewer, pointer, or reference). Practice safe structs. Don't let your member dangle.

16.11.2 Struct size and data structure alignment

Typically, the size of a struct is the sum of the size of all its members, but **not always!**

Let's say we have a struct `Foo` with members `a`, `b` and `c`:

```
The size of short is 2 bytes
The size of int is 4 bytes
The size of double is 8 bytes
The size of Foo is 16 bytes
```

Note that the size of `short + int + double` is **14 bytes**, but the size of `Foo` is **16 bytes!**

It turns out, **we can only say that the size of a struct will be at least as large as the size of all the variables it contains**. But it **could be larger!** For performance reasons, the compiler will sometimes add gaps into structures (this is called **padding**).

In the Foo struct above, the compiler is invisibly adding 2 bytes of padding after member a, making the size of the structure 16 bytes instead of 14. This can actually have a **pretty significant impact** on the size of the struct.

Tip:

You can **minimize padding** by **defining your members in decreasing order** of size.

The C++ **compiler is not allowed** to reorder members, so this **has to be done manually**.

16.12 Member selection with pointers and references

16.12.1 Member selection for structs and references to structs

Since references to an object act just like the object itself, we can also use the **member selection operator (.)** to select a member from a reference to a struct:

```
void printEmployee(const Employee& e)
{
    // Use member selection operator (.) to select member from reference to struct
    std::cout << "Id: " << e.id << '\n';
    std::cout << "Age: " << e.age << '\n';
    std::cout << "Wage: " << e.wage << '\n';
}
```

16.12.2 Member selection for pointers to structs

However, the member selection operator (.) **can't be used directly on a pointer to a struct**:

```
Employee joe{ 1, 34, 65000.0 };

++joe.age;
joe.wage = 68000.0;

Employee* ptr{ &joe };
std::cout << ptr.id << '\n'; // Compile error: can't use operator. with pointers
```

Because pointers hold addresses, we first need to **dereference the pointer** to get the object before we can do anything with it. So, one way to access a member from a pointer to a struct is as follows:

```
Employee* ptr{ &joe };  
std::cout << (*ptr).id << '\n'; // Not great but works: First dereference ptr, then use member selection
```

However, this is a bit ugly, especially because we need to parenthesize the dereference operation so it will take precedence over the member selection operation.

To make for a cleaner syntax, C++ offers a **member selection from pointer operator (->)** (also sometimes called the **arrow operator**) that can be used to select members from a pointer to an object:

```
Employee* ptr{ &joe };  
std::cout << ptr->id << '\n'; // Better: use -> to select member from pointer to object
```

This member selection from pointer operator (->) works identically to the member selection operator (.) but **does an implicit dereference of the pointer object before selecting the member**. Thus **ptr->id is equivalent to (*ptr).id**.

This arrow operator is not only easier to type, but is also **much less prone to error** because the indirection is implicitly done for you, so there are no precedence issues to worry about. Consequently, when doing member access through a pointer, always use the -> operator instead of the . operator.

Best practice: When using a pointer to access a member, use the member selection from pointer operator (->) instead of the member selection operator (.).

16.12.3 Chaining operator->

If the member accessed via operator-> is a pointer to a class type, **operator-> can be applied again in the same expression to access the member** of that class type.

The following example illustrates this (courtesy of reader Luna):

```
// ptr is a pointer to a Triangle, which contains members that are pointers to a Point
// To access member y of Point c of the Triangle pointed to by ptr, the following are equivalent:

// access via operator.
std::cout << (*(ptr).c).y << '\n'; // ugly!

// access via operator->
std::cout << ptr->c->y << '\n'; // much nicer
```

16.12.4 Mixing pointers and non-pointers to members

The member selection operator is always applied to the currently selected variable. If you have a mix of pointers and normal member variables, you can see member selections where . and -> are both used in sequence:

```
struct Paw
{
    int claws{};
};

struct Animal
{
    std::string name{};
    Paw paw{};
};

int main()
{
    Animal puma{ "Puma", { 5 } };

    Animal* ptr{ &puma };

    // ptr is a pointer, use ->
    // paw is not a pointer, use .

    std::cout << (ptr->paw).claws << '\n';
```

Note that in the case of (ptr->paw).claws, parentheses aren't necessary since both operator-> and operator. evaluate in left to right order, but it does help readability slightly.

16.13 Class templates

In lesson: Function templates, we introduced the challenge of having to create a separate (overloaded) function for each different set of types we want to work with. The **solution to this was to create a function template** that the compiler can use to instantiate normal functions for whichever set of types we need.

16.13.1 Aggregate types have similar challenges

We run into similar challenges with aggregate types (both structs/classes/unions and arrays).

For example, let's say we're writing a program where we need to work with pairs of int values, and need to determine which of the two numbers is larger. We might write a program like this:

```
struct Pair
{
    int first{};
    int second{};
};
```

Later, we discover that we also need pairs of double values. So, we update our program to the following:

```
struct Pair
{
    int first{};
    int second{};
};

struct Pair // compile error: erroneous redefinition of Pair
{
    double first{};
    double second{};
};
```

Unfortunately, this program **won't compile**, and has a **number of problems that need to be addressed**.

First, **unlike functions, type definitions can't be overloaded**. The compiler will treat double second definition of Pair as an erroneous redeclaration of the first definition of Pair. Second, although functions can be overloaded, our max(Pair) functions only differ by return type, and overloaded functions can't be differentiated solely by return type. Third, **there is a lot of redundancy here**. Each Pair struct is identical (except for the data type) and same with our max(Pair) functions (except for the return type).

We could solve the first two issues by giving our Pair structs different names (e.g. PairInt and PairDouble). But then **we both have to remember our naming scheme, and essentially clone a bunch of code** for each additional pair type we want, which doesn't solve the redundancy problem.

Fortunately, we can do better.

16.13.2 Class templates

Much like a function template is a template definition for instantiating functions, a **class template** is a template definition for instantiating **class types**.

A reminder: A "class type" is a **struct**, **class**, or **union** type. Although we'll be demonstrating "class templates" on structs for simplicity, everything here applies equally well to classes.

Definition:

```
template <typename T>
struct Pair
{
    T first{};
    T second{};
};
```

Declaration and initialization:

```
Pair<int> p1{ 5, 6 };           // instantiates Pair<int> and creates object p1
```

16.13.3 Using our class template in a function

```
template <typename T>
struct Pair
{
    T first{};
    T second{};
};

template <typename T>
constexpr T max(Pair<T> p)
{
    return (p.first < p.second ? p.second : p.first);
}
```

As with all calls to a function template, **we can either be explicit about the template type argument (e.g. `max<int>(p1)`) or we can be implicit (e.g. `max(p2)`)** and let the compiler use template argument deduction to determine what the template type argument should be.

16.13.4 Class templates with template type and non-template type members

Class templates can have some members using a template type and other members using a normal (non-template) type. For example:

```
template <typename T>
struct Foo
{
    T first{};    // first will have whatever type T is replaced with
    int second{}; // second will always have type int, regardless of what type T is
};
```

16.13.5 Class templates with multiple template types

Class templates can also have multiple template types. For example, if we wanted the two members of our Pair class to be able to have different types, we can define our Pair class template with two template types:

```
template <typename T, typename U>
struct Pair
{
    T first{};
    U second{};
};

template <typename T, typename U>
void print(Pair<T, U> p)
{
    std::cout << '[' << p.first << ", " << p.second << ']'<
}
```

To define multiple template types, in our template parameter declaration, we **separate each of our desired template types with a comma**.

16.13.6 Making a function template work with more than one class type

```
template <typename T, typename U>
struct Pair
{
    T first{};
    U second{};
};

struct Point
{
    int first{};
    int second{};
};

template <typename T>
void print(T p) // type template parameter will match anything
{
    std::cout << '[' << p.first << ", " << p.second << ']'< // will only compile if type has first and second members
}

int main()
{
    Pair<double, int> p1{ 4.5, 6 };
    print(p1); // matches print(Pair<double, int>)

    std::cout << '\n';

    Point p2 { 7, 8 };
    print(p2); // matches print(Point)
}
```

In the above example, we've rewritten `print()` so that it has only a single type template parameter (`T`), which will match any type. The body of the function will compile successfully for any class type that has a first and second member. We demonstrate this by calling `print()` with an object of type `Pair<double, int>`, and then again with an object of type `Point`.

There is **one case that can be misleading**. Consider the following version of `print()`:

```
template <typename T, typename U>
struct Pair // defines a class type named Pair
{
    T first{};
    U second{};
};

template <typename Pair> // defines a type template parameter named Pair (shadows Pair class type)
void print(Pair p)      // this refers to template parameter Pair, not class type Pair
{
    std::cout << '[' << p.first << ", " << p.second << ']'<
}
}
```

This is a **good reason to stick to simple template parameter names**, such as `T`, `U`, `N`, as they are **less likely to shadow a class type name**.

16.13.7 `std::pair`

Because working with pairs of data is common, the C++ standard library contains a class template named `std::pair` (in the `<utility>` header) that is defined identically to the `Pair` class template with multiple template types in the preceding section. In fact, we can swap out the `pair` struct we developed for `std::pair`:

```
template <typename T, typename U>
void print(std::pair<T, U> p)
{
    std::cout << '[' << p.first << ", " << p.second << ']'<
}
}
```

We developed our own `Pair` class in this lesson to show how things work, but in real code, you should **favor `std::pair` over writing your own**.

16.13.8 Using class templates in multiple files

Just like function templates, class templates are typically defined in header files so they can be included into any code file that needs them. Both template definitions and type definitions are exempt from the one-definition rule, so this won't cause problems:

```

1  #ifndef PAIR_H
2  #define PAIR_H
3
4  template <typename T>
5  struct Pair
6  {
7      T first{};
8      T second{};
9  };
10
11 template <typename T>
12 constexpr T max(Pair<T> p)
13 {
14     return (p.first < p.second ? p.second : p.first);
15 }
16
17 #endif

```

16.14 Class template argument deduction (CTAD) and deduction guides

16.14.1 Class template argument deduction (CTAD) C++17

Starting in C++17, when instantiating an object from a class template, the **compiler can deduce the template types from the types of the object's initializer** (this is called **class template argument deduction** or **CTAD** for short). For example:

```

std::pair<int, int> p1{ 1, 2 }; // explicitly specify class template std::pair<int, int> (C++11 onward)
std::pair p2{ 1, 2 };         // CTAD used to deduce std::pair<int, int> from the initializers (C++17)

```

CTAD is only performed if no template argument list is present. Therefore, both of the following are errors:

```

std::pair<> p1 { 1, 2 };    // error: too few template arguments, both arguments not deduced
std::pair<int> p2 { 3, 4 }; // error: too few template arguments, second argument not deduced

```

Since CTAD is a form of type deduction, we can use literal suffixes to change the deduced type:

```
std::pair p1 { 3.4f, 5.6f }; // deduced to pair<float, float>
std::pair p2 { 1u, 2u };    // deduced to pair<unsigned int, unsigned int>
```

16.14.2 Template argument deduction guides C++17

In most cases, CTAD works right out of the box. However, in certain cases, the compiler may need a little extra help understanding how to deduce the template arguments properly.

You may be surprised to find that the following program (which is almost identical to the example that uses `std::pair` above) doesn't compile in C++17 (only):

```
Pair<int, int> p1{ 1, 2 }; // ok: we're explicitly specifying the template arguments
Pair p2{ 1, 2 };         // compile error in C++17 (okay in C++20)
```

If you compile this in C++17, you'll likely get some error about "class template argument deduction failed" or "cannot deduce template arguments" or "No viable constructor or deduction guide". This is because in C++17, CTAD doesn't know how to deduce the template arguments for aggregate class templates. To address this, we can provide the compiler with a **deduction guide**, which tells the compiler how to deduce the template arguments for a given class template.

Here's the **same program with a deduction guide**:

```
// Here's a deduction guide for our Pair (needed in C++17 only)
// Pair objects initialized with arguments of type T and U should deduce to Pair<T, U>
template <typename T, typename U>
Pair(T, U) -> Pair<T, U>;

int main()
{
    Pair<int, int> p1{ 1, 2 }; // explicitly specify class template Pair<int, int> (C++11 onward)
    Pair p2{ 1, 2 };         // CTAD used to deduce Pair<int, int> from the initializers (C++17)
```

This example should compile under C++17.

The deduction guide for our Pair class is pretty simple, but let's take a closer look at how it works.

```
// Here's a deduction guide for our Pair (needed in C++17 only)
// Pair objects initialized with arguments of type T and U should deduce to Pair<T, U>
template <typename T, typename U>
Pair(T, U) -> Pair<T, U>;
```

First, we use the same template type definition as in our Pair class. This makes sense, because if our deduction guide is going to tell the compiler how to deduce the types for a Pair<T, U>, we have to define what T and U are (template types). Second, on the right-hand side of the arrow, we have the type that we're helping the compiler to deduce. In this case, we want the compiler to be able to deduce template arguments for objects of type Pair<T, U>, so that's exactly what we put here. Finally, on the left side of the arrow, we tell the compiler what kind of declaration to look for. In this case, we're telling it to look for a declaration of some object named Pair with two arguments (one of type T, the other of type U). We could also write this as Pair(T t, U u) (where t and u are the names of the parameters, but since we don't use t and u, we don't need to give them names).

Putting it all together, we're telling the compiler that if it sees a declaration of a Pair with two arguments (of types T and U respectively), it should deduce the type to be a Pair<T, U>.

So, when the compiler sees the definition Pair p2{ 1, 2 }; in our program, it will say, "oh, this is a declaration of a Pair and there are two arguments of type int and int, so using the deduction guide, I should deduce this to be a Pair<int, int>".

16.14.3 Type template parameters with default values

Just like function parameters can have default arguments, **template parameters can be given default values**. These will be used when the template parameter isn't explicitly specified and can't be deduced.

```
template <typename T=int, typename U=int> // default T and U to type int
struct Pair
{
    T first{};
    U second{};
};
```


16.14.4 CTAD doesn't work with non-static member initialization

When initializing the member of a class type using non-static member initialization, CTAD will not work in this context. All template arguments must be explicitly specified:

```
struct Foo
{
    std::pair<int, int> p1{ 1, 2 }; // ok, template arguments explicitly specified
    std::pair p2{ 1, 2 };          // compile error, CTAD can't be used in this context
};
```

16.14.5 CTAD doesn't work with function parameters

CTAD stands for class template **argument deduction**, not class template *parameter* deduction, so it will only deduce the type of template arguments, not template parameters. Therefore, CTAD can't be used in function parameters.

```
void print(std::pair p) // compile error, CTAD can't be used here
{
    std::cout << p.first << ' ' << p.second << '\n';
}
```

In such cases, you should use a template instead:

```
template <typename T, typename U>
void print(std::pair<T, U> p)
{
    std::cout << p.first << ' ' << p.second << '\n';
}
```

16.15 Alias templates

In lesson: Typedefs and type aliases, we discussed how type aliases let us define an alias for an existing type. Such aliases can be defined locally (e.g. inside a function) or globally.

Creating a type alias for a class template where all template arguments are explicitly specified works just like a normal type alias:

```
template <typename T>
struct Pair
{
    T first{};
    T second{};
};

template <typename T>
void print(const Pair<T>& p)
{
    std::cout << p.first << ' ' << p.second << '\n';
}

int main()
{
    using Point = Pair<int>; // create normal type alias
    Point p { 1, 2 };        // compiler replaces this with Pair<int>
```

16.15.1 Alias templates

In other cases, we might want a type alias for a template class where not all of the template arguments are defined as part of the alias (and will instead be provided by the user of the type alias). To do this, we can define an **alias template**, which is a template that can be used to instantiate type aliases. Just like type aliases do not define distinct types, alias templates do not define distinct types.

Here's an example of how this works:

```
template <typename T>
struct Pair
{
    T first{};
    T second{};
};

// Here's our alias template
// Alias templates must be defined in global scope
template <typename T>
using Coord = Pair<T>; // Coord is an alias for Pair<T>

// Our print function template needs to know that Coord's template parameter T is a type template parameter
template <typename T>
void print(const Coord<T>& c)
{
    std::cout << c.first << ' ' << c.second << '\n';
}

Coord<int> p1 { 1, 2 }; // Pre C++20: We must explicitly specify all type template argument
Coord p2 { 1, 2 };    // In C++20, we can use alias template deduction to deduce the template arguments in cases where CTAD works
```

There are a couple of things worth noting about this example.

First, unlike normal type aliases (which can be defined inside a block), alias templates must be defined in the global scope (as all templates must).

Second, prior to C++20, we must explicitly specify the template arguments when we instantiate an object using an alias template. As of C++20, we can use **alias template deduction**, which will deduce the type of the template arguments from an initializer in cases where the aliased type would work with CTAD.

Third, because CTAD doesn't work on function parameters, when we use an alias template as a function parameter, we must explicitly define the template arguments used by the alias template.

16.16 Summary

A **program-defined type** (also called a **user-defined type**) is a custom type that we can create for use in our own programs. The enumerated types and class types (including structs, classes and unions) allow for creation of program-defined types. Program-defined types must be defined before they can be used. The definition of a program-defined type is called a type definition. **Type definitions** are exempt from the one-definition rule.

An **enumeration** (also called an **enumerated type** or an **enum**) is a compound data type where every possible value is defined as a symbolic constant (called an **enumerator**). Enumerations are **distinct types**, meaning the compiler can differentiate them from other types (unlike type aliases).

Unscoped enumerations are named such because they put their enumerator names into the same scope as the enumeration definition itself (as opposed to creating a new scope region like a namespace does). Unscoped enumerations also provide a named scope region for their enumerators. Unscoped enumerations will implicitly convert to integral values.

Scoped enumerations work similarly to unscoped enumerations but won't implicitly convert to integers, and the enumerators are only placed into the scope region of the enumeration (not into the scope region where the enumeration is defined).

A **struct** (short for **structure**) is a program-defined data type that allows us to bundle multiple variables together into a single type. The variables that are part of the struct (or class) are called **data members** (or **member variables**). To access a specific member variable, we use the member selection operator (**operator.**) in between the struct variable name and the member name (for normal structs and references to structs), or the member selection from pointer operator (**operator->**) (for pointers to structs).

In general programming, an **aggregate data type** (also called an **aggregate**) is any type that can contain multiple data members. In C++, arrays and structs with only data members are **aggregates**.

Aggregates use a form of initialization called **aggregate initialization**, which allows us to directly initialize the members of aggregates. To do this, we provide an **initializer list** as an initializer, which is just a list of comma-separated values. Aggregate initialization does a **memberwise initialization**, which means each member in the struct is initialized in the order of declaration.

In C++20, **Designated initializers** allow you to explicitly define which initialization values map to which members. The members must be initialized in the order in which they are declared in the struct, otherwise an error will result.

When we define a struct (or class) type, we can provide a default initialization value for each member as part of the type definition. This process is called **non-static member initialization**, and the initialization value is called a **default member initializer**.

For performance reasons, the compiler will sometimes add gaps into structures (this is called **padding**), so the size of a structure may be larger than the sum of the size of its members.

A **class template** is a template definition for instantiating class types (structs, classes, or unions). **Class template argument deduction (CTAD)** is a **C++17 feature** that allows the compiler to deduce the template type arguments from an initializer.

17 Summary Intermezzo

Attention: from this point forward, the summary will deviate from the learning path that Learncpp provides. The summary will still use Learncpp as it's main resource for information but will take another path. E.g. instead of going into C++ OOP and classes we'll cover memory management and other C-related information.

18 C-style: Arrays and strings

18.1 Introduction to C-style arrays

C-style arrays were inherited from the C language, and are built-in to the core language of C++ (unlike the rest of the array types, which are standard library container classes). This means we don't need to #include a header file to use them.

As an aside...: Because they are the only array type natively supported by the language, the standard library array container types (e.g. `std::array` and `std::vector`) are typically implemented using a C-style array.

18.1.1 Declaring a C-style array

Because they are part of the core language, C-style arrays have their own special declaration syntax. In an C-style array declaration, we use **square brackets ([])** to tell the compiler that a declared object is a C-style array. Inside the square brackets, we can **optionally provide the length of the array**, which is an integral value of type `std::size_t` that tells the compiler how many elements are in the array.

The array length of a c-style array must be a constant expression.

```
int testScore[30] {};
```

Defines a C-style array named `testScore` that contains 30 value-initialized `int` elements (no include required).

The length of a **C-style array must be at least 1**. The compiler will error if the array length is zero, negative, or a non-integral value.

For advanced readers: C-style arrays dynamically allocated on the heap are allowed to have length 0.

Subscripting a C-style array

Just like with a `std::array`, C-style arrays can be **indexed using the subscript operator (`operator[]`)**:

```
arr[1] = 7; // use subscript operator to index array element 1
std::cout << arr[1]; // prints 7
```

`operator[]` does not do any bounds checking, and passing in an out-of-bounds index will result in undefined behavior.

18.1.2 Aggregate initialization of C-style arrays

Just like `std::array`, **C-style arrays are aggregates**, which means they **can be initialized using aggregate initialization**.

As a quick recap, **aggregate initialization allows us to directly initialize the members of aggregates**. To do this, we provide an initializer list, which is a brace-enclosed list of comma-separated initialization values.

```
int fibonacci[6] = { 0, 1, 1, 2, 3, 5 }; // copy-list initialization using braced list
int prime[5] { 2, 3, 5, 7, 11 }; // list initialization using braced list (preferred)
```

One downside of using a C-style array is that the **element's type must be explicitly specified**. **CTAD doesn't work** because C-style arrays aren't class templates. And **using `auto` to try to deduce the element type of an array from the list of initializers doesn't work either**.

18.1.3 Omitted length

```
const int prime[5] { 2, 3, 5, 7, 11 }; // prime has length 5
```

We're explicitly telling the compiler the array has length 5, and then we're also initializing it with 5 elements. When we initialize a C-style array with an initializer list, we can omit the length (in the array definition) and let the compiler deduce the length of the array from the number of initializers.

Best practice: Prefer omitting the length of a C-style array when explicitly initializing every array element with a value.

18.1.4 *Const and constexpr C-style arrays*

Just like `std::array`, C-style arrays can be `const` or `constexpr`. Just like other `const` variables, **const arrays must be initialized**, and the value of the elements cannot be changed afterward.

18.1.5 *The sizeof a C-style array*

In previous lessons, we used the `sizeof()` operator to get the size of an object or type in bytes. Applied to a C-style array, `sizeof()` returns the number of bytes used by the entire array:

```
const int prime[] { 2, 3, 5, 7, 11 }; // the compiler will deduce prime to have length 5
std::cout << sizeof(prime); // prints 20 (assuming 4 byte ints)
```

Assuming 4-byte ints, the above program prints 20. The prime array contains **5 int elements that are 4 bytes each, so $5 * 4 = 20$ bytes**.

Note that there is **no overhead here**. An C-style array object contains its elements and nothing more.

18.1.6 *Getting the length of a C-style array*

In **C++17**, we can use `std::size()` (defined in the `<iterator>` header), which **returns the array length as an unsigned integral value (of type `std::size_t`)**. In **C++20**, we can also use `std::ssize()`, which **returns the array length as a signed integral value (of a large signed integral type, probably `std::ptrdiff_t`)**.

```
std::cout << std::size(prime) << '\n'; // C++17, returns unsigned integral value 5
std::cout << std::ssize(prime) << '\n'; // C++20, returns signed integral value 5
```

18.1.7 *Getting the length of a C-style array (C++14 or older)*

Older codebases:

```
int array[8] {};
std::cout << "The array has: " << sizeof(array) / sizeof(array[0]) << " elements\n";
```


More modern codebases:

```
template <typename T, std::size_t N>
constexpr std::size_t length(const T(&)[N]) noexcept
{
    return N;
}
```

18.1.8 C-style arrays don't support assignment

Perhaps surprisingly, C++ arrays don't support assignment:

```
int arr[] { 1, 2, 3 }; // okay: initialization is fine
arr[0] = 4;           // assignment to individual elements is fine
arr = { 5, 6, 7 };    // compile error: array assignment not valid
```

18.2 C-style array decay

The designers of the C language had a problem. Consider the following simple program:

```
int x { 5 };
print(x);
```

Because `x` is cheap to copy, there's no problem here. Now consider the following similar program, which uses a 1000 element C-style int array instead of a single int:

```
int x[1000] { 5 }; // define an array with 1000 elements, x[0] is initialized to 5
printElementZero(x);
```

This program also compiles and prints the expected value (5) to the console.

While the code in this example is similar to the code in the prior example, **mechanically it works a bit different than you might expect**. And that is **due to the solution that the C designers came up to solve for two major challenges**.

First, **copying a 1000 element array every time a function is called is expensive** (and even more so if the elements are an expensive-to-copy type), so we want to avoid that. But how? **C doesn't have references, so using pass by reference to avoid making a copy of function arguments wasn't an option.**

Second, **we want to be able to write a single function that can accept array arguments of different lengths.** Ideally, our `printElementZero()` function in the example above should be callable with arrays arguments of any length (since element 0 is guaranteed to exist). We don't want to have to write a different function for every possible array length that we want to use as an argument. But how? C has no syntax to specify "any length" arrays, nor does it support templates, nor can arrays of one length be converted to arrays of another length (presumably because doing so would involve making an expensive copy).

The designers of the C language came up with a clever solution (inherited by C++ for compatibility reasons) that solves for both of these issues:

```
void printElementZero(int arr[1000]) // doesn't make a copy
{
    std::cout << arr[0]; // print the value of the first element
}

int main()
{
    int x[7] { 5 }; // define an array with 7 elements
    printElementZero(x); // somehow works!

    return 0;
}
```

Somehow, the above example passes a 7-element array to a function expecting a 1000 element array, without any copies being made. In this lesson, we'll explore how this works.

We'll also take a look at why the solution the C designers picked is dangerous, and not well suited for use in modern C++.

But **first, we need to cover two subtopics.**

18.2.1 Array to pointer conversions (array decay)

In most cases, when a C-style array is used in an expression, the **array will be implicitly converted into a pointer to the element type, initialized with the address of the first element** (with index 0). Colloquially, this is called **array decay** (or just **decay** for short).

There is nothing special about the pointer that an array decays into. It is a normal pointer that holds the address of the first element.

Similarly, a const array (e.g. `const int arr[5]`) decays into a pointer-to-const (`const int*`).

In C++, there are a few common cases where an C-style array doesn't decay:

- When used as an **argument to `sizeof()` or `typeid()`**.
- When **taking the address of the array using operator `&`**.
- When **passed as a member of a class type**.
- When **passed by reference**.

Because C-style arrays decay into a pointer in most cases, **it's a common fallacy to believe arrays are pointers**. This is **not the case**. An **array object is a sequence of elements**, whereas a pointer object just holds an address.

Key insight: A decayed array pointer does not know how long the array it is pointing to is. The term **"decay"** indicates this loss of length type information.

18.2.2 Subscripting a C-style array actually applies operator[] to the decayed pointer

Because a C-style arrays decays to a pointer when evaluated, when a C-style array is subscripted, the **subscript is actually operating on the decayed array pointer**:

```
const int arr[] { 9, 7, 5, 3, 1 };  
std::cout << arr[2]; // subscript decayed array to get element 2, prints 5
```

18.2.3 Array decay solves our C-style array passing issue

Array decay solves both challenges we encountered at the top of the lesson.

```
void printElementZero(const int* arr) // pass by const address
{
    std::cout << arr[0];
}
```

Key insight: C-style arrays are passed by address, even when it looks like they are passed by value.

Two C-style arrays with the same element type but different lengths will decay into the same pointer type.

18.2.4 C-style array function parameter syntax

One problem with declaring the function parameter as `int* arr` is that it's not obvious that `arr` is supposed to be a pointer to an array of values rather than a pointer to a single integer. For this reason, when passing a C-style array, it's preferable to use the alternate declaration form `int arr[]`:

```
void printElementZero(const int arr[]) // treated the same as const int*
{
    std::cout << arr[0];
}
```

Best practice: A function parameter expecting a C-style array type should use the array syntax (e.g. `int arr[]`) rather than pointer syntax (e.g. `int *arr`).

18.2.5 The problems with array decay

Although array decay was a clever solution to ensure C-style arrays of different lengths could be passed to a function without making expensive copies, the **loss of array length information makes it easy for several types of mistakes to be made.**

- **Sizeof()** will return different values for arrays and decayed arrays.
- array decay **can make refactoring** (breaking long functions into shorter, more modular functions) **difficult**. Code that works as expected with a non-decayed array may not compile (or worse, may silently malfunction) when the same code is using a decayed array.
- **Not having length information creates several programmatic challenges**. Without length information, it is difficult to sanity check the length of the array. Users can easily pass in arrays that are shorter than expected (or even pointers to a single value), which will then cause undefined behavior when they are subscripted with an invalid index.

18.2.6 Working around array length issues

Historically, programmers have worked around the lack of array length information via one of **two methods**.

First, we can **pass in both the array and the array length as separate arguments**:

```
void printElement2(const int arr[], int length)
```

However, this **still has a number of issues**.

Second, if there is an **element value that is not semantically valid** (e.g. a test score of -1), **we can instead mark the end of the array using an element of that value**. That way, the length of the array can be calculated by counting how many elements exist between the start of the array and this terminating element. The array can also be traversed by iterating from the start until we hit the terminating element. The nice thing about this method is that it works even with implicit function calls.

Key insight: C-style strings (which are C-style arrays) use a null-terminator to mark the end of the string, so that they can be traversed even if they have decayed.

But this method **also has a number of issues**.

18.2.7 C-style arrays should be avoided in most cases

Because of the non-standard passing semantics (pass by address is used instead of pass by value) and risks associated with decayed arrays losing their length information, C-style arrays have generally fallen out of favor. We recommend avoiding them as much as possible.

Best practice: Avoid C-style arrays whenever practical.

- Prefer **std::string_view** for read-only strings (string literal symbolic constants and string parameters).
- Prefer **std::string** for modifiable strings.
- Prefer **std::array** for non-global constexpr arrays. (Closest to C-Style arrays)
- Prefer **std::vector** for non-constexpr arrays. (E.g. array instantiated at runtime)

It is okay to use C-style arrays for global constexpr arrays. We'll discuss this in a moment.

18.2.8 So, when are C-style arrays used in modern C++?

1. To store constexpr global (or constexpr static local) program data. Because such arrays can be accessed directly from anywhere in the program, we do not need to pass the array, which avoids decay-related issues. The syntax for defining C-style arrays can be a little less wonky than std::array. More importantly, indexing such arrays does not have sign conversion issues like the standard library container classes do.
2. As parameters to functions or classes that want to handle non-constexpr C-style string arguments directly (rather than requiring a conversion to std::string_view). There are two possible reasons for this: First, converting a non-constexpr C-style string to a std::string_view requires traversing the C-style string to determine its length. If the function is in a performance critical section of code and the length isn't needed (e.g. because the function is going to traverse the string anyway) then avoiding the conversion may be useful. Second, if the function (or class) calls other functions that expect C-style strings, converting to a std::string_view just to convert back may be suboptimal (unless you have other reasons for wanting a std::string_view).

18.3 Pointer arithmetic and subscripting

In this lesson, we'll take a deeper look at how arrays indexing math works.

Although we won't use the indexing math in future lessons, the topics covered in this lesson will give your insight into how range-based for loops actually work, and will come in handy again later when we cover iterators.

Pointer arithmetic returns the address of the next/previous object (based on the type being pointed to), not the next/previous address.

18.3.1 What is pointer arithmetic?

Pointer arithmetic is a feature that allows us to **apply certain integer arithmetic operators** (addition, subtraction, increment, or decrement) **to a pointer to produce a new memory address**.

Given some pointer `ptr`, **`ptr + 1` returns the address of the next object in memory** (based on the type being pointed to). So, if `ptr` is an `int*`, and an `int` is 4 bytes, `ptr + 1` will return the memory address that is 4 bytes after `ptr`, and `ptr + 2` will return the memory address that is 8 bytes after `ptr`.

```
int x {};  
const int* ptr{ &x }; // assume 4 byte ints  
  
std::cout << ptr << ' ' << (ptr + 1) << ' ' << (ptr + 2) << '\n';
```

On the author's machine, this printed: **00AFFD80 00AFFD84 00AFFD88**

Note that **each memory address is 4 bytes greater** than the previous.

Although less common, pointer arithmetic **also works with subtraction**.

Applying the increment (`++`) and decrement (`--`) operators to a pointer do the same thing as pointer addition and pointer subtraction respectively, but actually modify the address held by the pointer.

Given some `int` value `x`, `++x` is shorthand for `x = x + 1`. Similarly, given some pointer `ptr`, `++ptr` is shorthand for `ptr = ptr + 1`, which does pointer arithmetic and assigns the result back to `ptr`.

```
int x {};  
const int* ptr{ &x }; // assume 4 byte ints  
  
std::cout << ptr << '\n';  
  
++ptr; // ptr = ptr + 1  
std::cout << ptr << '\n';  
  
--ptr; // ptr = ptr - 1  
std::cout << ptr << '\n';
```

On the author's machine, this printed: **00AFFD80 00AFFD84 00AFFD80**

Warning: Technically, the above is undefined behavior. According to the C++ standard, pointer arithmetic is only defined behavior when the pointer and the result are within the same array (or one-past-the-end). However, modern C++ implementations generally do not enforce this, and will typically not disallow you from using pointer arithmetic outside of arrays.

18.3.2 *Subscripting is implemented via pointer arithmetic*

In the prior lesson (C-style array decay), we noted that operator[] can be applied to a pointer:

```
const int arr[] { 9, 7, 5, 3, 1 };  
  
const int* ptr{ arr }; // a normal pointer holding the address of element 0  
std::cout << ptr[2]; // subscript ptr to get element 2, prints 5
```

It turns out that subscript operation **ptr[n]** is a **concise syntax equivalent to the more verbose expression `*((ptr) + (n))`**. You'll note that this is **just pointer arithmetic, with some additional parenthesis to ensure things evaluate in the correct order, and an implicit dereference to get the object at that address.**

First, we initialize ptr with arr. When arr is used as an initializer, it decays into a pointer holding the address of the element with index 0. So ptr now holds the address of element 0.

Next, we print ptr[2]. ptr[2] is equivalent to `*((ptr) + (2))`, which is equivalent to `*(ptr + 2)`. ptr + 2 returns the address of the object that is two objects past ptr, which is the element with index 2. The object at that address is then returned to the caller.

This is the primary reason arrays are 0-based rather than 1-based. It makes the math more efficient (because the compiler doesn't have to subtract 1 whenever subscripting)!

18.3.3 *Pointer arithmetic and subscripting are relative addresses*

When first learning about array subscripting, **it's natural to assume that the index represents a fixed element within the array**: Index 0 is always the first element, index 1 is always the second element, etc...

This is an illusion. Array **indices are actually relative positions**. The indices just appear fixed because we almost always index from the start (element 0) of the array!

Remember, given some pointer `ptr`, **both `*(ptr + 1)` and `ptr[1]` return the next object in memory** (based on the type being pointed to). Next is a relative term, not an absolute one. Thus, if `ptr` is pointing to element 0, then both `*(ptr + 1)` and `ptr[1]` will return element 1. **But if `ptr` is pointing to element 3 instead, then both `*(ptr + 1)` and `ptr[1]` will return element 4!**

```
const int arr[] { 9, 8, 7, 6, 5 };
const int *ptr { arr }; // arr decays into a pointer to element 0

// Prove that we're pointing at element 0
std::cout << *ptr << ptr[0] << '\n'; // prints 99
// Prove that ptr[1] is element 1
std::cout << *(ptr+1) << ptr[1] << '\n'; // prints 88

// Now set ptr to point at element 3
ptr = &arr[3];

// Prove that we're pointing at element 3
std::cout << *ptr << ptr[0] << '\n'; // prints 66
// Prove that ptr[1] is element 4!
std::cout << *(ptr+1) << ptr[1] << '\n'; // prints 55
```

However, you'll also note that **our program is a lot more confusing if we can't assume that `ptr[1]` is always the element with index 1**. For this reason, **we recommend using subscripting only when indexing from the start of the array (element 0)**. Use pointer arithmetic only when doing relative positioning.

Best practice:

- Favor subscripting when indexing from the start of the array (element 0), so the array indices line up with the element.
- Favor pointer arithmetic when doing relative positioning from a given element.

18.3.4 Negative indices

In the last lesson, we mentioned that (unlike the standard library container classes) the **index of a C-style array can be either an unsigned integer or a signed integer**. This wasn't done just for convenience -- **it's actually possible to index a C-style array with a negative subscript**. It sounds funny, but it makes sense.

We just covered that `*(ptr+1)` returns the next object in memory. And `ptr[1]` is just a convenient syntax to do the same.

At the top of this lesson, we noted that `*(ptr-1)` returns the previous object in memory. **Want to guess what the subscript equivalent is? Yup, `ptr[-1]`.**

```
const int arr[] { 9, 8, 7, 6, 5 };

// Set ptr to point at element 3
const int* ptr { &arr[3] };

// Prove that we're pointing at element 3
std::cout << *ptr << ptr[0] << '\n'; // prints 66
// Prove that ptr[-1] is element 2!
std::cout << *(ptr-1) << ptr[-1] << '\n'; // prints 77
```

18.3.5 Pointer arithmetic can be used to traverse an array

One of the most **common uses of pointer arithmetic is to iterate through a C-style array without explicit indexing**. The following example illustrates how this is done:

```
constexpr int arr[] { 9, 7, 5, 3, 1 };

const int* begin{ arr };           // begin points to start element
const int* end{ arr + std::size(arr) }; // end points to one-past-the-end element

for (; begin != end; ++begin)      // iterate from begin up to (but excluding) end
{
    std::cout << *begin << ' ';    // dereference our loop variable to get the current element
}
```

Note that `end` is set to one-past-the-end of the array. Having `end` hold this address is fine (so long as we don't dereference `end`, as there isn't a valid element at that address). We do this because it makes our math and comparisons as simple as possible (no need to add or subtract 1 anywhere).

18.3.6 Summary

Why is `arr[0]` the same as `*arr`?

`arr[0]` is shorthand for `*((arr) + (0))`, which is equal to `*(arr + 0)`, which equals `*arr`.

18.4 C-style strings

In lesson 5: Literals, we defined a string as a collection of sequential characters (such as “Hello, world!”), and introduced C-style string literals. We also noted that the C-style string literal “Hello, world!” has type `const char[14]` (13 explicit characters plus 1 hidden null-terminator character).

If you hadn’t connected the dots before, it should be obvious now that **C-style strings are just C-style arrays whose element type is `char` or `const char`!**

18.4.1 Defining C-style strings

```
char str1[8]{};           // an array of 8 char, indices 0 through 7
const char str2[] { "string" }; // an array of 7 char, indices 0 through 6
constexpr char str3[] { "hello" }; // an array of 6 const char, indices 0 through 5
```

Remember that we need an extra character for the implicit null terminator.

When defining C-style strings with an initializer, we **highly recommend omitting the array length and letting the compiler calculate the length**. That way if the initializer changes in the future, **you won’t have to remember to update the length, and there is no risk in forgetting to include an extra element to hold the null terminator**.

18.4.2 C-style strings will decay

Because C-style strings are C-style arrays, they will decay -- **C-style string literals decay into a `const char*`**, and C-style string arrays decay into either a `const char*` or `char*` depending on whether the array is `const`. And when a C-style string decays into a pointer, the length of the string (encoded in the type information) is lost.

This loss of length information is the reason C-style strings have a null-terminator. The length of the string can be (inefficiently) regenerated by counting the number of elements between the start of the string and the null terminator.

18.4.3 Other C-style string manipulating functions

Because C-style strings are the primary string type in C, the C language provides many functions for manipulating C-style strings. These functions have been inherited by C++ as part of the `<cstring>` header.

Here are a few of the most useful that you may see in older code:

- `strlen()` -- returns the length of a C-style string
- `strcpy()`, `strncpy()`, `strcpy_s()` -- overwrites one C-style string with another
- `strcat()`, `strncat()` -- Appends one C-style string to the end of another
- `strcmp()`, `strncmp()` -- Compares two C-style strings (returns 0 if equal)

Except for `strlen()`, we generally recommend avoiding these.

18.4.4 Avoid non-const C-style string objects

Unless you have a specific, compelling reason to use non-const C-style strings, they are best avoided, as they are awkward to work with and are prone to overruns, which will cause undefined behavior (and are potential security issues).

In the rare case that you do need to work with C-style strings or fixed buffer sizes (e.g. for memory-limited devices), we'd recommend using a well-tested 3rd party fixed-length string library designed for the purpose.

Best practice: Avoid non-const C-style string objects in favor of `std::string`.

18.5 C-style string symbolic constants

C++ supports **two different ways to create C-style string symbolic constants**:

```
const char name[] { "Alex" }; // case 1: const C-style string initialized with C-style string literal
const char* const color{ "Orange" }; // case 2: const pointer to C-style string literal
```

While the above two methods produce the same results, **C++ deals with the memory allocation** for these slightly differently.

In **case 1**, “Alex” is **put into** (probably **read-only**) **memory** somewhere. Then the program allocates memory for a C-style array of length 5 (four explicit characters plus the null terminator), and initializes that memory with the string “Alex”. So, **we end up with 2 copies of “Alex”** -- one in global memory somewhere, and the other owned by name. Since name is const (and will never be modified), **making a copy is inefficient**.

In **case 2**, how the compiler handles this is implementation defined. What usually happens is that the **compiler places the string “Orange” into read-only memory** somewhere, and **then initializes the pointer** with the address of the string.

18.5.1 Type deduction with const C-style strings

Type deduction using a C-style string literal is fairly straightforward:

```
1 auto s1{ "Alex" }; // type deduced as const char*
2 auto* s2{ "Alex" }; // type deduced as const char*
3 auto& s3{ "Alex" }; // type deduced as const char(&)[5]
```

18.5.2 Outputting pointers and C-style strings

You may have noticed something interesting about the way `std::cout` handles pointers of different types.

```
int narr[] { 9, 7, 5, 3, 1 };
char carr[] { "Hello!" };
const char* ptr { "Alex" };

std::cout << narr << '\n'; // narr will decay to type int*
std::cout << carr << '\n'; // carr will decay to type char*
std::cout << ptr << '\n'; // name is already type char*
```

On the author's machine, this printed:

003AF738

Hello!

Alex

Why did the int array print an address, but the character arrays print as strings?

The answer is that the **output streams** (e.g. `std::cout`) **make** some **assumptions about your intent**. If you pass it a non-char pointer, it will simply print the contents of that pointer (the address that the pointer is holding). However, if you pass it an object of type `char*` or `const char*`, it will assume you're intending to print a string. Consequently, instead of printing the pointer's value (an address), it will print the string being pointed to instead!

Best practice: Avoid C-style string symbolic constants in favor of `constexpr std::string_view`.

19 Dynamic Allocation

19.1 Dynamic memory allocation with new and delete

19.1.1 *The need for dynamic memory allocation*

C++ supports three basic types of memory allocation, of which you've already seen two.

- **Static memory allocation** happens for static and global variables. Memory for these types of variables is allocated once when your program is run and persists throughout the life of your program.
- **Automatic memory allocation** happens for function parameters and local variables. Memory for these types of variables is allocated when the relevant block is entered, and freed when the block is exited, as many times as necessary.
- **Dynamic memory allocation** is the topic of this article.

Both static and automatic allocation have two things in common:

- The **size** of the variable / array **must be known at compile time**.
- Memory **allocation and deallocation happens automatically** (when the variable is instantiated / destroyed).

Most of the time, this is just fine. However, you will come across situations where one or both of these constraints cause problems, **usually when dealing with external (user or file) input**.

If we have to declare the size of everything at compile time, the **best we can do is try to make a guess the maximum size** of variables we'll need and hope that's enough:

```
char name[25]; // let's hope their name is less than 25 chars!
Record record[500]; // let's hope there are less than 500 records!
Monster monster[40]; // 40 monsters maximum
Polygon rendering[30000]; // this 3d rendering better not have more than 30,000 polygons!
```

This is a **poor solution** for at least **four reasons**:

- First, it leads to **wasted memory if the variables aren't actually used**.
- Second, **how do we tell which bits of memory are actually used?** That necessitates having some way to tell the status of each monster, which adds complexity and can use up additional memory.
- Third, most normal variables (including fixed arrays) are allocated in a portion of memory called the **stack**. The **amount of stack memory** for a program is generally **quite small** -- Visual Studio defaults the stack size to 1MB. If you exceed this number, stack overflow will result, and the operating system will probably close down the program.
- Fourth, and most importantly, **it can lead to artificial limitations and/or array overflows**.

Fortunately, **these problems are easily addressed via dynamic memory allocation**. Dynamic memory allocation is a way for running programs to request memory from the operating system when needed. This memory does not come from the program's limited stack memory -- **instead, it is allocated from a much larger pool of memory managed by the operating system called the heap**. On modern machines, the **heap can be gigabytes in size**.

19.1.2 Dynamically allocating single variables

To allocate a *single* variable dynamically, we use the **scalar (non-array) form** of the **new** operator:

```
new int; // dynamically allocate an integer (and discard the result)
```

In the above case, we're requesting an integer's worth of memory from the operating system. The **new** operator creates the object using that memory, and then **returns a pointer containing the address of the memory that has been allocated**.

Most often, we'll assign the return value to our own pointer variable so we can access the allocated memory later.

```
int* ptr{ new int }; // dynamically allocate an integer and assign the address to ptr so we can access it later
```

We can then dereference the pointer to access the memory:

```
*ptr = 7; // assign value of 7 to allocated memory
```


If it wasn't before, it should now be clear **at least one case in which pointers are useful**. Without a pointer to hold the address of the memory that was just allocated, we'd have no way to access the memory that was just allocated for us!

Note that **accessing heap-allocated objects is generally slower** than accessing stack-allocated objects. Because the compiler knows the address of stack-allocated objects, it can go directly to that address to get a value. Heap allocated objects are typically accessed via pointer. This requires two steps: one to get the address of the object (from the pointer), and another to get the value.

19.1.3 How does dynamic memory allocation work?

Your computer has memory (probably lots of it) that is available for applications to use. When you run an application, your **operating system loads the application into some of that memory**. This **memory used by your application is divided into different areas**, each of which serves a different purpose. **One area contains your code. Another area is used for normal operations** (keeping track of which functions were called, creating and destroying global and local variables, etc...). We'll talk more about those later. **However, much of the memory available just sits there, waiting to be handed out to programs that request it.**

When you dynamically allocate memory, you're asking the operating system to reserve some of that memory for your program's use. If it can fulfill this request, it will return the address of that memory to your application. From that point forward, your application can use this memory as it wishes. When your application is done with the memory, it can return the memory back to the operating system to be given to another program.

Unlike static or automatic memory, the **program itself is responsible for requesting and disposing of dynamically allocated memory.**

Key insight:

The allocation and deallocation for stack objects is done automatically. There is no need for us to deal with memory addresses -- the code the compiler writes can do this for us.

The **allocation and deallocation for heap objects is not done automatically**. We need to be involved. That means we need some unambiguous way to refer to a specific heap allocated object, so that we can request its destruction when we're ready. And the way we reference such objects is via memory address.

When we use operator new, it returns a pointer containing the memory address of the newly allocated object. We generally want to store that in a pointer so we can use that address later to access the object (and eventually, request its destruction).

19.1.4 Initializing a dynamically allocated variable

When you dynamically allocate a variable, you can also initialize it via direct initialization or uniform initialization:

```
1 int* ptr1{ new int (5) }; // use direct initialization
2 int* ptr2{ new int { 6 } }; // use uniform initialization
```

Deleting a single variable

When we are done with a dynamically allocated variable, we need to explicitly tell C++ to free the memory for reuse. For single variables, this is done via the scalar (non-array) form of the **delete** operator:

```
1 // assume ptr has previously been allocated with operator new
2 delete ptr; // return the memory pointed to by ptr to the operating system
3 ptr = nullptr; // set ptr to be a null pointer
```

19.1.5 What does it mean to delete memory?

The **delete operator does not actually delete** anything. It **simply returns the memory being pointed to back to the operating system**. The **operating system is then free to reassign that memory to another application** (or to this application again later).

Although it looks like we're deleting a variable, this is not the case! The pointer variable still has the same scope as before, and can be assigned a new value just like any other variable.

Note that **deleting a pointer that is not pointing to dynamically allocated memory may cause bad things to happen**.

Dangling pointers

C++ does not make any guarantees about what will happen to the contents of deallocated memory, or to the value of the pointer being deleted. In most cases, the memory returned to the operating system will contain the same values it had before it was returned, and the pointer will be left pointing to the now deallocated memory.

A pointer that is pointing to deallocated memory is called a **dangling pointer**. Dereferencing or deleting a dangling pointer will lead to undefined behavior. Consider the following program:

```
int* ptr{ new int }; // dynamically allocate an integer
*ptr = 7; // put a value in that memory location

delete ptr; // return the memory to the operating system. ptr is now a dangling pointer.

std::cout << *ptr; // Dereferencing a dangling pointer will cause undefined behavior
delete ptr; // trying to deallocate the memory again will also lead to undefined behavior.
```

Deallocating memory may create multiple dangling pointers. Consider the case where you have a pointer to a pointer or to multiple pointers.

There are a **few best practices that can help here**.

First, **try to avoid having multiple pointers point at the same piece of dynamic memory**. If this is not possible, be clear about which pointer “owns” the memory (and is responsible for deleting it) and which pointers are just accessing it.

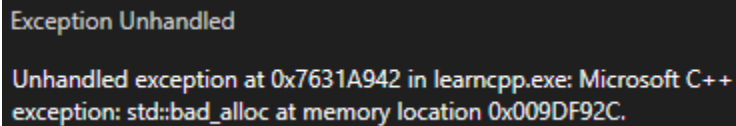
Second, **when you delete a pointer, if that pointer is not going out of scope immediately afterward, set the pointer to nullptr**. We'll talk more about null pointers, and why they are useful in a bit.

Best practice: Set deleted pointers to nullptr unless they are going out of scope immediately afterward.

19.1.6 Operator new can fail

When requesting memory from the operating system, **in rare circumstances, the operating system may not have any memory to grant the request with.**

By default, **if new fails, a bad_alloc exception is thrown** (usually when the heap is out of memory). If this exception isn't properly handled (and it won't be, since we haven't covered exceptions or exception handling yet), the program will simply terminate (crash) with an unhandled exception error.



```
Exception Unhandled
Unhandled exception at 0x7631A942 in learncpp.exe: Microsoft C++
exception: std::bad_alloc at memory location 0x009DF92C.
```

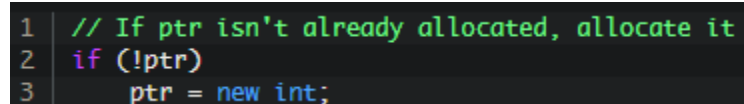
In many cases, having new throw an exception (or having your program crash) is undesirable, so there's an **alternate form of new that can be used instead to tell new to return a null pointer if memory can't be allocated.** This is **done by adding the constant std::nothrow between the new keyword and the allocation type:**

```
int* value { new (std::nothrow) int }; // value will be set to a null pointer if the integer allocation fails
```

In the above example, if new fails to allocate memory, it will return a null pointer instead of the address of the allocated memory.

19.1.7 Null pointers and dynamic memory allocation

Null pointers (pointers set to nullptr) are particularly useful when dealing with dynamic memory allocation. **In the context of dynamic memory allocation, a null pointer basically says “no memory has been allocated to this pointer”.** This allows us to do things like conditionally allocate memory:



```
1 // If ptr isn't already allocated, allocate it
2 if (!ptr)
3     ptr = new int;
```

Best practice: Deleting a null pointer is okay, and does nothing. There is no need to conditionalize your delete statements.

19.1.8 Memory leaks

Dynamically allocated memory stays allocated until it is explicitly deallocated or until the program ends (and the operating system cleans it up, assuming your operating system does that). However, the **pointers used to hold dynamically allocated memory addresses follow the normal scoping rules for local variables**. This mismatch can create interesting problems.

Consider the following function:

```
void doSomething()
{
    int* ptr{ new int{} };
}
```

This function allocates an integer dynamically, but never frees it using `delete`. Because pointers variables are just normal variables, **when the function ends, `ptr` will go out of scope**. And **because `ptr` is the only variable holding the address of the dynamically allocated integer, when `ptr` is destroyed there are no more references to the dynamically allocated memory**. This means **the program has now “lost” the address of the dynamically allocated memory**. As a result, this **dynamically allocated integer cannot be deleted**.

This is called a **memory leak**. Memory leaks happen **when your program loses the address of some bit of dynamically allocated memory before giving it back to the operating system**. When this happens, your program can't delete the dynamically allocated memory, because it no longer knows where it is. The **operating system also can't use this memory, because that memory is considered to be still in use by your program**.

Memory leaks eat up free memory while the program is running, making less memory available not only to this program, but to other programs as well. Programs with severe memory leak problems can eat all the available memory, **causing the entire machine to run slowly or even crash**. **Only after your program terminates is the operating system able to clean up and “reclaim” all leaked memory**.

Although memory leaks can result from a pointer going out of scope, there are other ways that memory leaks can result. For example, a memory leak can occur if a pointer holding the address of the dynamically allocated memory is assigned another value:

```
int value = 5;
int* ptr{ new int{} }; // allocate memory
ptr = &value; // old address lost, memory leak results
```

This can be fixed by deleting the pointer before reassigning it:

```
int value{ 5 };  
int* ptr{ new int{} }; // allocate memory  
delete ptr; // return memory back to operating system  
ptr = &value; // reassign pointer to address of value
```

Relatedly, it is also possible to get a memory leak via double-allocation. The address returned from the second allocation overwrites the address of the first allocation. Consequently, **the first allocation becomes a memory leak!**

Similarly, **this can be avoided by ensuring you delete the pointer before reassigning.**

19.1.9 Conclusion

Operators new and delete allow us to dynamically allocate single variables for our programs.

Dynamically allocated memory has dynamic duration and will stay allocated until you deallocate it or the program terminates.

Be careful not to perform dereference a dangling or null pointers.

In the next lesson, we'll take a look at using new and delete to allocate and delete arrays.

19.2 Dynamically allocating arrays

In addition to dynamically allocating single values, **we can also dynamically allocate arrays of variables**. Unlike a fixed array, where the array size must be fixed at compile time, dynamically allocating an array allows us to choose an array length at runtime (**meaning our length does not need to be constexpr**).

To allocate an array dynamically, we use the array form of new and delete (often called new[] and delete[]):

```
std::cout << "Enter a positive integer: ";
std::size_t length{};
std::cin >> length;

int* array{ new int[length]{} }; // use array new. Note that length does not need to be constant!

std::cout << "I just allocated an array of integers of length " << length << '\n';

array[0] = 5; // set element 0 to value 5

delete[] array; // use array delete to deallocate array

// we don't need to set array to nullptr/0 here because it's going out of scope immediately after this anyway
```

Because we are allocating an array, C++ knows that it should use the array version of new instead of the scalar version of new. Essentially, the new[] operator is called, even though the [] isn't placed next to the new keyword.

The length of dynamically allocated arrays has type std::size_t. If you are using a non-constexpr int, you'll need to static_cast to std::size_t since that is considered a narrowing conversion and your compiler will warn otherwise.

Note that because this memory is allocated from a different place than the memory used for fixed arrays, the size of the array can be quite large. You can run the program above and allocate an array of length 1,000,000 (or probably even 100,000,000) without issue. Try it! Because of this, programs that need to allocate a lot of memory in C++ typically do so dynamically.

19.2.1 Dynamically deleting arrays

When deleting a dynamically allocated array, we have to use the array version of delete, which is delete[].

This tells the CPU that it needs to clean up multiple variables instead of a single variable. One of the most common mistakes that new programmers make when dealing with dynamic memory allocation is to use delete instead of delete[] when deleting a dynamically allocated array. Using the scalar version of delete on an array will result in undefined behavior, such as data corruption, memory leaks, crashes, or other problems.

One often asked question of array delete[] is, “How does array delete know how much memory to delete?” The answer is that array new[] keeps track of how much memory was allocated to a variable, so that array delete[] can delete the proper amount. Unfortunately, this size/length isn’t accessible to the programmer.

```
int* array{ new int[length]{} }; // use array new. Note that length does not need to be constant!
std::cout << "I just allocated an array of integers of length " << length << '\n';
array[0] = 5; // set element 0 to value 5
delete[] array; // use array delete to deallocate array
```

19.2.2 Dynamic arrays are almost identical to fixed arrays

In lesson: C-style array decay, you learned that a fixed array holds the memory address of the first array element. You also learned that a fixed array can decay into a pointer that points to the first element of the array. In this decayed form, the length of the fixed array is not available (and therefore neither is the size of the array via sizeof()), but otherwise there is little difference.

A dynamic array starts its life as a pointer that points to the first element of the array. Consequently, it has the same limitations in that it doesn’t know its length or size. A dynamic array functions identically to a decayed fixed array, with the exception that the programmer is responsible for deallocating the dynamic array via the delete[] keyword.

19.2.3 Initializing dynamically allocated arrays

If you want to initialize a dynamically allocated array to 0, the syntax is quite simple:

```
1 | int* array{ new int[length]{} };
```

Prior to C++11, there was no easy way to initialize a dynamic array to a non-zero value (initializer lists only worked for fixed arrays). This means you had to loop through the array and assign element values explicitly.

```
1 | int* array = new int[5];
2 | array[0] = 9;
3 | array[1] = 7;
4 | array[2] = 5;
5 | array[3] = 3;
6 | array[4] = 1;
```

Super annoying!

However, **starting with C++11, it's now possible to initialize dynamic arrays using initializer lists!**

```
1 | int fixedArray[5] = { 9, 7, 5, 3, 1 }; // initialize a fixed array before C++11
2 | int* array{ new int[5]{ 9, 7, 5, 3, 1 } }; // initialize a dynamic array since C++11
3 | // To prevent writing the type twice, we can use auto. This is often done for types with long names.
4 | auto* array{ new int[5]{ 9, 7, 5, 3, 1 } };
```

Note that this syntax has no operator= between the array length and the initializer list.

For consistency, fixed arrays can also be initialized using uniform initialization:

```
1 | int fixedArray[]{ 9, 7, 5, 3, 1 }; // initialize a fixed array in C++11
2 | char fixedArray[]{ "Hello, world!" }; // initialize a fixed array in C++11
```

Explicitly stating the size of the array is optional.

19.2.4 Resizing arrays

Dynamically allocating an array allows you to set the array length at the time of allocation. However, **C++ does not provide a built-in way to resize an array that has already been allocated**. It is possible to work around this limitation by dynamically allocating a new array, copying the elements over, and deleting the old array. However, this is error prone, especially when the element type is a class (which have special rules governing how they are created).

Consequently, we recommend avoiding doing this yourself. Use `std::vector` instead.

19.3 RAI

RAII (Resource Acquisition Is Initialization) is a programming technique whereby **resource use is tied to the lifetime of objects with automatic duration** (e.g. non-dynamically allocated objects). In C++, **RAII is implemented via classes with constructors and destructors**. A resource (such as memory, a file or database handle, etc...) is **typically acquired in the object's constructor** (though it can be acquired after the object is created if that makes sense). That resource can then be used while the object is alive. The resource is released in the destructor, when the object is destroyed. The **primary advantage of RAII is that it helps prevent resource leaks** (e.g. memory not being deallocated) as all resource-holding objects are cleaned up automatically.

19.4 Pointers to pointers and dynamic multidimensional arrays

This lesson is optional, for advanced readers who want to learn more about C++. No future lessons build on this lesson.

A pointer to a pointer is exactly what you'd expect: a pointer that holds the address of another pointer.

19.4.1 Pointers to pointers

A normal pointer to an int is declared using a single asterisk:

```
1 | int* ptr; // pointer to an int, one asterisk
```

A pointer to a pointer to an int is declared using two asterisks:

```
1 | int** ptrptr; // pointer to a pointer to an int, two asterisks
```

A pointer to a pointer works just like a normal pointer — you can dereference it to retrieve the value pointed to. And because that value is itself a pointer, you can dereference it again to get to the underlying value. These dereferences can be done consecutively:

```
6 | int** ptrptr { &ptr };  
7 | std::cout << **ptrptr << '\n'; // dereference to get pointer to int, dereference again to get int value
```

Note that you cannot set a pointer to a pointer directly to a value:

```
int** ptrptr { &&value }; // not valid
```

19.4.2 Arrays of pointers

Pointers to pointers have a few uses. The most common use is to dynamically allocate an array of pointers:

```
int** array { new int*[10] }; // allocate an array of 10 int pointers
```

This works just like a standard dynamically allocated array, except the array elements are of type “pointer to integer” instead of integer.

19.4.3 Passing a pointer by address

Much like we can use a pointer parameter to change the actual value of the underlying argument passed in, **we can pass a pointer to a pointer to a function and use that pointer to change the value of the pointer it points to** (confused yet?).

However, if we want a function to be able to modify what a pointer argument points to, this is generally better done using a reference to a pointer instead. This is covered in lesson: Pass by address (part 2).

19.4.4 Pointer to a pointer to a pointer to...

It’s also possible to declare a pointer to a pointer to a pointer. You can even declare a pointer to a pointer to a pointer to a pointer or higher, if you wish.

This can be used to dynamically allocate a three-dimensional array. However, doing so would require a loop inside a loop, and is extremely complicated to get correct.

Conclusion: We recommend avoiding using pointers to pointers unless no other options are available, because they’re complicated to use and potentially dangerous. It’s easy enough to dereference a null or dangling pointer with normal pointers — it’s doubly easy with a pointer to a pointer since you have to do a double-dereference to get to the underlying value!

19.5 Void pointers

The **void pointer**, also known as the **generic pointer**, is a special type of pointer that can be pointed at objects of any data type! A void pointer is declared like a normal pointer, using the void keyword as the pointer's type:

```
void* ptr {}; // ptr is a void pointer
```

A void pointer **can point to objects of any data type**. However, because the void pointer does not know what type of object it is pointing to, **dereferencing a void pointer is illegal**. Instead, the **void pointer must first be cast to another pointer type** before the dereference can be performed.

```
int value{ 5 };
void* voidPtr{ &value };

// std::cout << *voidPtr << '\n'; // illegal: dereference of void pointer

int* intPtr{ static_cast<int*>(voidPtr) }; // however, if we cast our void pointer to an int pointer...
std::cout << *intPtr << '\n'; // then we can dereference the result
```

The next obvious question is: If a void pointer doesn't know what it's pointing to, how do we know what to cast it to? **Ultimately, that is up to you to keep track of.**

19.5.1 Void pointer miscellany

Void pointers can be set to a null value:

```
void* ptr{ nullptr }; // ptr is a void pointer that is currently a null pointer
```

Because a void pointer does not know what type of object it is pointing to, **deleting a void pointer will result in undefined behavior**. If you need to delete a void pointer, **static_cast** it back to the appropriate type first.

It is **not possible to do pointer arithmetic on a void pointer**. This is because **pointer arithmetic requires the pointer to know what size object it is pointing to**, so it can increment or decrement the pointer appropriately.

Note that **there is no such thing as a void reference**. This is because a **void reference would be of type void &**, and would not know what type of value it referenced.

19.5.2 Conclusion

In general, it is a **good idea to avoid using void pointers** unless absolutely necessary, **as they effectively allow you to avoid type checking**. This **allows you to inadvertently do things that make no sense**, and the **compiler won't complain about it**.

However, very **occasionally, you may still find a reasonable use for the void pointer**. Just make sure there isn't a better (safer) way to do the same thing using other language mechanisms first!

20 Functions

20.1 Function Pointers

In lesson: Introduction to pointers, you learned that a pointer is a variable that holds the address of another variable. **Function pointers are similar, except that instead of pointing to variables, they point to functions!**

Identifier `foo` is the function's name. But what type is the function? Functions have their own l-value function type -- in this case, **a function type that returns an integer and takes no parameters**. Much like variables, functions live at an assigned address in memory.

When a function is called (via the `()` operator), execution jumps to the address of the function being called:

```
int foo() // code for foo starts at memory address 0x002717f0
{
    return 5;
}

int main()
{
    foo(); // jump to address 0x002717f0

    return 0;
}
```

At some point in your programming career (if you haven't already), you'll probably make a simple mistake:

```
std::cout << foo << '\n';
```

Instead of calling function `foo()` and printing the return value, we've unintentionally sent function `foo` directly to `std::cout`. What happens in this case?

operator<< does not know how to output a function pointer (because there are an infinite number of possible function pointers). The standard says that in this case, **foo should be converted to a bool** (which operator<< does know how to print). And **since the function pointer for foo is a non-void pointer**, it should always evaluate to Boolean true. Thus, **this should print: 1.**

Tip: Some compilers (e.g. **Visual Studio**) have a compiler extension that prints the address of the function instead: **0x002717f0**

Just like it is possible to declare a non-constant pointer to a normal variable, it's also possible to declare a non-constant pointer to a function. In the rest of this lesson, we'll examine these function pointers and their uses. Function pointers are a fairly advanced topic, and the rest of this lesson can be safely skipped or skimmed by those only looking for C++ basics.

20.1.1 Pointers to functions

The syntax for creating a non-const function pointer is **one of the ugliest things you will ever see in C++:**

```
// fcnPtr is a pointer to a function that takes no arguments and returns an integer
int (*fcnPtr)();
```

In the above snippet, fcnPtr is a pointer to a function that has no parameters and returns an integer. **fcnPtr can point to any function that matches this type.**

The parentheses around *fcnPtr are necessary for precedence reasons, as **int* fcnPtr()** would be interpreted as a **forward declaration** for a function named fcnPtr that takes no parameters and returns a pointer to an integer.

To make a const function pointer, the const goes after the asterisk:

```
int (*const fcnPtr)();
```

If you put the const before the int, then that would indicate the function being pointed to would return a const int.

Here are some legal and illegal examples:

<code>int i;</code>	an int
<code>int *p;</code>	an int pointer (ptr to an int)
<code>int a[];</code>	an array of ints
<code>int f();</code>	a function returning an int
<code>int **pp;</code>	a pointer to an int pointer (ptr to a ptr to an int)
<code>int (*pa)[];</code>	a pointer to an array of ints
<code>int (*pf)();</code>	a pointer to a function returning an int
<code>int *ap[];</code>	an array of int pointers (array of ptrs to ints)
<code>int aa[][];</code>	an array of arrays of ints
<code>int af[]();</code>	an array of functions returning an int (ILLEGAL)
<code>int *fp();</code>	a function returning an int pointer
<code>int fa()[];</code>	a function returning an array of ints (ILLEGAL)
<code>int ff()();</code>	a function returning a function returning an int (ILLEGAL)
<code>int ***ppp;</code>	a pointer to a pointer to an int pointer
<code>int (**ppa)[];</code>	a pointer to a pointer to an array of ints
<code>int (**ppf)();</code>	a pointer to a pointer to a function returning an int
<code>int *(*pap)[];</code>	a pointer to an array of int pointers
<code>int (*paa)[][];</code>	a pointer to an array of arrays of ints
<code>int (*paf)[]();</code>	a pointer to a an array of functions returning an int (ILLEGAL)
<code>int *(*pfp)();</code>	a pointer to a function returning an int pointer
<code>int (*pfa)()[];</code>	a pointer to a function returning an array of ints (ILLEGAL)
<code>int (*pff)()();</code>	a pointer to a function returning a function returning an int (ILLEGAL)
<code>int **app[];</code>	an array of pointers to int pointers
<code>int (*apa[])[];</code>	an array of pointers to arrays of ints
<code>int (*apf[])();</code>	an array of pointers to functions returning an int
<code>int *aap[][];</code>	an array of arrays of int pointers
<code>int aaa[][][];</code>	an array of arrays of arrays of ints
<code>int aaf[][]();</code>	an array of arrays of functions returning an int (ILLEGAL)
<code>int *afp[]();</code>	an array of functions returning int pointers (ILLEGAL)
<code>int afa[]()[];</code>	an array of functions returning an array of ints (ILLEGAL)
<code>int aff[]()();</code>	an array of functions returning functions returning an int (ILLEGAL)
<code>int **fpp();</code>	a function returning a pointer to an int pointer
<code>int (*fpa())[];</code>	a function returning a pointer to an array of ints
<code>int (*fpf)()();</code>	a function returning a pointer to a function returning an int
<code>int *fap()[];</code>	a function returning an array of int pointers (ILLEGAL)
<code>int faa()[][];</code>	a function returning an array of arrays of ints (ILLEGAL)
<code>int faf>[]()();</code>	a function returning an array of functions returning an int (ILLEGAL)
<code>int *ffp()();</code>	a function returning a function returning an int pointer (ILLEGAL)

20.1.2 Assigning a function to a function pointer

Function pointers can be initialized with a function (and non-const function pointers can be assigned a function). Like with pointers to variables, we can also use `&foo` to get a function pointer to `foo`.

```
int (*fcnPtr)() { &foo }; // fcnPtr points to function foo
fcnPtr = &goo; // fcnPtr now points to function goo
```

One common mistake is to do this:

```
1 | fcnPtr = goo();
```

This tries to assign the return value from a call to function `goo()` (which has type `int`) to `fcnPtr` (which is expecting a value of type `int(*)()`), which isn't what we want. We want `fcnPtr` to be assigned the address of function `goo`, not the return value from function `goo()`. So, no parentheses are needed.

Note that the **type (parameters and return type)** of the function pointer must match the type of the function.

```
1 // function prototypes
2 int foo();
3 double goo();
4 int hoo(int x);
5
6 // function pointer initializers
7 int (*fcnPtr1)() { &foo }; // okay
8 int (*fcnPtr2)() { &goo }; // wrong -- return types don't match!
9 double (*fcnPtr4)() { &goo }; // okay
10 fcnPtr1 = &hoo; // wrong -- fcnPtr1 has no parameters, but hoo() does
11 int (*fcnPtr3)(int) { &hoo }; // okay
```

Unlike fundamental types, C++ **will** implicitly convert a function into a function pointer if needed (so you don't need to use the address-of operator (`&`) to get the function's address). However, function pointers will not convert to void pointers, or vice-versa (though some compilers like Visual Studio may allow this anyway).

```
// function prototypes
int foo();

// function initializations
int (*fcnPtr5)() { foo }; // okay, foo implicitly converts to function pointer to foo
void* vPtr { foo }; // not okay, though some compilers may allow
```


Function pointers can also be initialized or assigned the value `nullptr`:

```
int (*fcnPtr)() { nullptr }; // okay
```

20.1.3 Calling a function using a function pointer

The other primary thing you can do with a function pointer is use it to actually call the function. There are **two ways to do this**.

The first is via **explicit dereference**:

```
int (*fcnPtr)(int){ &foo }; // Initialize fcnPtr with function foo
(*fcnPtr)(5); // call function foo(5) through fcnPtr.
```

The second way is via **implicit dereference**:

```
int (*fcnPtr)(int){ &foo }; // Initialize fcnPtr with function foo
fcnPtr(5); // call function foo(5) through fcnPtr.
```

As you can see, the **implicit dereference method** looks just like a normal function call -- which is what you'd expect, since **normal function names are pointers to functions anyway!** However, **some older compilers do not support the implicit dereference method**, but all modern compilers should.

Also **note that because function pointers can be set to `nullptr`, it's a good idea to assert or conditionally test whether your function pointer is a null pointer before calling it**. Just like with normal pointers, dereferencing a null function pointer leads to undefined behavior.

```
int (*fcnPtr)(int){ &foo }; // Initialize fcnPtr with function foo
if (fcnPtr) // make sure fcnPtr isn't a null pointer
    fcnPtr(5); // otherwise this will lead to undefined behavior
```

20.1.4 *Passing functions as arguments to other functions*

One of the most useful things to do with function pointers is **pass a function as an argument to another function**. Functions used as arguments to another function are sometimes called **callback functions**.

Consider a case where you are writing a function to perform a task (such as sorting an array), but you want the user to be able to define how a particular part of that task will be performed (such as whether the array is sorted in ascending or descending order). Let's take a closer look at this problem as applied specifically to sorting, as an example that can be generalized to other similar problems.

Many **comparison-based sorting algorithms work on a similar concept**: the sorting algorithm iterates through a list of numbers, does comparisons on pairs of numbers, and reorders the numbers based on the results of those comparisons. Consequently, by varying the comparison, we can change the way the algorithm sorts without affecting the rest of the sorting code.

```
#include <utility> // for std::swap

void SelectionSort(int* array, int size)
{
    if (!array)
        return;

    // Step through each element of the array
    for (int startIndex{ 0 }; startIndex < (size - 1); ++startIndex)
    {
        // smallestIndex is the index of the smallest element we've encountered so far.
        int smallestIndex{ startIndex };

        // Look for smallest element remaining in the array (starting at startIndex+1)
        for (int currentIndex{ startIndex + 1 }; currentIndex < size; ++currentIndex)
        {
            // If the current element is smaller than our previously found smallest
            if (ascending(array[smallestIndex], array[currentIndex])) // COMPARISON DONE HERE
            {
                // This is the new smallest number for this iteration
                smallestIndex = currentIndex;
            }
        }

        // Swap our start element with our smallest element
        std::swap(array[startIndex], array[smallestIndex]);
    }
}
```

Now, in order to let the caller decide how the sorting will be done, instead of using our own hard-coded comparison function, we'll allow the caller to provide their own sorting function! This is done via a function pointer.

Because the caller's comparison function is going to compare two integers and return a boolean value, a pointer to such a function would look something like this:

```
1 | bool (*comparisonFcn)(int, int);
```

So, we'll allow the caller to pass our sort routine a pointer to their desired comparison function as the third parameter, and then we'll use the caller's function to do the comparison.

```
// Note our user-defined comparison is the third parameter
void selectionSort(int* array, int size, bool (*comparisonFcn)(int, int))
{
    if (!array || !comparisonFcn)
        return;

    // Step through each element of the array
    for (int startIndex{ 0 }; startIndex < (size - 1); ++startIndex)
    {
        // bestIndex is the index of the smallest/largest element we've encountered so far.
        int bestIndex{ startIndex };

        // Look for smallest/largest element remaining in the array (starting at startIndex+1)
        for (int currentIndex{ startIndex + 1 }; currentIndex < size; ++currentIndex)
        {
            // If the current element is smaller/larger than our previously found smallest
            if (comparisonFcn(array[bestIndex], array[currentIndex])) // COMPARISON DONE HERE
            {
                // This is the new smallest/largest number for this iteration
                bestIndex = currentIndex;
            }
        }

        // Swap our start element with our smallest/largest element
        std::swap(array[startIndex], array[bestIndex]);
    }
}

// Here is a comparison function that sorts in ascending order
// (Note: it's exactly the same as the previous ascending() function)
bool ascending(int x, int y)
{
    return x > y; // swap if the first element is greater than the second
}

// Here is a comparison function that sorts in descending order
bool descending(int x, int y)
{
    return x < y; // swap if the second element is greater than the first
}
```

```
// This function prints out the values in the array
void printArray(int* array, int size)
{
    if (!array)
        return;

    for (int index{ 0 }; index < size; ++index)
    {
        std::cout << array[index] << ' ';
    }

    std::cout << '\n';
}

int main()
{
    int array[9]{ 3, 7, 9, 5, 6, 1, 8, 2, 4 };

    // Sort the array in descending order using the descending() function
    selectionSort(array, 9, descending);
    printArray(array, 9);

    // Sort the array in ascending order using the ascending() function
    selectionSort(array, 9, ascending);
    printArray(array, 9);

    return 0;
}
```

This program produces the result:

9 8 7 6 5 4 3 2 1

1 2 3 4 5 6 7 8 9

Is that cool or what? We've given the caller the ability to control how our selection sort does its job.

The caller can even define their own “strange” comparison functions:

```
bool evensFirst(int x, int y)
{
    // if x is even and y is odd, x goes first (no swap needed)
    if ((x % 2 == 0) && !(y % 2 == 0))
        return false;

    // if x is odd and y is even, y goes first (swap needed)
    if (!(x % 2 == 0) && (y % 2 == 0))
        return true;

    // otherwise sort in ascending order
    return ascending(x, y);
}
```

As you can see, using a function pointer in this context provides a nice way to allow a caller to “hook” their own functionality into something you’ve previously written and tested, which helps facilitate code reuse! Previously, if you wanted to sort one array in descending order and another in ascending order, you’d need multiple versions of the sort routine. Now you can have one version that can sort any way the caller desires!

Note: If a function parameter is of a function type, it will be converted to a pointer to the function type. This means:

```
1 | void selectionSort(int* array, int size, bool (*comparisonFcn)(int, int))
```

can be equivalently written as:

```
1 | void selectionSort(int* array, int size, bool comparisonFcn(int, int))
```

This **only works for function parameters**, and so is **of somewhat limited use**. On a non-function parameter, the latter is interpreted as a forward declaration:

```
1 | bool (*ptr)(int, int); // definition of function pointer ptr
2 | bool fcn(int, int);    // forward declaration of function fcn
```

20.1.5 Providing default functions

If you're going to allow the caller to pass in a function as a parameter, it can often be useful to provide some standard functions for the caller to use for their convenience. For example, in the selection sort example above, providing the `ascending()` and `descending()` function along with the `selectionSort()` function would make the caller's life easier, as they wouldn't have to rewrite `ascending()` or `descending()` every time they want to use them.

You can even **set one of these as a default parameter**:

```
1 // Default the sort to ascending sort
2 void selectionSort(int* array, int size, bool (*comparisonFcn)(int, int) = ascending);
```

20.1.6 Making function pointers prettier with type aliases

Let's face it -- the syntax for pointers to functions is ugly. However, **type aliases can be used to make pointers to functions look more like regular variables**:

```
1 using ValidateFunction = bool (*)(int, int);
```

This defines a type alias called "**ValidateFunction**" that is a **pointer to a function** that takes two ints and returns a bool.

Now instead of doing this:

```
1 bool validate(int x, int y, bool (*fcnPtr)(int, int)); // ugly
```

You can do this:

```
1 bool validate(int x, int y, ValidateFunction pfcn) // clean
```

20.1.7 Using `std::function`

An **alternate method of defining and storing function pointers** is to use `std::function`, which is part of the standard library `<functional>` header. To define a function pointer using this method, declare a `std::function` object like so:

```
1 #include <functional>
2 bool validate(int x, int y, std::function<bool(int, int)> fcn); // std::function method that returns a bool and takes two int parameters
```

As you see, **both the return type and parameters go inside angled brackets**, with the **parameters inside parentheses**. If there are no parameters, the parentheses can be left empty.

Updating our earlier example with `std::function`:

```
int foo()
{
    return 5;
}

int goo()
{
    return 6;
}

int main()
{
    std::function<int()> fcnPtr{ &foo }; // declare function pointer that returns an int and takes no parameters
    fcnPtr = &goo; // fcnPtr now points to function goo
    std::cout << fcnPtr() << '\n'; // call the function just like normal

    std::function fcnPtr2{ &foo }; // can also use CTAD to infer template arguments
}
```

However, **type aliasing `std::function` can still be helpful for readability**.

Also note that **`std::function` only allows calling the function via implicit dereference** (e.g. `fcnPtr()`), not explicit dereference (e.g. `(*fcnPtr)()`).

When defining a type alias, we must **explicitly specify any template arguments**. We **can't use CTAD in this case since there is no initializer to deduce the template arguments from**.

20.1.8 Type inference for function pointers

Much like the *auto* keyword can be used to infer the type of normal variables, the ***auto* keyword** can also infer the type of a function pointer.

```
3 int foo(int x)
4 {
5     return x;
6 }
7
8 int main()
9 {
10     auto fcnPtr{ &foo };
11     std::cout << fcnPtr(5) << '\n';
12
13     return 0;
14 }
```

This works exactly like you'd expect, and the syntax is very clean. The **downside** is, of course, that **all of the details about the function's parameters types and return type are hidden**, so it's **easier to make a mistake when making a call with the function**, or using its return value.

20.1.9 Conclusion

Function pointers are **useful primarily when you want to store functions in an array** (or other structure), **or when you need to pass a function to another function**. Because the native syntax to declare function pointers is ugly and error prone, we recommend using `std::function`. In places where a function pointer type is only used once (e.g. a single parameter or return value), `std::function` can be used directly. In places where a function pointer type is used multiple times, a type alias to a `std::function` is a better choice (to prevent repeating yourself).

20.2 The stack and the heap

The **memory** that a program uses is **typically divided into a few different areas, called segments**:

- The **code segment** (also called a **text segment**), where the **compiled program** sits in memory. The code segment is **typically read-only**.
- The **bss segment** (also called the **uninitialized data segment**), where **zero-initialized global and static variables are stored**.
- The **data segment** (also called the **initialized data segment**), where **initialized global and static variables are stored**.
- The **heap**, where **dynamically allocated variables are allocated from**.
- The **call stack**, where **function parameters, local variables, and other function-related information are stored**.

For this lesson, we'll focus primarily on the heap and the stack, as that is where most of the interesting stuff takes place.

20.2.1 The heap segment

The heap segment (also known as the “free store”) **keeps track of memory used for dynamic memory allocation**. In C++, when you use the **new operator** to allocate memory, **this memory is allocated in the application's heap segment**.

The address of this memory is passed back by operator new, and can then be stored in a pointer. You do not have to worry about the mechanics behind the process of how free memory is located and allocated to the user. However, **it is worth knowing that sequential memory requests may not result in sequential memory addresses** being allocated!

```
1 int* ptr1 { new int };
2 int* ptr2 { new int };
3 // ptr1 and ptr2 may not have sequential addresses
```

When a dynamically allocated variable is deleted, the memory is “returned” to the heap and can then be reassigned as future allocation requests are received. **Remember that deleting a pointer does not delete the variable**, it just returns the memory at the associated address back to the operating system.

The **heap** has advantages and disadvantages:

- Allocating memory on the heap is comparatively **slow**.
- Allocated **memory stays allocated until it is specifically deallocated** (beware memory leaks) or the application ends (at which point the OS should clean it up).
- Dynamically allocated memory **must be accessed through a pointer**. Dereferencing a pointer is **slower than accessing a variable directly**.
- Because the heap is a **big pool of memory**, large arrays, structures, or classes can be allocated here.

20.2.2 The call stack

The **call stack** (usually referred to as “**the stack**”) has a much more interesting role to play. The call stack **keeps track of all the active functions** (those that have been called but have not yet terminated) **from the start of the program to the current point of execution**, and **handles allocation of all function parameters and local variables**.

The **call stack is implemented as a stack data structure**. So, before we can talk about how the call stack works, we need to understand what a stack data structure is.

20.2.3 The stack data structure

A **data structure** is a programming **mechanism for organizing data** so that it can be used efficiently. You’ve already seen several types of data structures, **such as arrays and structs**. Both of these data structures **provide mechanisms for storing data and accessing that data** in an efficient way. There are many additional data structures that are commonly used in programming, quite a few of which are implemented in the standard library, and a stack is one of those.

Consider a stack of plates in a cafeteria. **Because each plate is heavy and they are stacked, you can really only do one of three things:**

1. Look at the surface of the top plate
2. Take the top plate off the stack (exposing the one underneath, if it exists)
3. Put a new plate on top of the stack (hiding the one underneath, if it exists)

In computer programming, a stack is a container data structure that holds multiple variables (much like an array). However, whereas an array lets you access and modify elements in any order you wish (called **random access**), a stack is more limited. The operations that can be performed on a stack correspond to the three things mentioned above:

1. Look at the top item on the stack (usually done via a function called **top()**, but sometimes called **peek()**)
2. Take the top item off of the stack (done via a function called **pop()**)
3. Put a new item on top of the stack (done via a function called **push()**)

A stack is a **last-in, first-out (LIFO)** structure. The last item pushed onto the stack will be the first item popped off. If you put a new plate on top of the stack, the first plate removed from the stack will be the plate you just pushed on last. Last on, first off. **As items are pushed onto a stack, the stack grows larger -- as items are popped off, the stack grows smaller.**

```
Stack: empty
Push 1
Stack: 1
Push 2
Stack: 1 2
Push 3
Stack: 1 2 3
Pop
Stack: 1 2
Pop
Stack: 1
```

The plate analogy is a pretty good analogy as to how the call stack works, but we can make a better analogy. Consider a bunch of mailboxes, all stacked on top of each other. Each mailbox can only hold one item, and all mailboxes start out empty. Furthermore, each mailbox is nailed to the mailbox below it, so the number of mailboxes cannot be changed. If we can't change the number of mailboxes, how do we get a stack-like behavior?

First, we use a marker (like a post-it note) to keep track of where the bottom-most empty mailbox is. In the beginning, this will be the lowest mailbox (on the bottom of the stack). When we push an item onto our mailbox stack, we put it in the mailbox that is marked (which is the first empty mailbox), and move the marker up one mailbox. When we pop an item off the stack, we move the marker down one mailbox (so **it's pointed at the top non-empty mailbox**) and remove the item from that mailbox. Anything below the marker is considered "**on the stack**". Anything at the marker or above the marker is not on the stack.

20.2.4 The call stack segment

The **call stack segment holds the memory used for the call stack**. When the application starts, the `main()` function is pushed on the call stack by the operating system. Then the program begins executing.

When a function call is encountered, the function is pushed onto the call stack. When the current function ends, that function is popped off the call stack (this process is sometimes called **unwinding the stack**). Thus, **by looking at the functions that are currently on the call stack, we can see all of the functions that were called to get to the current point of execution**.

Our mailbox analogy above is fairly analogous to how the call stack works. The stack itself is a fixed-size chunk of memory addresses. The mailboxes are memory addresses, and the “items” we’re pushing and popping on the stack are called **stack frames**. A stack frame **keeps track of all of the data associated with one function call**. We’ll talk more about stack frames in a bit. The “marker” is a register (a small piece of memory in the CPU) **known as the stack pointer** (sometimes abbreviated “SP”). The stack pointer **keeps track of where the top of the call stack currently is**.

We can make one further optimization: When we pop an item off the call stack, we only have to move the stack pointer down -- we don’t have to clean up or zero the memory used by the popped stack frame (the equivalent of emptying the mailbox). **This memory is no longer considered to be “on the stack” (the stack pointer will be at or below this address)**, so it won’t be accessed. **If we later push a new stack frame to this same memory, it will overwrite the old value we never cleaned up**.

20.2.5 The call stack in action

Let's examine in more detail how the call stack works. Here is the sequence of steps that takes place when a function is called:

1. The program **encounters a function call**.
2. A **stack frame is constructed and pushed on the stack**. The stack frame consists of:
 - The address of the instruction beyond the function call (called the **return address**). This is how the CPU remembers where to return to after the called function exits.
 - All **function arguments**.
 - **Memory for any local variables**
 - **Saved copies of any registers modified by the function that need to be restored when the function returns**
3. The CPU jumps to the function's start point.
4. The instructions inside of the function begin executing.

When the function terminates, the following steps happen:

1. **Registers are restored** from the call stack
2. The **stack frame is popped off the stack**. This **frees the memory for all local variables and arguments**.
3. The **return value is handled**.
4. The **CPU resumes execution at the return address**.

Return values can be handled in a number of different ways, depending on the computer's architecture. **Some architectures include the return value as part of the stack frame. Others use CPU registers.**

Typically, it is not important to know all the details about how the call stack works. However, understanding that functions are effectively pushed on the stack when they are called and popped off (unwound) when they return gives you the fundamentals needed to understand recursion, as well as some other concepts that are useful when debugging.

A technical note: on some architectures, the call stack grows away from memory address 0. On others, it grows towards memory address 0. As a consequence, newly pushed stack frames may have a higher or a lower memory address than the previous ones.

20.2.6 A quick and dirty call stack example

Consider the following simple application:

```
1  int foo(int x)
2  {
3      // b
4      return x;
5  } // foo is popped off the call stack here
6
7  int main()
8  {
9      // a
10     foo(5); // foo is pushed on the call stack here
11     // c
12
13     return 0;
14 }
```

The call stack looks like the following at the labeled points:

a:

main()

b:

foo() (including parameter x)

main()

c:

main()

20.2.7 Stack overflow

The stack has a limited size, and consequently can only hold a limited amount of information. On **Visual Studio for Windows, the default stack size is 1MB**. With **g++/Clang** for Unix variants, it **can be as large as 8MB**. If the program tries to put too much information on the stack, stack overflow will result. Stack overflow happens when all the memory in the stack has been allocated -- in that case, further allocations begin overflowing into other sections of memory.

Stack overflow is generally the result of allocating too many variables on the stack, and/or making too many nested function calls (where function A calls function B calls function C calls function D etc...) On modern operating systems, overflowing the stack will generally cause your OS to issue an access violation and terminate the program.

20.2.8 The stack has advantages and disadvantages:

- Allocating memory on the stack is comparatively **fast**.
- Memory allocated on the stack stays **in scope as long as it is on the stack**. It is destroyed when it is popped off the stack.
- All **memory allocated on the stack is known at compile time**. Consequently, this memory can be accessed directly through a variable.
- Because the **stack is relatively small**, it is generally not a good idea to do anything that eats up lots of stack space. This includes allocating or copying large arrays or other memory-intensive structures.

20.3 Recursion

A **recursive function** in C++ is a function that calls itself. Here is an example of a poorly-written recursive function:

```
void countDown(int count)
{
    std::cout << "push " << count << '\n';
    countDown(count-1); // countDown() calls itself recursively
}
```

Because the `countDown()` function never returns (it just calls `countDown()` again), **this information is never being popped off the stack!** Consequently, at some point, the computer will run out of stack memory, stack overflow will result, and the program will crash or terminate.

20.3.1 Recursive termination conditions

Recursive function calls generally work just like normal function calls. However, the program above illustrates the most important difference with recursive functions: **you must include a recursive termination condition**, or they will run “forever” (actually, until the call stack runs out of memory). A **recursive termination** is a condition that, when met, will cause the recursive function to stop calling itself.

```
if (count > 1) // termination condition
    countDown(count-1);
```

20.3.2 Recursive algorithms

Recursive functions typically solve a problem by first finding the solution to a subset of the problem (recursively), and then modifying that sub-solution to get to a solution. In the above algorithm, `sumTo(value)` first solves `sumTo(value-1)`, and then adds the value of variable `value` to find the solution for `sumTo(value)`.

In many recursive algorithms, some inputs produce trivial outputs. For example, `sumTo(1)` has the trivial output 1 (you can calculate this in your head), and does not benefit from further recursion. Inputs for which an algorithm trivially produces an output is called a base case. Base cases act as termination conditions for the algorithm. Base cases can often be identified by considering the output for an input of 0, 1, “”, “”, or null.

```
int fibonacci(int count)
{
    if (count == 0)
        return 0; // base case (termination condition)
    if (count == 1)
        return 1; // base case (termination condition)
    return fibonacci(count-1) + fibonacci(count-2);
}
```


20.3.3 Memoization algorithms

The above recursive Fibonacci algorithm isn't very efficient, in part because each call to a Fibonacci **non-base case results in two more Fibonacci calls**. This **produces an exponential number of function calls** (in fact, the above example calls fibonacci() 1205 times!). There are techniques that can be used to reduce the number of calls necessary. One technique, called **memoization**, **caches the results of expensive function calls so the result can be returned when the same input occurs again**.

```
// h/t to potterman28wxcv for a variant of this code
// count is now a std::size_t to make indexing the std::vector easier
int fibonacci(std::size_t count)
{
    // We'll use a static std::vector to cache calculated results
    static std::vector results{ 0, 1 };

    // If we've already seen this count, then use the cache'd result
    if (count < std::size(results))
        return results[count];

    // Otherwise calculate the new result and add it
    results.push_back(fibonacci(count - 1) + fibonacci(count - 2));
    return results[count];
}
```

This memoized version makes **35** function calls, which is much better than the **1205** of the original algorithm.

THIS IS **EXPONENTIALLY BETTER!**

20.3.4 Recursive vs iterative

One question that is often asked about recursive functions is, “**Why use a recursive function if you can do many of the same tasks iteratively (using a for loop or while loop)?**”. It turns out that **you can always solve a recursive problem iteratively -- however, for non-trivial problems, the recursive version is often much simpler to write (and read)**. For example, while it’s possible to write the Fibonacci function iteratively, it’s a little more difficult! (Try it!)

Iterative functions (those using a for-loop or while-loop) are almost always more efficient than their recursive counterparts. This is because every time you call a function there is some amount of overhead that takes place in pushing and popping stack frames. Iterative functions avoid this overhead.

That’s not to say iterative functions are always a better choice. Sometimes the **recursive implementation of a function is so much cleaner and easier to follow** that incurring a little extra overhead is more than worth it for the benefit in maintainability, **particularly if the algorithm doesn’t need to recurse too many times to find a solution.**

In general, recursion is a good choice when most of the following are true:

- The recursive code is much simpler to implement.
- The recursion depth can be limited (e.g. there’s no way to provide an input that will cause it to recurse down 100,000 levels).
- The iterative version of the algorithm requires managing a stack of data.
- This isn’t a performance-critical section of code.

However, **if the recursive algorithm is simpler to implement, it may make sense to start recursively and then optimize to an iterative algorithm later.**

Best practice: Generally, favor iteration over recursion, except when recursion really makes sense.

20.4 Ellipsis (and why to avoid them)

In all of the functions we’ve seen so far, the number of parameters a function will take must be known in advance (even if they have default values). However, **there are certain cases where it can be useful to be able to pass a variable number of parameters to a function.** C++ provides a **special specifier known as ellipsis** (aka “...”) that allow us to do precisely this.

Because ellipsis are **rarely used, potentially dangerous, and we recommend avoiding their use**, this section can be considered optional reading.

20.4.1 Ellipsis example

```
1  #include <iostream>
2  #include <stdarg.h> // needed to use ellipsis
3
4  // The ellipsis must be the last parameter
5  // count is how many additional arguments we're passing
6  double findAverage(int count, ...)
7  {
8      int sum{ 0 };
9
10     // We access the ellipsis through a va_list, so let's declare one
11     std::va_list list;
12
13     // We initialize the va_list using va_start. The first argument is
14     // the list to initialize. The second argument is the last non-ellipsis
15     // parameter.
16     va_start(list, count);
17
18     // Loop through all the ellipsis values
19     for (int arg{ 0 }; arg < count; ++arg)
20     {
21         // We use va_arg to get values out of our ellipsis
22         // The first argument is the va_list we're using
23         // The second argument is the type of the value
24         sum += va_arg(list, int);
25     }
26
27     // Cleanup the va_list when we're done.
28     va_end(list);
29
30     return static_cast<double>(sum) / count;
31 }
32
33 int main()
34 {
35     std::cout << findAverage(5, 1, 2, 3, 4, 5) << '\n';
36     std::cout << findAverage(6, 1, 2, 3, 4, 5, 6) << '\n';
37
38     return 0;
39 }
```

20.4.2 *Why ellipsis are dangerous: Type checking is suspended*

Ellipsis offer the programmer a lot of flexibility to implement functions that can take a variable number of parameters. However, this flexibility comes with some downsides.

With regular function parameters, the compiler uses type checking to ensure the types of the function arguments match the types of the function parameters (or can be implicitly converted so they match). This helps ensure you don't pass a function an integer when it was expecting a string, or vice versa. However, note that **ellipsis parameters have no type declarations**. When using ellipsis, **the compiler completely suspends type checking for ellipsis parameters**. This means it is possible to send arguments of any type to the ellipsis! However, the downside is that the compiler will no longer be able to warn you if you call the function with ellipsis arguments that do not make sense. When using the ellipsis, it is completely up to the caller to ensure the function is called with ellipsis arguments that the function can handle. Obviously that leaves quite a bit of room for error (especially if the caller wasn't the one who wrote the function).

```
1 | std::cout << findAverage(6, 1.0, 2, 3, 4, 5, 6) << '\n';
```

20.4.3 *Why ellipsis are dangerous: ellipsis don't know how many parameters were passed*

Not only do the ellipsis throw away the *type* of the parameters, it also throws away the *number* of parameters in the ellipsis. This means **we have to devise our own solution for keeping track of the number of parameters passed into the ellipsis**. Typically, this is done in one of three ways:

- Pass a length parameter
- Use a sentinel value
- Use a decoder string

20.4.4 Recommendations for safer use of ellipsis

First, if possible, do not use ellipsis at all! Oftentimes, other reasonable solutions are available, even if they require slightly more work. For example, in our `findAverage()` program, we could have passed in a dynamically sized array of integers instead. This would have provided both strong type checking (to make sure the caller doesn't try to do something nonsensical) while preserving the ability to pass a variable number of integers to be averaged.

Second, if you do use ellipsis, it is better if all values passed to the ellipsis parameter are the same type (e.g. all `int`, or all `double`, not a mix of each). Intermixing different types vastly increases the possibility of the caller inadvertently passing in data of the wrong type and `va_arg()` producing a garbage result.

Third, using a count parameter or decoder string parameter is generally safer than using a sentinel value. This forces the user to pick an appropriate value for the count/decoder parameter, which ensures the ellipsis loop will terminate after a reasonable number of iterations even if it produces a garbage value.

20.5 Introduction to lambdas (anonymous functions)

20.5.1 Lambdas are anonymous functions

A **lambda expression** (also called a **lambda** or **closure**) allows us to define an anonymous function inside another function. **The nesting is important**, as it allows us both to **avoid namespace naming pollution**, and to **define the function as close to where it is used as possible** (providing additional context).

The syntax for lambdas is one of the weirder things in C++, and takes a bit of getting used to. Lambdas take the form:

```
[ captureClause ] ( parameters ) -> returnType
{
    statements;
}
```

- The capture clause can be empty if no captures are needed.
- The parameter list can be empty if no parameters are required. It can also be omitted entirely unless a return type is specified.
- The return type is optional, and if omitted, auto will be assumed (thus using type deduction used to determine the return type). While we previously noted that type deduction for function return types should be avoided, in this context, it's fine to use (because these functions are typically so trivial).

Also note that lambdas (being anonymous) have no name, so we don't need to provide one.

This means a trivial lambda definition looks like this:

```
[] {}; // a lambda with an omitted return type, no captures, and omitted parameters.
```

Best practice: Following the best practice of defining things in the smallest scope and closest to first use, lambdas are preferred over normal functions when we need a trivial, one-off function to pass as an argument to some other function.

20.5.2 Type of a lambda

In the above example, we defined a lambda right where it was needed. This use of a lambda is sometimes called a function literal.

However, writing a lambda in the same line as it's used can sometimes make code harder to read. Much like we can initialize a variable with a literal value (or a function pointer) for use later, we can also initialize a lambda variable with a lambda definition and then use it later. A named lambda along with a good function name can make code easier to read.

For example, in the following snippet, we're using `std::all_of` to check if all elements of an array are even:

```
1 // Bad: We have to read the lambda to understand what's happening.
2 return std::all_of(array.begin(), array.end(), [](int i){ return ((i % 2) == 0); });
```

```
// Good: Instead, we can store the lambda in a named variable and pass it to the function.
auto isEven{
    [](int i)
    {
        return (i % 2) == 0;
    }
};

return std::all_of(array.begin(), array.end(), isEven);
```

Key insight:

Storing a lambda in a variable provides a way for us to give the lambda a useful name, which can help make our code more readable.

Storing a lambda in a variable also provides us with a way to use that lambda more than once.

Best practice

When storing a lambda in a variable, use auto as the variable's type.

When passing a lambda to a function:

- If C++20 capable, use auto as the parameter's type.
- Otherwise, use a function with a type template parameter or std::function parameter (or a function pointer if the lambda has no captures).

20.5.3 Generic lambdas

For the most part, lambda parameters work by the same rules as regular function parameters.

One notable exception is that since C++14 we're allowed to use auto for parameters (note: in C++20, regular functions are able to use auto for parameters too). When a lambda has one or more auto parameter, the compiler will infer what parameter types are needed from the calls to the lambda.

Because lambdas with one or more auto parameter can potentially work with a wide variety of types, they are called **generic lambdas**.

20.5.4 *Constexpr lambdas*

As of C++17, lambdas are **implicitly constexpr** if the result satisfies the requirements of a constant expression. This **generally requires two things**:

- The lambda must either have no captures, or all captures must be constexpr.
- The functions called by the lambda must be constexpr. Note that many standard library algorithms and math functions weren't made constexpr until C++20 or C++23.

20.5.5 *Return type deduction and trailing return types*

If return type deduction is used, a lambda's return type is deduced from the return-statements inside the lambda, and all return statements in the lambda must return the same type (otherwise the compiler won't know which one to prefer).

In the case where we're returning different types, we have two options:

1. Do explicit casts to make all the return types match, or
2. **explicitly specify a return type** for the lambda, and let the compiler do implicit conversions.

The second case is usually the better choice:

```
// note: explicitly specifying this returns a double
auto divide{ [](int x, int y, bool intDivision) -> double {
    if (intDivision)
        return x / y; // will do an implicit conversion of result to double
    else
        return static_cast<double>(x) / y;
} };
```

20.5.6 *Standard library function objects*

For common operations (e.g. addition, negation, or comparison) you don't need to write your own lambdas, because the **standard library comes with many basic callable objects** that can be used instead. These are defined in the **<functional> header**.

20.5.7 Conclusion

Lambdas and the algorithm library may seem unnecessarily complicated when compared to a solution that uses a loop. However, this combination can allow some very powerful operations in just a few lines of code, and can be more readable than writing your own loops. On top of that, the algorithm library features powerful and easy-to-use parallelism, which you won't get with loops. Upgrading source code that uses library functions is easier than upgrading code that uses loops.

Lambdas are great, but they don't replace regular functions for all cases. Prefer regular functions for non-trivial and reusable cases.